

Basic Image Operations

Images as functions

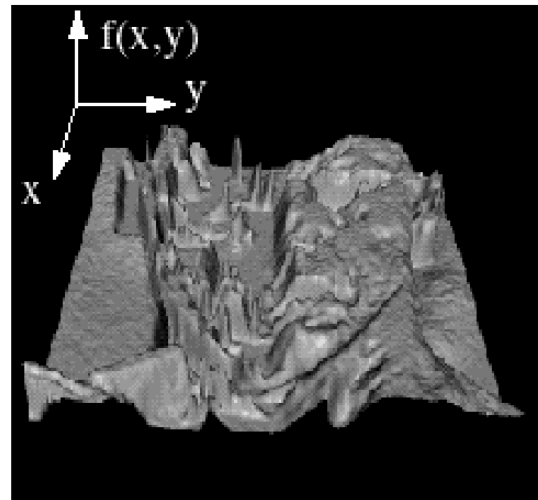
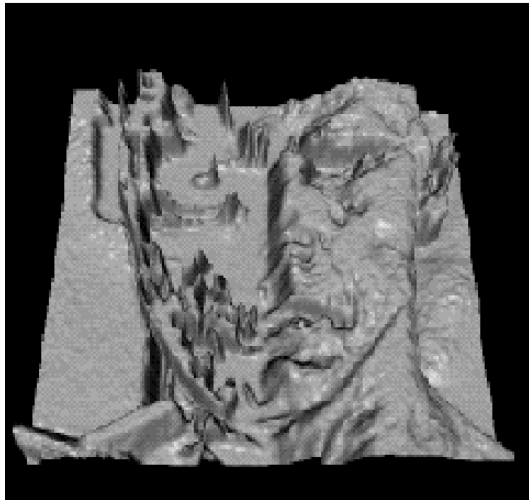
We can think of an **image** as a function, f , from $\mathbb{R}^2$ to $\mathbb{R}$:

- ◆ $f(x, y)$ gives the **intensity** at position (x, y)
- ◆ Realistically, we expect the image only to be defined over a rectangle, with a finite range:
 - $f: [a, b] \times [c, d] \rightarrow [0, 1]$

A color image is just three functions pasted together.
We can write this as a “vector-valued” function:

$$f(x, y) = \begin{bmatrix} r(x, y) \\ g(x, y) \\ b(x, y) \end{bmatrix}$$

Images as functions



What is a digital image?

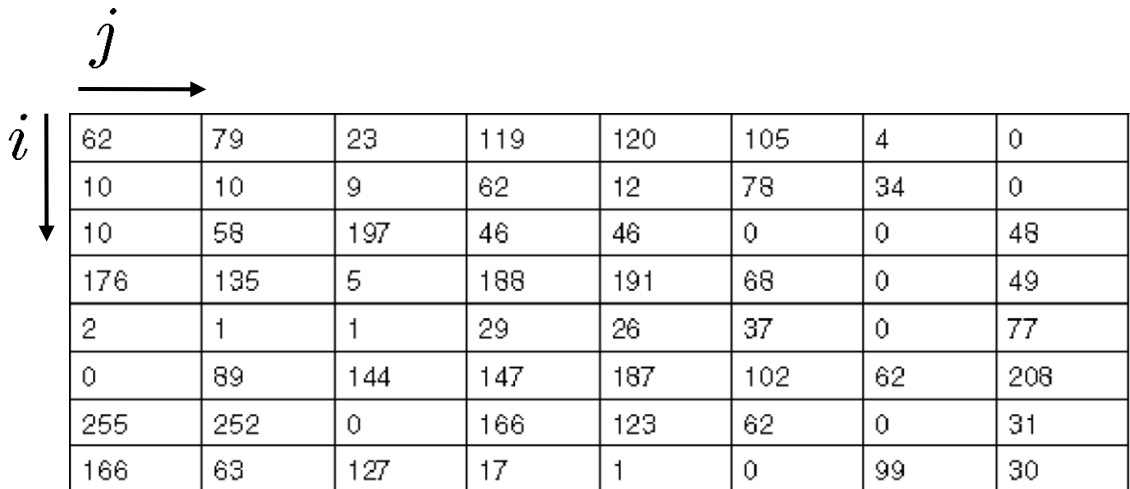
We usually work with **digital (discrete)** images:

- ◆ **Sample** the 2D space on a regular grid
- ◆ **Quantize** each sample (round to nearest integer)

If our samples are Δ apart, we can write this as:

$$f[i, j] = \text{Quantize}\{ f(i \Delta, j \Delta) \}$$

The image can now be represented as a matrix of integer values



62	79	23	119	120	105	4	0
10	10	9	62	12	78	34	0
10	58	197	46	46	0	0	48
176	135	5	188	191	68	0	49
2	1	1	29	26	37	0	77
0	89	144	147	187	102	62	208
255	252	0	166	123	62	0	31
166	63	127	17	1	0	99	30

Noise Removal - Filtering

Let's write this down as an equation. Assume the averaging window is $(2k+1) \times (2k+1)$:

$$G[i, j] = \frac{1}{(2k + 1)^2} \sum_{u=-k}^k \sum_{v=-k}^k F[i + u, j + v]$$

We can generalize this idea by allowing different weights for different neighboring pixels:

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

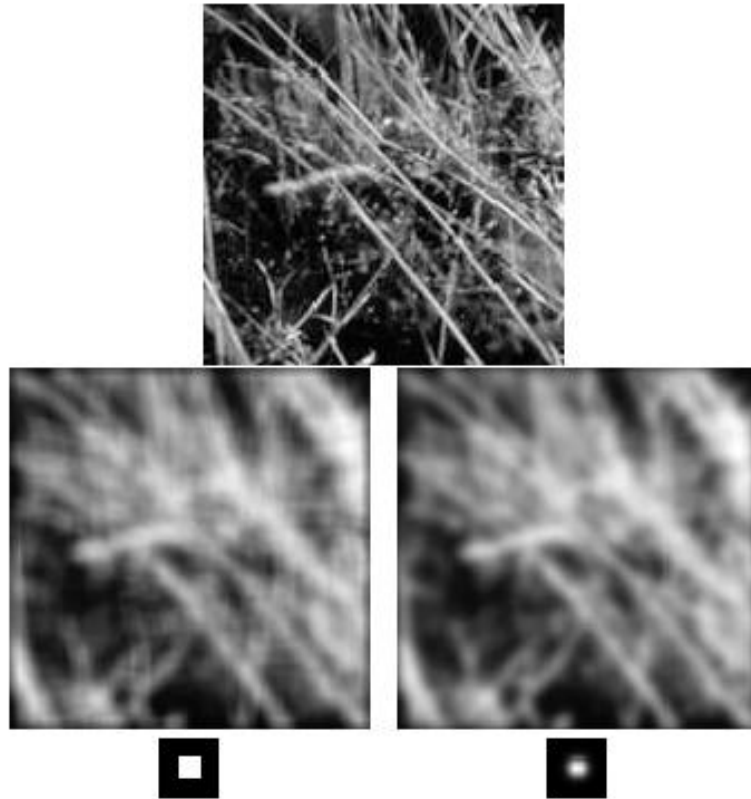
This is called a **cross-correlation** operation and written:

$$G = H \otimes F$$

H is called the “filter,” “kernel,” or “mask.”

The above allows negative filter indices. When you implement need to use: $H[u+k, v+k]$ instead of $H[u, v]$

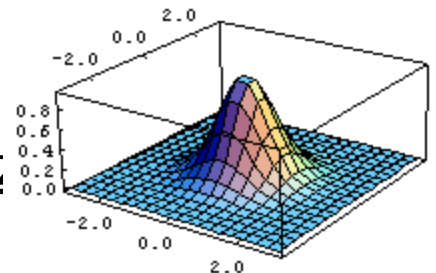
Mean vs Gaussian filtering



This kernel is an approximation of a Gaussian function:

$$h(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{\sigma^2}}$$

What happens if you increase σ ?



$\frac{1}{16}$	1	2	1
	2	4	2
	1	2	1

Convolution

A **convolution** operation is a cross-correlation where the filter is flipped both horizontally and vertically before being applied to the image:

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

It is written: $G = H \star F$

How does convolution differ from cross-correlation?

Suppose F is an impulse function. What will G look like?

Image gradient

How can we differentiate a *digital* image $F[x,y]$?

- ◆ Option 1: reconstruct a continuous image, f , then take gradient
- ◆ Option 2: take discrete derivative (finite difference)

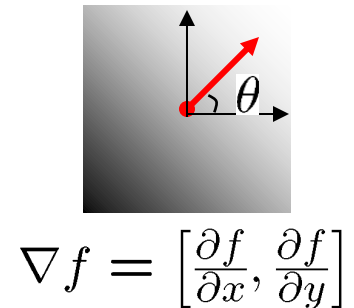
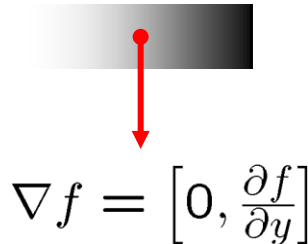
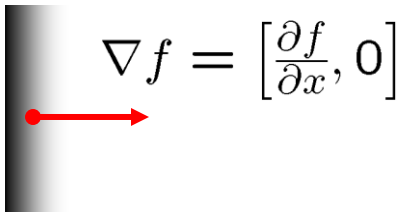
$$\frac{\partial f}{\partial x}[x, y] \approx F[x + 1, y] - F[x, y]$$

How would you implement this as a cross-correlation?

Image gradient

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

It points in the direction of most rapid change in intensity



The gradient direction is given by:

$$\theta = \tan^{-1} \left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x} \right)$$

- ◆ how does this relate to the direction of the edge?

The *edge strength* is given by the gradient magnitude

$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$



original



smoothed (5x5 Gaussian)



smoothed – original
(scaled by 4, offset +128)

Why does
this work?

Continuous Filters

We can also apply filters to *continuous* images.

In the case of cross correlation: $g = h \otimes f$

$$g(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} h(u, v) f(x + u, y + v) du dv$$

In the case of convolution: $g = h \star f$

$$g(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} h(u, v) f(x - u, y - v) du dv$$

Edges and Lines

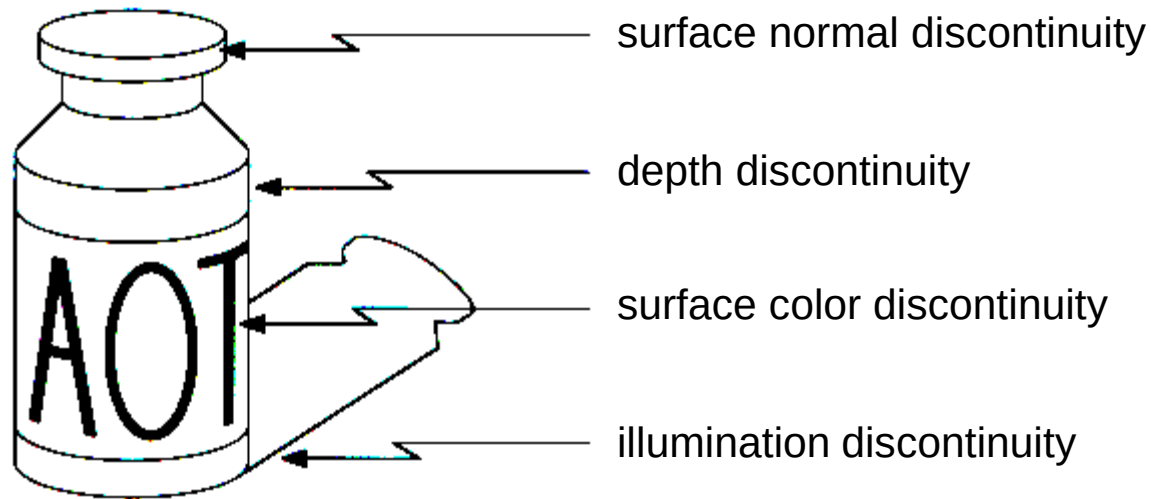
Detecting edges

What's an edge?

- intensity discontinuity (= rapid change)

How can we find large changes in intensity?

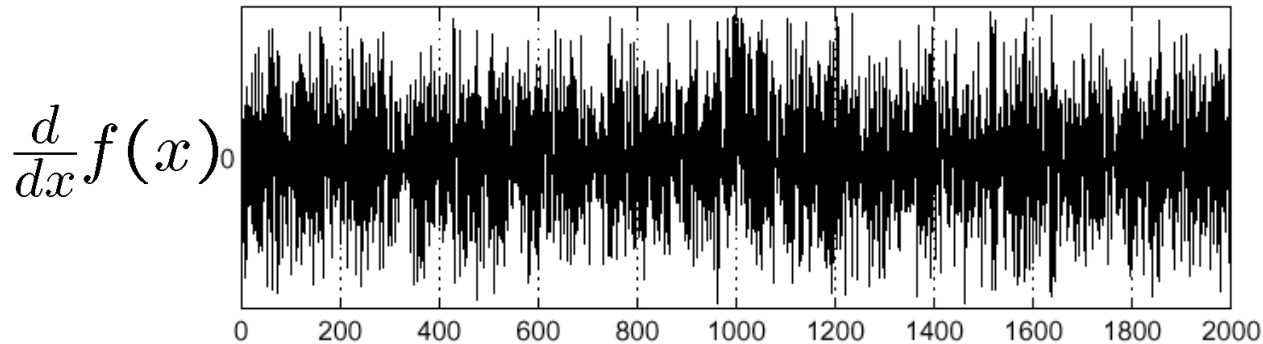
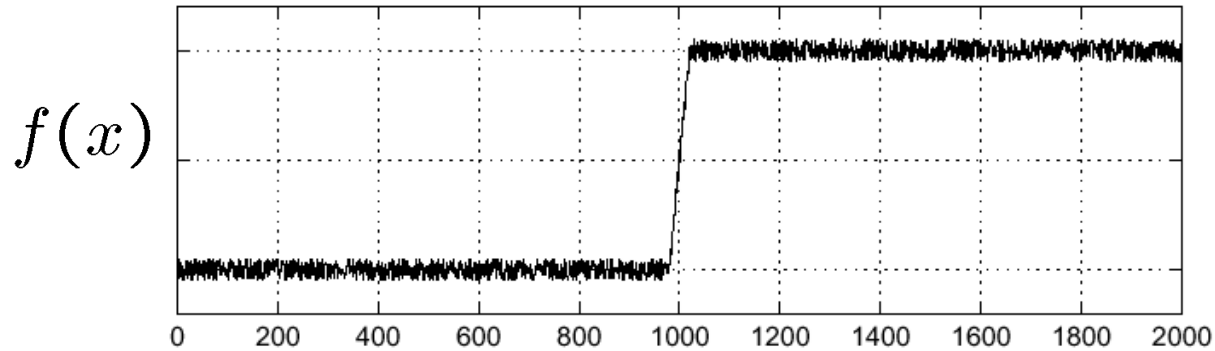
- gradient operator seems like the right solution



Effects of noise

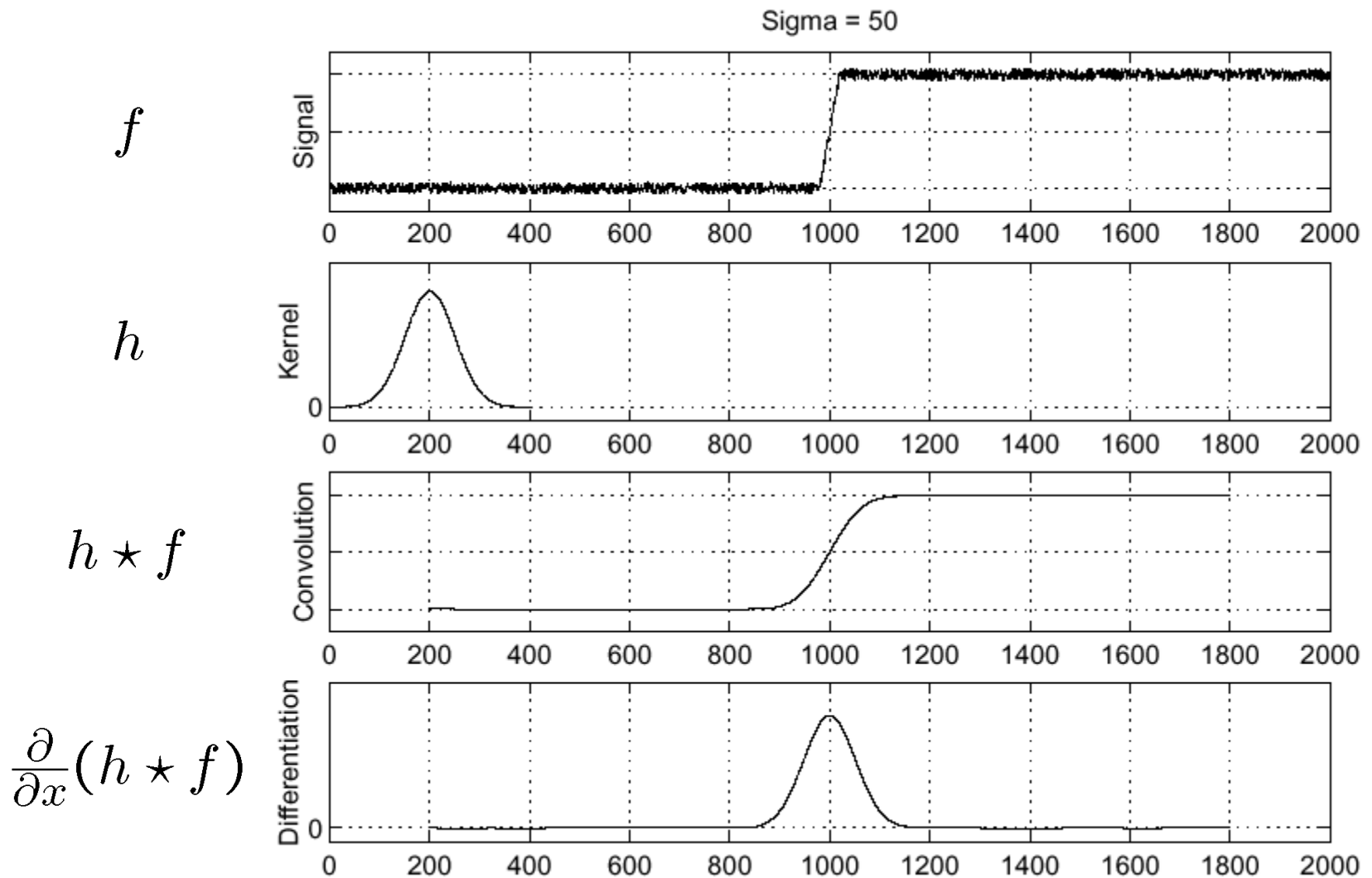
Consider a single row or column of the image

- Plotting intensity as a function of position gives a signal



Where is the edge?

Solution: smooth first

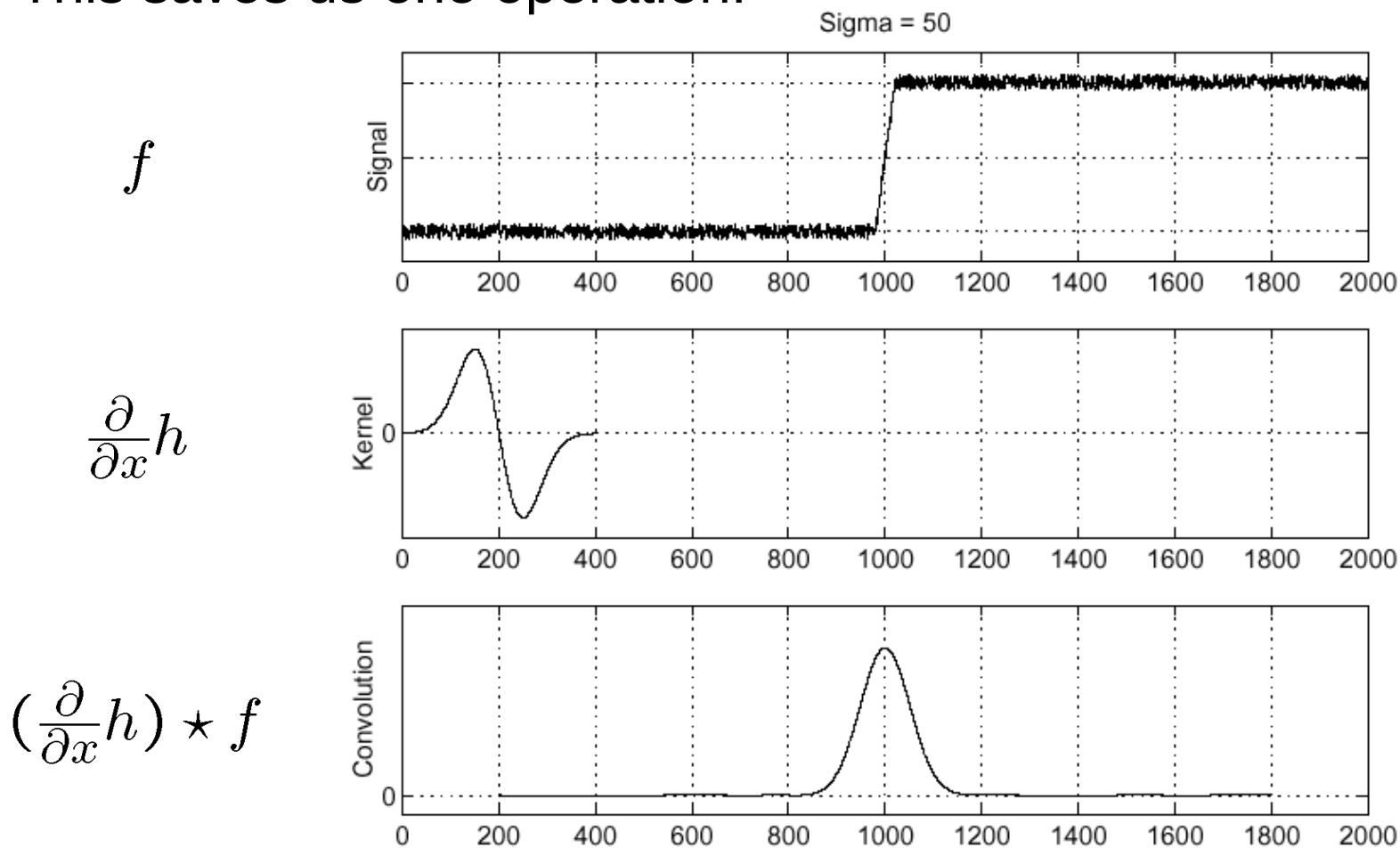


Where is the edge? Look for peaks in $\frac{\partial}{\partial x}(h \star f)$

Associative property of convolution

$$\frac{\partial}{\partial x}(h \star f) = (\frac{\partial}{\partial x}h) \star f$$

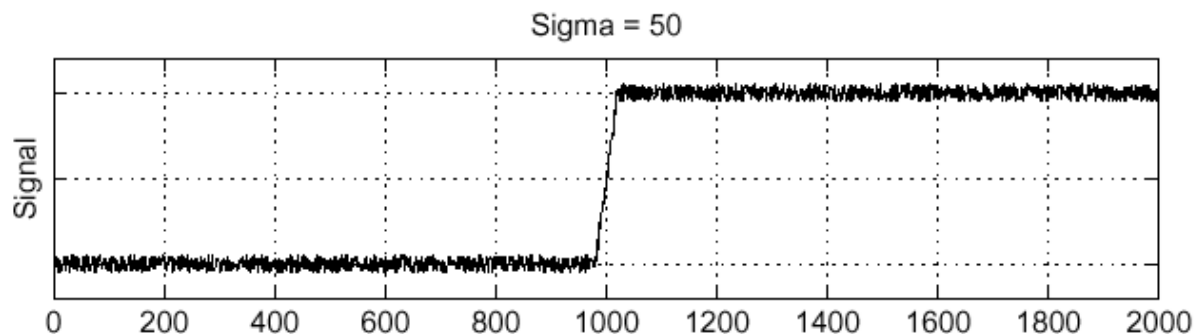
This saves us one operation:



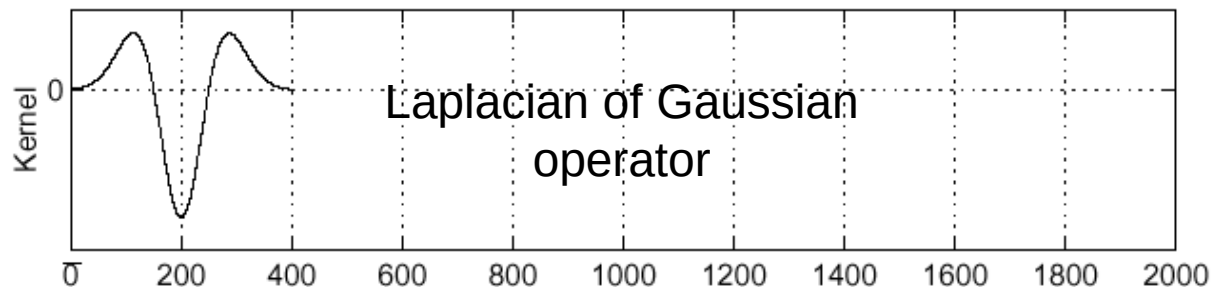
Laplacian of Gaussian

Consider $\frac{\partial^2}{\partial x^2}(h \star f)$

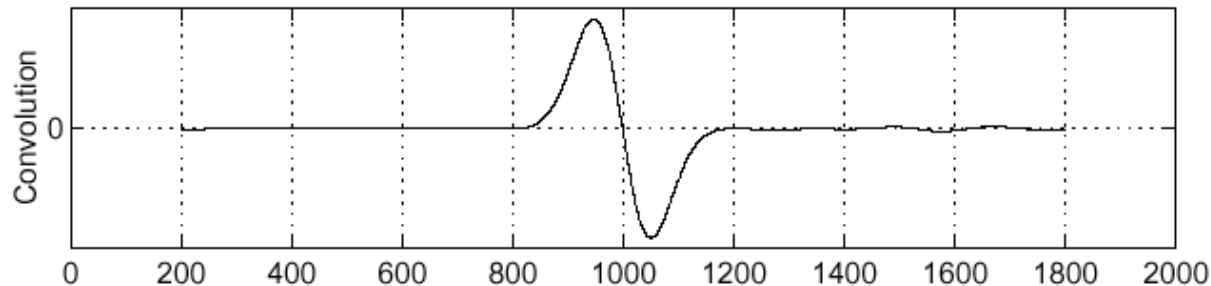
f



$\frac{\partial^2}{\partial x^2}h$

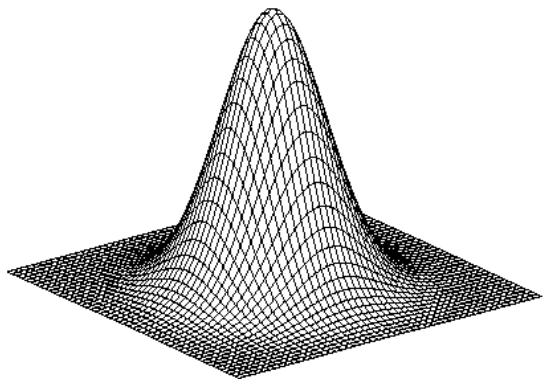


$(\frac{\partial^2}{\partial x^2}h) \star f$



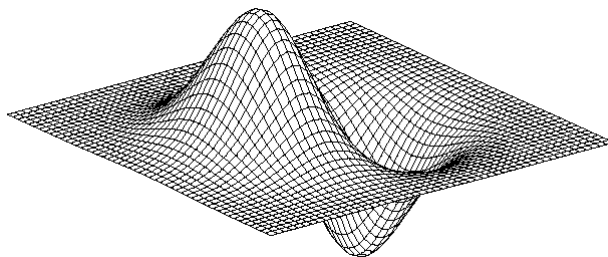
Where is the edge? Zero-crossings of bottom graph

2D edge detection filters



Gaussian

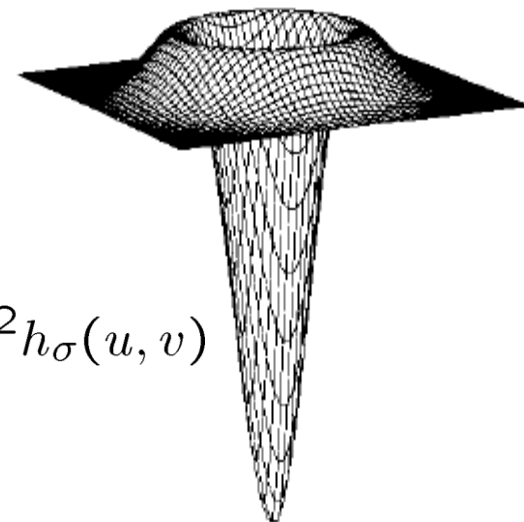
$$h_{\sigma}(u, v) = \frac{1}{2\pi\sigma^2} e^{-\frac{u^2+v^2}{2\sigma^2}}$$



derivative of Gaussian

$$\frac{\partial}{\partial x} h_{\sigma}(u, v)$$

Laplacian of Gaussian



$$\nabla^2 h_{\sigma}(u, v)$$

∇^2 is the **Laplacian** operator:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Discrete Approximations

Common approximation of derivative of Gaussian

$$\frac{1}{8} \begin{array}{|c|c|c|} \hline -1 & 0 & 1 \\ \hline -2 & 0 & 2 \\ \hline -1 & 0 & 1 \\ \hline \end{array}$$

s_x

$$\frac{1}{8} \begin{array}{|c|c|c|} \hline 1 & 2 & 1 \\ \hline 0 & 0 & 0 \\ \hline -1 & -2 & -1 \\ \hline \end{array}$$

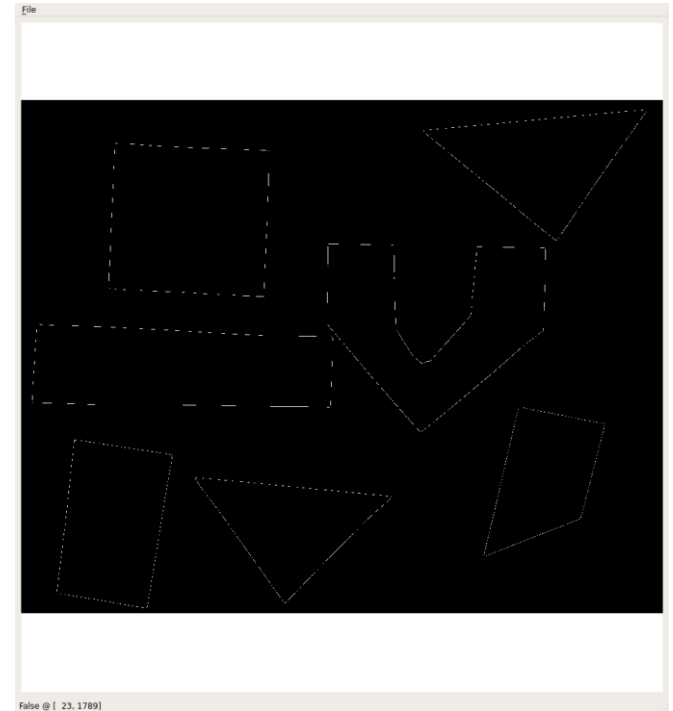
s_y

- The standard defn. of the Sobel operator omits the $1/8$ term
 - doesn't make a difference for edge detection
 - the $1/8$ term **is** needed to get the right gradient value, however

Laplacian Filter

0	-1	0
-1	4	-1
0	-1	0

Edge Detection Results - Canny Edge Detection



Low Threshold + High Threshold

Fitting

Choose a parametric object/some objects to represent a set of tokens

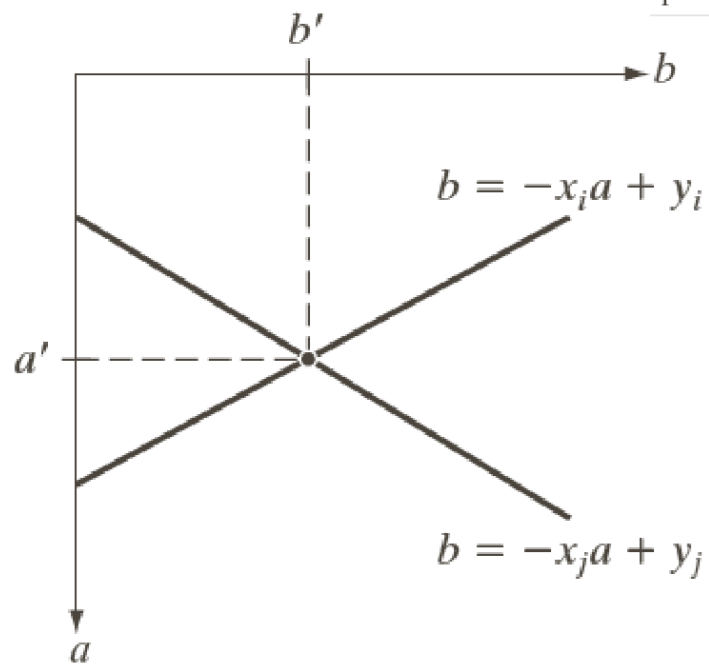
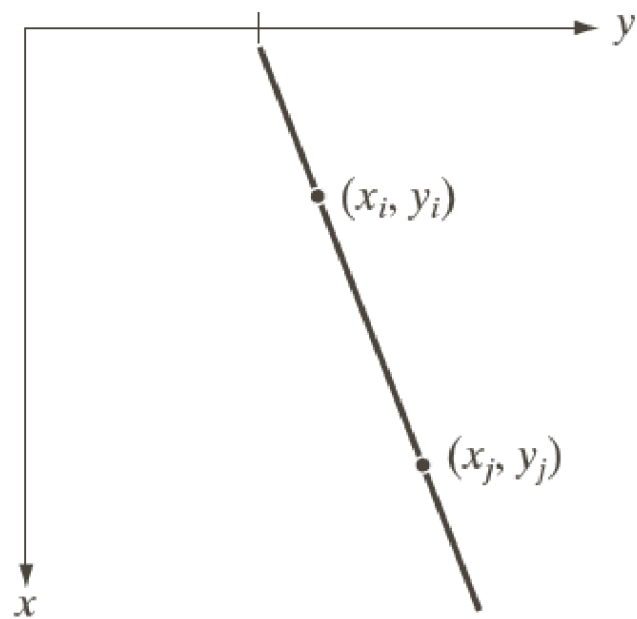
Most interesting case is when criterion is not local

- can't tell whether a set of points lies on a line by looking only at each point and the next.

Three main questions:

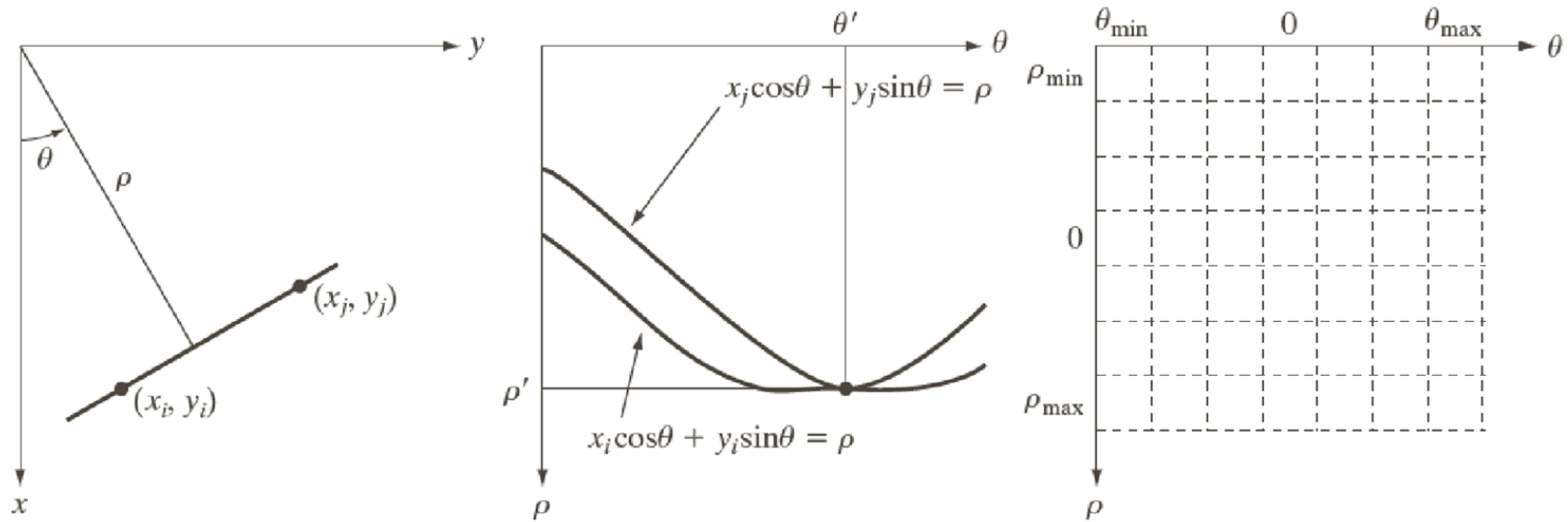
- what object represents this set of tokens best?
- which of several objects gets which token?
- how many objects are there?

(you could read line for object here, or circle, or ellipse or...)



a b

FIGURE 10.31
(a) xy -plane.
(b) Parameter space.



a b c

FIGURE 10.32 (a) (ρ, θ) parameterization of line in the xy -plane. (b) Sinusoidal curves in the $\rho\theta$ -plane; the point of intersection (ρ', θ') corresponds to the line passing through points (x_i, y_i) and (x_j, y_j) in the xy -plane. (c) Division of the $\rho\theta$ -plane into accumulator cells.

Mechanics of the Hough transform

Construct an array representing θ, d

For each point, render the curve (θ, d) into this array, adding one at each cell

Difficulties

- how big should the cells be? (too big, and we cannot distinguish between quite different lines; too small, and noise causes lines to be missed)

How many lines?

- count the peaks in the Hough array

Who belongs to which line?

- tag the votes

Hardly ever satisfactory in practice, because problems with noise and cell size defeat it

RANSAC

Choose a small subset
uniformly at random

Fit to that

Anything that is close to result is
signal; all others are noise

Refit

Do this many times and choose
the best

Issues

- How many times?
 - Often enough that we are likely to have a good line
- How big a subset?
 - Smallest possible
- What does close mean?
 - Depends on the problem
- What is a good line?
 - One where the number of nearby points is so big it is unlikely to be all outliers

Algorithm

RANSAC: fitting lines using random sample consensus

Determine:

n — the smallest number of points required

k — the number of iterations required

t — the threshold used to identify a point that fits well

d — the number of nearby points required
to assert a model fits well

Until k iterations have occurred

Draw a sample of n points from the data
uniformly and at random

Fit to that set of n points

For each data point outside the sample

Test the distance from the point to the line
against t ; if the distance from the point to the line
is less than t , the point is close

end

If there are d or more points close to the line
then there is a good fit. Refit the line using all
these points.

end

Use the best fit from this collection, using the
fitting error as a criterion

The 2D Fourier Transform

- Represent function on a new basis
 - Think of functions as vectors, with many components
 - We now apply a linear transformation to transform the basis
 - dot product with each basis element
- In the expression, u and v select the basis element, so a function of x and y becomes a function of u and v
- basis elements have the form
$$e^{-i2\pi(ux+vy)}$$

$$F(g(x,y))(u,v) = \iint_{\mathbb{R}^2} g(x,y) e^{-i2\pi(ux+vy)} dx dy$$

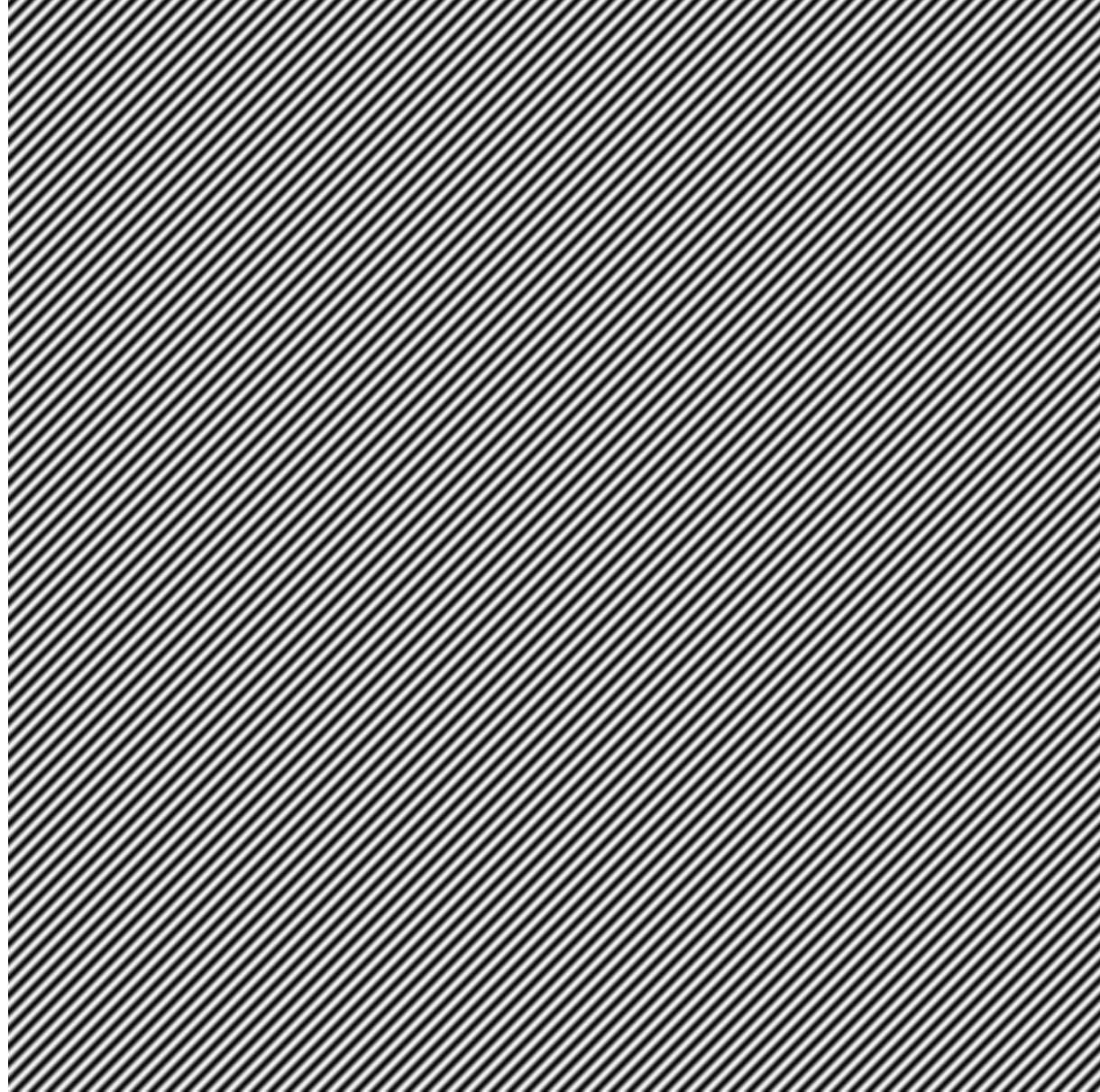
To get some sense of what basis elements look like, we plot a basis element --- or rather, its real part --- as a function of x, y for some fixed u, v . We get a function that is constant when $(ux+vy)$ is constant. The magnitude of the vector (u, v) gives a frequency, and its direction gives an orientation. The function is a sinusoid with this frequency along the direction, and constant perpendicular to the direction.



Here u and v
are larger than
in the previous
slide.



And larger still...



Phase and Magnitude

- Fourier transform of a real function is complex
 - difficult to plot, visualize
 - instead, we can think of the phase and magnitude of the transform
- Phase is the phase of the complex transform
- Magnitude is the magnitude of the complex transform
- **Curious fact**
 - all natural images have about the same magnitude transform
 - hence, phase seems to matter, but magnitude largely doesn't
- Demonstration
 - Take two pictures, swap the phase transforms, compute the inverse - what does the result look like?

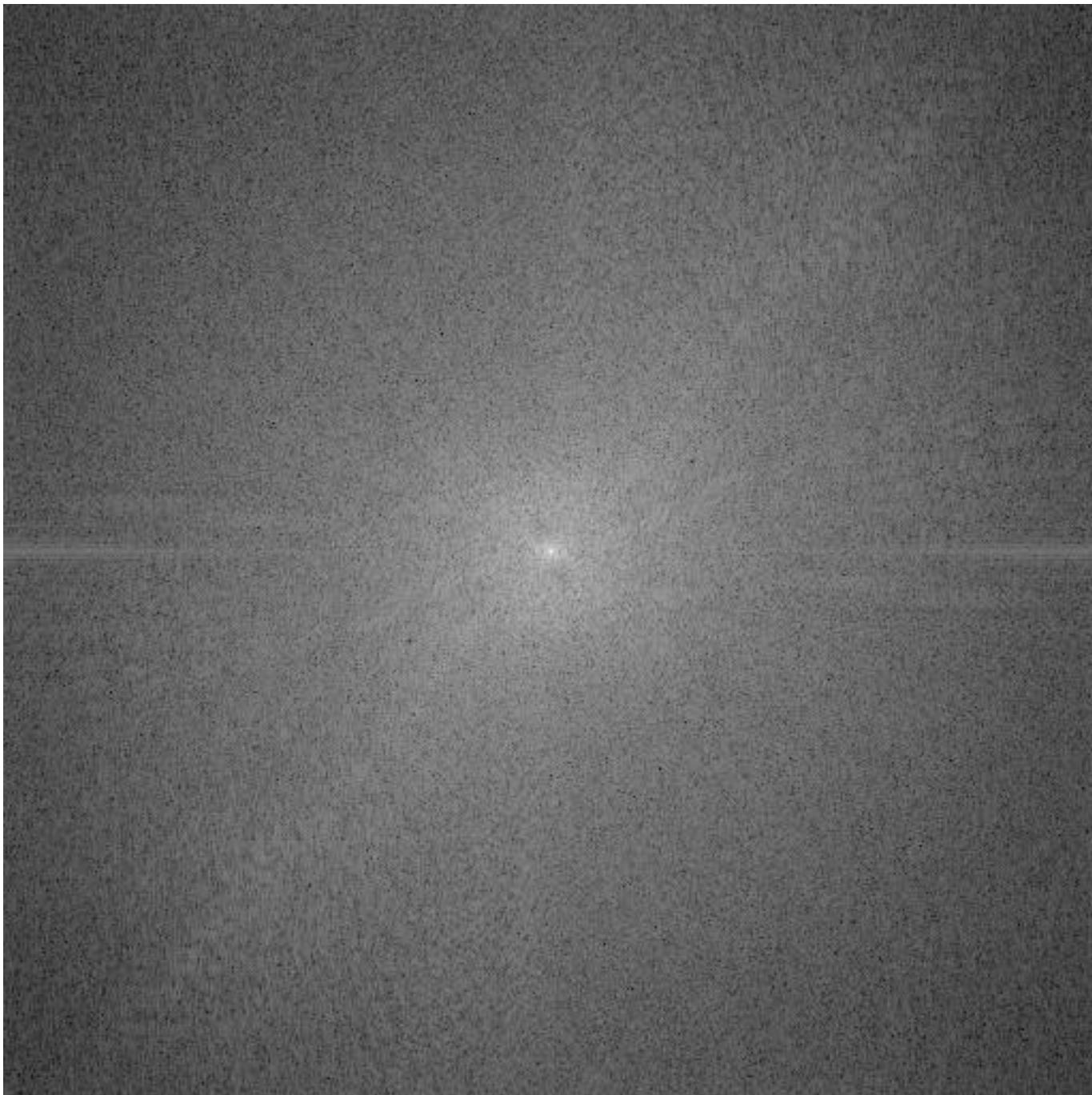
Read:

https://openaccess.thecvf.com/content_CVPR_2020/papers/Yang_FDA_Fourier_Domain_Adaptation_for_Semantic_Segmentation_CVP_R_2020_paper.pdf

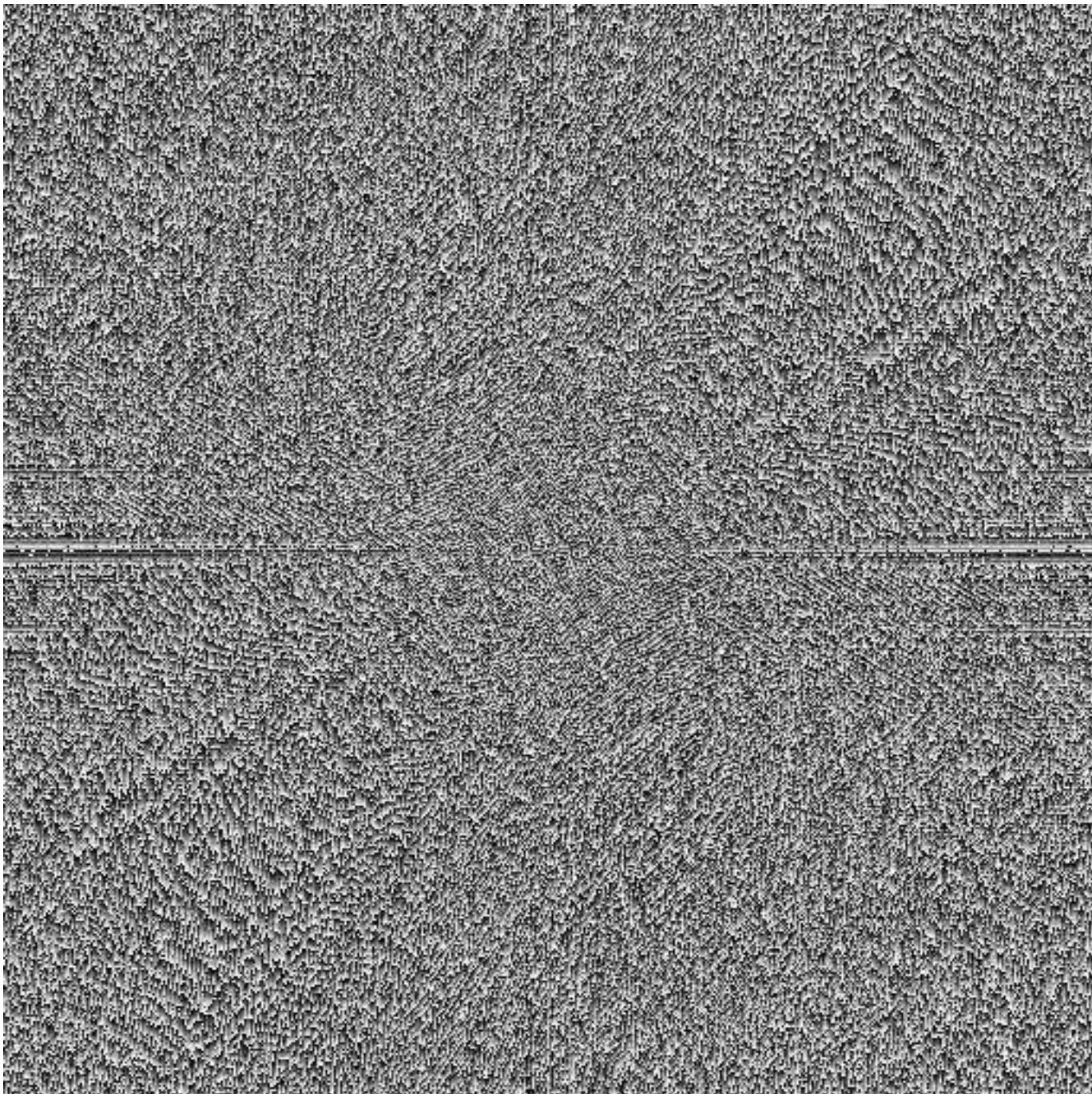


Shades by D.A. Forsyth

This is the
magnitude
transform
of the
cheetah pic



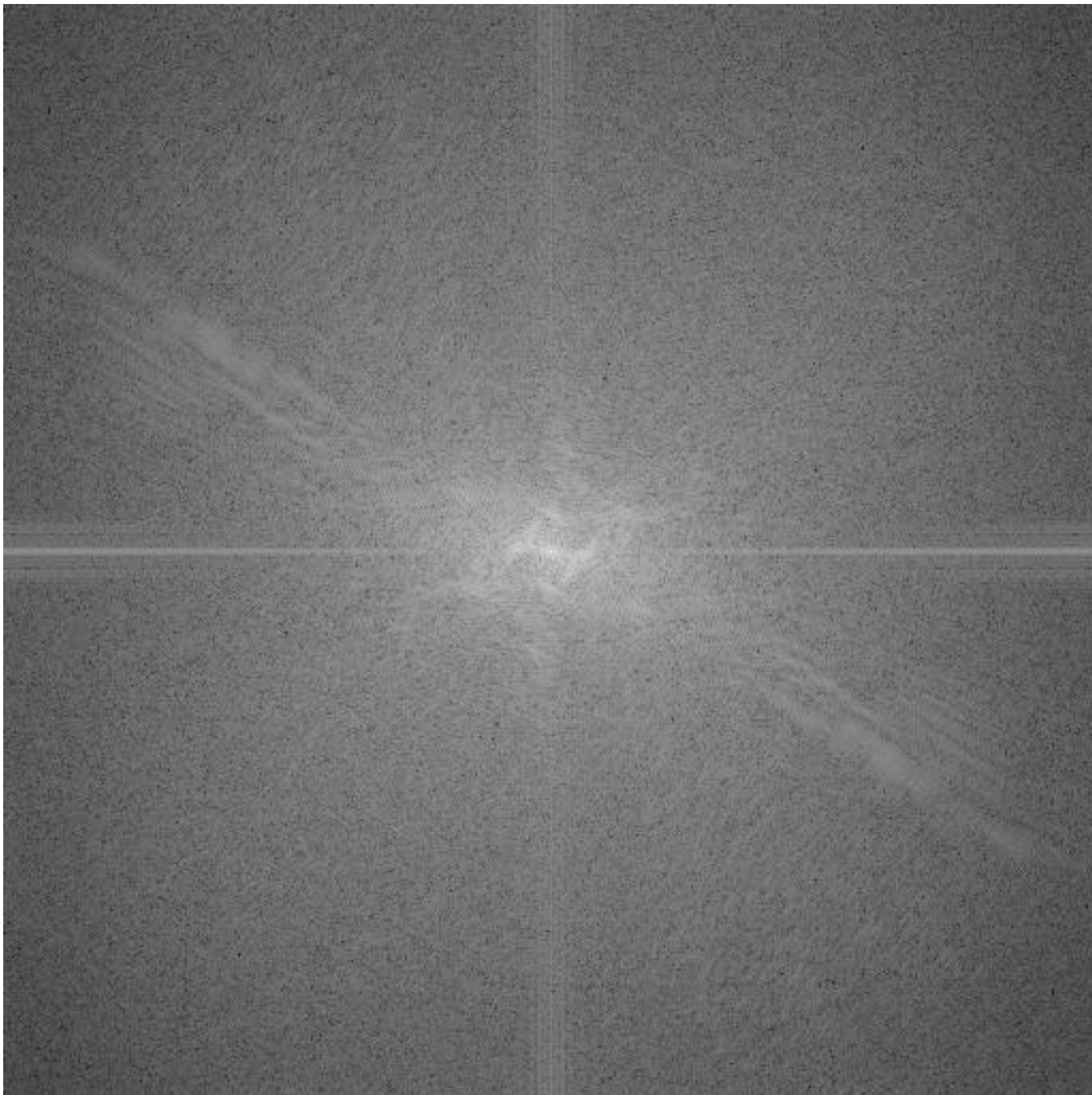
This is the
phase
transform
of the
cheetah pic



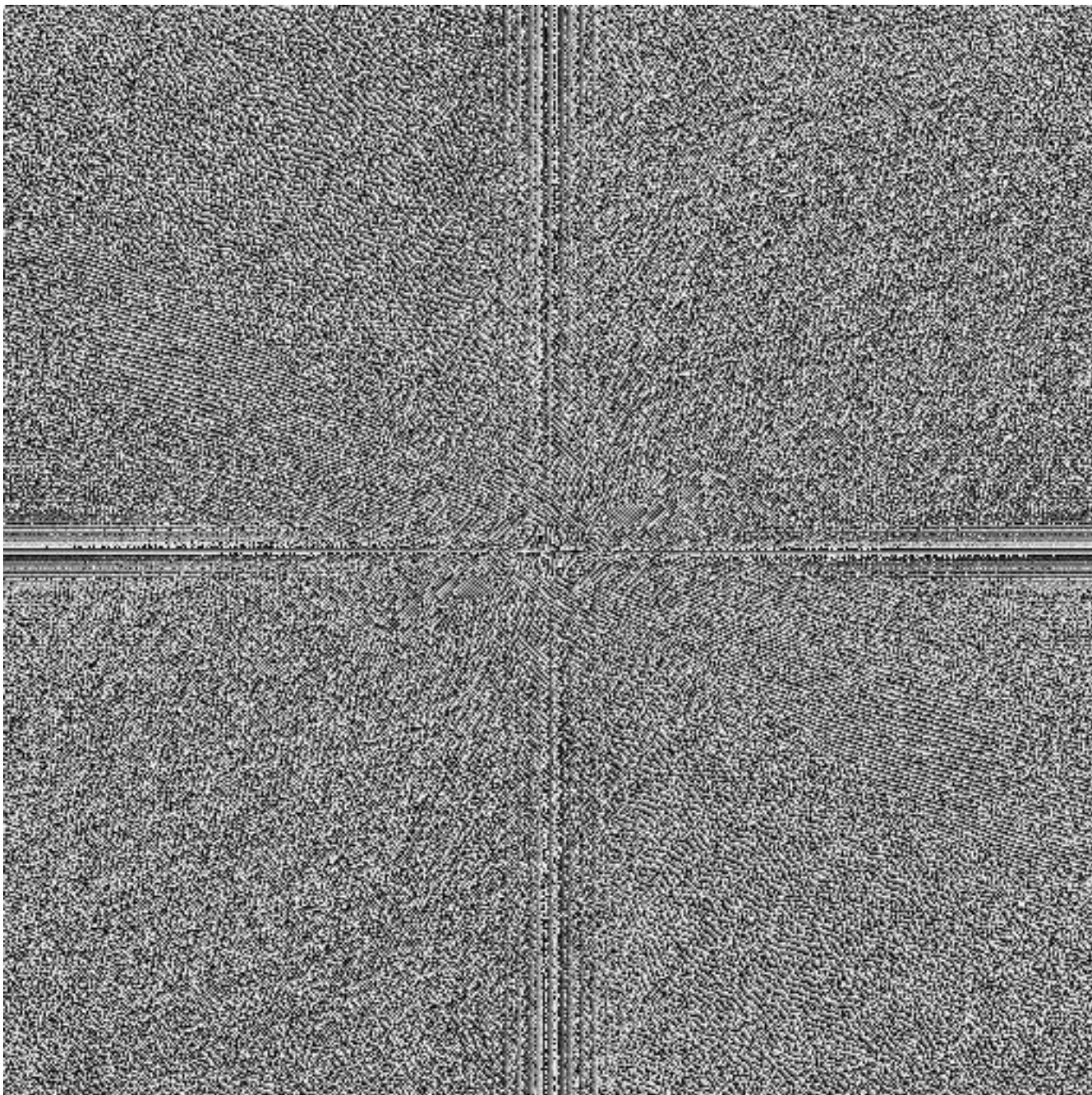


Slides by D.A. Forsyth

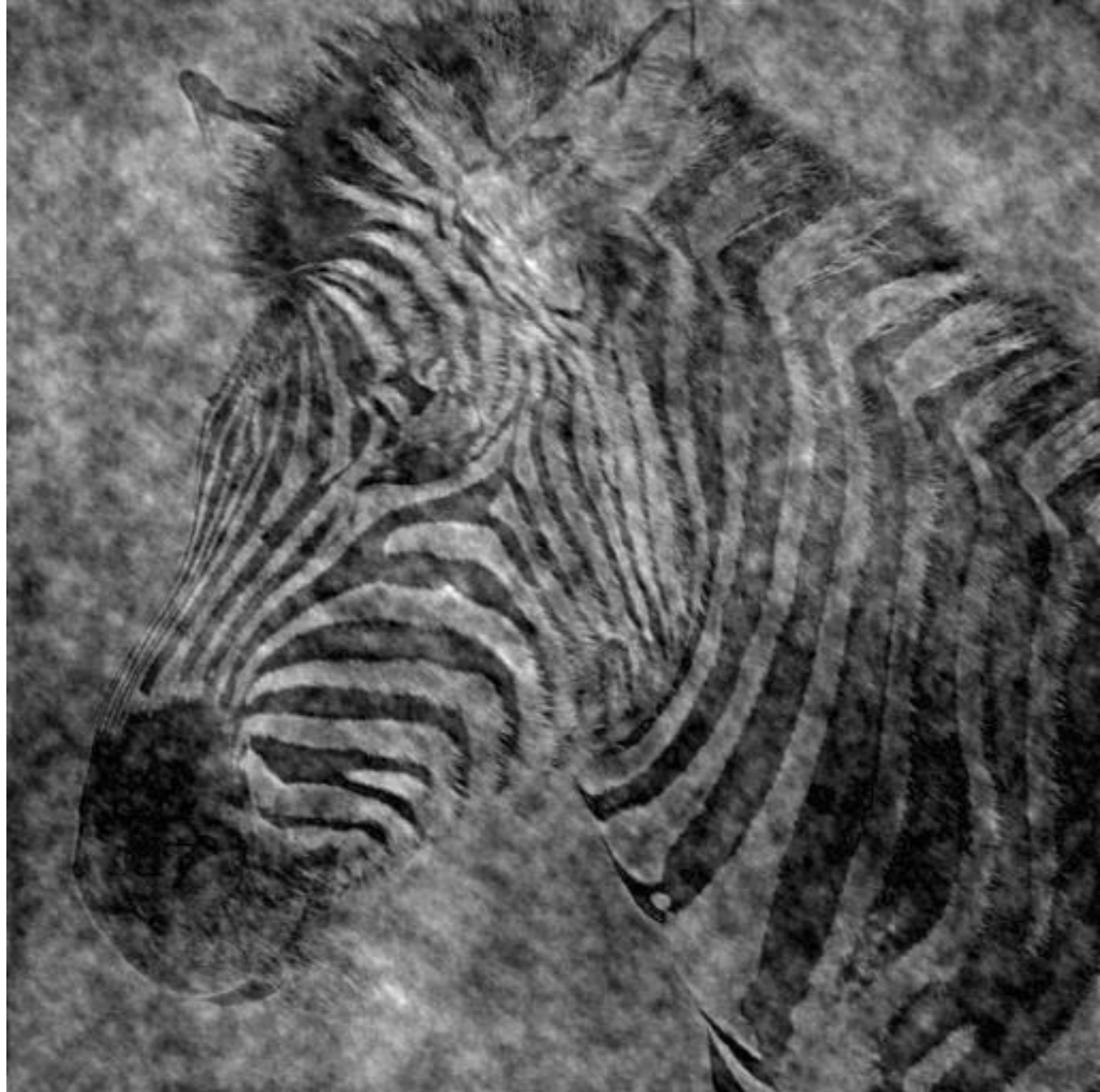
This is the
magnitude
transform
of the zebra
pic



This is the
phase
transform
of the zebra
pic

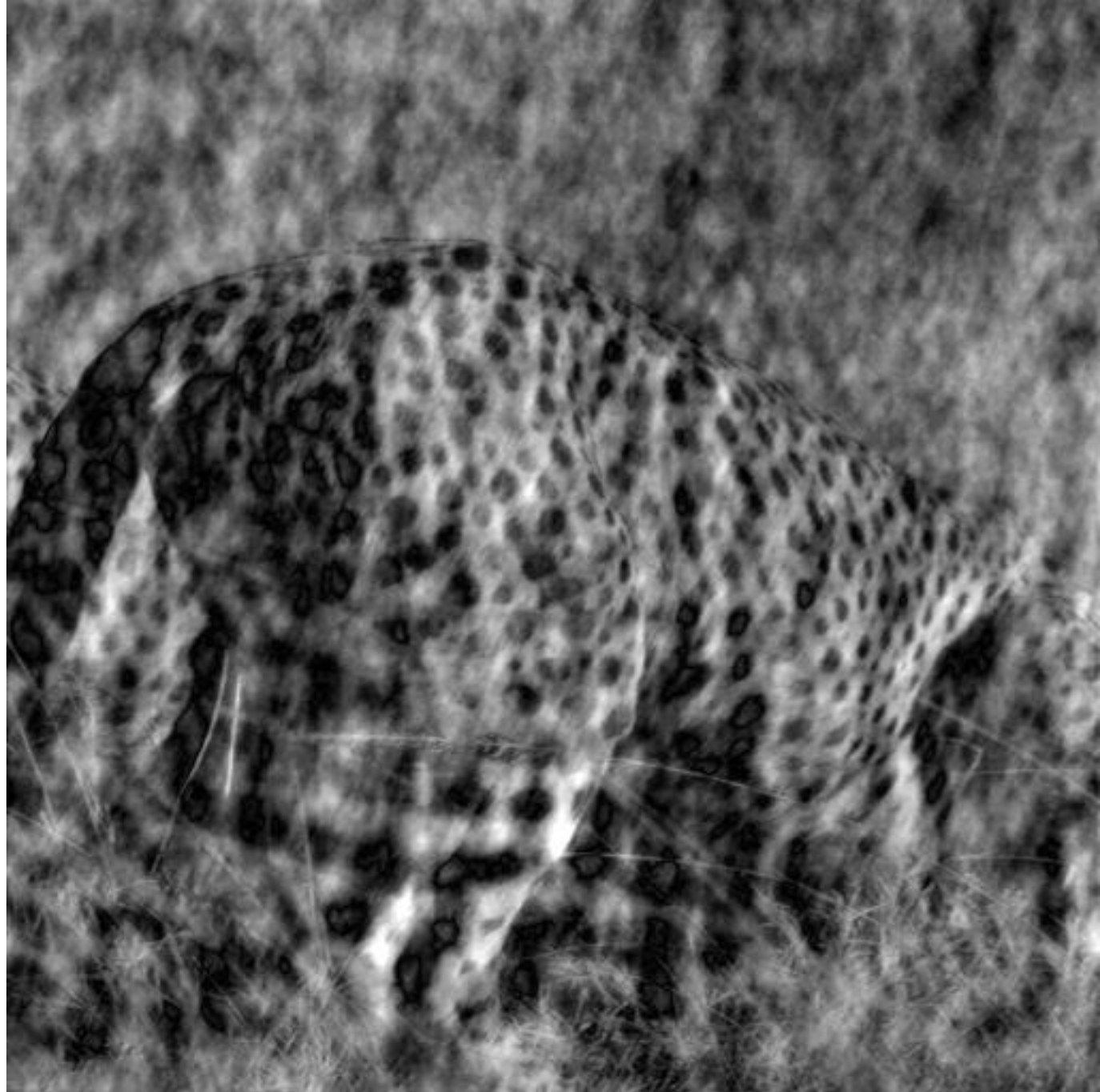


Reconstruction
with zebra
phase, cheetah
magnitude



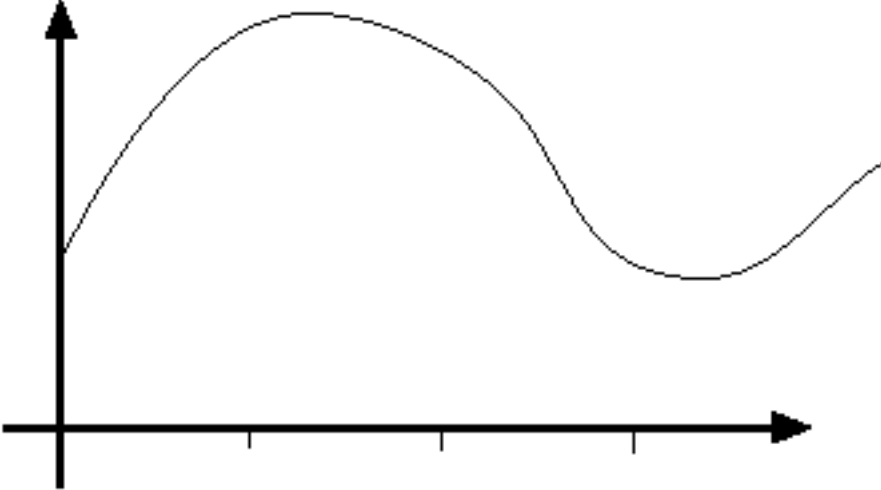
Set: Pyramids and Texture
Slides by D.A. Forsyth

Reconstruction
with cheetah
phase, zebra
magnitude

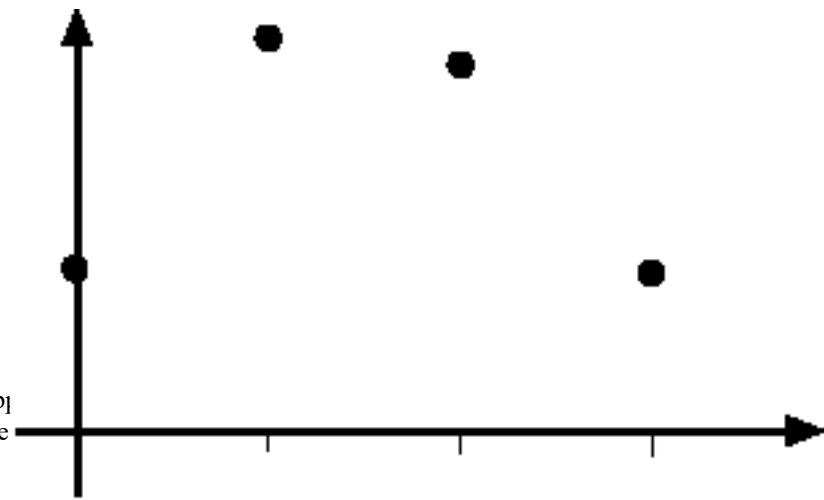


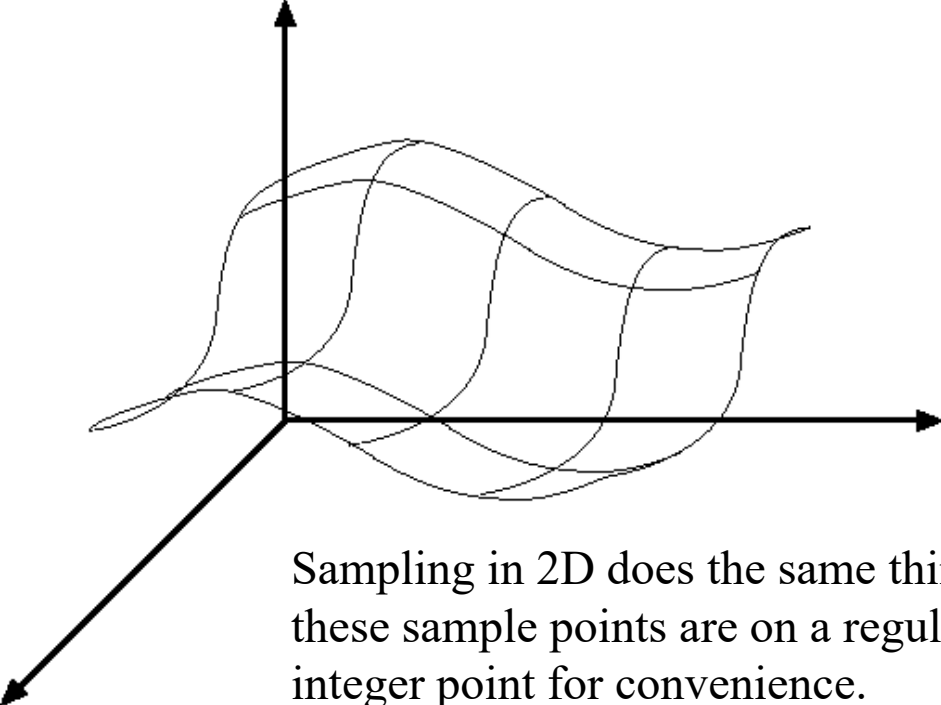
Set: Pyramids and Texture
Slides by D.A. Forsyth

Sampling and Aliasing

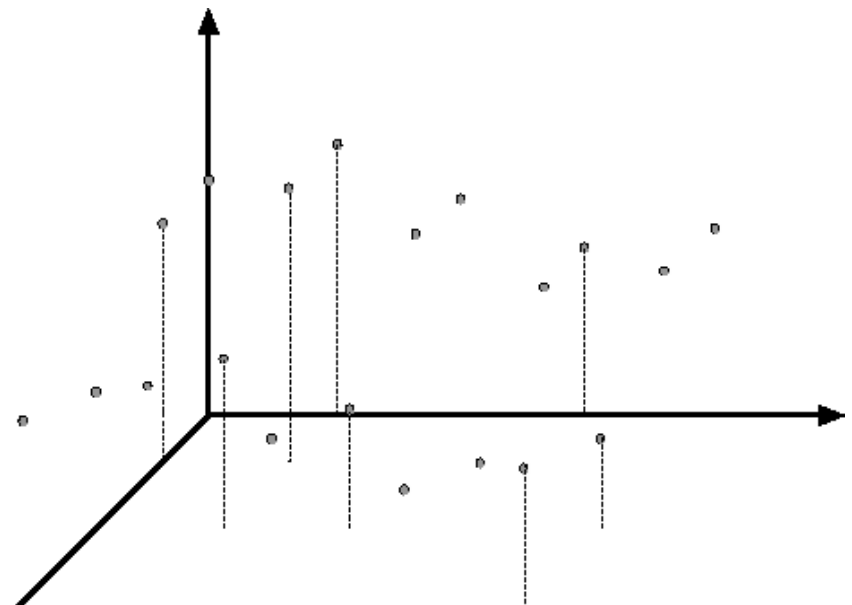


Sampling in 1D takes a continuous function and replaces it with a vector of values, consisting of the function's values at a set of sample points. We'll assume that these sample points are on a regular grid, and can place one at each integer for convenience.





Sampling in 2D does the same thing, only in 2D. We'll assume that these sample points are on a regular grid, and can place one at each integer point for convenience.



A continuous model for a sampled function

- We want to be able to approximate integrals sensibly
- Leads to
 - the delta function
 - model on right

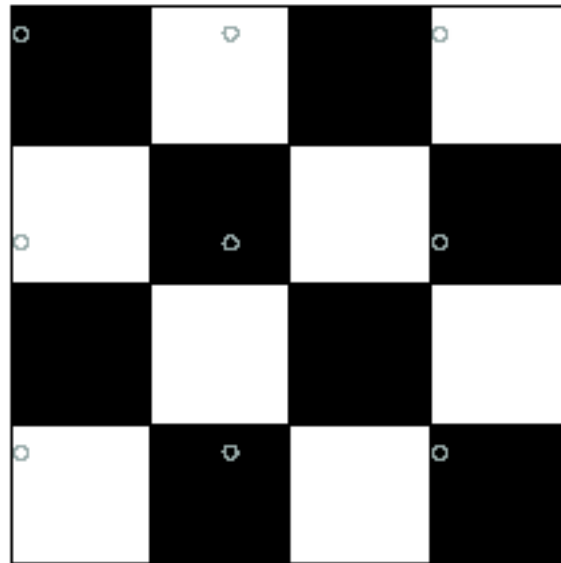
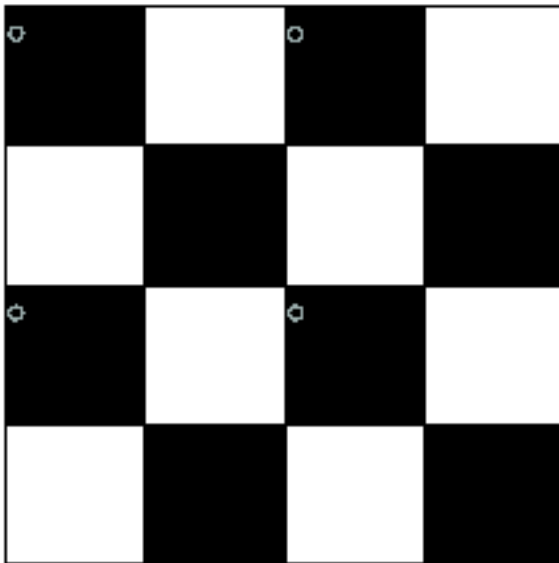
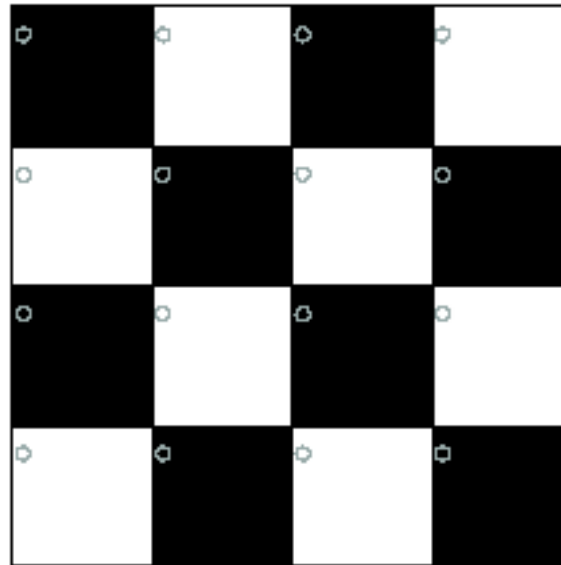
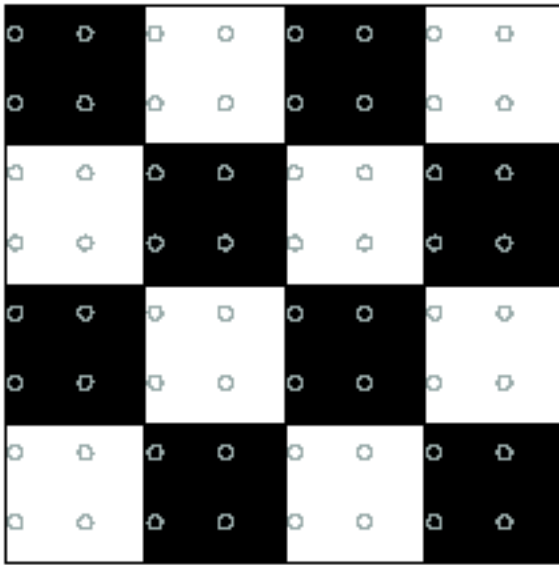
$$\begin{aligned}\text{Sample}_{2D}(f(x,y)) &= \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} f(x,y) \delta(x-i, y-j) \\ &= f(x,y) \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} \delta(x-i, y-j)\end{aligned}$$

The Fourier transform of a sampled signal

$$\begin{aligned} F(\text{Sample}_{2D}(f(x,y))) &= F\left(f(x,y) \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} \delta(x-i, y-j)\right) \\ &= F(f(x,y)) * * F\left(\sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} \delta(x-i, y-j)\right) \\ &= \sum_{i=-\infty}^{\infty} \sum_{j=-\infty}^{\infty} F(u-i, v-j) \end{aligned}$$

Aliasing

- Can't shrink an image by taking every second pixel
- If we do, characteristic errors appear
 - In the next few slides
 - Typically, small phenomena look bigger; fast phenomena can look slower
 - Common phenomenon
 - Wagon wheels rolling the wrong way in movies
 - Checkerboards misrepresented in ray tracing
 - Striped shirts look funny on colour television



Resample the checkerboard by taking one sample at each circle. In the case of the top left board, new representation is reasonable. Top right also yields a reasonable representation. Bottom left is all black (dubious) and bottom right has checks that are too big.

Smoothing as low-pass filtering

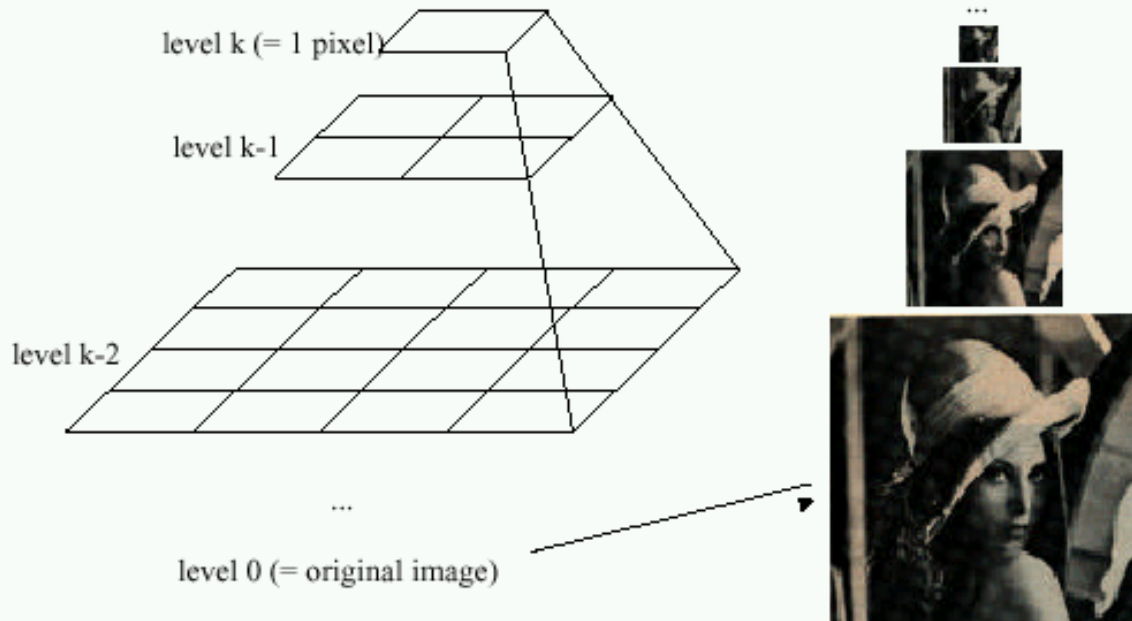
- The message of the FT is that high frequencies lead to trouble with sampling.
- Solution: suppress high frequencies before sampling
 - multiply the FT of the signal with something that suppresses high frequencies
 - or convolve with a low-pass filter
- A filter whose FT is a box is bad, because the filter kernel has infinite support
- Common solution: use a Gaussian
 - multiplying FT by Gaussian is equivalent to convolving image with Gaussian.

The Gaussian pyramid

- Smooth with gaussians, because
 - a gaussian*gaussian=another gaussian
- Synthesis
 - smooth and sample
- Analysis
 - take the top image

Gaussian Pyramid

Idea: Represent $N \times N$ image as a “pyramid” of $1 \times 1, 2 \times 2, 4 \times 4, \dots, 2^k \times 2^k$ images (assuming $N=2^k$)



Known as a **Gaussian Pyramid** [Burt and Adelson, 1983]

- In computer graphics, a *mip map* [Williams, 1983]
- A precursor to *wavelet transform*

Gaussian Pyramids have all sorts of applications in computer vision



512

256

128

64

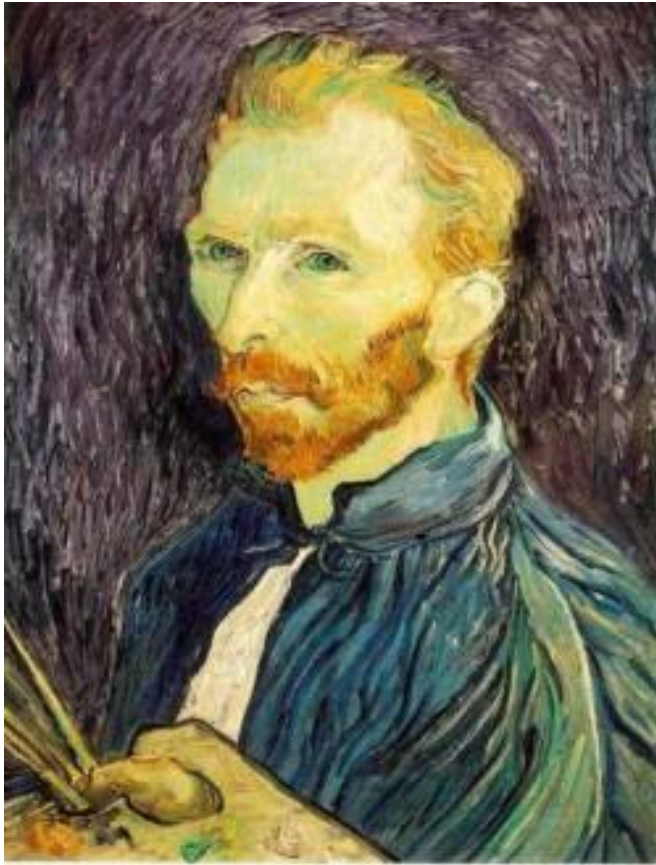
32

16

8



Subsampling with Gaussian pre-filtering



Gaussian 1/2



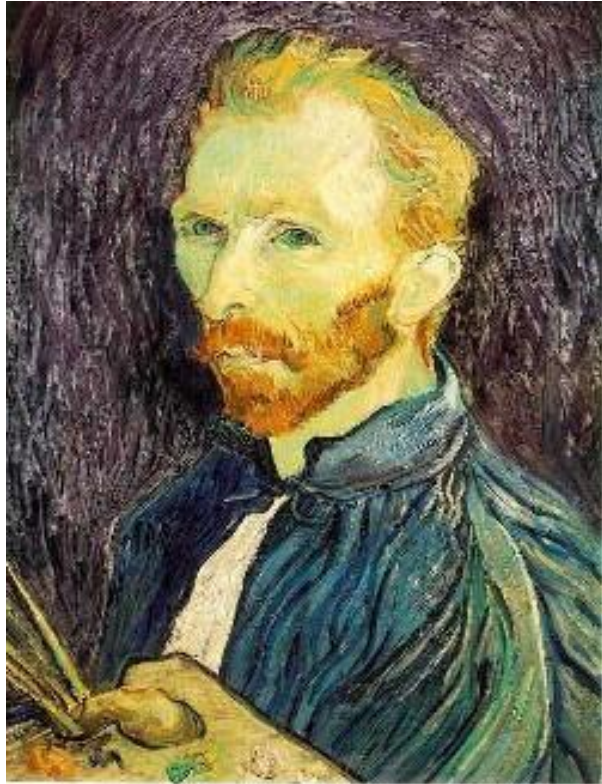
G 1/4



G 1/8

Filter the image, *then* subsample

Subsampling *without* pre-filtering



$1/2$

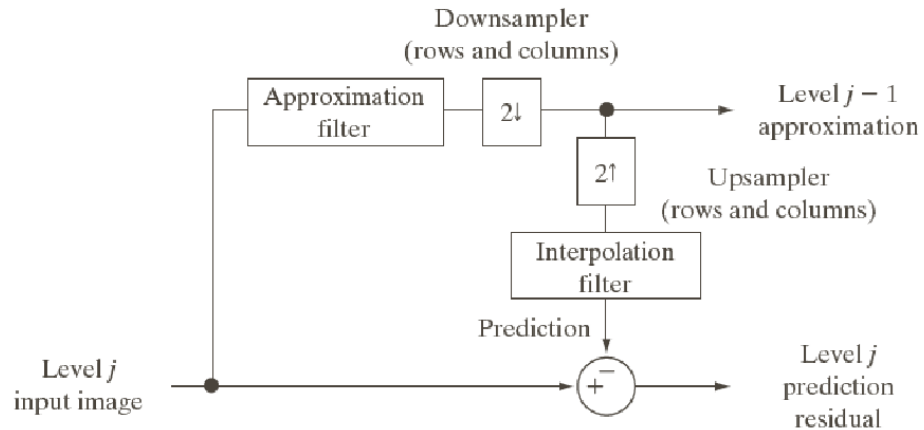
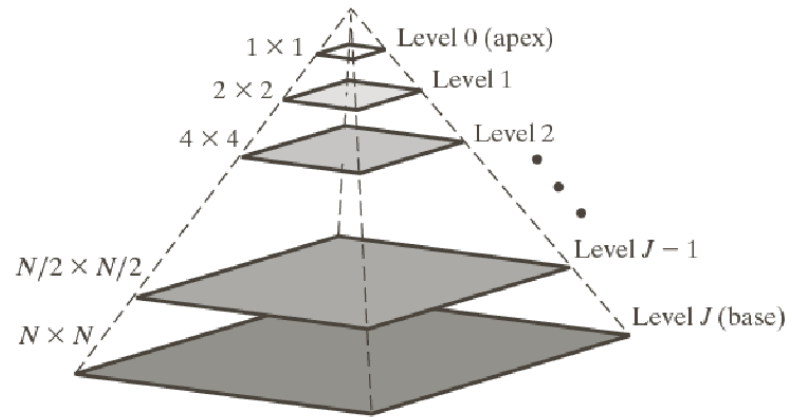


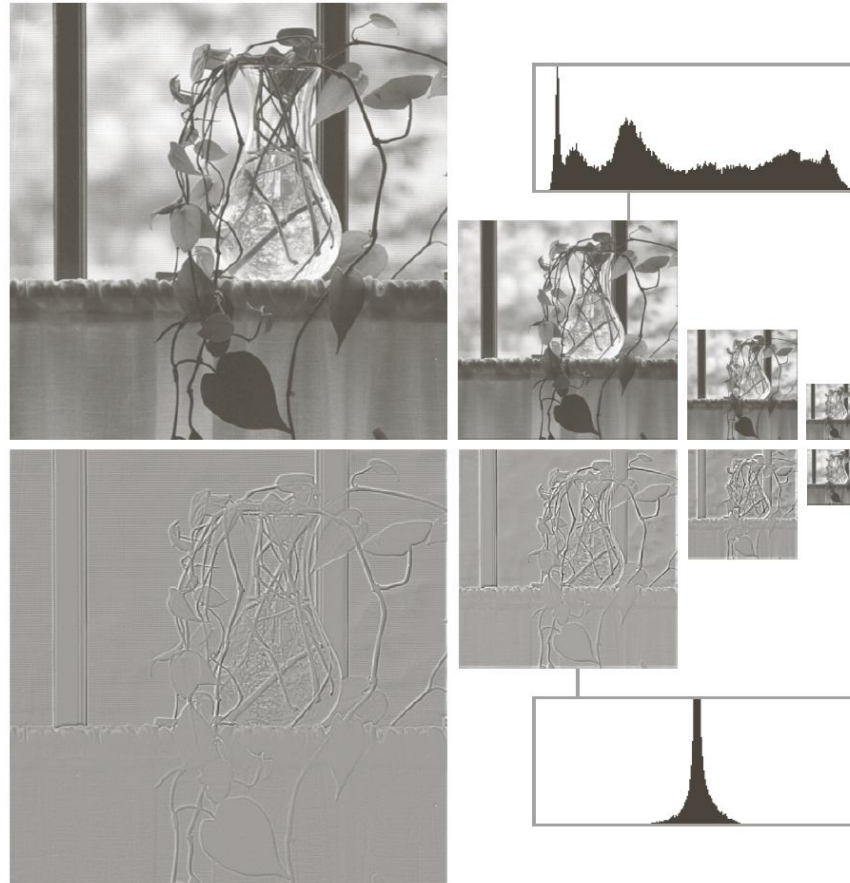
$1/4$ (2x zoom)



$1/8$ (4x zoom)

Image Pyramid – Approximation and Residual

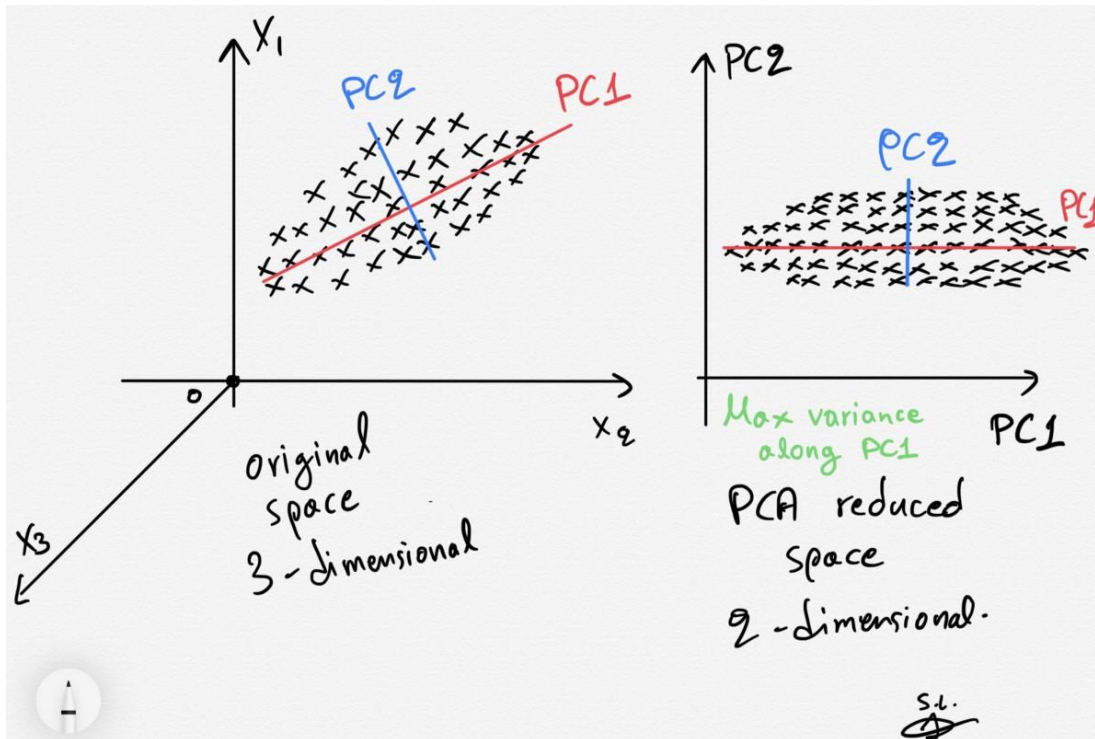




a
b

FIGURE 7.3
Two image
pyramids and
their histograms:
(a) an
approximation
pyramid;
(b) a prediction
residual pyramid.

Principal Components Analysis (PCA)



- Reduce dimension
- PC component describe the largest data variance

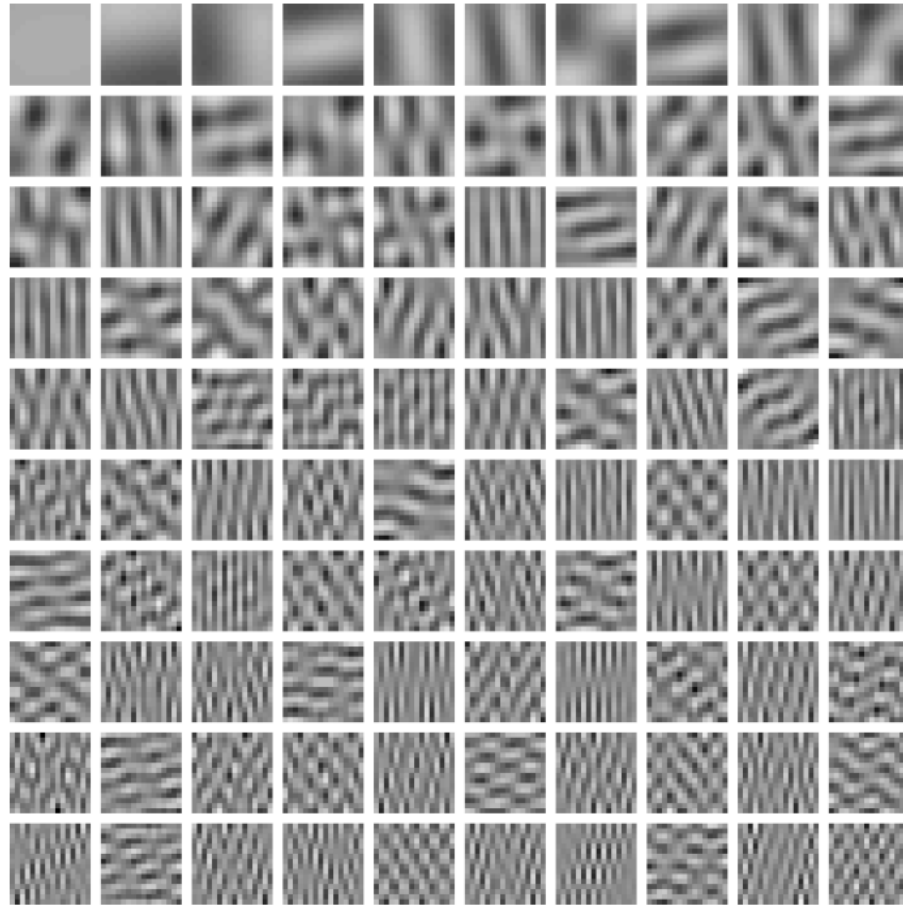
- Mathematically, the principal components are the **eigenvectors of the covariance matrix** of the original dataset.
- Because the **covariance matrix is symmetric**, the eigenvectors are orthogonal.
- The **principal components** (eigenvectors) correspond to the **direction** (in the original n-dimensional space) **with the greatest variance** in the data.

PCA

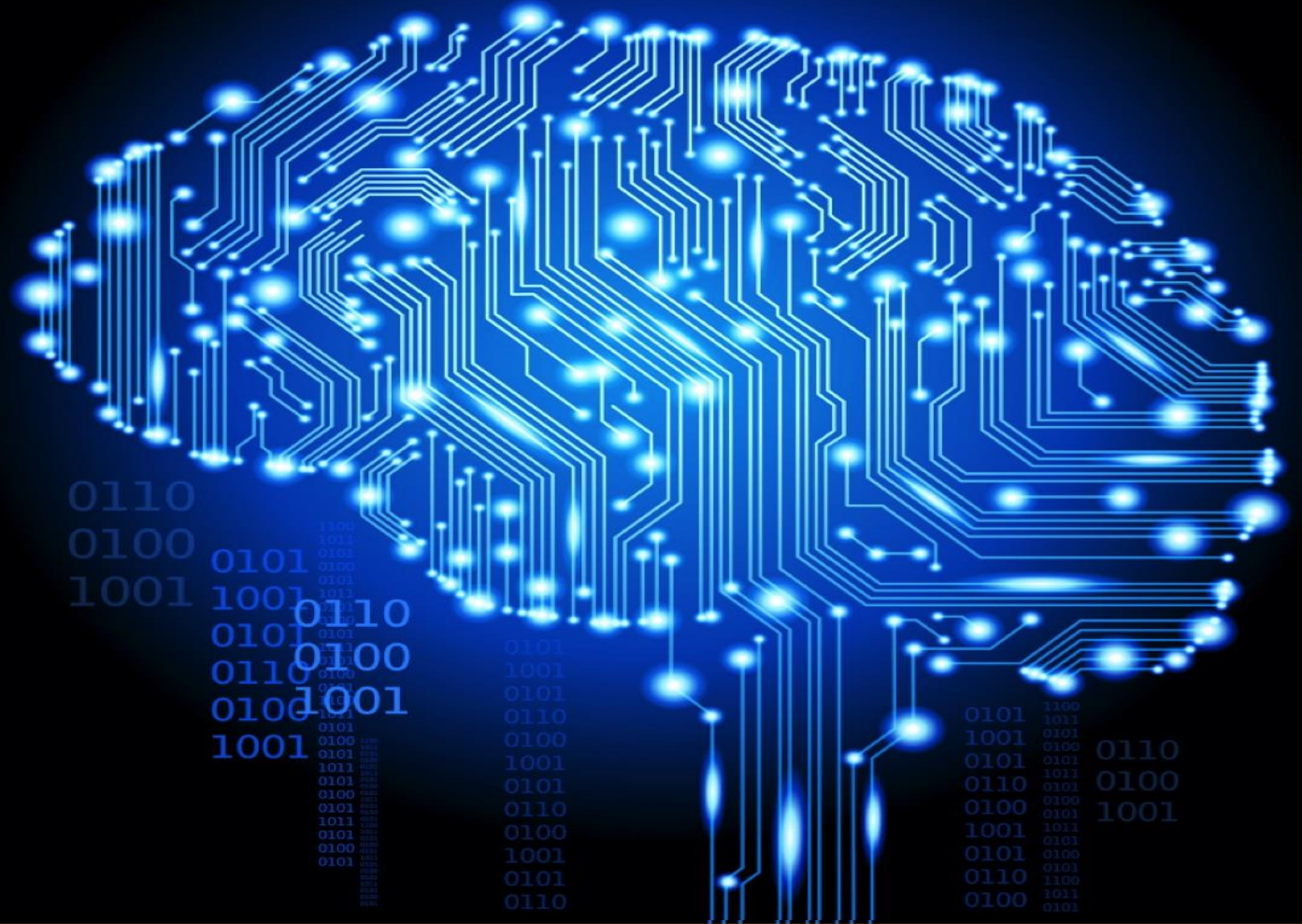
- eigen-decomposition of Σ : $\Sigma = U\Lambda U^T$
 - U : orthonormal matrix (rotation)
 $U^T U = U U^T = I$
 - Λ : diagonal matrix, $\Lambda_{ii} \geq 0$ -- eigenvalue

$$\begin{aligned} E\{U^T \vec{x} (U^T \vec{x})^T\} &= U^T E\{\vec{x} \vec{x}^T\} U \\ &= U^T U \Lambda U^T U = \Lambda \end{aligned}$$

- $s = U^T x$, or $x = Us$, s is independent Gaussian
- principal component analysis (PCA)
 - Karhunen Loeve transform



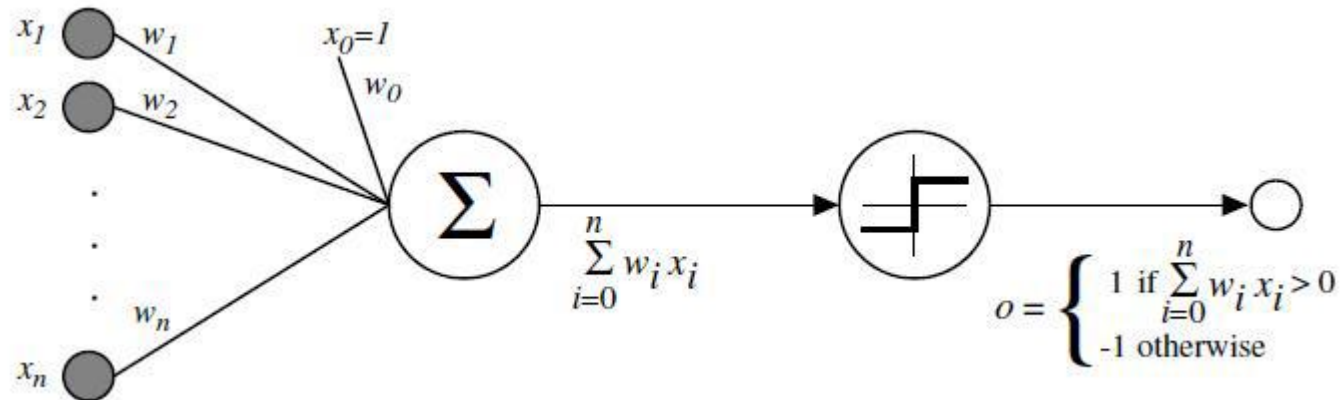
PCA bases learned from natural images (U)



Deep Learning for Feature Extraction

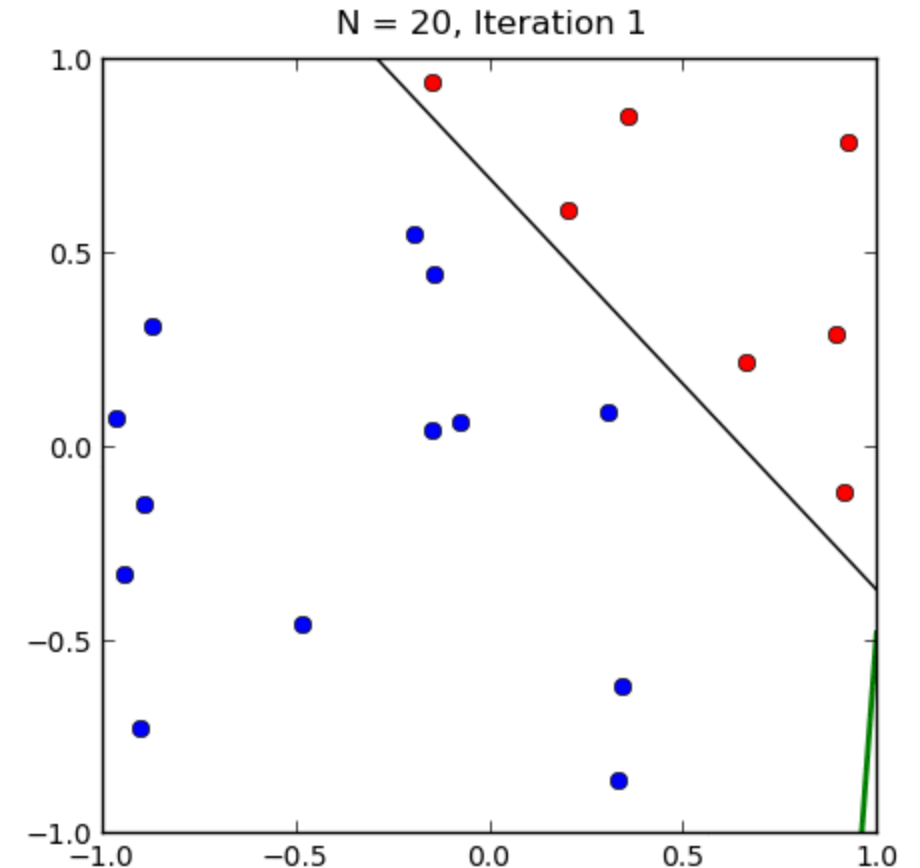
Perceptrons

- Rosenblatt proposed a machine for binary classifications
- Main idea
 - One weight w_i per input x_i
 - Multiply weights with respective inputs and add bias $x_0 = +1$
 - If result larger than threshold return 1, otherwise 0



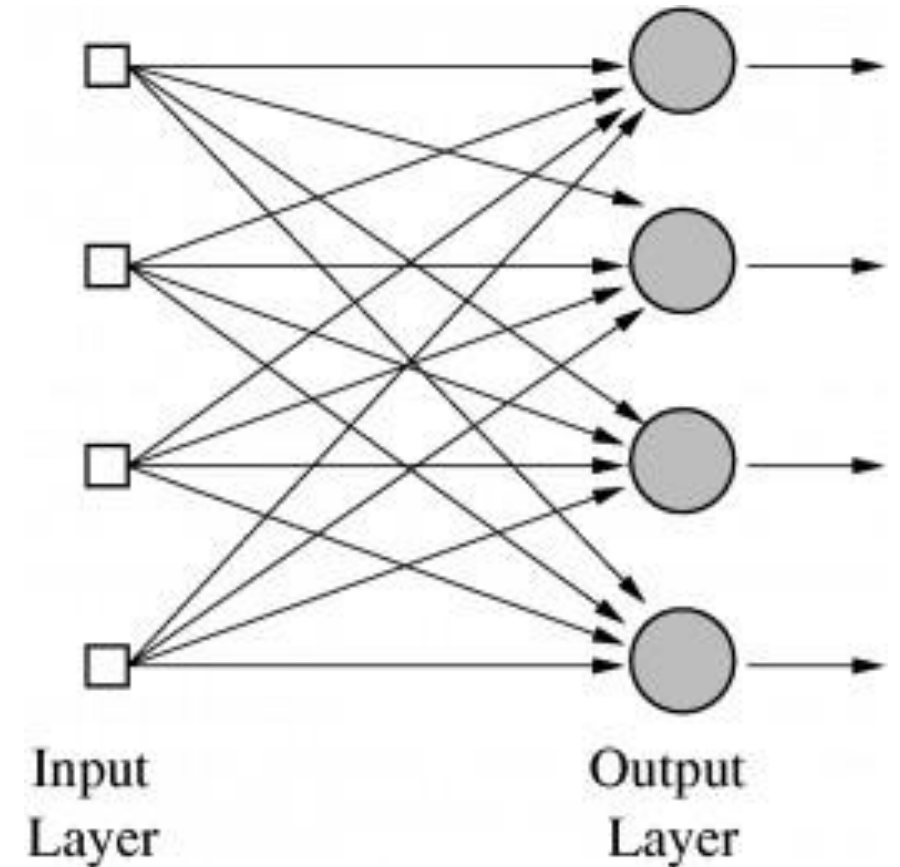
Training a perceptron

- Rosenblatt's innovation was mainly the learning algorithm for perceptrons
- Learning algorithm
 - Initialize weights randomly
 - Take one sample x_i and predict y_i
 - For erroneous predictions update weights
 - If the output was $\hat{y}_i = 0$ and $y_i = 1$, increase weights
 - If the output was $\hat{y}_i = 1$ and $y_i = 0$, decrease weights
 - Repeat until no errors are made

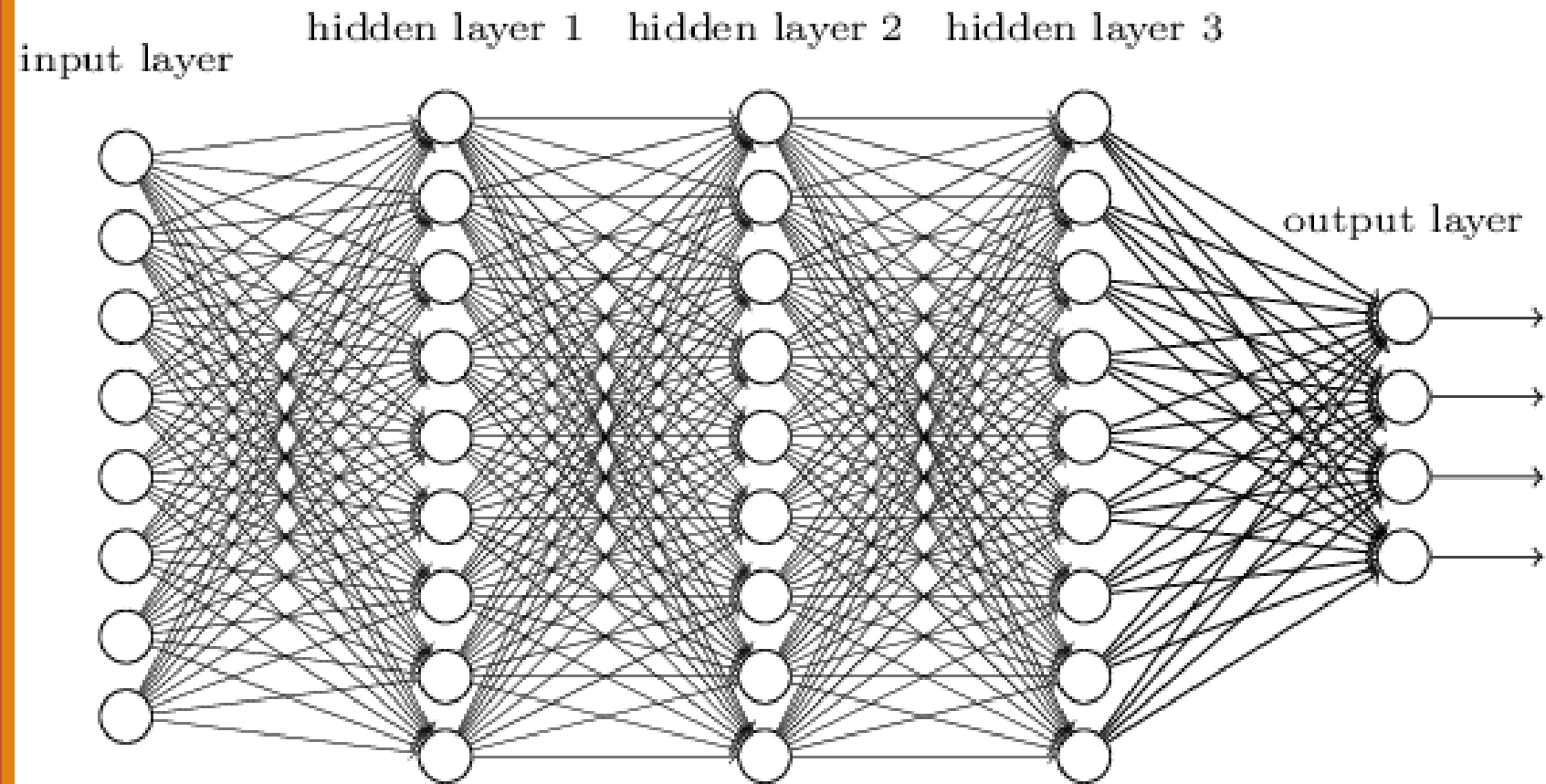


From a perceptron to a neural network

- One perceptron = one decision
- What about multiple decisions?
 - E.g. digit classification
- Stack as many outputs as the possible outcomes into a layer
 - Neural network
- Use one layer as input to the next layer
 - Multi-layer perceptron (MLP)



Deep Networks



Long story short

- A family of **parametric**, **non-linear** and **hierarchical representation learning functions**, which are **massively optimized with stochastic gradient descent** to **encode domain knowledge**, i.e. domain invariances, stationarity.
- $a_L(x; \theta_1, \dots, \theta_L) = h_L(h_{L-1}(\dots h_1(x, \theta_1), \theta_{L-1}), \theta_L)$
 - x : input, θ_l : parameters for layer l , $a_l = h_l(x, \theta_l)$: (non-)linear function
- Given training corpus $\{X, Y\}$ find optimal parameters

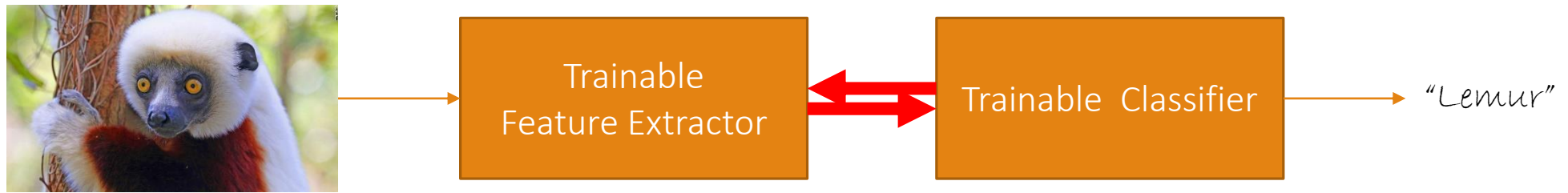
$$\theta^* \leftarrow \arg \min_{\theta} \sum_{(x,y) \in (X,Y)} \ell(y, a_L(x; \theta_1, \dots, \theta_L))$$

Learning Representations & Features

- Traditional pattern recognition



- End-to-end learning → Features are also learned from data

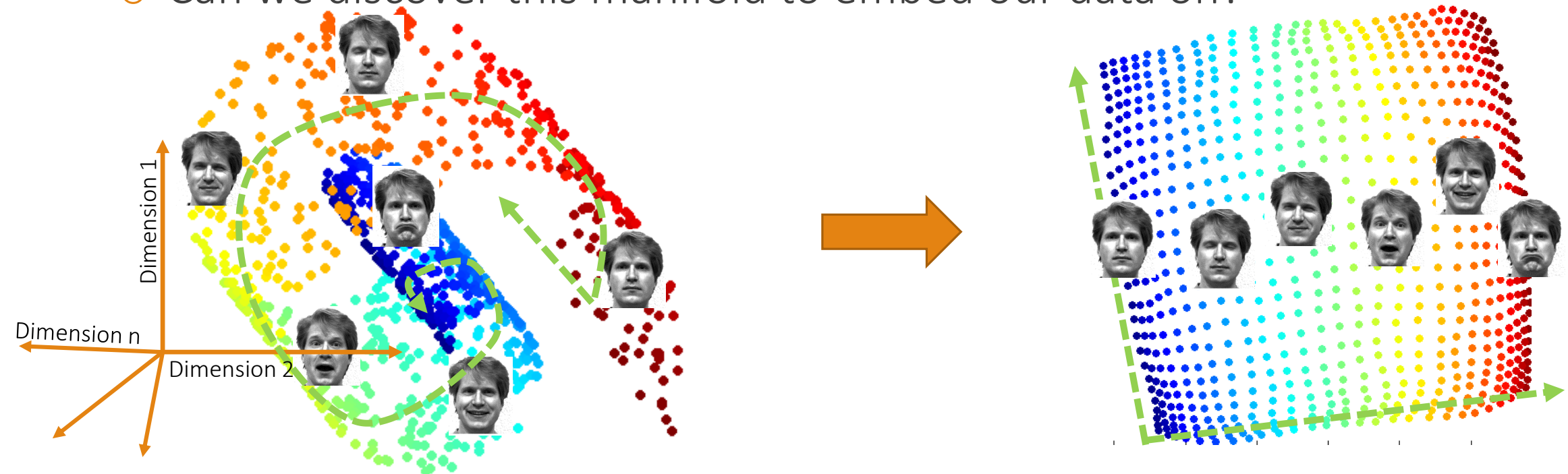


How to get good features?

- High-dimensional data (e.g. faces) lie in lower dimensional manifolds
 - **Goal:** discover these lower dimensional manifolds
 - These manifolds are most probably highly non-linear
- **Hypothesis (1):** Compute the coordinates of the input (e.g. a face image) to this non-linear manifold → data become separable
- **Hypothesis (2):** Semantically similar things lie closer together than semantically dissimilar things

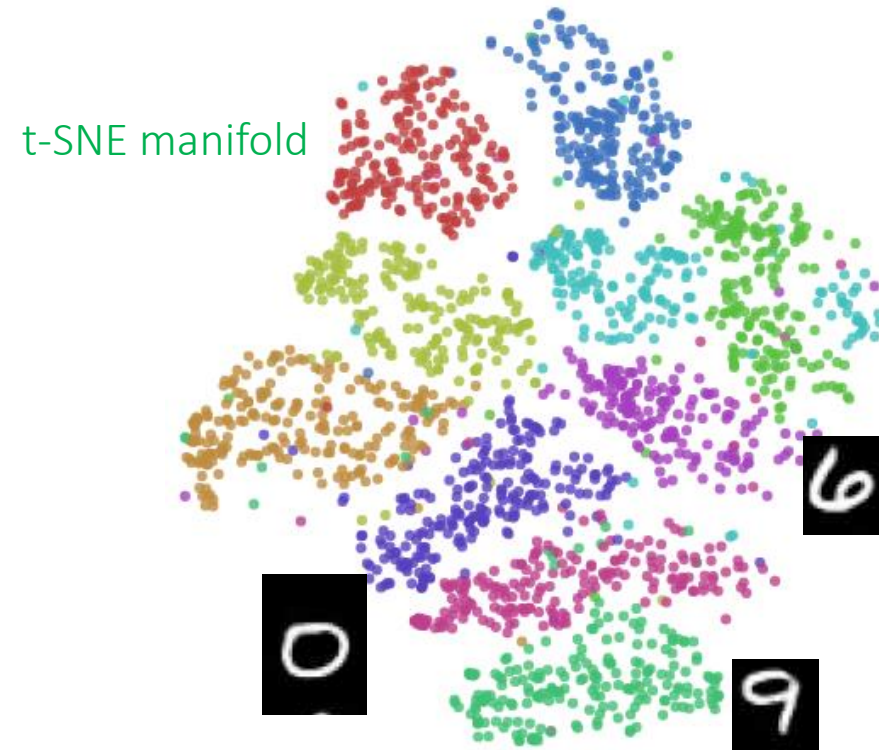
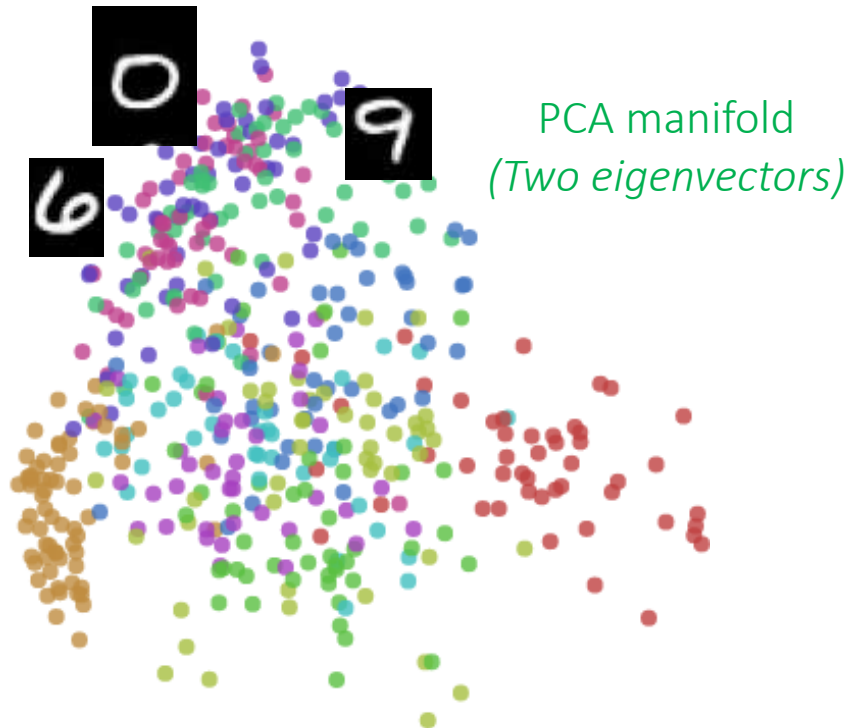
Feature manifold example

- Raw data live in huge dimensionalities
- Semantically meaningful raw data prefer lower dimensional manifolds
 - Which still live in the same huge dimensionalities
- Can we discover this manifold to embed our data on?



The digits manifold

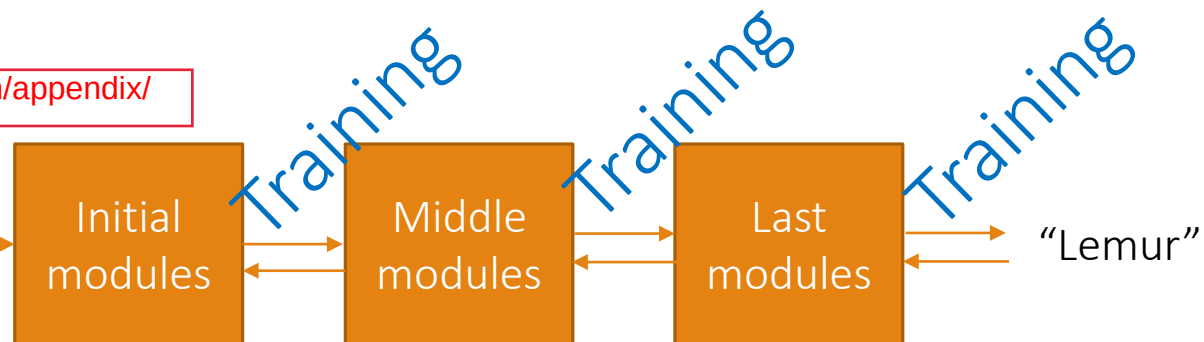
- There are good features and bad features, good manifold representations and bad manifold representations
- $28 \text{ pixels} \times 28 \text{ pixels} = 784 \text{ dimensions}$



End-to-end learning of feature hierarchies

- A pipeline of successive modules
- Each module's output is the input for the next module
- Modules produce features of higher and higher abstractions
 - Initial modules capture low-level features (e.g. edges or corners)
 - Middle modules capture mid-level features (e.g. circles, squares, textures)
 - Last modules capture high level, class specific features (e.g. face detector)

<https://distill.pub/2017/feature-visualization/appendix/>



Why learn the features?

- Manually designed features
 - Often take a lot of time to come up with and implement
 - Often take a lot of time to validate
 - Often they are incomplete, as one cannot know if they are optimal for the task
- Learned features
 - Are easy to adapt
 - Very compact and specific to the task at hand
 - Given a basic architecture in mind, it is relatively easy and fast to optimize
- Time spent for designing features now spent for designing architectures

vs.

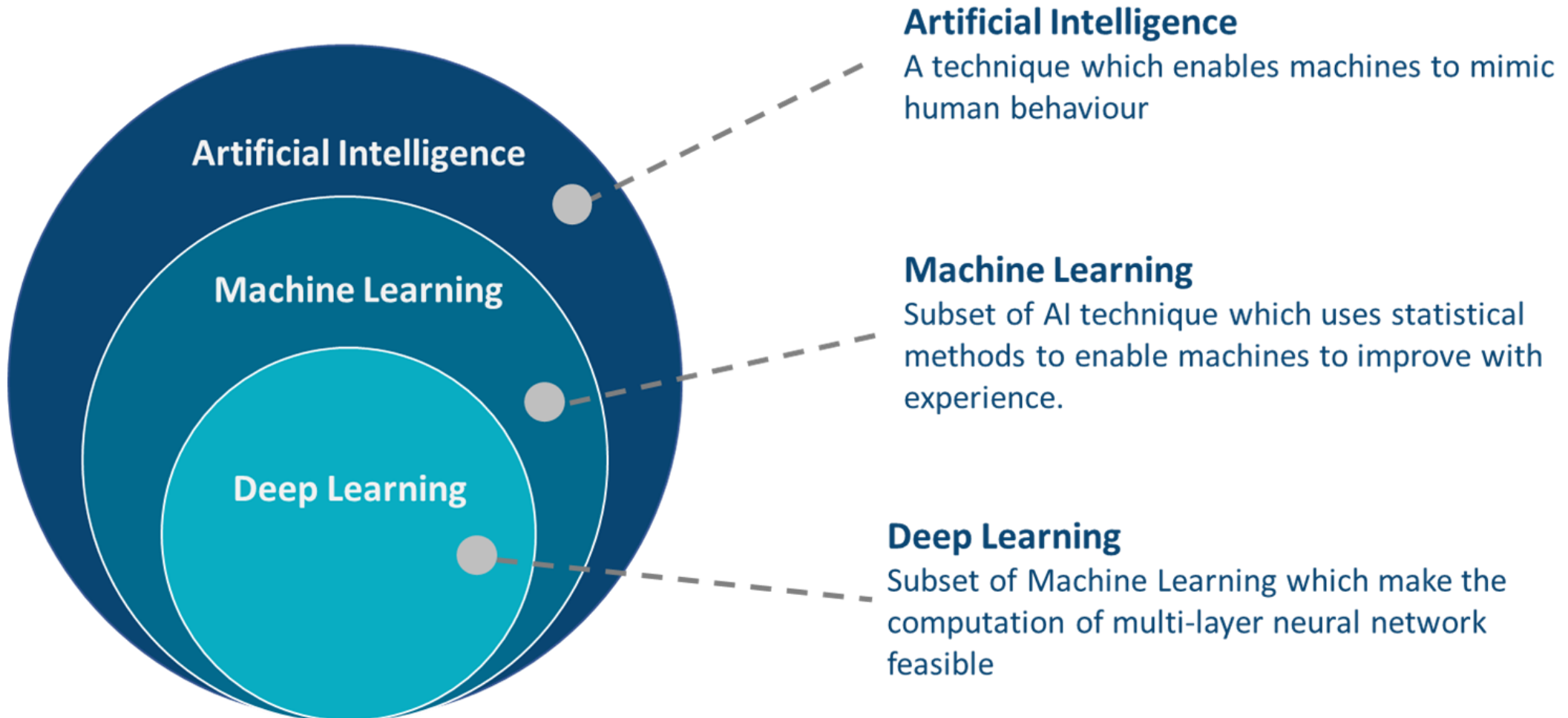


Deep Learning Architectures

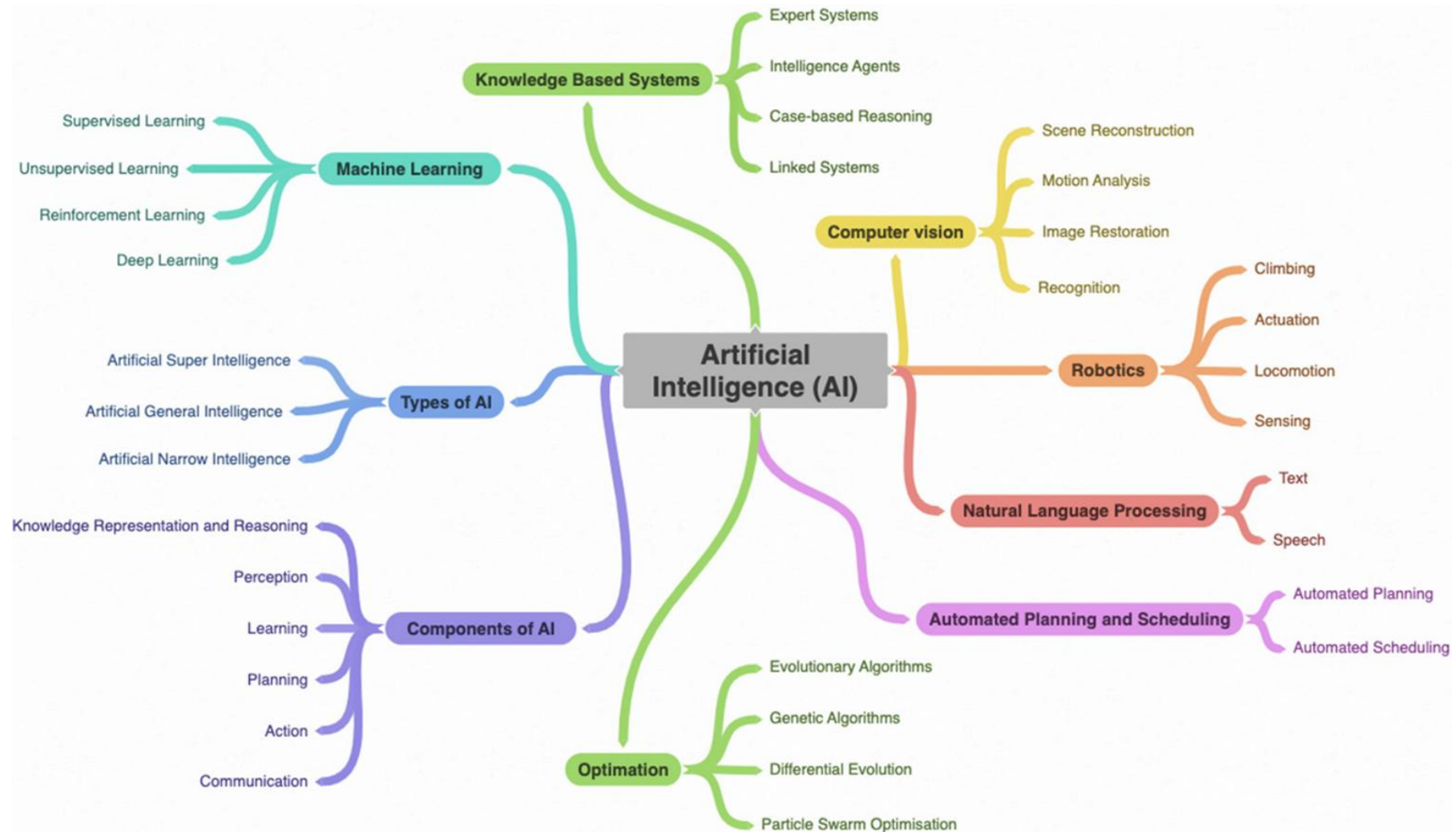
<https://developer.ibm.com/articles/cc-machine-learning-deep-learning-architectures/>

EE/CS 228 – Yinglun Zhu

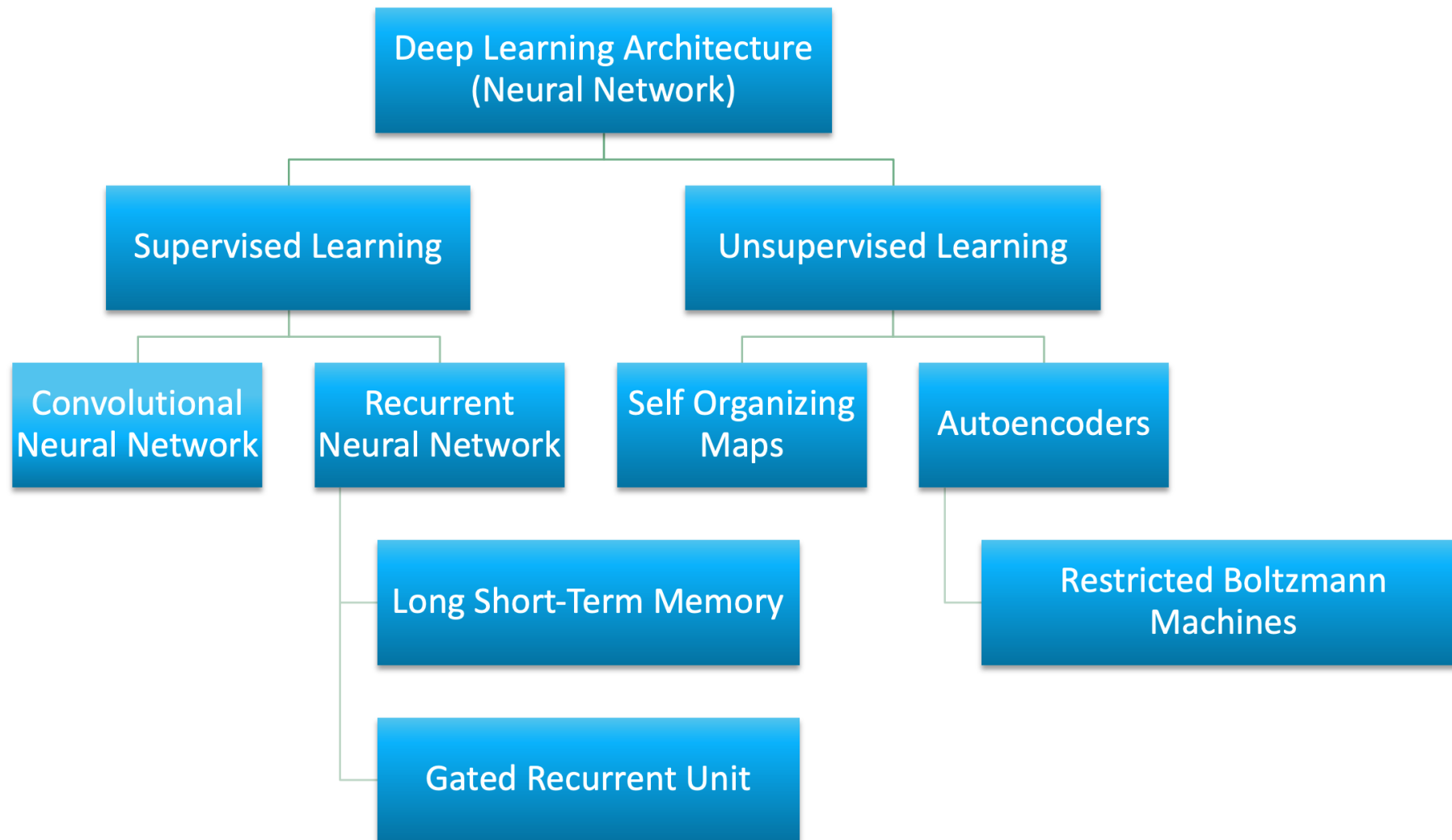
What is AI? ML? DL?



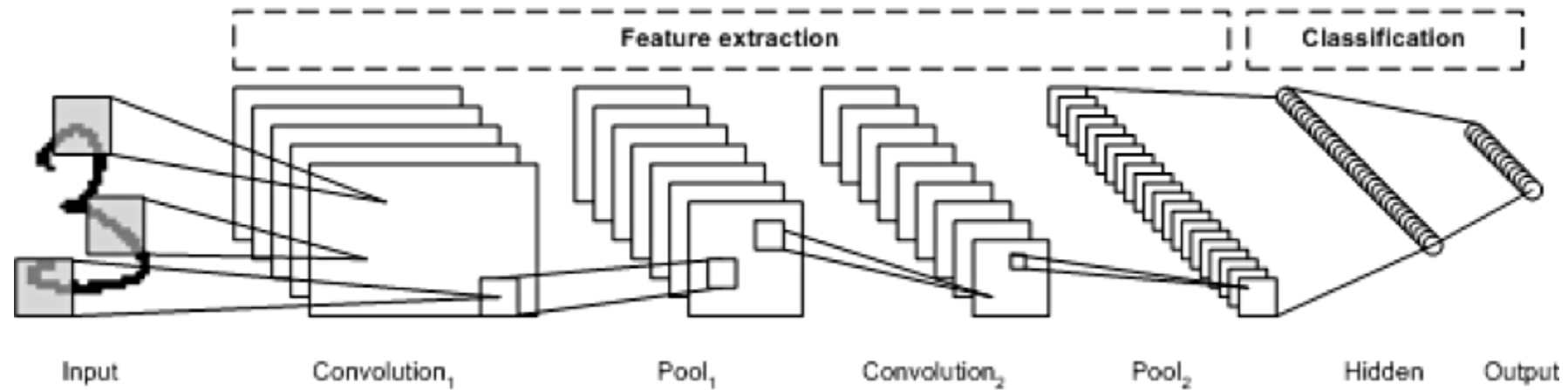
Overview of AI



Hierarchy of Learning Approaches



Convolutional Neural Networks

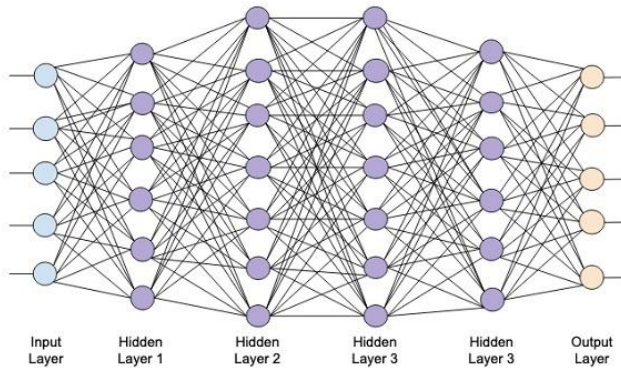


Convolution, Pooling, Dropout

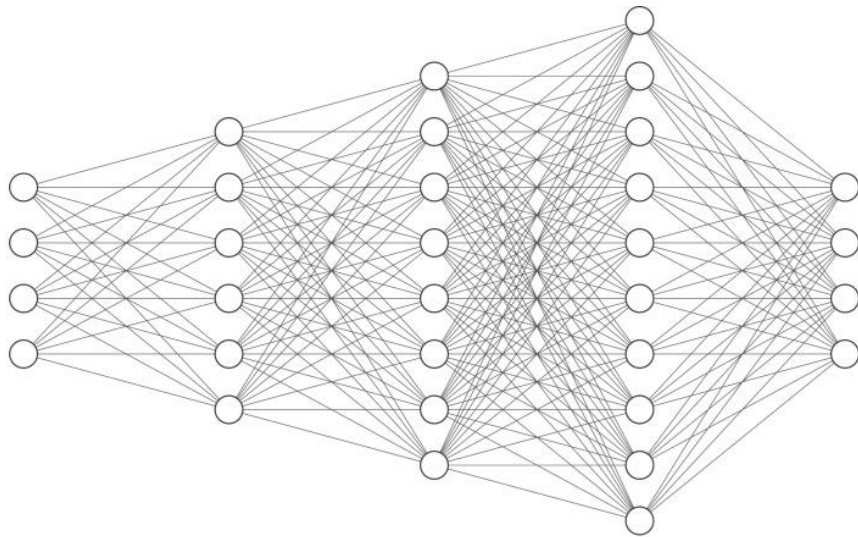
96 convolutional kernels of size
11x11 learned by the first convolutional
layer on the 224x224 input images. – AlexNet, NeurIPS 2012



Larger Networks?

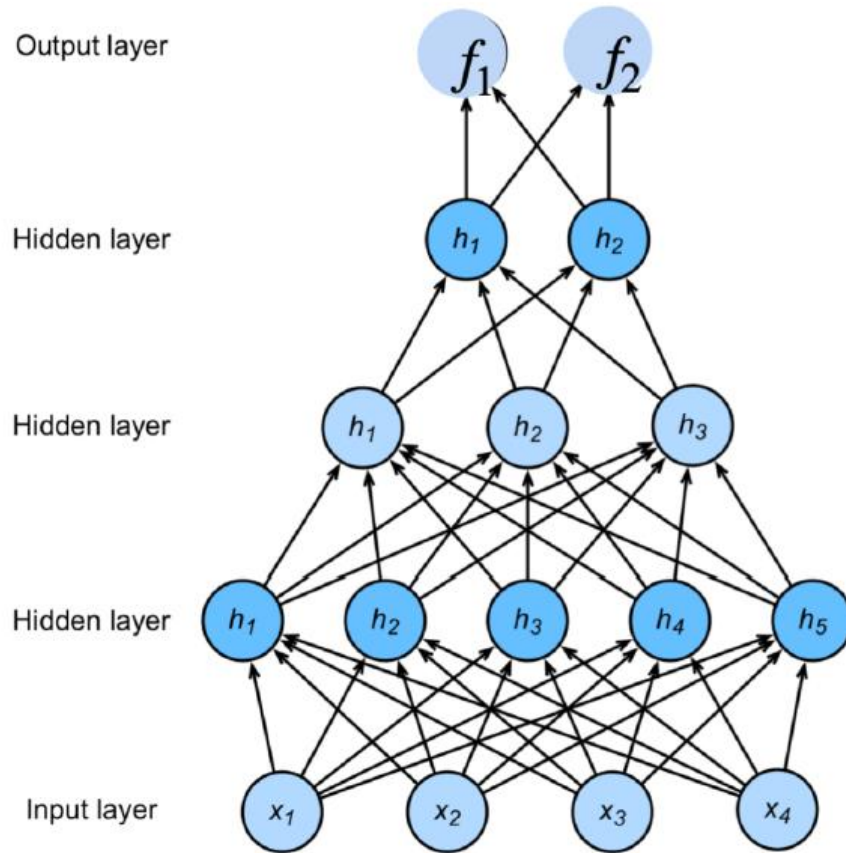


- The larger the better???
- No overfitting (layman term – memorize training data only, poor performance when testing data is used)
- What the reason of overfitting??? One of the reason is we have too many parameters

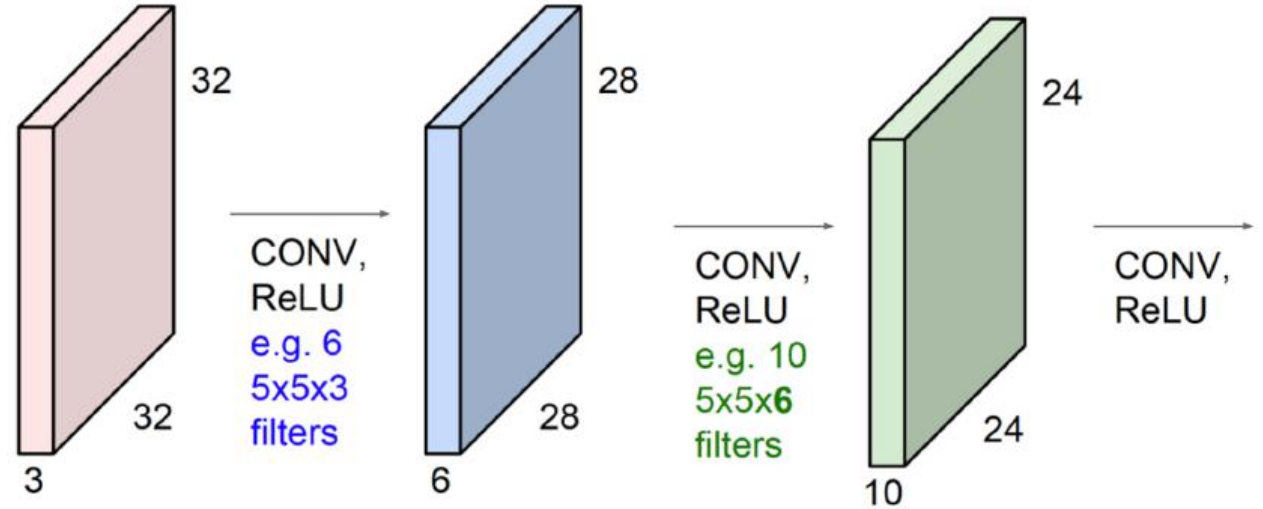


Input Layer $\in \mathbb{R}^4$ Hidden Layer $\in \mathbb{R}^6$ Hidden Layer $\in \mathbb{R}^8$ Hidden Layer $\in \mathbb{R}^{10}$ Output Layer $\in \mathbb{R}^4$

Feed-forward Networks

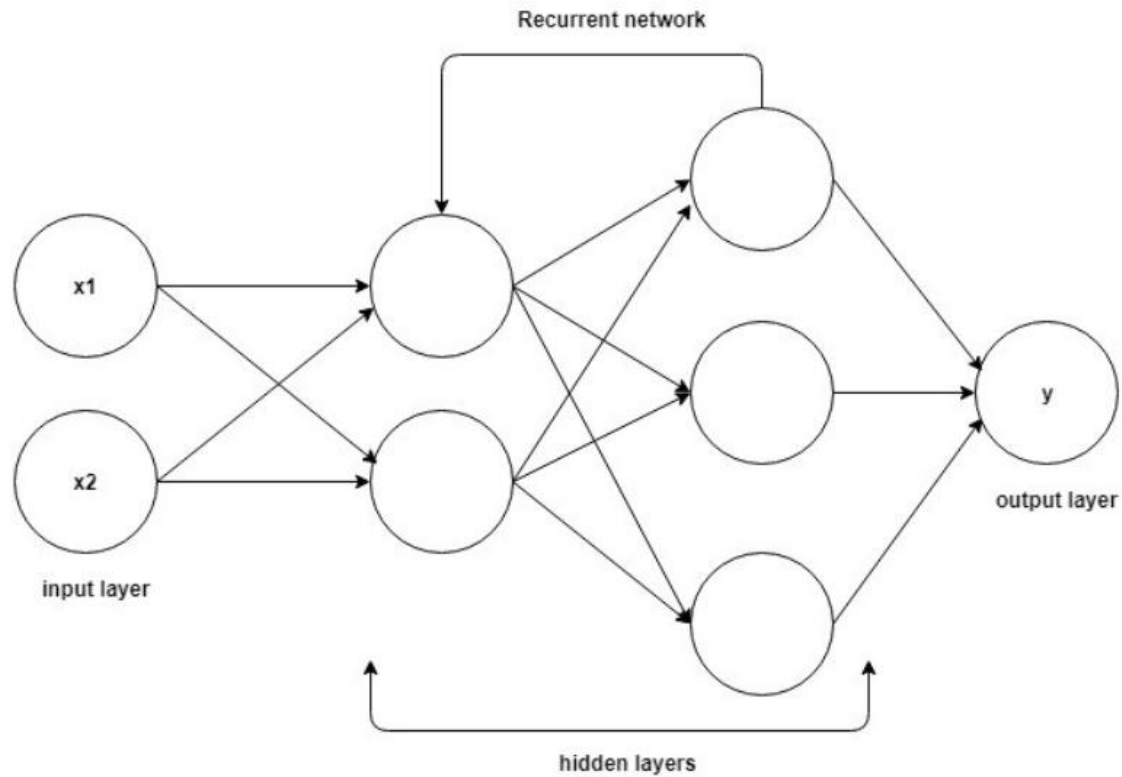


MLP



CNN

Recurrent Neural Networks

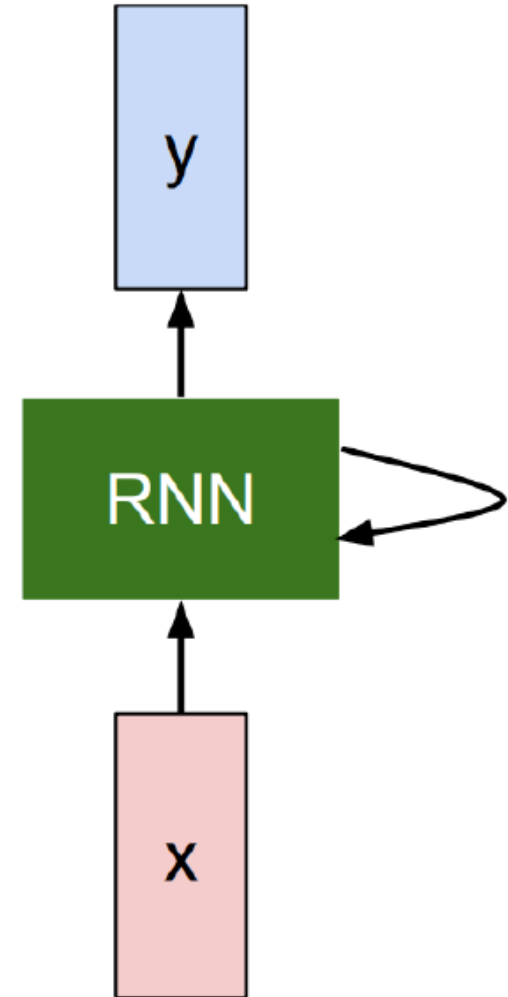


$$h_t = f_W(h_{t-1}, x_t)$$

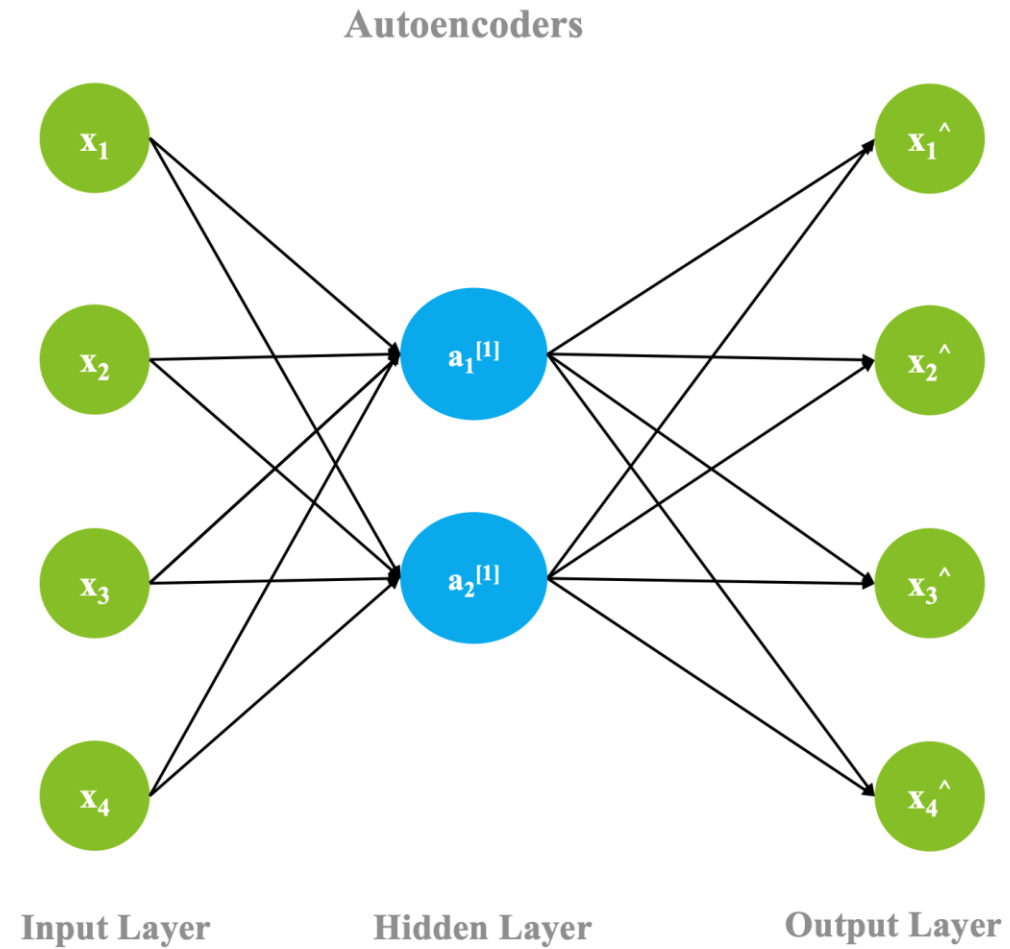
new state

old state

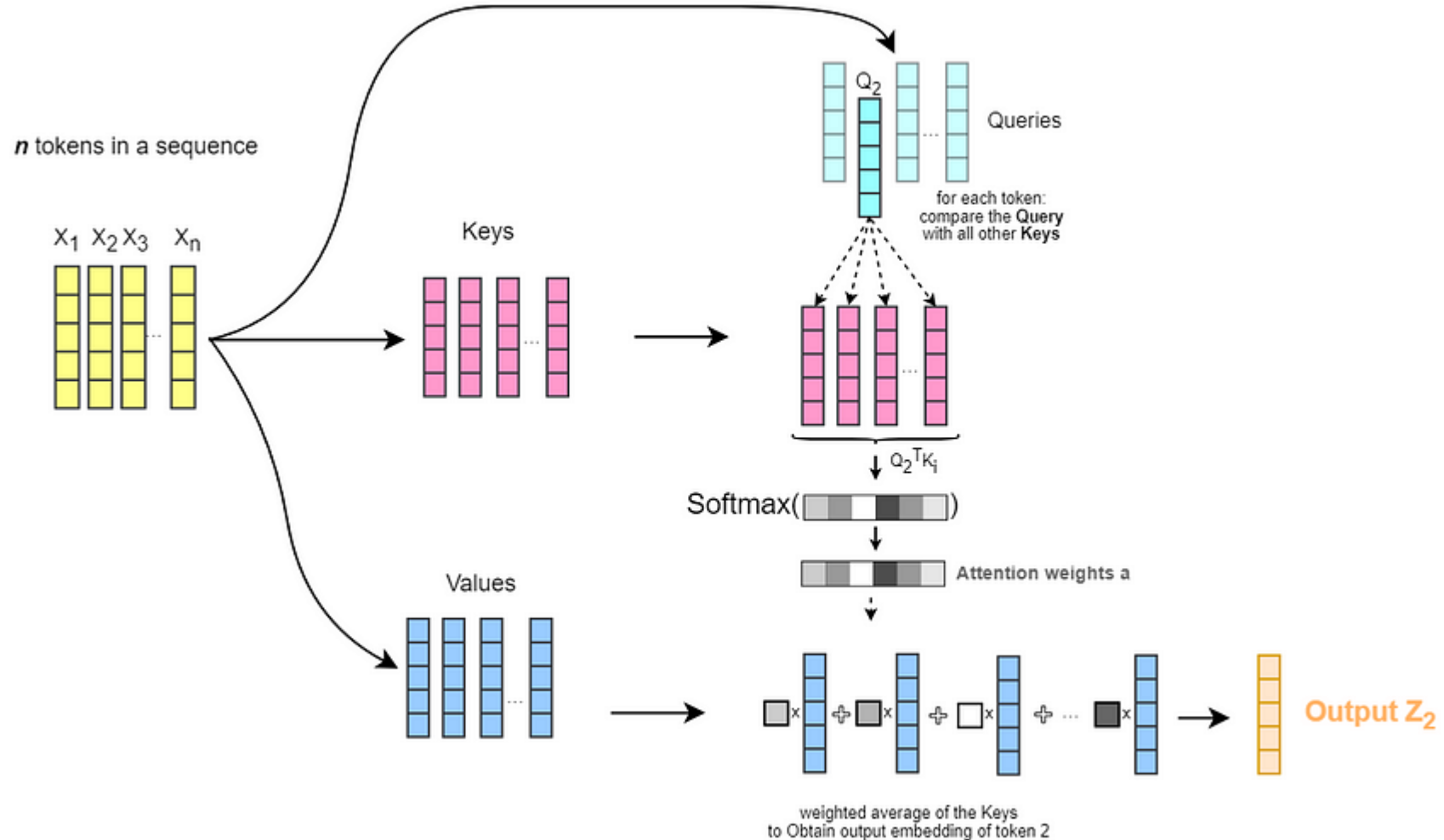
input at some
time step



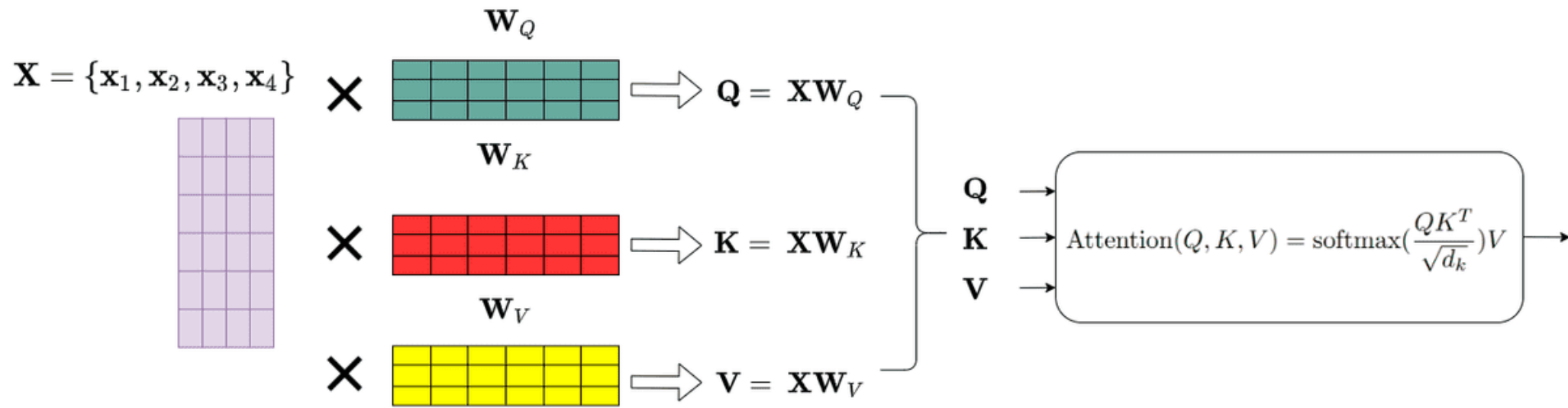
Autoencoders



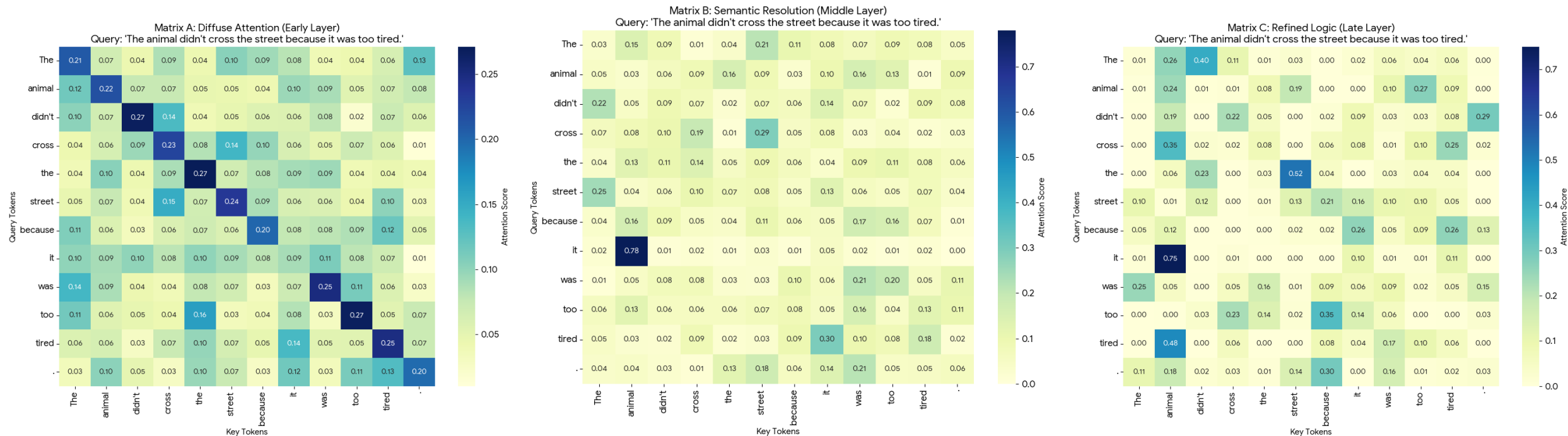
Transformers – Query, Key, Value



Self-Attention

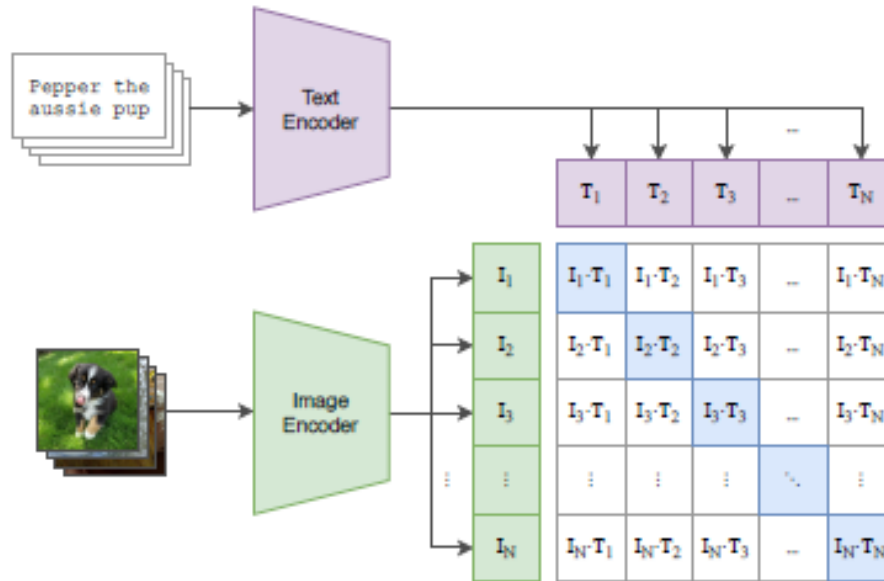


Attention Matrices

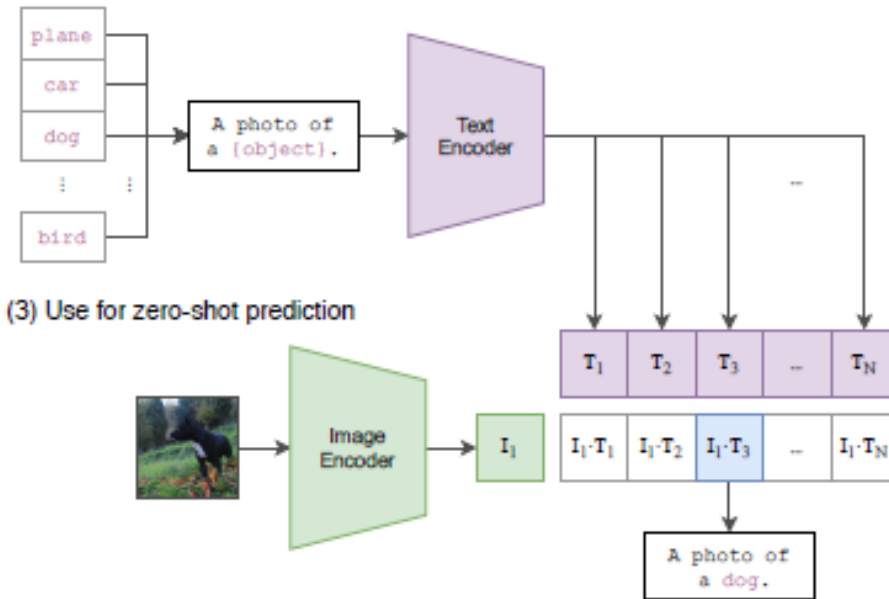


Vision-Language Representation

(1) Contrastive pre-training



(2) Create dataset classifier from label text



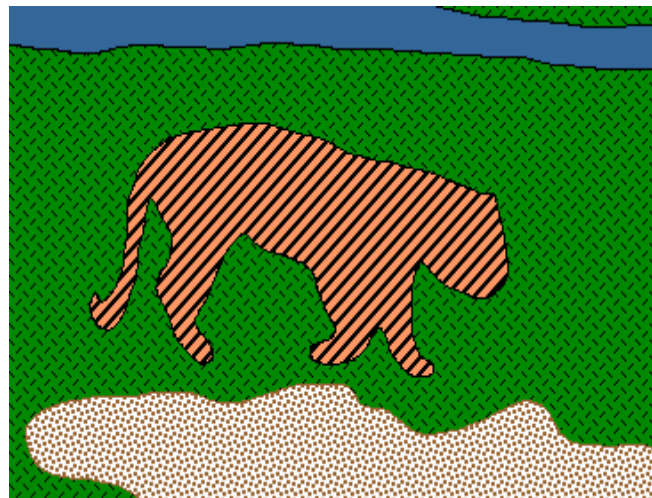
(3) Use for zero-shot prediction

Advantages and Issues with LLMs/VLMs

- Zero shot recognition
- Catastrophic forgetting
- Hallucinations
- Security

Segmentation and Clustering

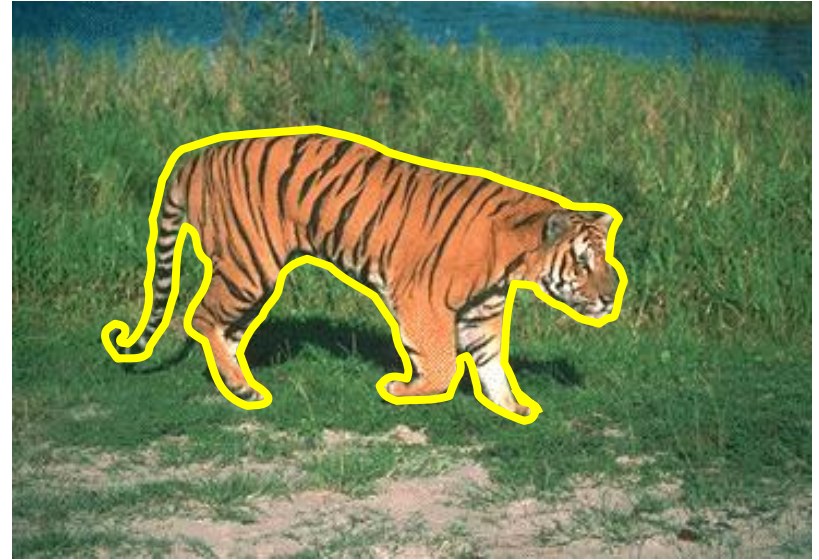
From images to objects



What Defines an Object?

- Subjective problem, but has been well-studied
- Gestalt Laws seek to formalize this
 - proximity, similarity, continuation, closure, common fate
 - see [notes](#) by Steve Joordens, U. Toronto

Extracting objects



How could this be done?

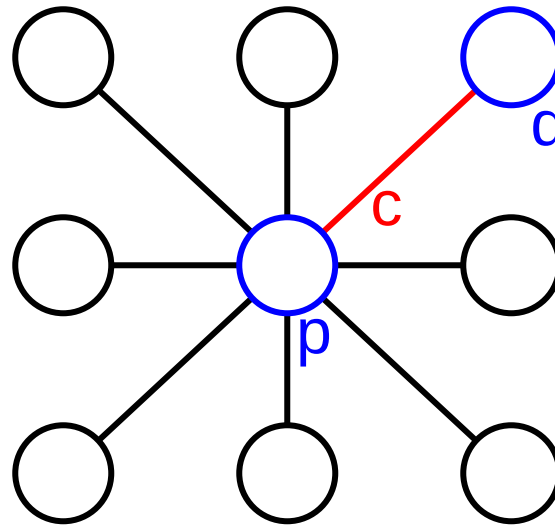
Image Segmentation

Many approaches proposed

- cues: color, regions, contours
- automatic vs. user-guided
- no clear winner
- we'll consider several approaches

Image as a graph

Treat the image as a graph



Graph

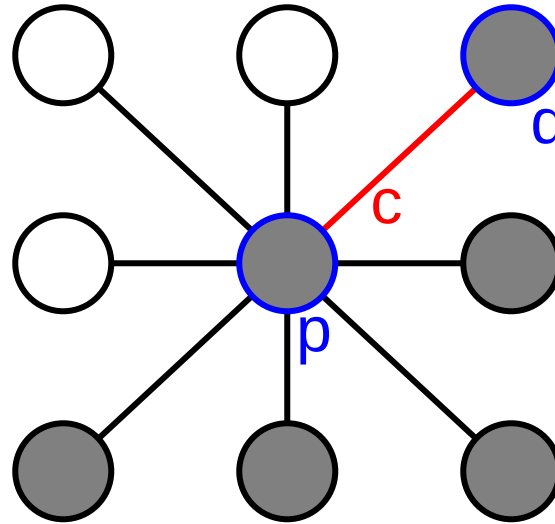
- node for every pixel **p**
- link between every adjacent pair of pixels, **p,q**
- cost **c** for each link

Note: each *link* has a cost

- this is a little different than the figure before where each pixel had a cost

Defining the costs

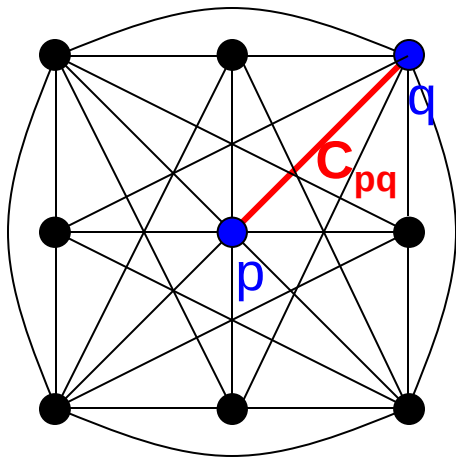
Treat the image as a graph



Want to hug image edges: how to define cost of a link?

- the link should follow the intensity edge
 - want intensity to change rapidly $\perp$ to the link
- $c \approx - |\text{difference of intensity } \perp \text{ to link}|$

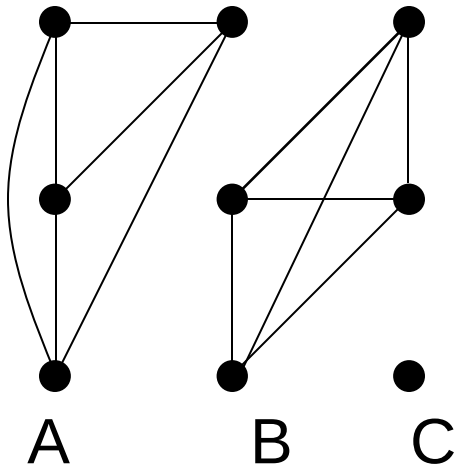
Automatic graph cut [Shi & Malik]



Fully-connected graph

- node for every pixel
- link between every pair of pixels, p, q
- cost C_{pq} for each link
 - C_{pq} measures *similarity*
 - » similarity is *inversely proportional* to difference in color and position

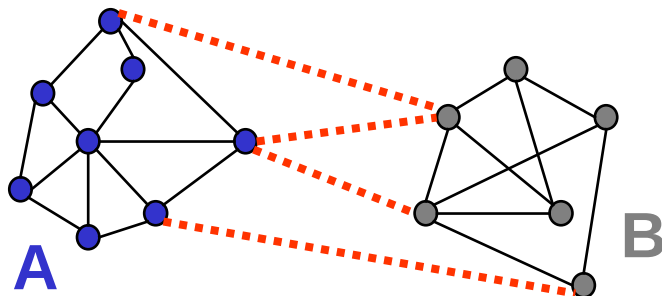
Segmentation by Graph Cuts



Break Graph into Segments

- Delete links that cross between segments
- Easiest to break links that have low cost (similarity)
 - similar pixels should be in the same segments
 - dissimilar pixels should be in different segments

Cuts in a graph



Link Cut

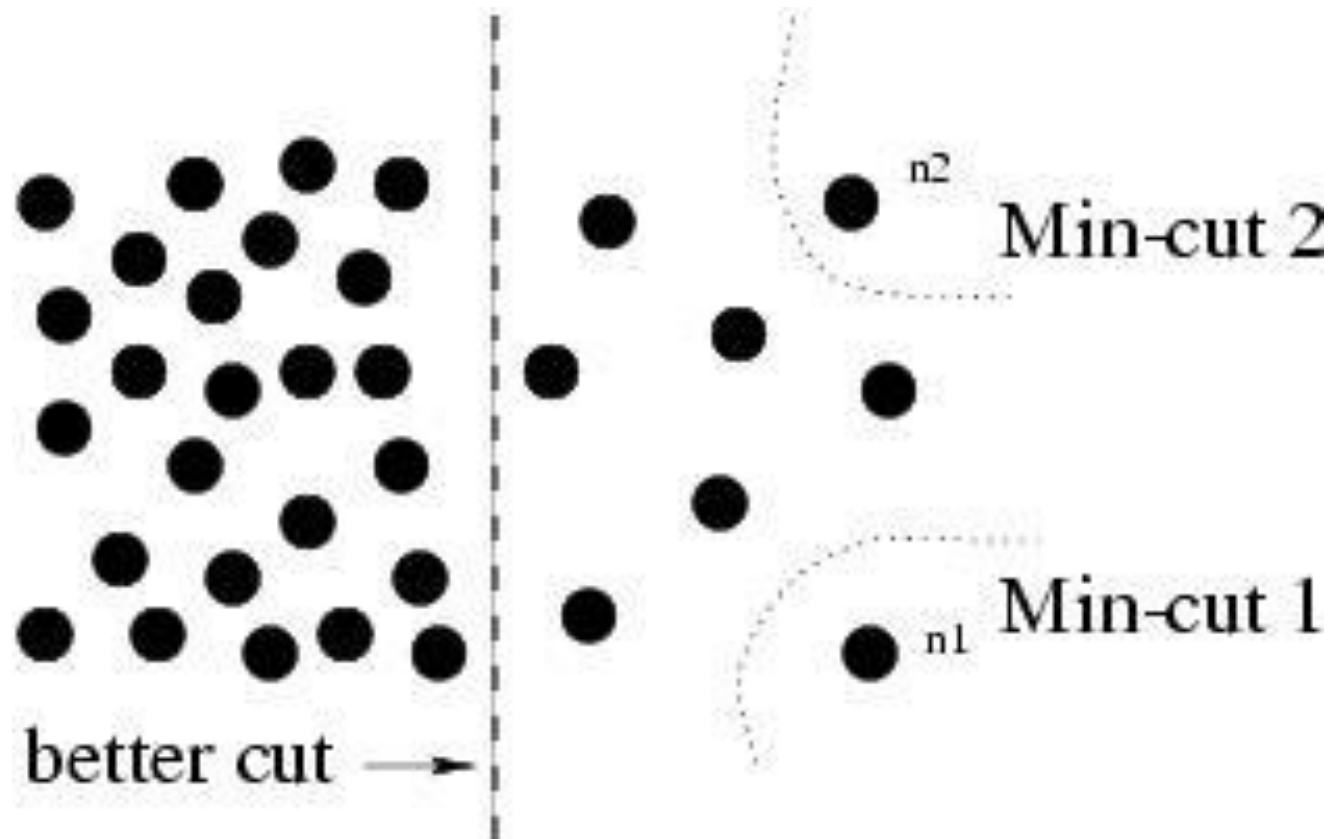
- set of links whose removal makes a graph disconnected
- cost of a cut:

$$cut(A, B) = \sum_{p \in A, q \in B} c_{p,q}$$

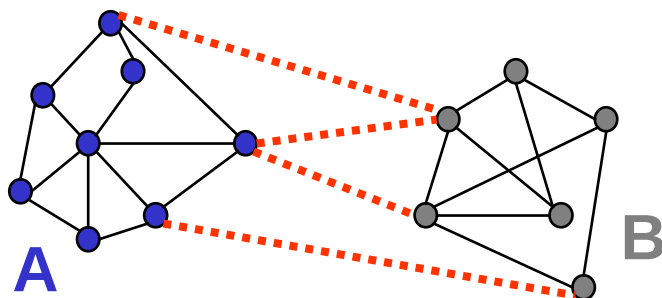
Find minimum cut

- gives you a segmentation

But min cut is not always the best cut...



Cuts in a graph



Normalized Cut

- a cut penalizes large segments
- fix by normalizing for size of segments

$$Ncut(A, B) = \frac{cut(A, B)}{volume(A)} + \frac{cut(A, B)}{volume(B)}$$

- $volume(A)$ = sum of costs of all edges that touch A

Further Reading: <https://arxiv.org/abs/2312.04713>

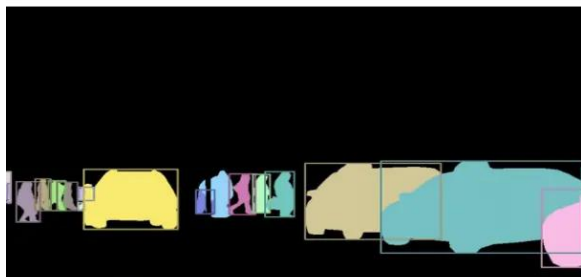
Types of Segmentation



(a) image



(b) semantic segmentation



(c) instance segmentation



(d) panoptic segmentation

<https://www.labellerr.com/blog/semantic-vs-instance-vs-panoptic-which-image-segmentation-technique-to-choose/>

Semantic Segmentation - Intersection Over Union: similarity between predicted and ground truth masks.

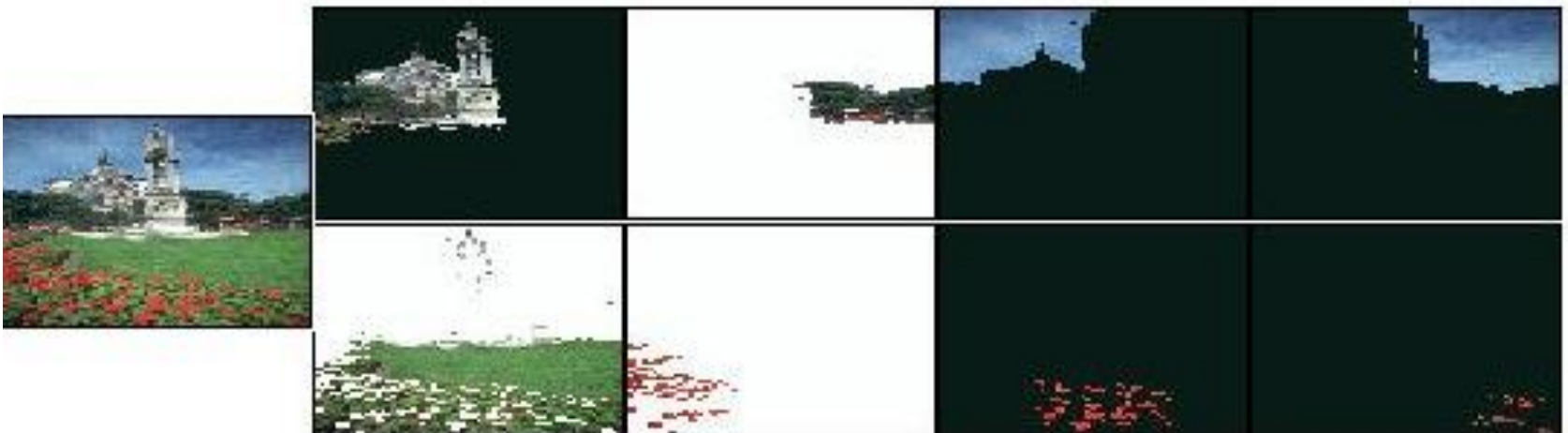
Instance Segmentation - Average Precision: IoU for each object instance

Panoptic Quality: Segmentation Quality (SQ) * Recognition Quality (RQ)

SQ: average IoU for matched segments

RQ: F1 score using precision and recall values of predicted masks

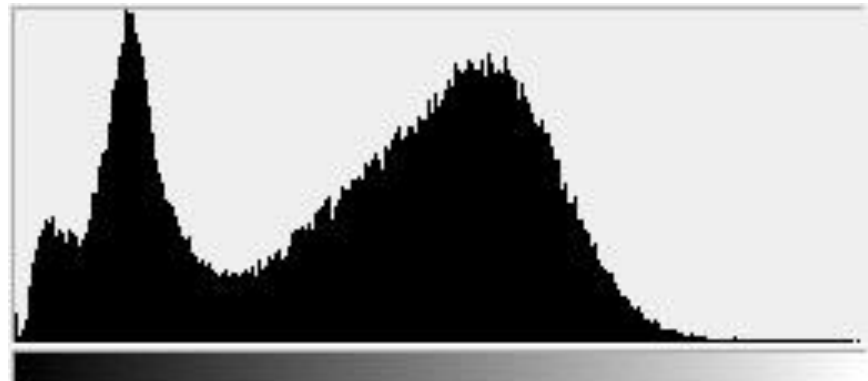
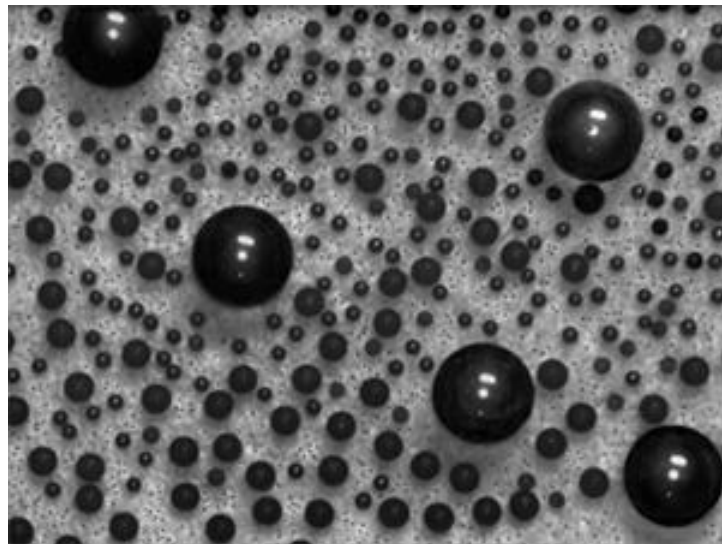
Color Image Segmentation



Histogram-based segmentation

Goal

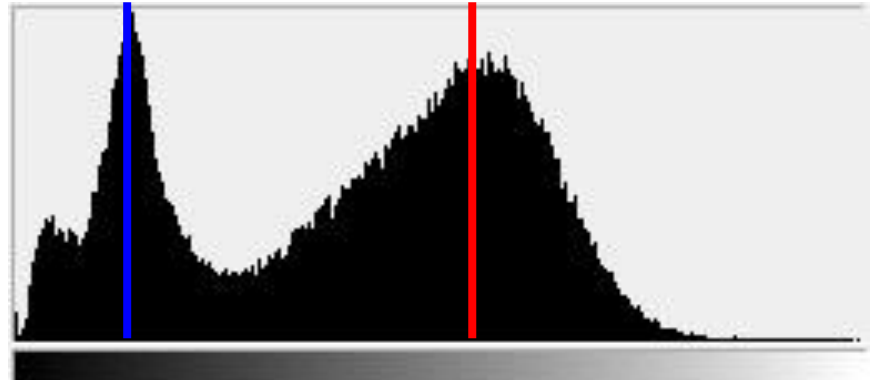
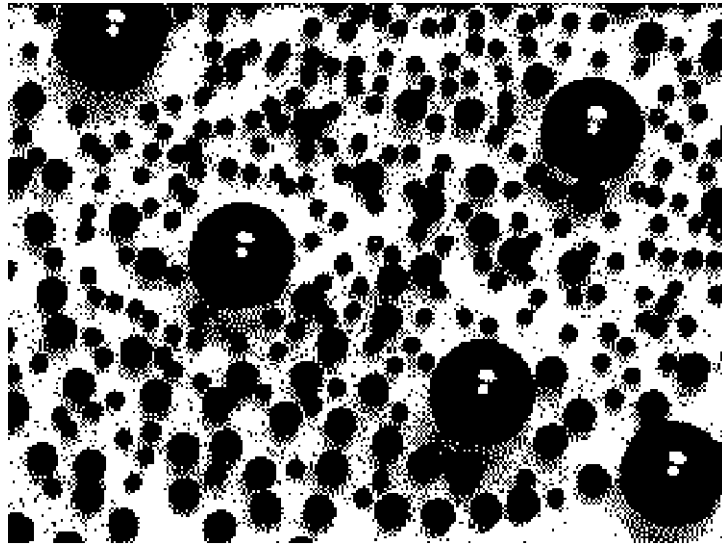
- Break the image into K regions (segments)
- Solve this by reducing the number of colors to K and mapping each pixel to the closest color



Histogram-based segmentation

Goal

- Break the image into K regions (segments)
- Solve this by reducing the number of colors to K and mapping each pixel to the closest color

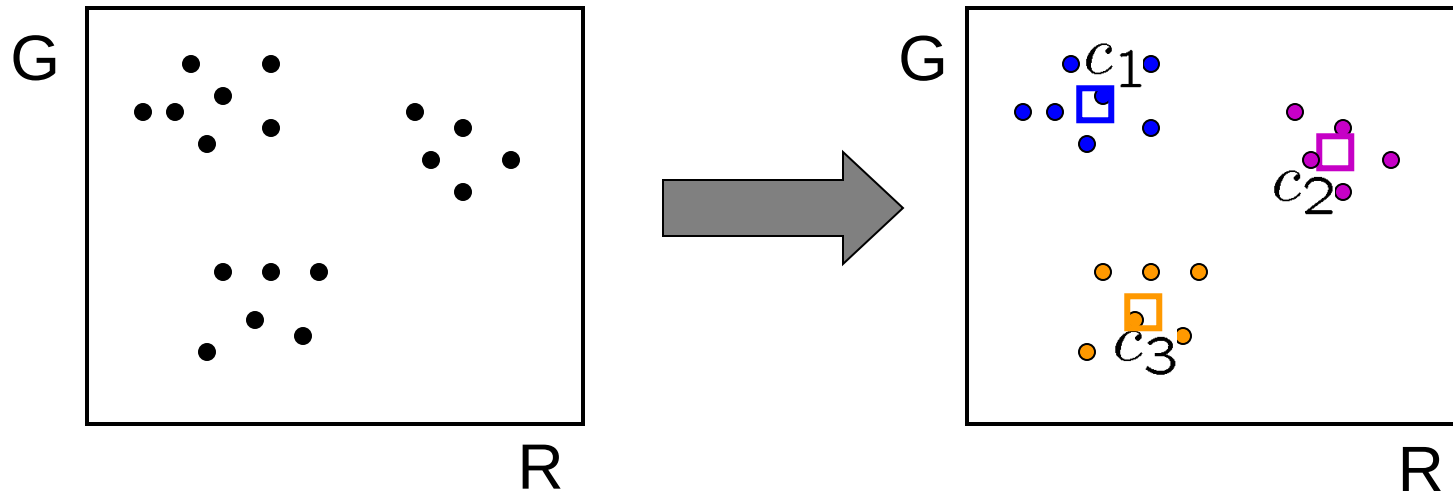


Here's what it looks like if we use two colors

Clustering

How to choose the representative colors?

- This is a clustering problem!



Objective

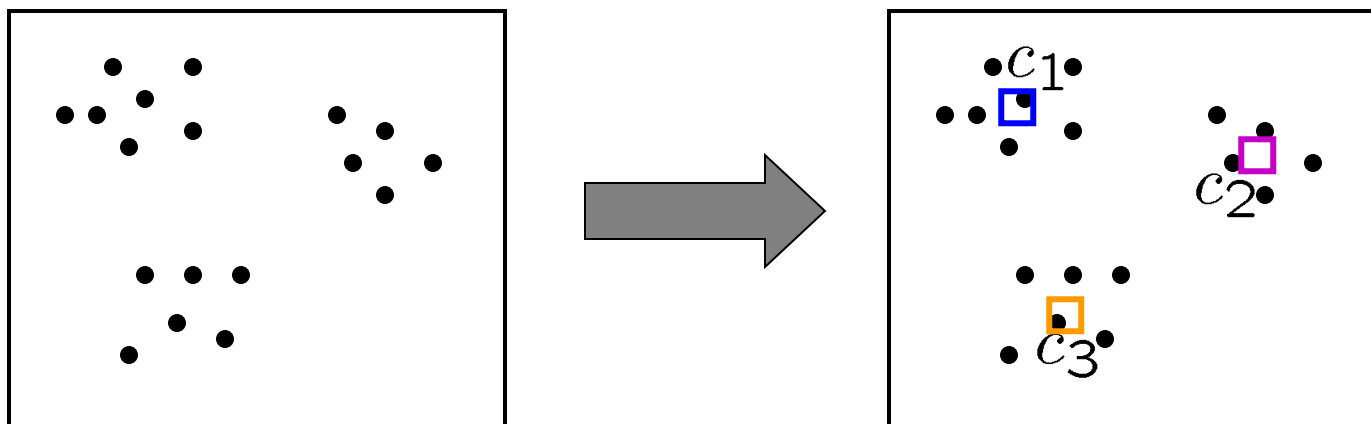
- Each point should be as close as possible to a cluster center
 - Minimize sum squared distance of each point to closest center

$$\sum_{\text{clusters } i} \sum_{\text{points } p \text{ in cluster } i} \|p - c_i\|^2$$

Break it down into subproblems

Suppose I tell you the cluster centers c_i

- Q: how to determine which points to associate with each c_i ?
- A: for each point p , choose closest c_i



Suppose I tell you the points in each cluster

- Q: how to determine the cluster centers?
- A: choose c_i to be the mean of all points in the cluster

K-means clustering

K-means clustering algorithm

1. Randomly initialize the cluster centers, $c_1, \dots, c_K$
2. Given cluster centers, determine points in each cluster
 - For each point p , find the closest c_i . Put p into cluster i
3. Given points in each cluster, solve for c_i
 - Set c_i to be the mean of points in cluster i
4. If c_i have changed, repeat Step 2

Properties

- Will always converge to *some* solution
- Can be a “local minimum”
 - does not always find the global minimum of objective function:

$$\sum_{\text{clusters } i} \sum_{\text{points } p \text{ in cluster } i} \|p - c_i\|^2$$

Probabilistic clustering

Basic questions

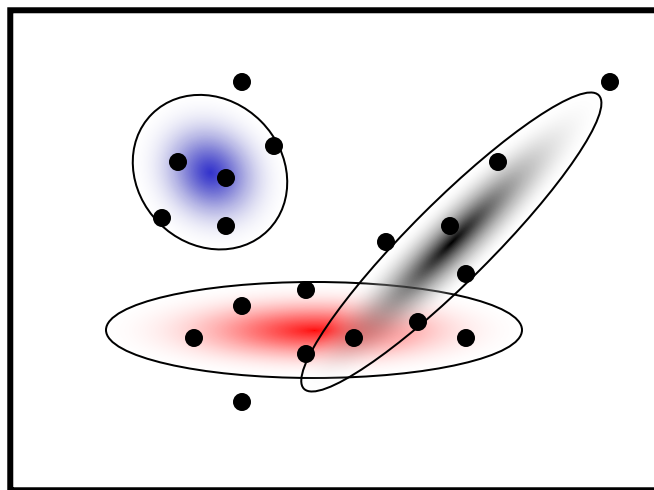
- what's the probability that a point $\mathbf{x}$ is in cluster m ?
- what's the shape of each cluster?

K-means doesn't answer these questions

Basic idea

- instead of treating the data as a bunch of points, assume that they are all generated by sampling a continuous function
- This function is called a **generative model**
 - defined by a vector of parameters θ

Mixture of Gaussians



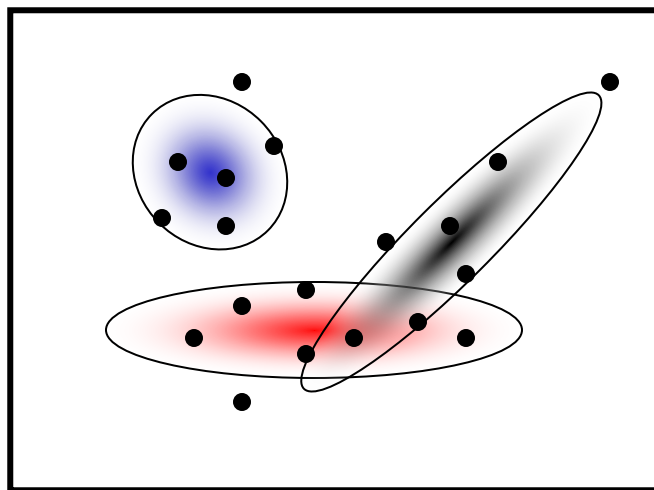
One generative model is a mixture of Gaussians (MOG)

- K Gaussian blobs with means μ_b covariance matrices V_b , dimension d
 - blob b defined by:
$$P(x|\mu_b, V_b) = \frac{1}{\sqrt{(2\pi)^d |V_b|}} e^{-\frac{1}{2}(x-\mu_b)^T V_b^{-1} (x-\mu_b)}$$
- blob b is selected with probability α_b
- the likelihood of observing $\mathbf{x}$ is a weighted mixture of Gaussians

$$P(x|\theta) = \sum_{b=1}^K \alpha_b P(x|\theta_b)$$

- where $\theta = [\mu_1, \dots, \mu_n, V_1, \dots, V_n]$

Expectation maximization (EM)



Goal

- find blob parameters θ that maximize the likelihood function:

$$P(data|\theta) = \prod_x P(x|\theta)$$

Approach:

1. E step: given current guess of blobs, compute ownership of each point
2. M step: given ownership probabilities, update blobs to maximize likelihood function
3. repeat until convergence

EM details

E-step

- compute probability that point $\mathbf{x}$ is in blob i , given current guess of $\boldsymbol{\theta}$

$$P(b|x, \mu_b, V_b) = \frac{\alpha_b P(x|\mu_b, V_b)}{\sum_{i=1}^K \alpha_i P(x|\mu_i, V_i)}$$

M-step

- compute probability that blob b is selected

$$\alpha_b^{new} = \frac{1}{N} \sum_{i=1}^N P(b|x_i, \mu_b, V_b) \quad \text{N data points}$$

- mean of blob b

$$\mu_b^{new} = \frac{\sum_{i=1}^N x_i P(b|x_i, \mu_b, V_b)}{\sum_{i=1}^N P(b|x_i, \mu_b, V_b)}$$

- covariance of blob b

$$V_b^{new} = \frac{\sum_{i=1}^N (x_i - \mu_b^{new})(x_i - \mu_b^{new})^T P(b|x_i, \mu_b, V_b)}{\sum_{i=1}^N P(b|x_i, \mu_b, V_b)}$$

Applications of EM

Turns out this is useful for all sorts of problems

- any clustering problem
- any model estimation problem
- missing data problems
- finding outliers
- segmentation problems
 - segmentation based on color
 - segmentation based on motion
 - foreground/background separation
- ...

Problems with EM

1. Local minima

k-means is NP-hard even with $k=2$

2. Need to know number of segments

solutions: AIC, BIC, Dirichlet process mixture

3. Need to choose generative model

(Courtesy : Univ of Wisconsin, Madison)

Normalized Cut Method for Image Segmentation

- J. Shi and J. Malik, *IEEE Trans. Pattern Analysis and Machine Intelligence* **22**(8), 1997
- Divisive (aka splitting, partitioning) method
- Graph-theoretic criterion for measuring goodness of an image partition
- Hierarchical partitioning
 - dendrogram type representation of all regions

- Criterion for measuring a candidate partitioning: Affinity measure between elements **within** each region is **high**, and the affinity **between** elements across regions is **low**
- **Affinity**: element $\times$ element $\rightarrow \mathbb{R}^+$ Examples of components of an affinity function: spatial position, intensity, color, texture, motion. Defines the similarity of a pair of data elements.

Affinity (Similarity) Measures

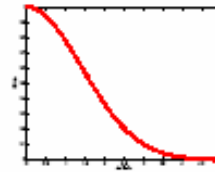
- Intensity

$$\text{aff}(\mathbf{x}, \mathbf{y}) = e^{-\|I(\mathbf{x}) - I(\mathbf{y})\|^2 / 2\sigma_I^2}$$

- Distance

$$\text{aff}(\mathbf{x}, \mathbf{y}) = e^{-\|\mathbf{x} - \mathbf{y}\|^2 / 2\sigma_d^2}$$

- Color
- Texture
- Motion



Problem Formulation

- Given an undirected graph $G = (V, E)$, where V is a set of nodes, one for each data element (e.g., pixel), and E is a set of edges with weights representing the affinity between connected nodes
- Find the image partition that maximizes the “association” within each region *and* minimizes the “disassociation” between regions
- Finding the optimal partition is NP-complete

- Let A, B partition G. Therefore, $A \cup B = V$, and $A \cap B = \emptyset$
- The **affinity** or **similarity** between A and B is defined as

$$\text{cut}(A,B) = \sum_{i \in A, j \in B} w_{ij}$$

= total weight of edges removed

- The **optimal bi-partition** of G is the one that minimizes *cut*
- *Cut* is biased towards small regions

- So, instead define the **normalized** similarity, called the *normalized-cut*(A,B), as

$$\text{ncut}(A,B) = \frac{\text{cut}(A,B)}{\text{assoc}(A,V)} + \frac{\text{cut}(B,A)}{\text{assoc}(B,V)}$$

$$\text{where } \text{assoc}(A,V) = \sum_{i \in A, k \in V} w_{ik}$$

= total connection weight from nodes in A
to all nodes in G

- *Ncut* measures the dissimilarity between regions
("disassociation" measure)
- *Ncut* removes the bias based on region size (usually)

- Similarly, define the “**normalized association:**”

$$nassoc(A, B) = \frac{assoc(A, A)}{assoc(A, V)} + \frac{assoc(B, B)}{assoc(B, V)}$$

- *Nassoc* measures how similar, on average, nodes within the groups are to each other
- New **goal**: Find the bi-partition that minimizes $ncut(A, B)$ **and** maximizes $nassoc(A, B)$
- But, it can be proved that $ncut(A, B) = 2 - nassoc(A, B)$, so we can just **minimize $ncut$** : $y = \arg \min ncut$

- Let $\mathbf{y}$ be a $P = |V|$ dimensional vector where $y_i = \begin{cases} 1, & \text{if node } i \in A \\ -1, & \text{otherwise} \end{cases}$

- Let $d(i) = \sum_j w_{ij}$
define the affinity of node i with all other nodes

- Let $\mathbf{D} = P \times P$ diagonal matrix:

$$\mathbf{D} = \begin{bmatrix} d_1 & 0 & \dots & 0 \\ 0 & d_2 & \dots & 0 \\ & & \dots & \\ 0 & 0 & \dots & d_p \end{bmatrix} \quad \text{“degree matrix”}$$

- Let $\mathbf{A} = P \times P$ symmetric matrix:

“affinity matrix”

$$\mathbf{A} = \begin{bmatrix} w_{11} & w_{12} & \dots & w_{1P} \\ w_{21} & w_{22} & \dots & w_{2P} \\ & & \dots & \\ w_{P1} & w_{P2} & \dots & w_{PP} \end{bmatrix}$$

- It can be shown that

$$\mathbf{y} = \arg \min_{\mathbf{x}} ncut(\mathbf{x})$$

$$= \arg \min_{\mathbf{y}} \frac{\mathbf{y}^T (\mathbf{D} - \mathbf{A}) \mathbf{y}}{\mathbf{y}^T \mathbf{D} \mathbf{y}} \text{ subject to } \mathbf{y}^T \mathbf{D} \mathbf{1} = 0$$

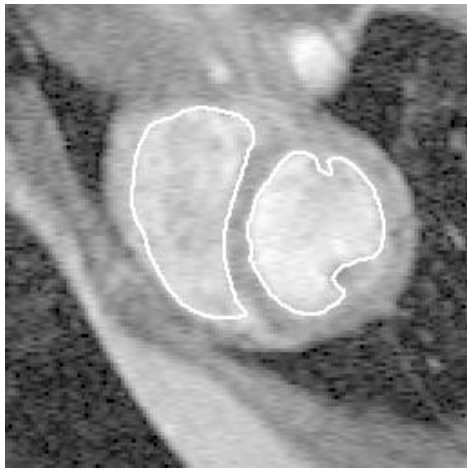
- Relaxing the constraint on $\mathbf{y}$ so as to allow it to have real values means that we can **approximate the solution** by solving an equation of the form: $(\mathbf{D} - \mathbf{A}) \mathbf{y} = \lambda \mathbf{D} \mathbf{y}$

- The solution, $\mathbf{y}$, is an eigenvector of $(\mathbf{D} - \mathbf{A})$
- An eigenvector is a characteristic vector of a matrix and specifies a segmentation based on the values of its components; similar points will hopefully have similar eigenvector components.
- Theorem: If $\mathbf{M}$ is any real, symmetric matrix and $\mathbf{x}$ is orthogonal to the $j-1$ smallest eigenvectors $\mathbf{x}_1, \dots, \mathbf{x}_{j-1}$, then $\mathbf{x}^T \mathbf{M} \mathbf{x} / \mathbf{x}^T \mathbf{x}$ is minimized by the next smallest eigenvector $\mathbf{x}_j$ and its minimum value is the eigenvalue λ_j

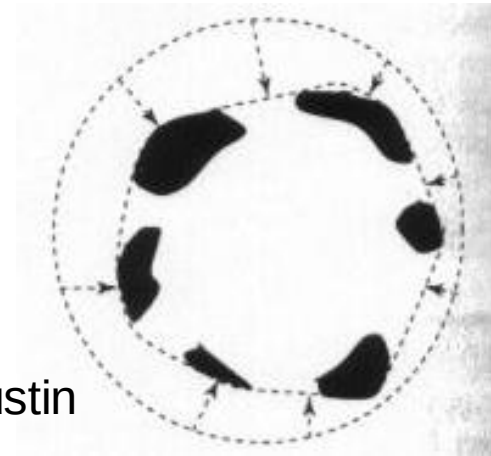
- Smallest eigenvector is always 0
because $A=V$, $B=\{\}$ means $ncut(A,B)=0$
- Second smallest eigenvector is the real-valued $\mathbf{y}$ that minimizes $ncut$
- Third smallest eigenvector is the real-valued $\mathbf{y}$ that optimally sub-partitions the first two regions
- Etc.
- Note: Converting from the real-valued $\mathbf{y}$ to a binary-valued $\mathbf{y}$ introduces errors that will propagate to each sub-partition

NCUT Segmentation Algorithm

1. Set up problem as $G = (V, E)$ and define affinity matrix $\mathbf{A}$ and degree matrix $\mathbf{D}$
2. Solve $(\mathbf{D} - \mathbf{A})\mathbf{x} = \lambda\mathbf{D}\mathbf{x}$ for the eigenvectors with the smallest eigenvalues
3. Let $\mathbf{x}_2$ = eigenvector with the 2nd smallest eigenvalue λ_2
4. Threshold $\mathbf{x}_2$ to obtain the binary-valued vector $\mathbf{x}'_2$ such that $ncut(\mathbf{x}'_2) \geq ncut(\mathbf{x}^t_2)$ for all possible thresholds t
5. For each of the two new regions, if $ncut < \text{threshold } T$, then recurse on the region



Deformable contours

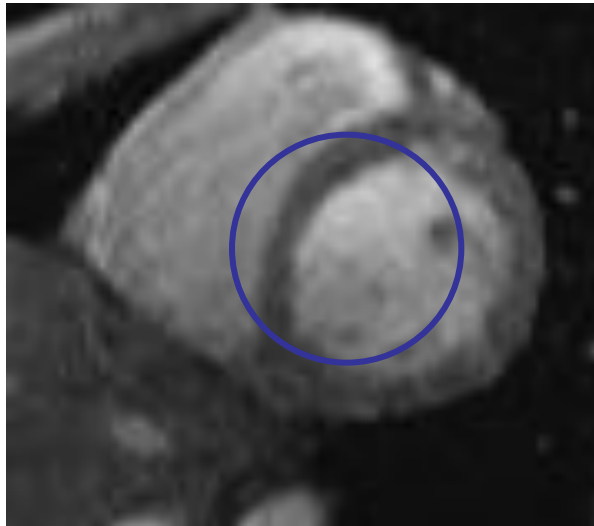


Adapted from slides by Kristen Grauman, UT Austin

Deformable contours

a.k.a. active contours, snakes

Given: initial contour (model) near desired object



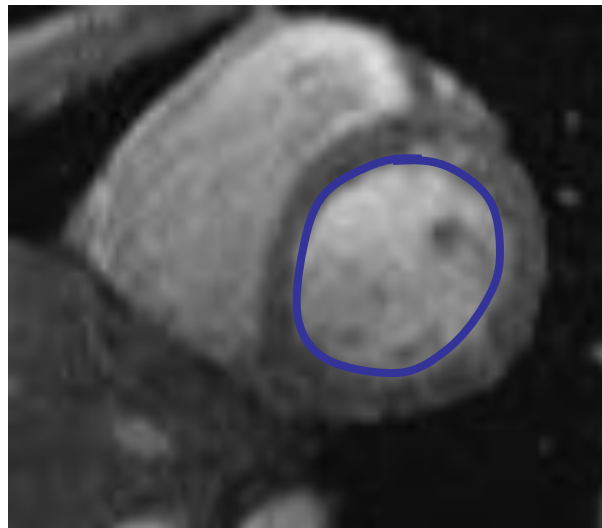
(Single frame)

Deformable contours

a.k.a. active contours, snakes

Given: initial contour (model) near desired object

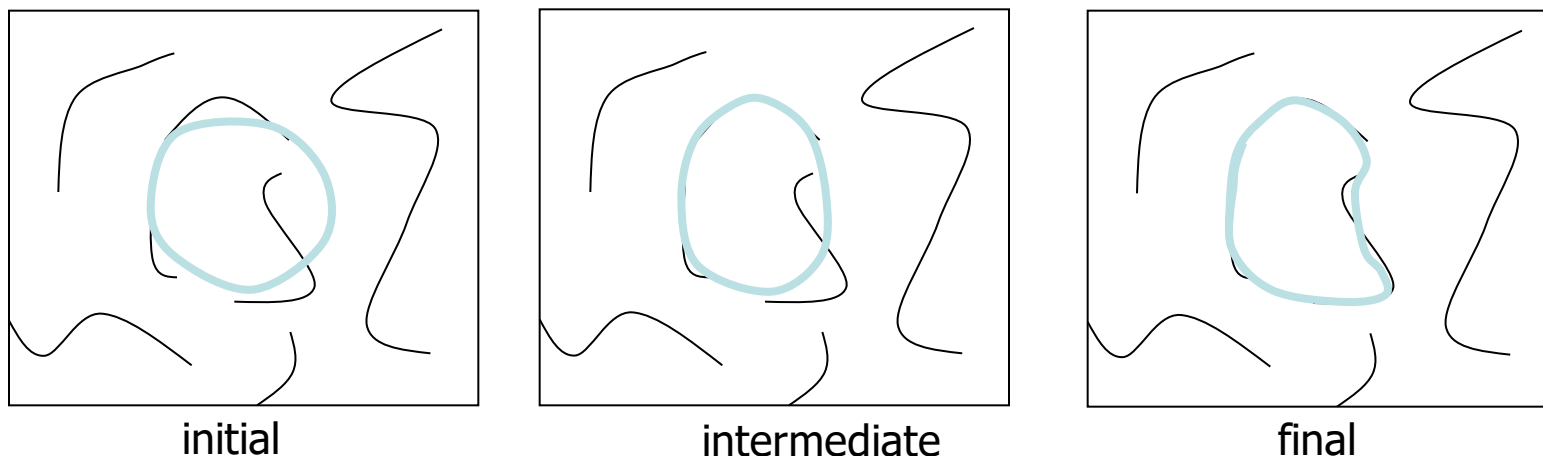
Goal: evolve the contour to fit exact object boundary



(Single frame)

Deformable contours

a.k.a. active contours, snakes



Initialize near contour of interest

Iteratively refine: elastic band is adjusted so as to

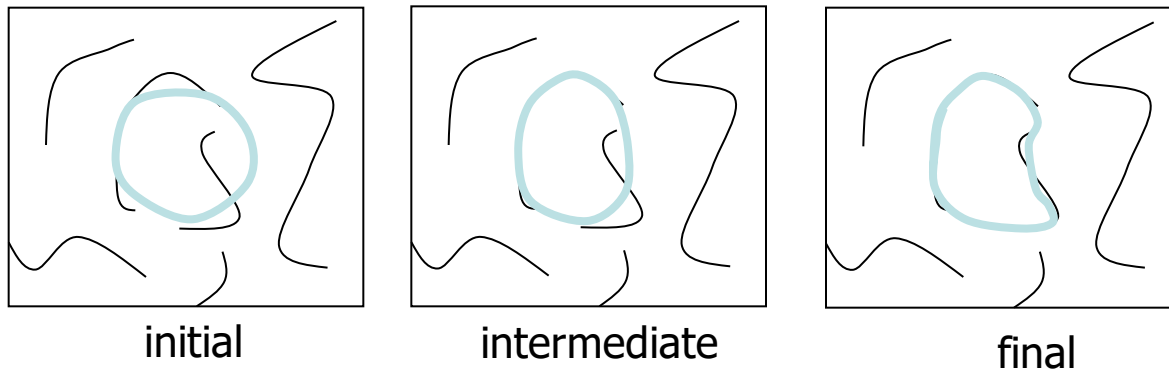
- be near image positions with high gradients, and
- satisfy shape “preferences” or contour priors

Deformable contours

a.k.a. active contours, snakes

How is the current contour adjusted to find the new contour at each iteration?

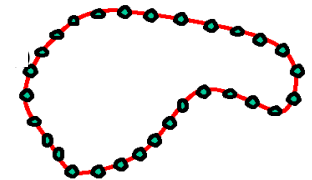
- Define a cost function (“energy” function) that says how good a possible configuration is.
- Seek next configuration that minimizes that cost function.



What are examples of problems with energy functions that we have seen previously?

Snakes energy function

The total energy (cost) of the current snake is defined as:



$$E_{total} = E_{internal} + E_{external}$$

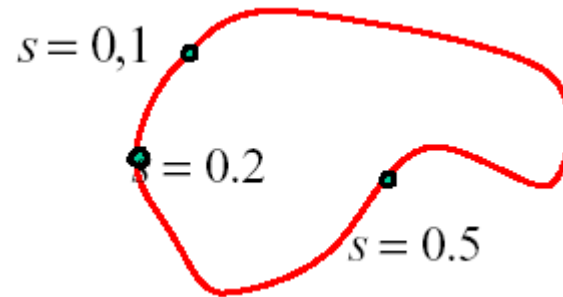
Internal energy: encourage prior shape preferences: e.g., smoothness, elasticity, particular known shape.

External energy (“image” energy): encourage contour to fit on places where image structures exist, e.g., edges.

A good fit between the current deformable contour and the target shape in the image will yield a **low** value for this cost function.

Parametric curve representation (continuous case)

$$\nu(s) = (x(s), y(s)) \quad 0 \leq s \leq 1$$



External energy: intuition

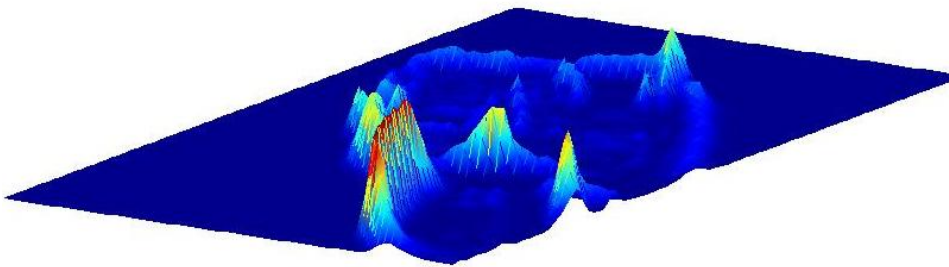
- Measure how well the curve matches the image data
- “Attract” the curve toward different image features
 - Edges, lines, etc.

External image energy



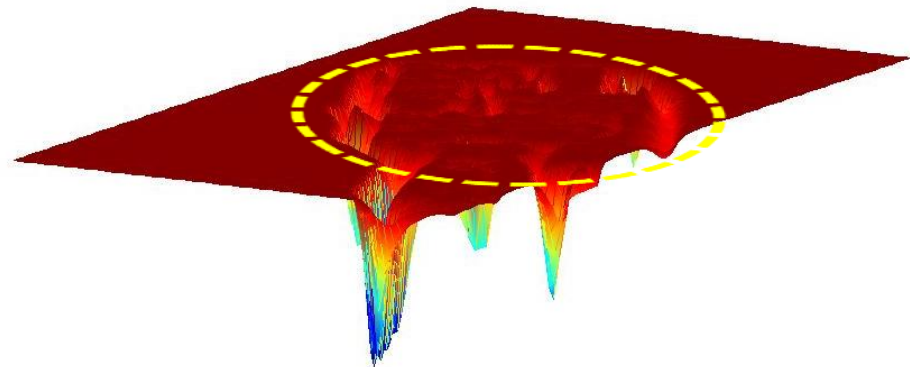
How do edges affect “snap” of rubber band?

Think of external energy from image as gravitational pull towards areas of high contrast



Magnitude of gradient

$$G_x(I)^2 + G_y(I)^2$$



- (Magnitude of gradient)

$$-\left(G_x(I)^2 + G_y(I)^2\right)$$

External image energy

- Image $I(x,y)$
- Gradient images $G_x(x,y)$ and $G_y(x,y)$
- External energy at a point $v(s)$ on the curve is

$$E_{external}(v(s)) = -(|G_x(v(s))|^2 + |G_y(v(s))|^2)$$

- External energy for the whole curve:

$$E_{external} = \int_0^1 E_{external}(v(s)) ds$$

Internal energy: intuition



Internal energy: intuition

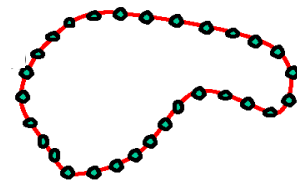


A priori, we want to favor smooth shapes, contours with low curvature, contours similar to a known shape, etc. to balance what is actually observed (i.e., in the gradient image).





Internal energy



For a continuous curve, a common internal energy term is the “bending energy”.

At some point $v(s)$ on the curve, this is:

$$E_{internal}(v(s)) = \alpha \left| \frac{dv}{ds} \right|^2 + \beta \left| \frac{d^2v}{ds^2} \right|^2$$

Elasticity,
Tension Stiffness,
Curvature

The more the curve bends $\rightarrow$ the larger this energy value is.

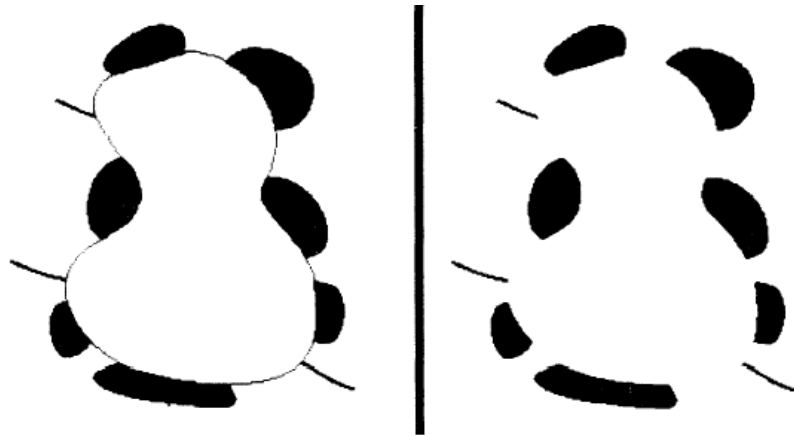
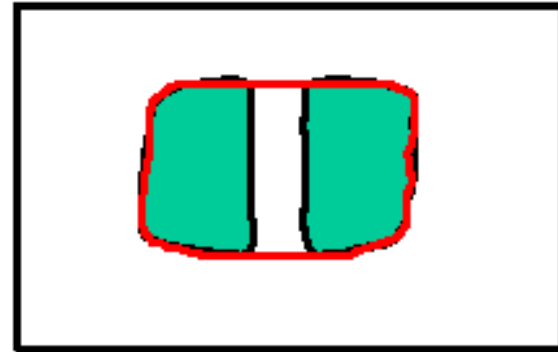
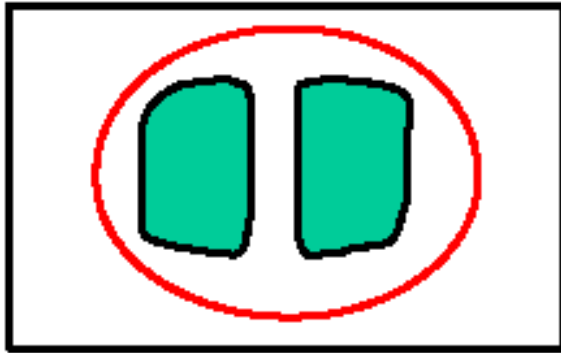
The weights α and β dictate how much influence each component has.

Internal energy
for whole curve:

$$E_{internal} = \int_0^1 E_{internal}(v(s)) ds$$

Dealing with missing data

- The smoothness constraint can deal with missing data:



Total energy (continuous form)

$$E_{total} = E_{internal} + \gamma E_{external}$$

$$E_{internal} = \int_0^1 E_{internal}(\nu(s)) ds \quad // \text{ bending energy}$$

$$E_{external} = \int_0^1 E_{external}(\nu(s)) ds \quad // \text{ total edge strength under curve}$$

Limitations

- External energy: snake does not really “see” object boundaries in the image unless it gets very close to it.

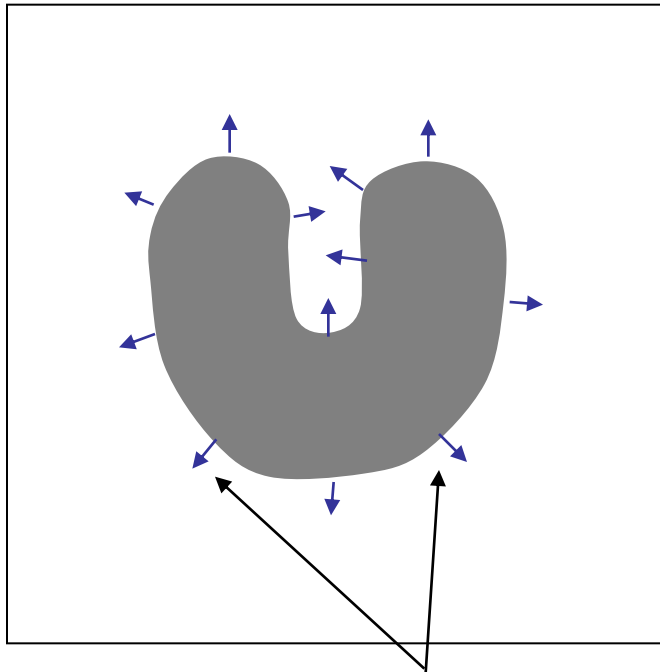
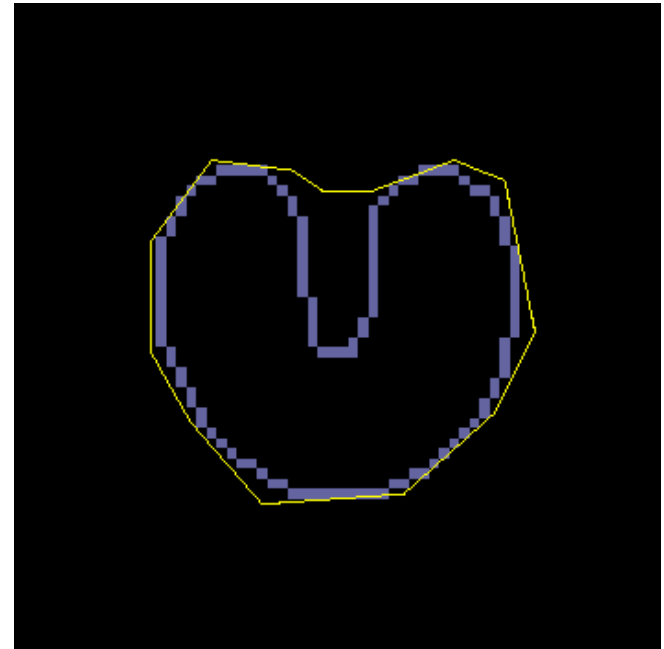
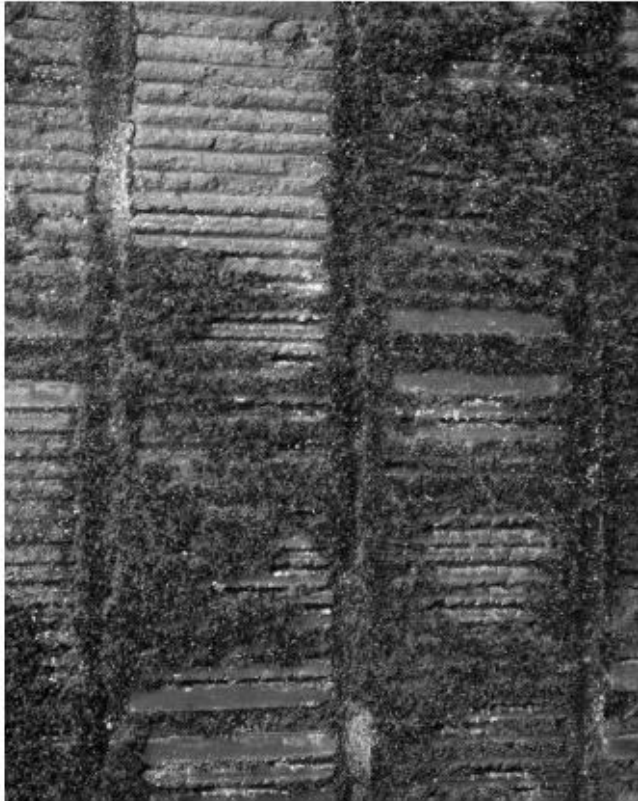


image gradients ∇I
are large only directly on the boundary

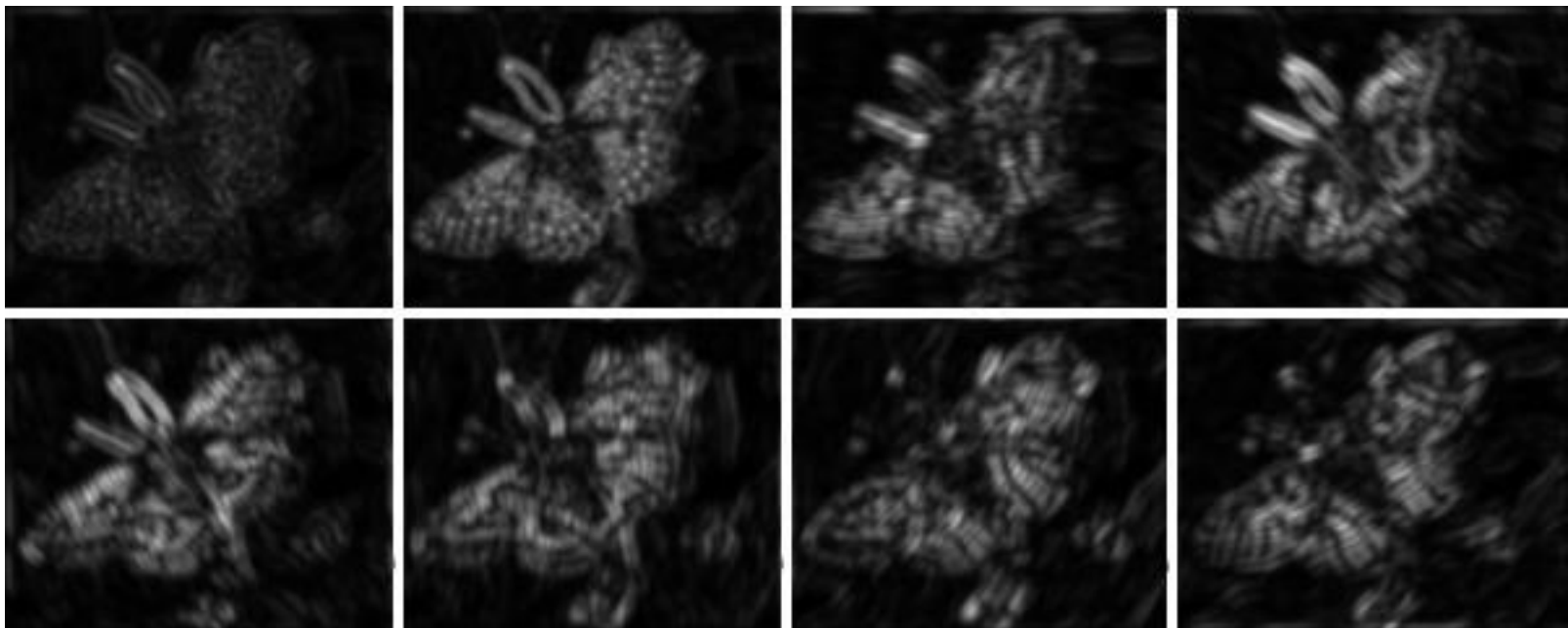
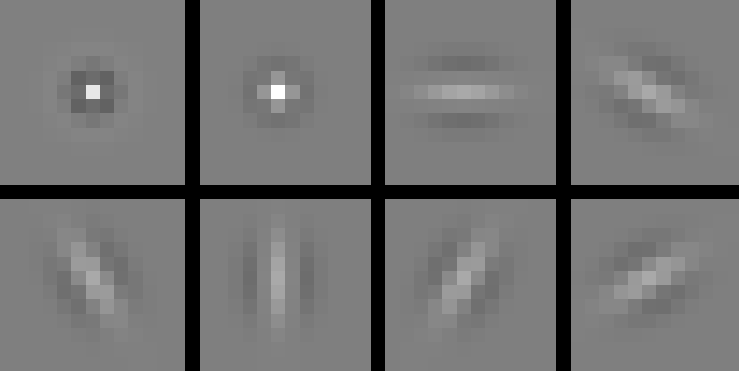


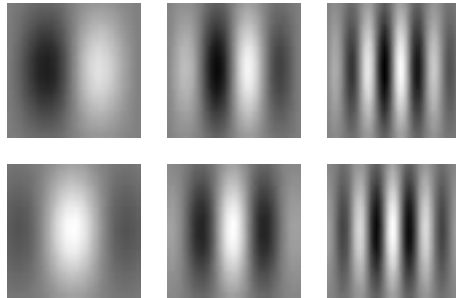
Texture



Representing textures

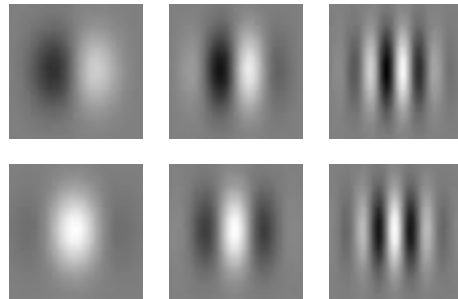
- Textures are made up of quite stylised subelements, repeated in meaningful ways
- Representation:
 - find the subelements, and represent their statistics
- But what are the subelements, and how do we find them?
 - recall normalized correlation
 - find subelements by applying filters, looking at the magnitude of the response
- What filters?
 - experience suggests spots and oriented bars at a variety of different scales
- What statistics?
 - within reason, the more the merrier.
 - At least, mean and standard deviation
 - better, various conditional histograms.





Gabor filters at different
scales and spatial frequencies

top row shows anti-symmetric
(or odd) filters, bottom row the
symmetric (or even) filters.



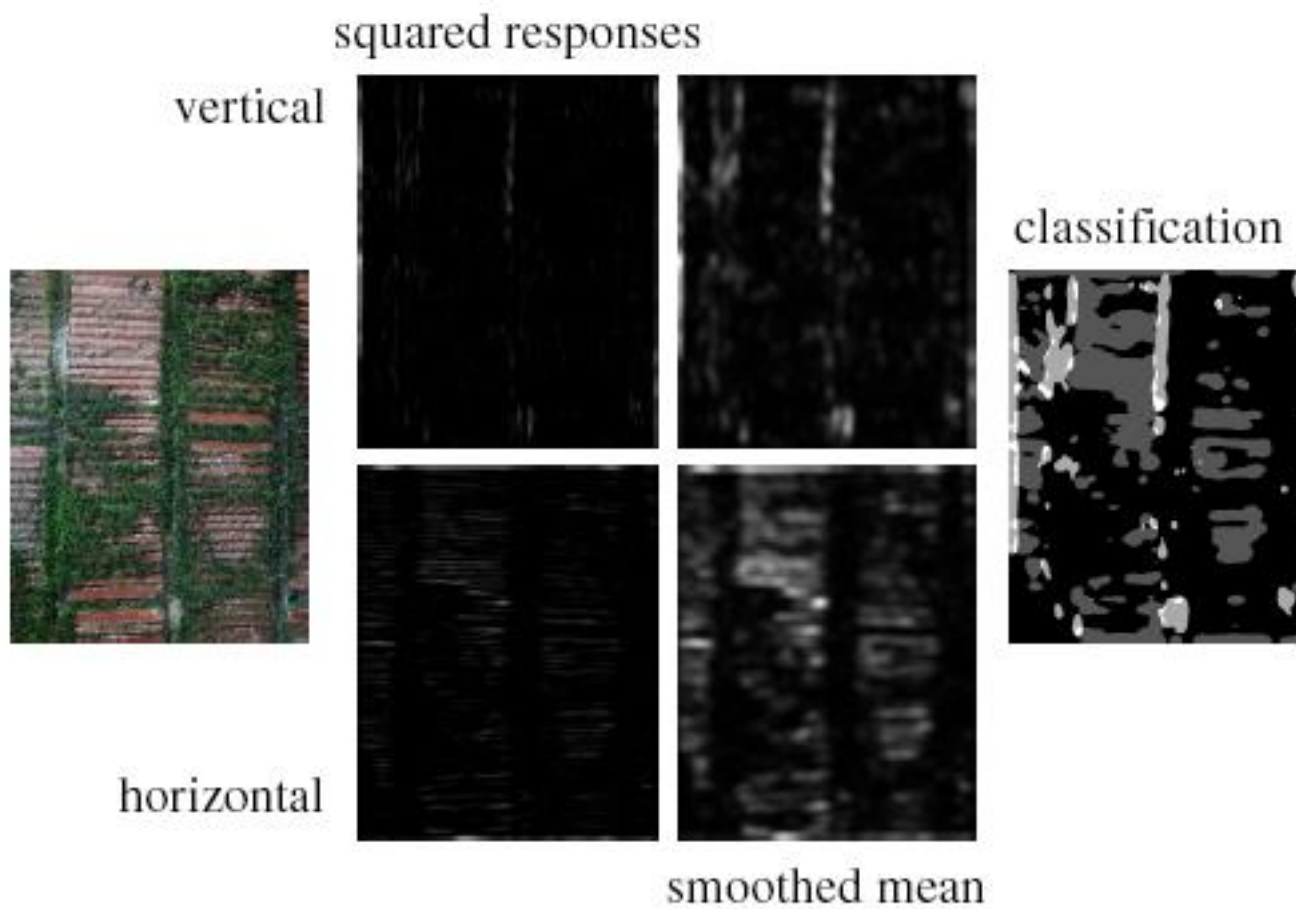
$$G_c[i, j] = B e^{-\frac{(i^2 + j^2)}{2\sigma^2}} \cos(2\pi f(i \cos \theta + j \sin \theta))$$

$$G_s[i, j] = C e^{-\frac{(i^2 + j^2)}{2\sigma^2}} \sin(2\pi f(i \cos \theta + j \sin \theta))$$

B, C are normalizing constants

Final texture representation

- Form an oriented pyramid (or equivalent set of responses to filters at different scales and orientations).
- Square the output
- Take statistics of responses
 - e.g. mean of each filter output (are there lots of spots)
 - std of each filter output
 - mean of one scale conditioned on other scale having a particular range of values (e.g. are the spots in straight rows?)



Motion estimation

Why estimate visual motion?

Visual Motion can be annoying

- Camera instabilities, jitter
- Measure it; remove it (stabilize)

Visual Motion indicates dynamics in the scene

- Moving objects, behavior
- Track objects and analyze trajectories

Visual Motion reveals spatial layout

- Motion parallax

Outline

Motion estimation

- Patch-based Motion Estimation
- Image warping
- Parametric (global) motion

Patch-based motion estimation

Classes of Techniques

Feature-based methods – next topic

- Extract visual features (corners, textured areas) and track them
- Sparse motion fields, but possibly robust tracking
- Suitable especially when image motion is large (10s of pixels)

Direct-methods

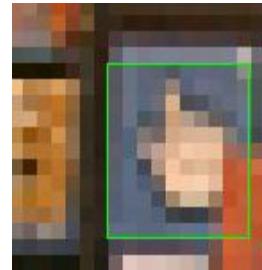
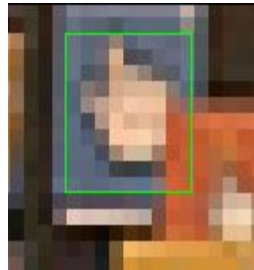
- Directly recover image motion from spatio-temporal image brightness variations
- Global motion parameters directly recovered without an intermediate feature motion calculation
- Dense motion fields, but more sensitive to appearance variations
- Suitable for video and when image motion is small (< 10 pixels)

Patch matching (revisited)

How do we determine correspondences?

- *block matching* or *SSD* (sum squared differences)

$$E(x, y; d) = \sum_{(x', y') \in N(x, y)} [I_L(x' + d, y') - I_R(x', y')]^2$$



The Brightness Constraint

Brightness Constancy Equation:

$$J(x, y) \approx I(x + u(x, y), y + v(x, y))$$

Or, equivalently, minimize :

$$E(u, v) = (J(x, y) - I(x + u, y + v))^2$$

Linearizing (assuming small (u, v))
using Taylor series expansion:

$$J(x, y) \approx I(x, y) + I_x(x, y) \cdot u(x, y) + I_y(x, y) \cdot v(x, y)$$

Gradient Constraint (or the Optical Flow Constraint)

$$E(u, v) = (I_x \cdot u + I_y \cdot v + I_t)^2$$

Minimizing: $\frac{\partial E}{\partial u} = \frac{\partial E}{\partial v} = 0$

$$I_x (I_x u + I_y v + I_t) = 0$$

$$I_y (I_x u + I_y v + I_t) = 0$$

In general $I_x, I_y \neq 0$

Hence, $I_x \cdot u + I_y \cdot v + I_t \approx 0$

Patch Translation [Lucas-Kanade]

Assume a single velocity for all pixels within an image patch

$$E(u, v) = \sum_{x, y \in \Omega} \left(I_x(x, y)u + I_y(x, y)v + I_t \right)^2$$

Minimizing

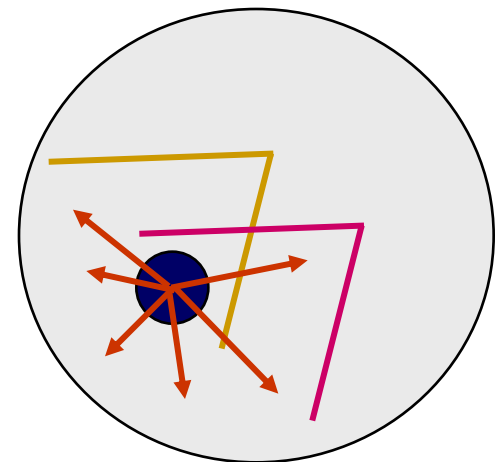
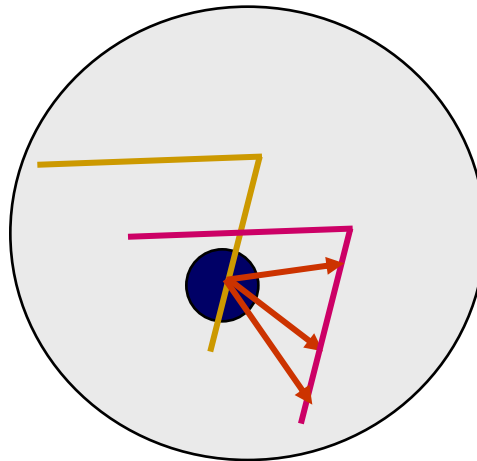
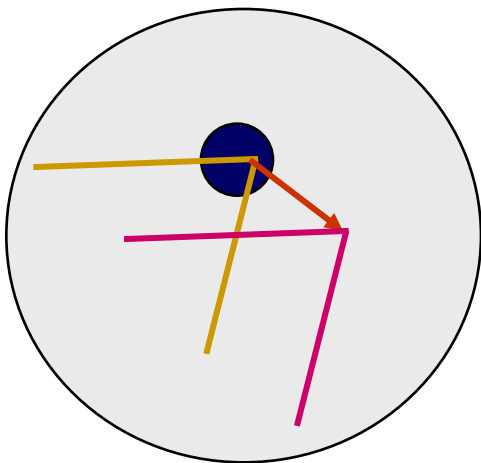
$$\begin{bmatrix} \sum I_x^2 & \sum I_x I_y \\ \sum I_x I_y & \sum I_y^2 \end{bmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = - \begin{pmatrix} \sum I_x I_t \\ \sum I_y I_t \end{pmatrix}$$

$$\left(\sum \nabla I \nabla I^T \right) \vec{U} = - \sum \nabla I I_t$$

LHS: sum of the 2x2 outer product of the gradient vector

Local Patch Analysis

How *certain* are the motion estimates?

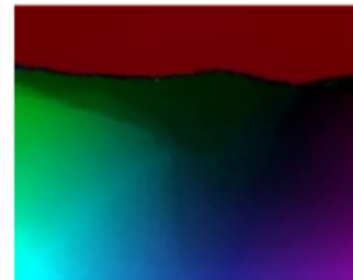
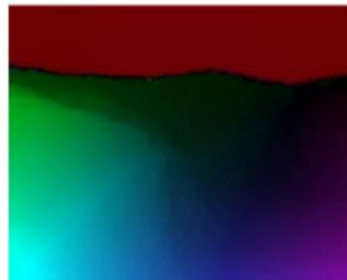
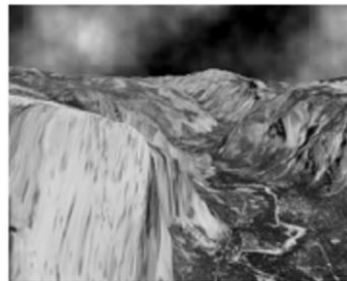
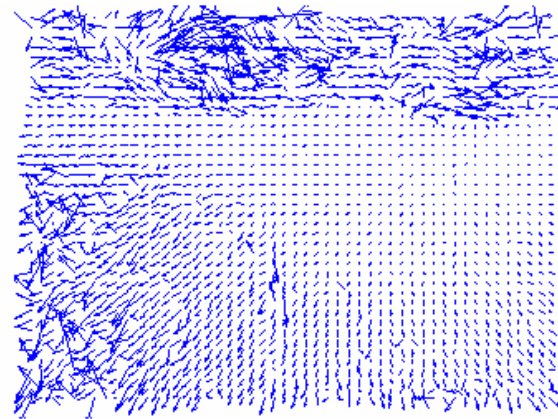
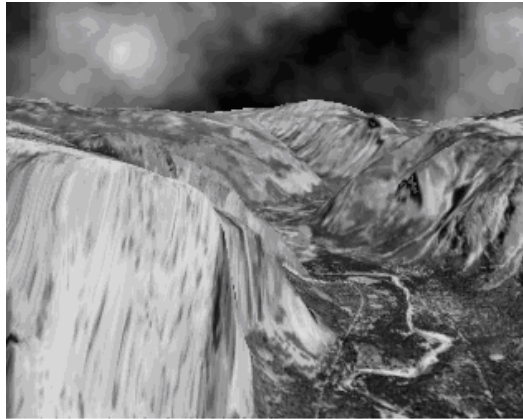


The Aperture Problem

$$\text{Let } M = \sum (\nabla I)(\nabla I)^T \quad \text{and} \quad b = \begin{bmatrix} -\sum I_x I_t \\ -\sum I_y I_t \end{bmatrix}$$

- Algorithm: At each pixel compute U by solving $M U = b$
- M is singular if all gradient vectors point in the same direction
 - e.g., along an edge
 - of course, trivially singular if the summation is over a single pixel or there is no texture
 - i.e., only *normal flow* is available (aperture problem)
- Corners and textured areas are OK

Visualization



Visualization



hue indicates orientation and saturation indicates magnitude

Outline

Motion estimation

- Patch-based Motion Estimation
- Image warping
- Parametric (global) motion

Image Warping

Image Warping

image filtering: change *range* of image

$$g(x) = h(f(x))$$

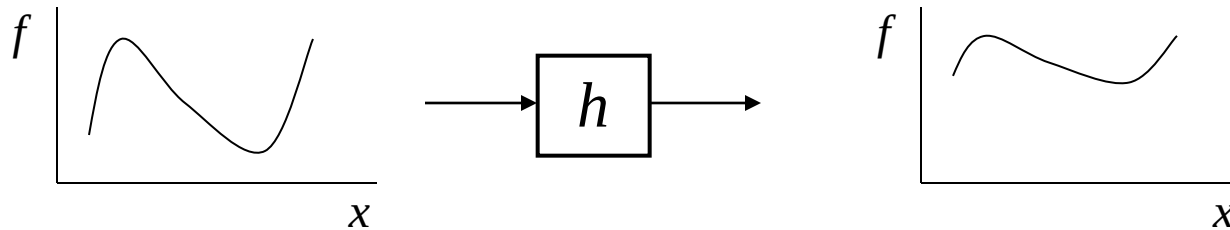


image warping: change *domain* of image

$$g(x) = f(h(x))$$

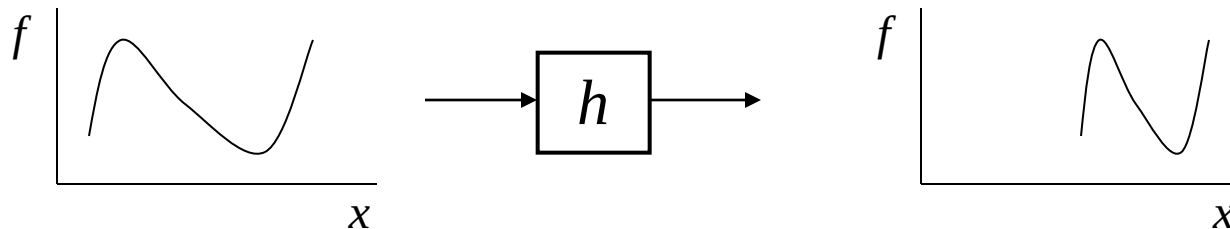


Image Warping

image filtering: change *range* of image

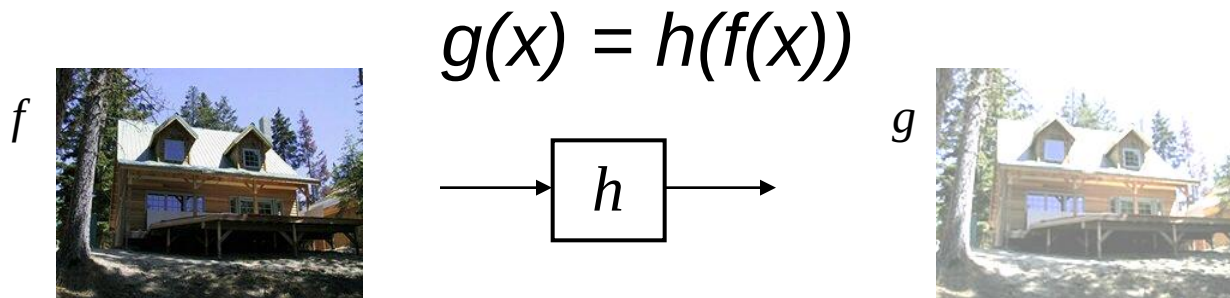
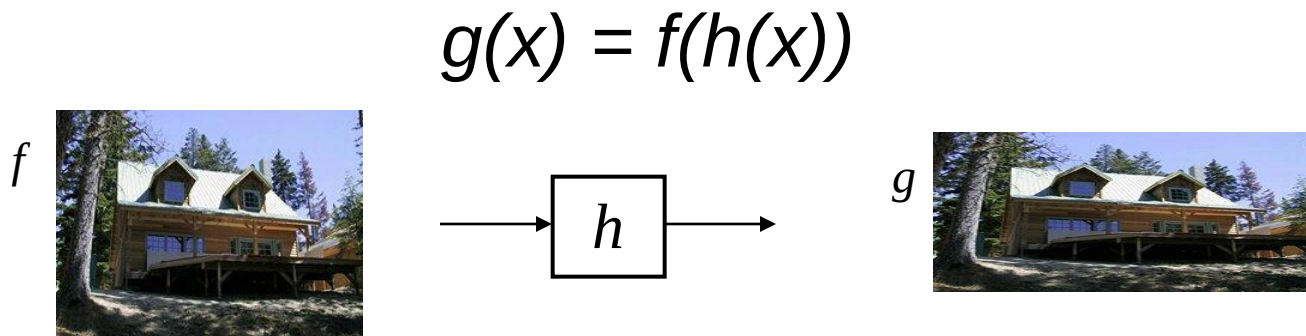


image warping: change *domain* of image



Parametric (global) warping

Examples of parametric warps:



translation



rotation



aspect



affine



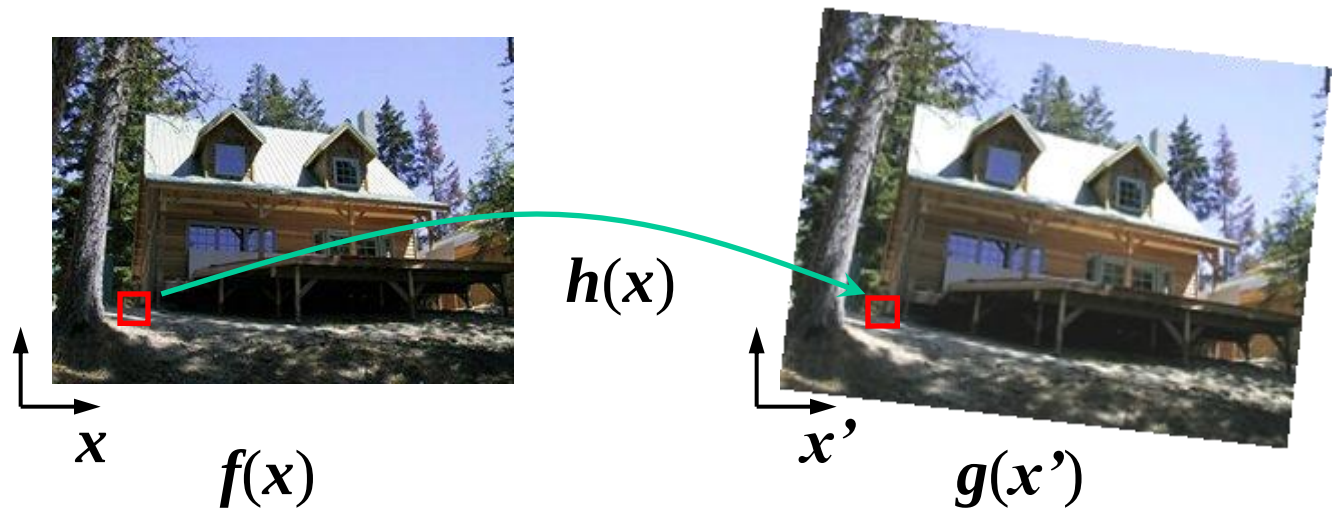
perspective



cylindrical

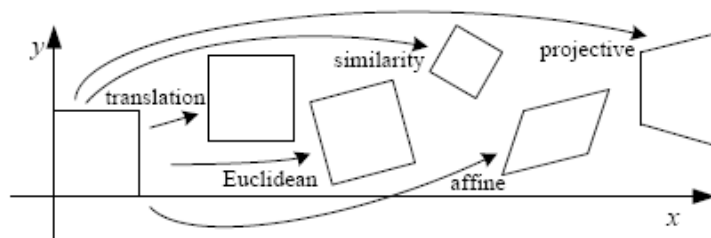
Image Warping

Given a coordinate transform $\mathbf{x}' = \mathbf{h}(\mathbf{x})$ and a source image $\mathbf{f}(\mathbf{x})$, how do we compute a transformed image $\mathbf{g}(\mathbf{x}') = \mathbf{f}(\mathbf{h}(\mathbf{x}))$?



Parametric motion estimation

Motion models



Translation

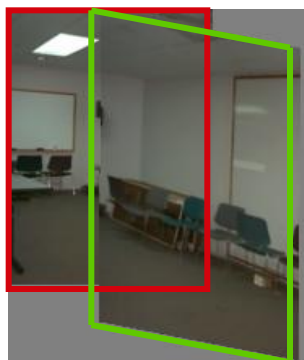
Affine

Perspective

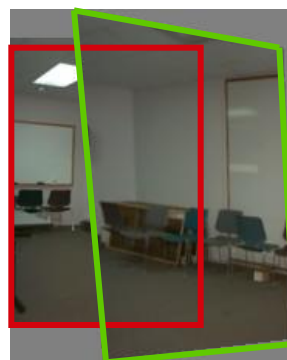
3D rotation



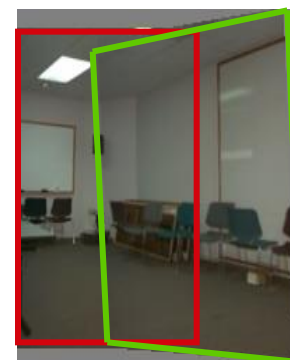
2 unknowns



6 unknowns



8 unknowns



3 unknowns

Motion Models – Mathematical Representation

Translation

$$\begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$

Translation and Rotation (Euclidean)

$$\begin{bmatrix} \cos(\theta) & -\sin(\theta) & t_x \\ \sin(\theta) & \cos(\theta) & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

Scale

$$\begin{bmatrix} a & 0 & 0 \\ 0 & a & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

Similarity Transformation

Translation + Rotation + Scale
(4 DOF)

Motion Models – Mathematical Representation

Aspect Ratio

$$\begin{bmatrix} a & 0 & 0 \\ 0 & \frac{1}{a} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

Shear

$$\begin{bmatrix} 1 & a & 0 \\ b & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

Affine Transformation (translation + rotation + scale
+ aspect ratio + shear)

$$\begin{bmatrix} a & b & c \\ d & e & f \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix}$$

Projective Transformation

Homogeneous Coordinates

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} u \\ v \\ w \end{bmatrix}$$

$$\begin{aligned} x' &= u/w \\ y' &= v/w \end{aligned}$$

Projective Transformation

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} u \\ v \\ w \end{bmatrix}$$

$$\begin{aligned} x' &= u/w \\ y' &= v/w \end{aligned}$$

Remember when we discuss homography later. Also, we will study 3D transformations later.

Example: Affine Motion

$u(x, y) = a_1 + a_2x + a_3y$ Substituting into the B.C. Equation:

$$v(x, y) = a_4 + a_5x + a_6y \quad \boxed{I_x \cdot u + I_y \cdot v + I_t \approx 0}$$

$$\boxed{I_x(a_1 + a_2x + a_3y) + I_y(a_4 + a_5x + a_6y) + I_t \approx 0}$$

Each pixel provides 1 linear constraint in 6 *global* unknowns

Least Square Minimization (over all pixels):

$$\boxed{Err(\vec{a}) = \sum \left[I_x(a_1 + a_2x + a_3y) + I_y(a_4 + a_5x + a_6y) + I_t \right]^2}$$

Other 2D Motion Models

Quadratic – instantaneous approximation to planar motion

$$\begin{aligned}u &= q_1 + q_2x + q_3y + q_7x^2 + q_8xy \\v &= q_4 + q_5x + q_6y + q_7xy + q_8y^2\end{aligned}$$

Projective – exact planar motion

$$x' = \frac{h_1 + h_2x + h_3y}{h_7 + h_8x + h_9y}$$

$$y' = \frac{h_4 + h_5x + h_6y}{h_7 + h_8x + h_9y}$$

and

$$u = x' - x, \quad v = y' - y$$

Bayesian Tracking: The three main steps

- **Prediction:** we have seen $\mathbf{y}_0, \dots, \mathbf{y}_{i-1}$ — what state does this set of measurements predict for the i 'th frame? to solve this problem, we need to obtain a representation of $P(\mathbf{X}_i | \mathbf{Y}_0 = \mathbf{y}_0, \dots, \mathbf{Y}_{i-1} = \mathbf{y}_{i-1})$.
- **Data association:** Some of the measurements obtained from the i -th frame may tell us about the object's state. Typically, we use $P(\mathbf{X}_i | \mathbf{Y}_0 = \mathbf{y}_0, \dots, \mathbf{Y}_{i-1} = \mathbf{y}_{i-1})$ to identify these measurements.
- **Correction:** now that we have $\mathbf{y}_i$ — the relevant measurements — we need to compute a representation of $P(\mathbf{X}_i | \mathbf{Y}_0 = \mathbf{y}_0, \dots, \mathbf{Y}_i = \mathbf{y}_i)$.

Simplifying Assumptions

- **Only the immediate past matters:** formally, we require

$$P(\mathbf{X}_i | \mathbf{X}_1, \dots, \mathbf{X}_{i-1}) = P(\mathbf{X}_i | \mathbf{X}_{i-1})$$

This assumption hugely simplifies the design of algorithms, as we shall see; furthermore, it isn't terribly restrictive if we're clever about interpreting $\mathbf{X}_i$ as we shall show in the next section.

- **Measurements depend only on the current state:** we assume that $\mathbf{Y}_i$ is conditionally independent of all other measurements given $\mathbf{X}_i$. This means that

$$P(\mathbf{Y}_i, \mathbf{Y}_j, \dots, \mathbf{Y}_k | \mathbf{X}_i) = P(\mathbf{Y}_i | \mathbf{X}_i) P(\mathbf{Y}_j, \dots, \mathbf{Y}_k | \mathbf{X}_i)$$

Again, this isn't a particularly restrictive or controversial assumption, but it yields important simplifications.

Base case

Firstly, we assume that we have $P(\mathbf{X}_0)$

$$\begin{aligned} P(\mathbf{X}_0 | \mathbf{Y}_0 = \mathbf{y}_0) &= \frac{P(\mathbf{y}_0 | \mathbf{X}_0) P(\mathbf{X}_0)}{P(\mathbf{y}_0)} \\ &= \frac{P(\mathbf{y}_0 | \mathbf{X}_0) P(\mathbf{X}_0)}{\int P(\mathbf{y}_0 | \mathbf{X}_0) P(\mathbf{X}_0) d\mathbf{X}_0} \\ &\propto P(\mathbf{y}_0 | \mathbf{X}_0) P(\mathbf{X}_0) \end{aligned}$$

Induction step

Given $P(\mathbf{X}_{i-1}|\mathbf{y}_0, \dots, \mathbf{y}_{i-1})$.

Prediction

Prediction involves representing

$$P(\mathbf{X}_i|\mathbf{y}_0, \dots, \mathbf{y}_{i-1})$$

Our independence assumptions make it possible to write

$$\begin{aligned} P(\mathbf{X}_i|\mathbf{y}_0, \dots, \mathbf{y}_{i-1}) &= \int P(\mathbf{X}_i, \mathbf{X}_{i-1}|\mathbf{y}_0, \dots, \mathbf{y}_{i-1})d\mathbf{X}_{i-1} \\ &= \int P(\mathbf{X}_i|\mathbf{X}_{i-1}, \mathbf{y}_0, \dots, \mathbf{y}_{i-1})P(\mathbf{X}_{i-1}|\mathbf{y}_0, \dots, \mathbf{y}_{i-1})d\mathbf{X}_{i-1} \\ &= \int P(\mathbf{X}_i|\mathbf{X}_{i-1})P(\mathbf{X}_{i-1}|\mathbf{y}_0, \dots, \mathbf{y}_{i-1})d\mathbf{X}_{i-1} \end{aligned}$$

Induction step

Correction

Correction involves obtaining a representation of

$$P(\mathbf{X}_i | \mathbf{y}_0, \dots, \mathbf{y}_i)$$

Our independence assumptions make it possible to write

$$\begin{aligned} P(\mathbf{X}_i | \mathbf{y}_0, \dots, \mathbf{y}_i) &= \frac{P(\mathbf{X}_i, \mathbf{y}_0, \dots, \mathbf{y}_i)}{P(\mathbf{y}_0, \dots, \mathbf{y}_i)} \\ &= \frac{P(\mathbf{y}_i | \mathbf{X}_i, \mathbf{y}_0, \dots, \mathbf{y}_{i-1}) P(\mathbf{X}_i | \mathbf{y}_0, \dots, \mathbf{y}_{i-1}) P(\mathbf{y}_0, \dots, \mathbf{y}_{i-1})}{P(\mathbf{y}_0, \dots, \mathbf{y}_i)} \\ &= P(\mathbf{y}_i | \mathbf{X}_i) P(\mathbf{X}_i | \mathbf{y}_0, \dots, \mathbf{y}_{i-1}) \frac{P(\mathbf{y}_0, \dots, \mathbf{y}_{i-1})}{P(\mathbf{y}_0, \dots, \mathbf{y}_i)} \\ &= \frac{P(\mathbf{y}_i | \mathbf{X}_i) P(\mathbf{X}_i | \mathbf{y}_0, \dots, \mathbf{y}_{i-1})}{\int P(\mathbf{y}_i | \mathbf{X}_i) P(\mathbf{X}_i | \mathbf{y}_0, \dots, \mathbf{y}_{i-1}) d\mathbf{X}_i} \end{aligned}$$

Linear dynamic models

- Use notation $\sim$ to mean “has the pdf of”, $N(a, b)$ is a normal distribution with mean a and covariance b .
- Then a linear dynamic model has the form

$$\mathbf{x}_i = N(\mathbf{D}_{i-1}\mathbf{x}_{i-1}; \Sigma_{d_i})$$

$$\mathbf{y}_i = N(\mathbf{M}_i\mathbf{x}_i; \Sigma_{m_i})$$

- This is much, much more general than it looks, and extremely powerful

Examples

- Drifting points
 - we assume that the new position of the point is the old one, plus noise.
 - For the measurement model, we may not need to observe the whole state of the object
 - e.g. a point moving in 3D, at the $3k$ 'th tick we see x , $3k+1$ 'th tick we see y , $3k+2$ 'th tick we see z
 - in this case, we can still make decent estimates of **all three** coordinates at each tick.
 - This property, which does not apply to every model, is called **Observability**

Examples

- Points moving with constant velocity
- Periodic motion
- Etc.
- Points moving with constant acceleration

Points moving with constant velocity

- We have

$$u_i = u_{i-1} + \Delta t v_{i-1} + \varepsilon_i$$

$$v_i = v_{i-1} + \zeta_i$$

- (the Greek letters denote noise terms)
- Stack (u, v) into a single state vector

$$\begin{pmatrix} u \\ v \end{pmatrix}_i = \begin{pmatrix} 1 & \Delta t \\ 0 & 1 \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix}_{i-1} + \text{noise}$$

- which is the form we had above

Points moving with constant acceleration

- We have

$$u_i = u_{i-1} + \Delta t v_{i-1} + \varepsilon_i$$

$$v_i = v_{i-1} + \Delta t a_{i-1} + \varsigma_i$$

$$a_i = a_{i-1} + \xi_i$$

- (the Greek letters denote noise terms)

- Stack (u, v) into a single state vector

$$\begin{pmatrix} u \\ v \\ a \end{pmatrix}_i = \begin{pmatrix} 1 & \Delta t & 0 \\ 0 & 1 & \Delta t \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} u \\ v \\ a \end{pmatrix}_{i-1} + \text{noise}$$

- which is the form we had above

The Kalman Filter

- Key ideas:
 - Linear models interact uniquely well with Gaussian noise - make the prior Gaussian, everything else Gaussian and the calculations are easy
 - Gaussians are really easy to represent --- once you know the mean and covariance, you're done

The Kalman Filter in 1D

- Dynamic Model

$$x_i \sim N(d_i x_{i-1}, \sigma_{d_i}^2)$$

- Notation

$$y_i \sim N(m_i x_i, \sigma_{m_i}^2)$$

mean of $P(X_i | y_0, \dots, y_{i-1})$ as $\overline{X}_i^-$ ————— Predicted mean

Corrected mean — mean of $P(X_i | y_0, \dots, y_i)$ as $\overline{X}_i^+$

the standard deviation of $P(X_i | y_0, \dots, y_{i-1})$ as σ_i^-

of $P(X_i | y_0, \dots, y_i)$ as σ_i^+

Dynamic Model:

$$x_i \sim N(d_i x_{i-1}, \sigma_{d_i})$$

$$y_i \sim N(m_i x_i, \sigma_{m_i})$$

Start Assumptions: $\bar{x}_0^-$ and σ_0^- are known

Update Equations: Prediction

$$\bar{x}_i^- = d_i \bar{x}_{i-1}^+$$

$$\sigma_i^- = \sqrt{\sigma_{d_i}^2 + (d_i \sigma_{i-1}^+)^2}$$

Update Equations: Correction

$$x_i^+ = \left(\frac{\bar{x}_i^- \sigma_{m_i}^2 + m_i y_i (\sigma_i^-)^2}{\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2} \right)$$

$$\sigma_i^+ = \sqrt{\left(\frac{\sigma_{m_i}^2 (\sigma_i^-)^2}{(\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2)} \right)}$$

Correction for 1D Kalman filter

- Let us look at the equations carefully:

- Notice:

- if measurement noise is small,
we rely mainly on the measurement,
if it's large, mainly on the
prediction

$$x_i^+ = \left(\frac{\bar{x}_i \sigma_{m_i}^2 + m_i y_i (\sigma_i^-)^2}{\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2} \right)$$

$$\sigma_i^+ = \sqrt{\left(\frac{\sigma_{m_i}^2 (\sigma_i^-)^2}{(\sigma_{m_i}^2 + m_i^2 (\sigma_i^-)^2)} \right)}$$

Dynamic Model:

$$\mathbf{x}_i \sim N(\mathcal{D}_i \mathbf{x}_{i-1}, \Sigma_{d_i})$$

$$\mathbf{y}_i \sim N(\mathcal{M}_i \mathbf{x}_i, \Sigma_{m_i})$$

Start Assumptions: $\overline{\mathbf{x}}_0^-$ and Σ_0^- are known

Update Equations: Prediction

$$\overline{\mathbf{x}}_i^- = \mathcal{D}_i \overline{\mathbf{x}}_{i-1}^+$$

$$\Sigma_i^- = \Sigma_{d_i} + \mathcal{D}_i \Sigma_{i-1}^+ \mathcal{D}_i$$

Update Equations: Correction

$$\mathcal{K}_i = \Sigma_i^- \mathcal{M}_i^T [\mathcal{M}_i \Sigma_i^- \mathcal{M}_i^T + \Sigma_{m_i}]^{-1}$$

$$\overline{\mathbf{x}}_i^+ = \overline{\mathbf{x}}_i^- + \mathcal{K}_i [\mathbf{y}_i - \mathcal{M}_i \overline{\mathbf{x}}_i^-]$$

$$\Sigma_i^+ = [Id - \mathcal{K}_i \mathcal{M}_i] \Sigma_i^-$$

In higher dimensions,
derivation follows the
same lines, but isn't as
easy. Expressions here.

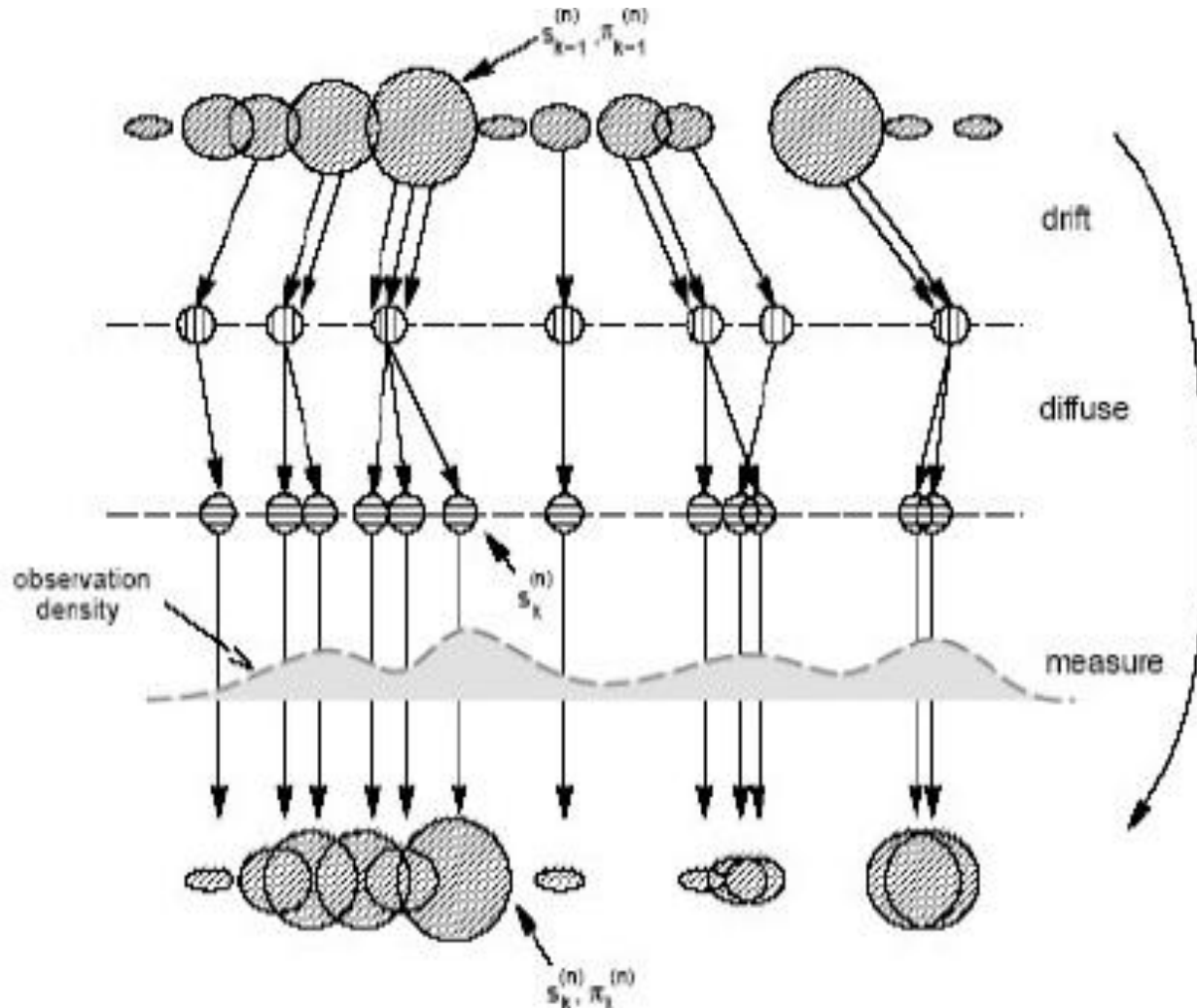
Data Association

- Nearest neighbours
 - choose the measurement with highest probability given predicted state
 - popular, but can lead to catastrophe
- Probabilistic Data Association
 - combine measurements, weighting by probability given predicted state
 - gate using predicted state

Particle Filters

- Have N samples from time $t - 1$,
 $S_{t-1} = \{\mathbf{s}_{t-1}^{(i)}, \pi_{t-1}^{(i)}, i = 1, \dots, N\}$
- Want to construct S_t
- For $n = 1 : N$,
 - Select a sample $\mathbf{s}_t'^{(n)}$ from S_{t-1} according to probability π_{t-1}
 - Predict $\mathbf{s}_t^{(n)}$ by sampling from $p(\mathbf{x}_t | \mathbf{x}_{t-1} = \mathbf{s}_t'^{(n)})$
 - Measure and weight the new position according to image features $\mathbf{z}_t^y$,
 $\pi_t^{(n)} = p(\mathbf{z}_t^y | \mathbf{x}_t = \mathbf{s}_t^{(n)})$

Particle Filters



ML for Computer vision: models, learning and inference

Adapted from Simon Prince's book
<http://www.computervisionmodels.com/>

ML for Computer Vision

- Computer vision models
 - Two types of model
 - Regression
 - Classification
- Which type should we choose?
- Applications

Computer vision models

- Observe **measured data**, $\mathbf{x}$
- Draw inferences from it about **state of world**, $\mathbf{w}$

Examples:

- Observe adjacent frames in video sequence
- Infer camera motion
- Observe image of face
- Infer identity
- Observe images from two displaced cameras
- Infer 3d structure of scene

Regression vs. Classification

- Observe measured data, $\mathbf{x}$
- Draw inferences from it about world, $\mathbf{w}$

When the world state $\mathbf{w}$ is **continuous** we'll call this **regression**

When the world state $\mathbf{w}$ is **discrete** we call this **classification**

Ambiguity of visual world

- Unfortunately visual measurements may be compatible with more than one world state $\mathbf{w}$
 - Measurement process is noisy
 - Inherent ambiguity in visual data
- Conclusion: the best we can do is compute a probability distribution $\Pr(\mathbf{w} | \mathbf{x})$ over possible states of world

Refined goal of computer vision

- Take observations $\mathbf{x}$
- Return probability distribution $\Pr(\mathbf{w} | \mathbf{x})$ over possible worlds compatible with data

(not always tractable – might have to settle for an approximation to this distribution, samples from it, or the best (MAP) solution for $\mathbf{w}$)

Components of solution

We need

- A **model** that mathematically relates the visual data $\mathbf{x}$ to the world state $\mathbf{w}$. Model specifies family of relationships, particular relationship depends on parameters θ
- A **learning algorithm**: fits parameters θ from paired training examples $\mathbf{x}_i, \mathbf{w}_i$
- An **inference algorithm**: uses model to return $\Pr(\mathbf{w} | \mathbf{x})$ given new observed data $\mathbf{x}$.

Types of Model

The **model** mathematically relates the visual data $\mathbf{x}$ to the world state $\mathbf{w}$. Two main categories of model

1. Model contingency of the world on the data $\Pr(\mathbf{w}|\mathbf{x})$
2. Model contingency of data on world $\Pr(\mathbf{x}|\mathbf{w})$

Generative vs. Discriminative

1. Model contingency of the world on the data

$$\Pr(\mathbf{w} | \mathbf{x})$$

(DISCRIMINATIVE MODEL)

2. Model contingency of data on world $\Pr(\mathbf{x} | \mathbf{w})$

(GENERATIVE MODELS)

Generative as probability model over data and so when we draw samples from model, we GENERATE new data

Type 1: Model $\Pr(\mathbf{w} | \mathbf{x})$ - Discriminative

How to model $\Pr(\mathbf{w} | \mathbf{x})$?

1. Choose an appropriate form for $\Pr(\mathbf{w})$
2. Make parameters a function of $\mathbf{x}$
3. Function takes parameters θ that define its shape

Learning algorithm: learn parameters θ from training data $\mathbf{x}, \mathbf{w}$

Inference algorithm: just evaluate $\Pr(\mathbf{w} | \mathbf{x})$

Type 2: $\Pr(\mathbf{x}|\mathbf{w})$ - Generative

How to model $\Pr(\mathbf{x}|\mathbf{w})$?

1. Choose an appropriate form for $\Pr(\mathbf{x})$
2. Make parameters a function of $\mathbf{w}$
3. Function takes parameters θ that define its shape

Learning algorithm: learn parameters θ from training data $\mathbf{x}, \mathbf{w}$

Inference algorithm: Define prior $\Pr(\mathbf{w})$ and then compute $\Pr(\mathbf{w}|\mathbf{x})$ using Bayes' rule

$$\Pr(\mathbf{w}|\mathbf{x}) = \frac{\Pr(\mathbf{x}|\mathbf{w})\Pr(\mathbf{w})}{\int \Pr(\mathbf{x}|\mathbf{w})\Pr(\mathbf{w})d\mathbf{w}}$$

Summary

Two different types of model depend on the quantity of interest:

1. $\Pr(\mathbf{w}|\mathbf{x})$ Discriminative
2. $\Pr(\mathbf{x}|\mathbf{w})$ Generative

Inference in discriminative models easy as we directly model posterior $\Pr(\mathbf{w}|\mathbf{x})$. Generative models require more complex inference process using Bayes' rule

ML for Computer Vision

- Computer vision models
 - Two types of model
 - Regression
 - Classification
- Which type should we choose?
- Applications

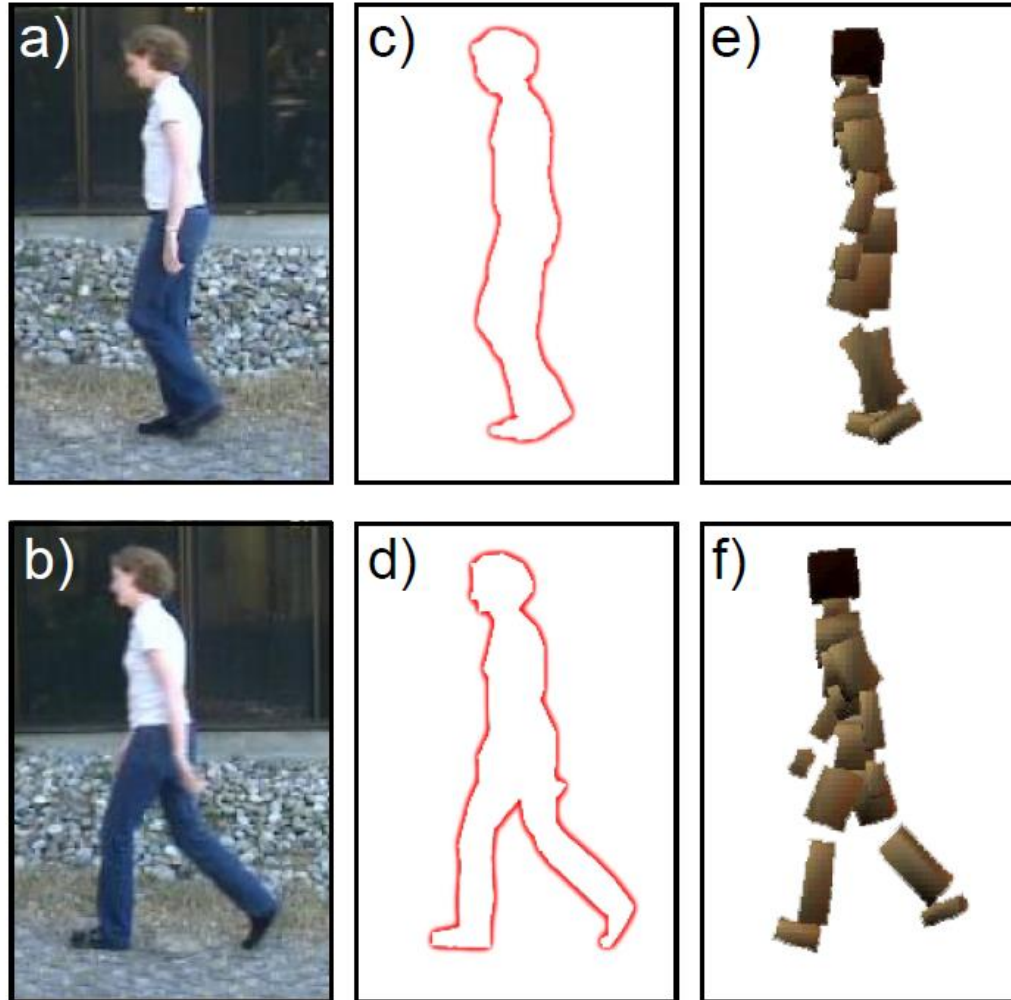
Regression

Consider simple case where

- we make a univariate continuous measurement $\mathbf{x}$
- use this to predict a univariate continuous state $\mathbf{w}$

(regression as world state is continuous)

Regression application 1: Pose from Silhouette



Regression application 2: Head pose estimation



-76°



-11°



2°



8°



43°



79°

ML for Computer Vision

- Computer vision models
 - Regression
 - Classification
- Which type should we choose?
- Applications

Classification

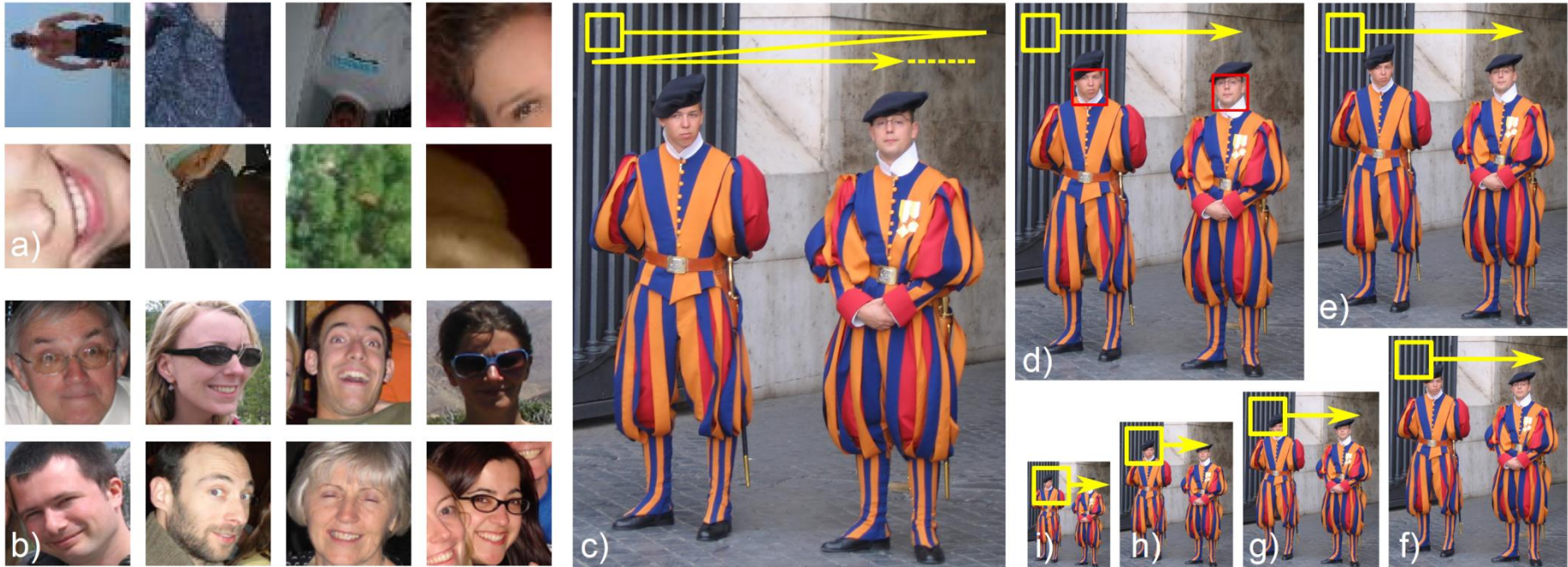
Consider simple case where

- we make a univariate continuous measurement x
- use this to predict a discrete binary world

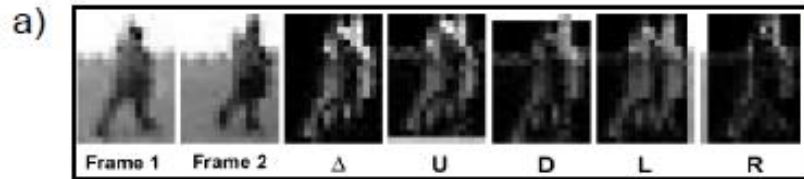
$$w \in \{0, 1\}$$

(classification as world state is discrete)

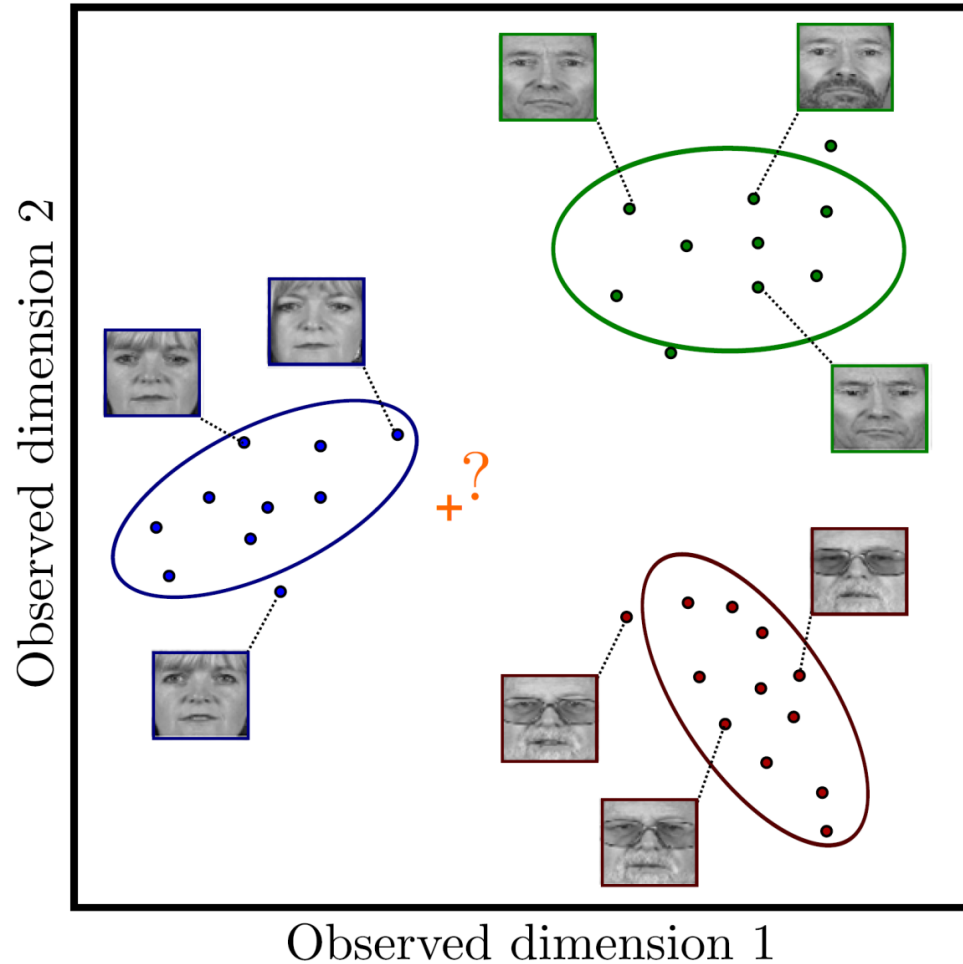
Classification Example 1: Face Detection



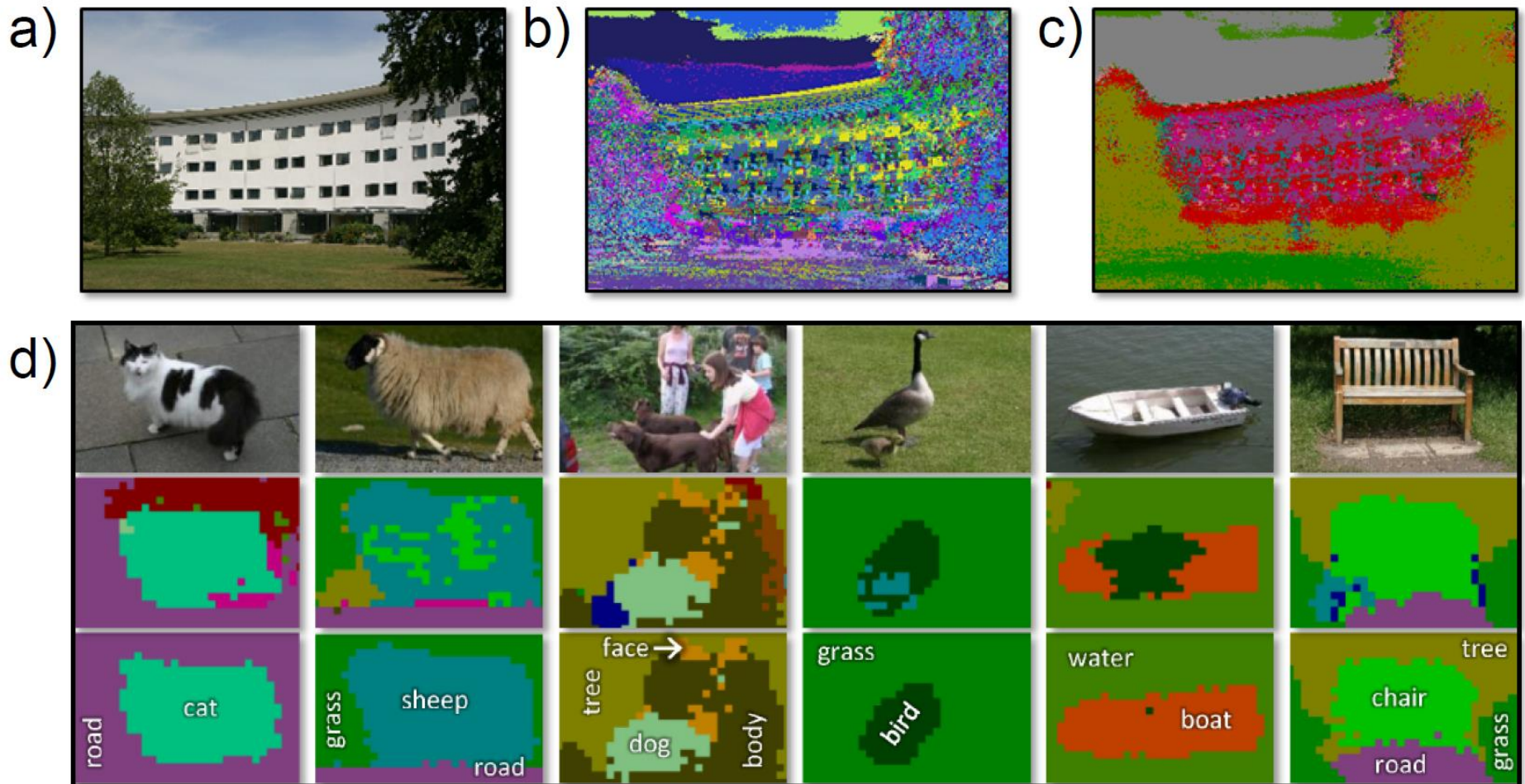
Classification Example 2: Pedestrian Detection



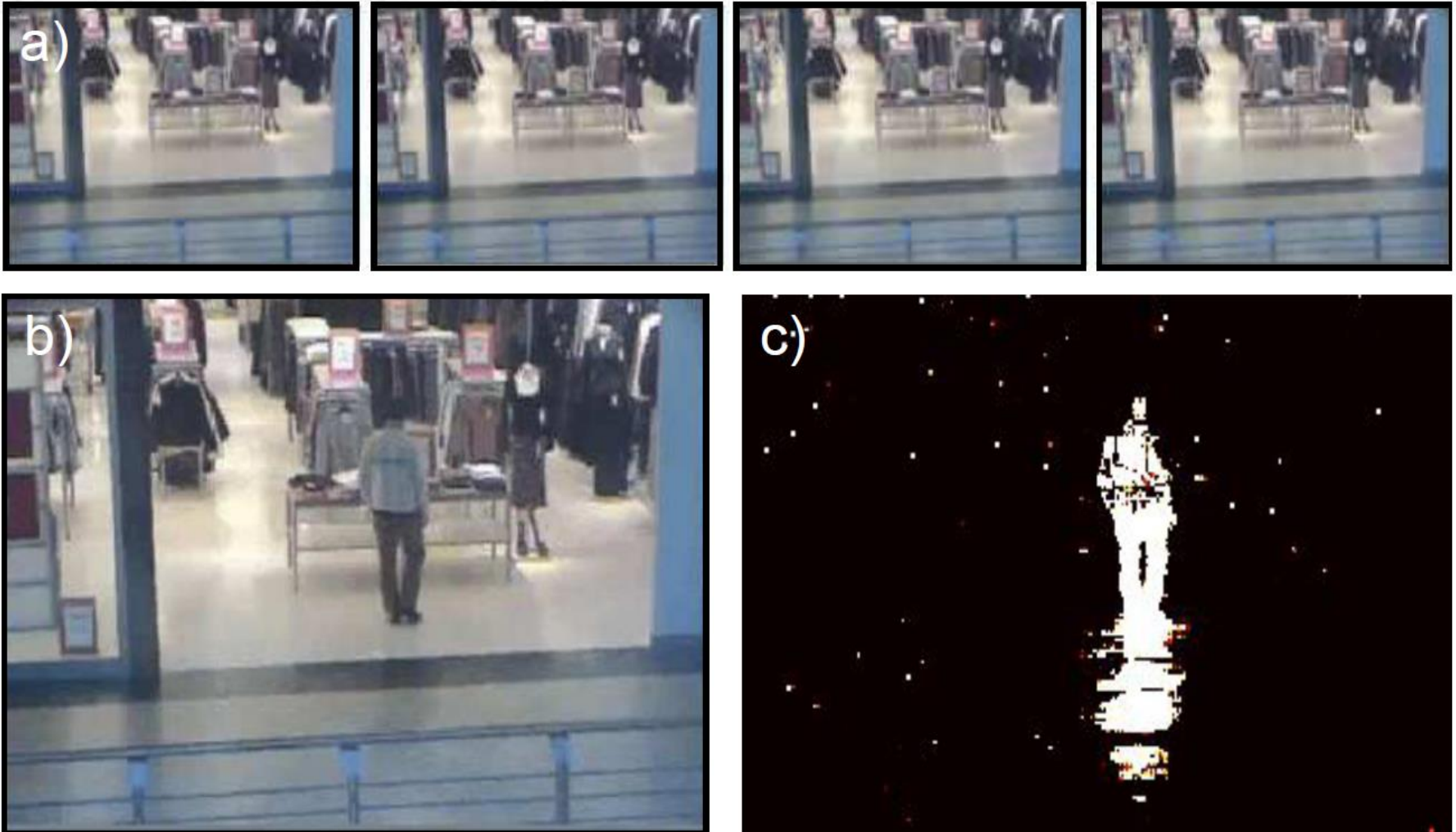
Classification Example 3: Face Recognition



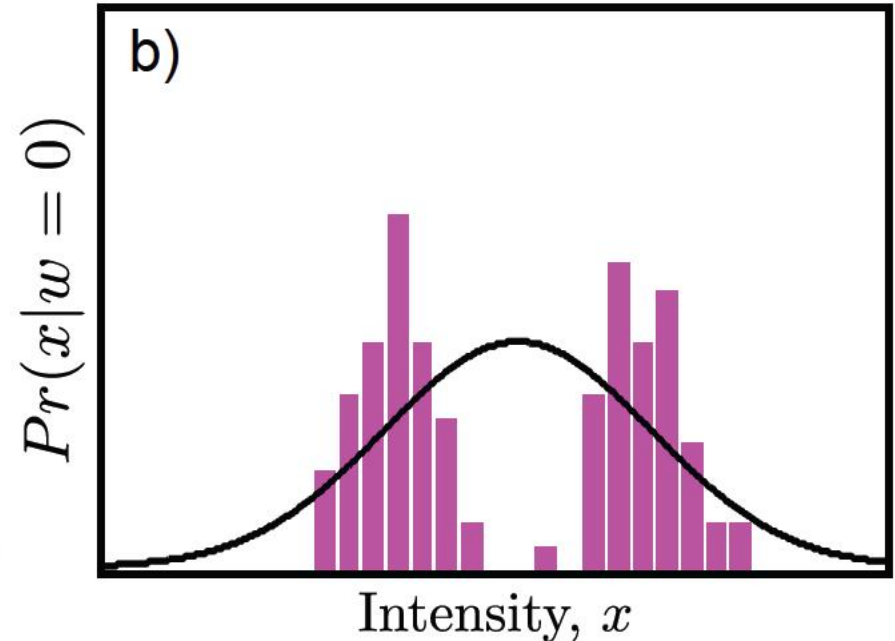
Classification Example 4: Semantic Segmentation



Application: Background subtraction



Application: Background subtraction



But consider this scene in which the foliage is blowing in the wind. A normal distribution is not good enough! Need a way to make more complex distributions

ML for Computer Vision

- Computer vision models
 - Regression
 - Classification
- Which type should we choose?
- Applications

Which type of model to use?

1. Generative methods model data – costly and many aspects of data may have no influence on world state
2. Inference simple in discriminative models
3. Data really is generated from world – generative matches this
4. If missing data, then generative preferred
5. Generative allows imposition of prior knowledge specified by user

Structure

- Computer vision models
 - Regression
 - Classification
- Which type should we choose?
- Applications

Summary

- Seen two types of model

	Model $Pr(w x)$	Model $Pr(x w)$
Regression $x \in [-\infty, \infty], w \in [-\infty, \infty]$	Linear regression	Linear regression
Classification $x \in [-\infty, \infty], w \in \{0, 1\}$	Logistic regression	Probability density function

- Probability density function
- Linear regression
- Logistic regression

Recognition in Computer Vision

Recognition Problems

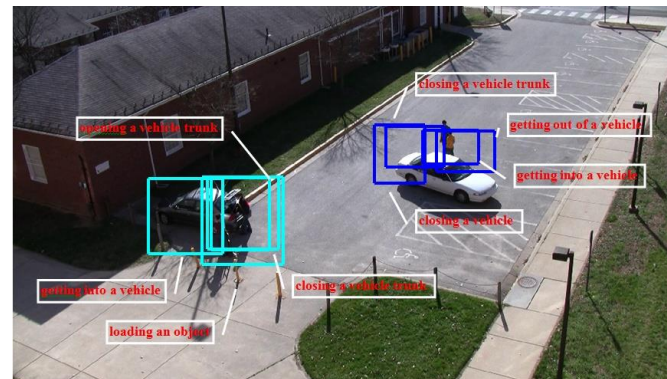
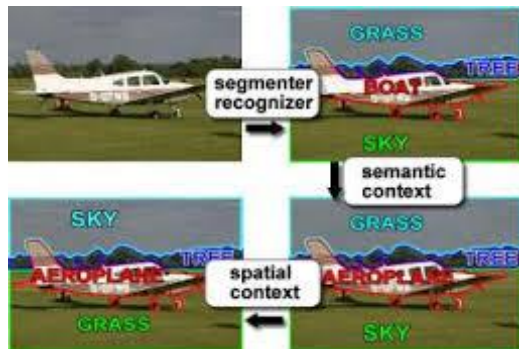
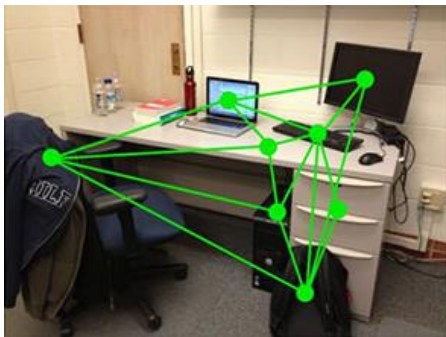
- What is it?
 - Object and scene recognition
- Who is it?
 - Identity recognition
- Where is it?
 - Object detection
- What are they doing?
 - Activities
- All of these are **classification** problems
 - Choose one class from a list of possible candidates

Types of Learning

- Supervised Learning – learning from labeled data
- Unsupervised Learning – clustering, anomaly detection, dimensionality reduction
- Self-supervised Learning – supervisory signal from the structure of the data
- Reinforcement Learning – actions to maximize reward; state, action, reward
 - Deep RL: Policy($a|s$) represented as a deep network
 - Imitation learning: expert provided demonstrations instead of reward function
- Domain Adaptation – domain shift or distribution shift between training and test data

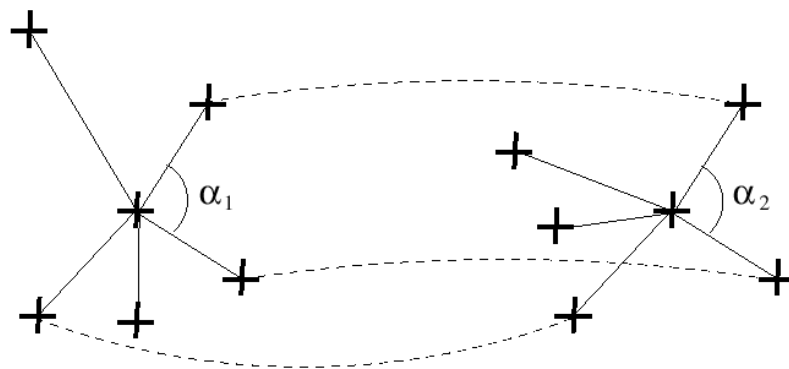
Context in Visual Recognition

- In nature, objects and events tend to coexist with each other in a particular configuration, which is often termed as **context**.
- Context can provide significant visual clue for recognition.
- Recognition of one object/event can help recognizing others.



Matching Using Relationships

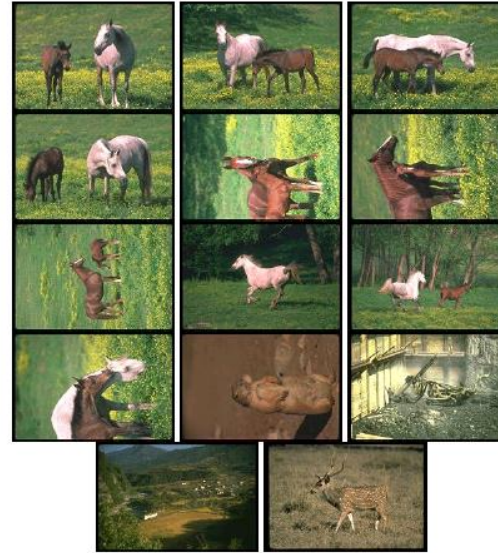
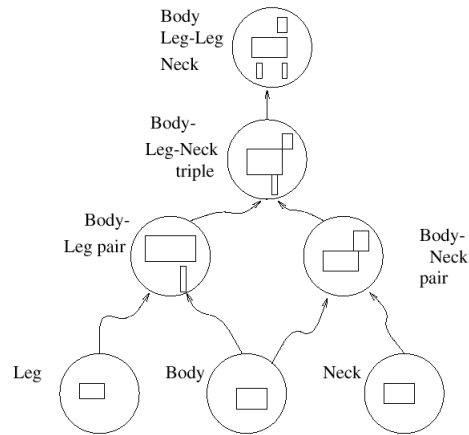
Employ spatial relations



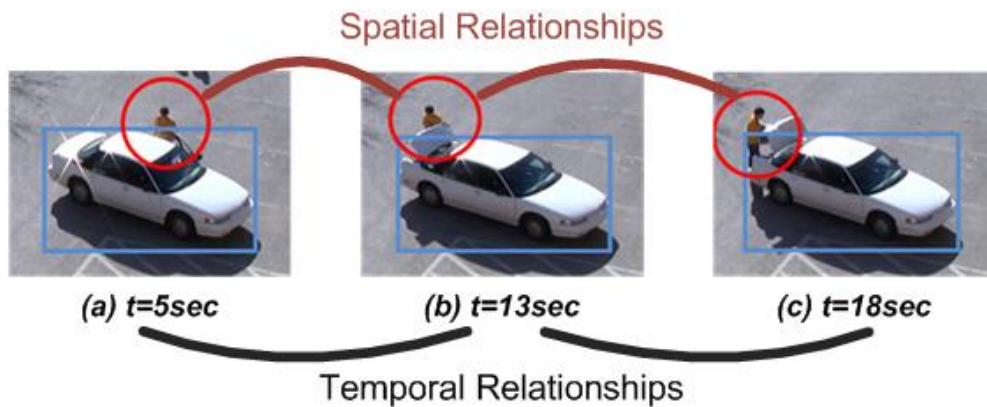
a database entry and
its p closest features

a match

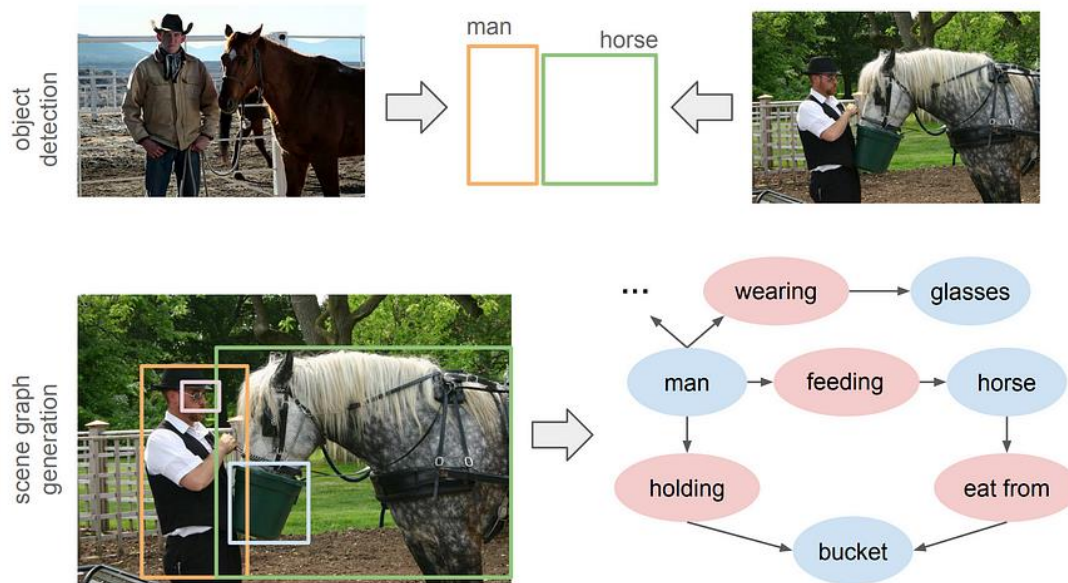
Recognition by parts



Spation-Temporal Relationships



Scene Graphs

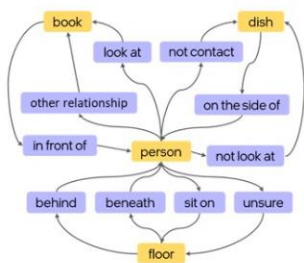


Dynamic Scene Graph

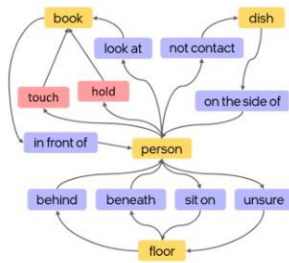
FRAME 1



GROUND
TRUTH



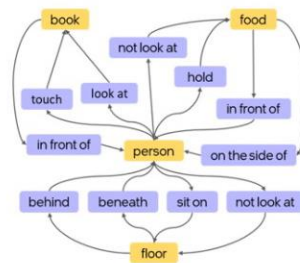
ESTIMATED
SCENE GRAPH



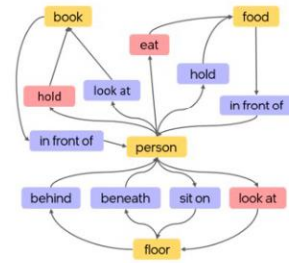
FRAME 2



GROUND
TRUTH



ESTIMATED
SCENE GRAPH



 : Object
 : Relationship
 : Relationship (False Positive)

Classification- Accuracy

		Predicted condition	
Total population = P + N		Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP), hit	False negative (FN), type II error, miss, underestimation
	Negative (N)	False positive (FP), type I error, false alarm, overestimation	True negative (TN), correct rejection

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{FN} + \text{FP} + \text{TN})$$

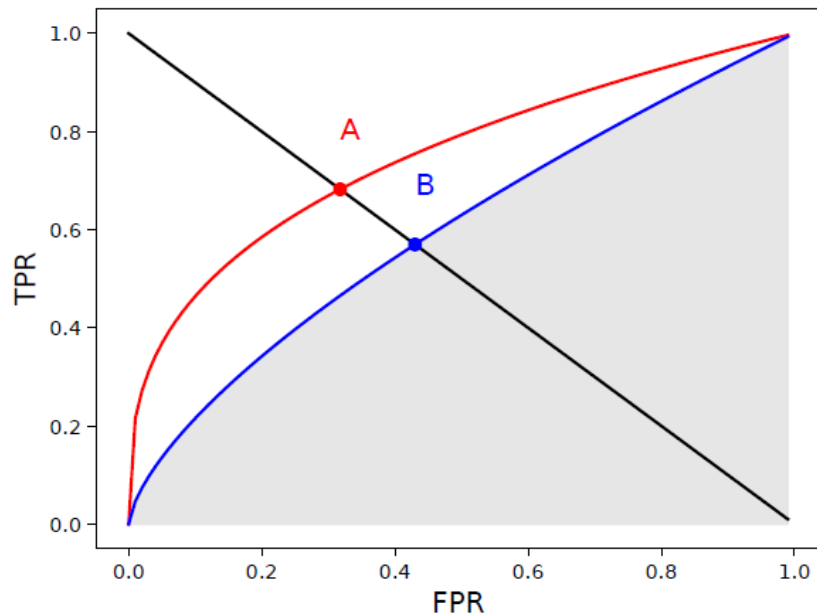
Recall / True Positive Rate (TPR) = predict how many detected correctly in cohort

$$\text{TPR} = \text{TP} / (\text{TP} + \text{FN})$$

False Positive Rate (FPR) = predict how many detected wrongly in cohort

$$\text{FPR} = \text{FP} / (\text{FP} + \text{TN})$$

Classification- ROC curve



EER: $FPR = FNR = 1 - TPR$

Adapted from <https://probml.github.io/pml-book/teaching1.html> by Prof. Saw Shier Nee

Area under ROC curve is computed.

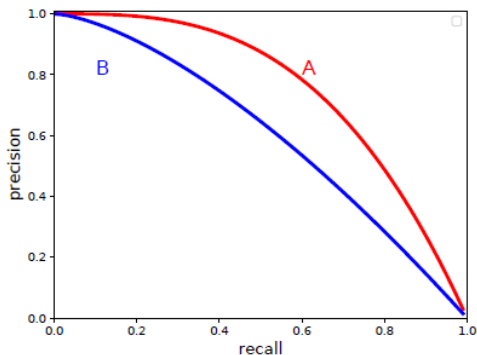
The higher the AUROC, the better the classifier.

		Predicted condition	
Total population = P + N		Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP), hit	False negative (FN), type II error, miss, underestimation
	Negative (N)	False positive (FP), type I error, false alarm, overestimation	True negative (TN), correct rejection

$TPR = TP / (TP + FN) \rightarrow$ predict how many disease correctly in disease cohort

$FPR = FP / (FP + TN) \rightarrow$ predict how many disease in healthy cohort

Classification- Precision-Recall Curve

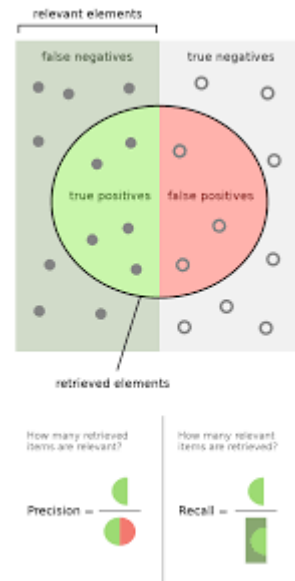


Recall = TPR = $TP / (TP + FN)$ → how many predicted items are correct among all relevant items
Precision = Positive Predictive Value (PPV) = $TP / (TP + FP)$ → how many predicted items are correct among all retrieved items.

Area under PR curve is computed.

The higher the AUC of PR curve, the better the classifier.

		Predicted condition	
		Positive (PP)	Negative (PN)
Actual condition	Positive (P)	True positive (TP), hit	False negative (FN), type II error, miss, underestimation
	Negative (N)	False positive (FP), type I error, false alarm, overestimation	True negative (TN), correct rejection



F1 score: Harmonic mean of Precision and Recall (encourages similar values of precision and recall)

Cumulative Match Characteristic

A Cumulative Match Characteristic (CMC) curve plots the probability of identification against the returned 1:N candidate list size.

It shows the probability that a given user appears in different sized candidate lists. The faster the CMC curve approaches 1, indicating that the user always appears in the candidate list of specified size, the better the matching algorithm.

What is light?

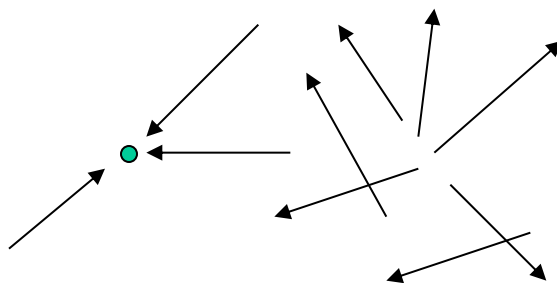
Electromagnetic radiation (EMR) moving along rays in space

- $R(\lambda)$ is EMR, measured in units of power (watts)
 - λ is wavelength



Light field

- We can describe all of the light in the scene by specifying the radiation (or “**radiance**” along all light rays) arriving at every point in space and from every direction



$$R(X, Y, Z, \theta, \phi, \lambda, t)$$

The light field

$$R(X, Y, Z, \theta, \phi, \lambda, t)$$

- Known as the **plenoptic function**
- If you know R , you can predict how the scene would appear from any viewpoint.

Additional Reading

The plenoptic function

- **Original reference:** E. Adelson and J. Bergen, "[The Plenoptic Function and the Elements of Early Vision](#)," in M. Landy and J. A. Movshon, (eds) Computational Models of Visual Processing, MIT Press 1991.
- L. McMillan and G. Bishop, "[Plenoptic Modeling: An Image-Based Rendering System](#)", Proc. SIGGRAPH, 1995, pp. 39-46.

The light field

- M. Levoy and P. Hanrahan, "[Light Field Rendering](#)", Proc SIGGRAPH 96, pp. 31-42.
- S. J. Gortler, R. Grzeszczuk, R. Szeliski, and M. F. Cohen, "[The lumigraph](#)," in Proc. SIGGRAPH, 1996, pp. 43-54.

The interaction of light and matter

What happens when a light ray hits a point on an object?

- Some of the light gets absorbed
 - converted to other forms of energy (e.g., heat)
- Some gets transmitted through the object
 - possibly bent, through “refraction”
- Some gets reflected
 - as we saw before, it could be reflected in multiple directions at once

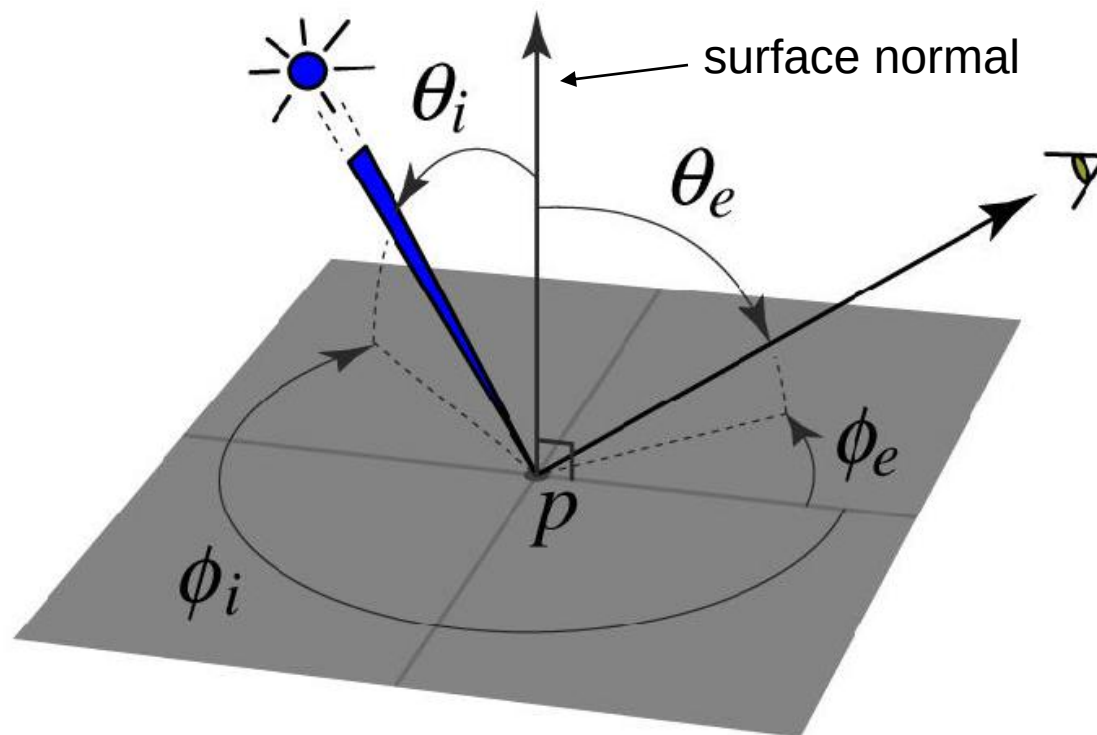
Let's consider the case of reflection in detail

- In the most general case, a single incoming ray could be reflected in all directions. How can we describe the amount of light reflected in each direction?

The BRDF

The Bidirectional Reflection Distribution Function

- Given an incoming ray (θ_i, ϕ_i) and outgoing ray (θ_e, ϕ_e)
what proportion of the incoming light is reflected along outgoing ray?

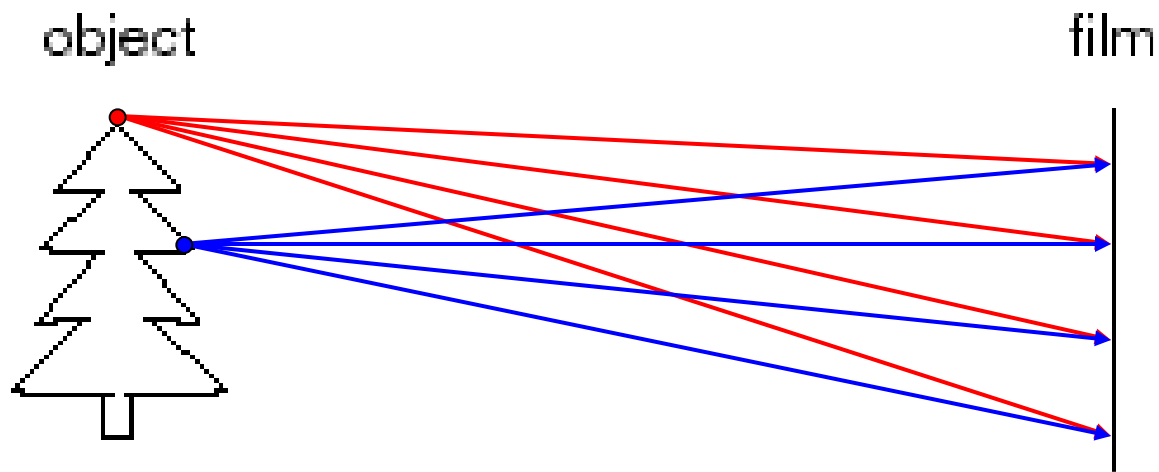


Answer given by the BRDF: $\rho(\theta_i, \phi_i, \theta_e, \phi_e)$

We will first study the geometry of image formation.

Later, we will look into illumination effects.

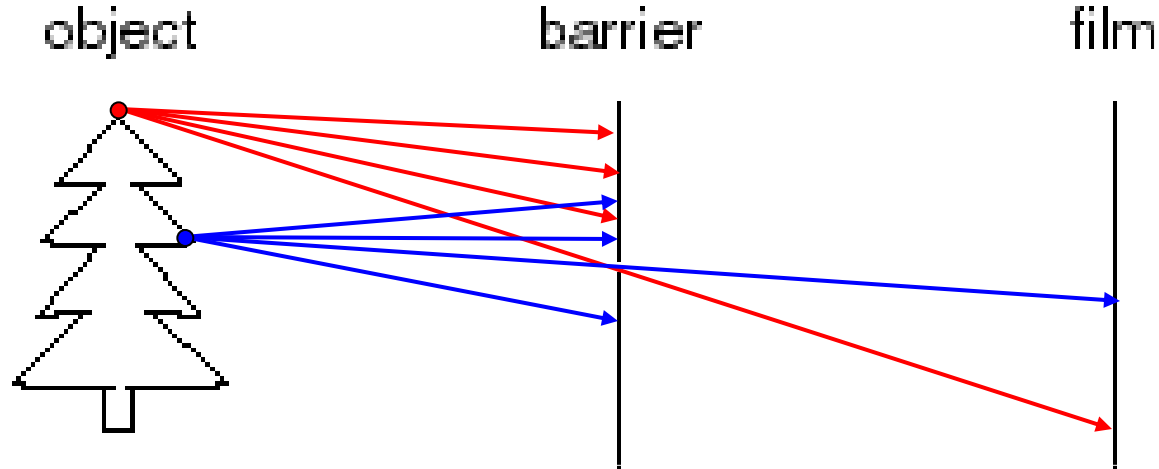
Image formation



Let's design a camera

- Idea 1: put a piece of film in front of an object
- Do we get a reasonable image?

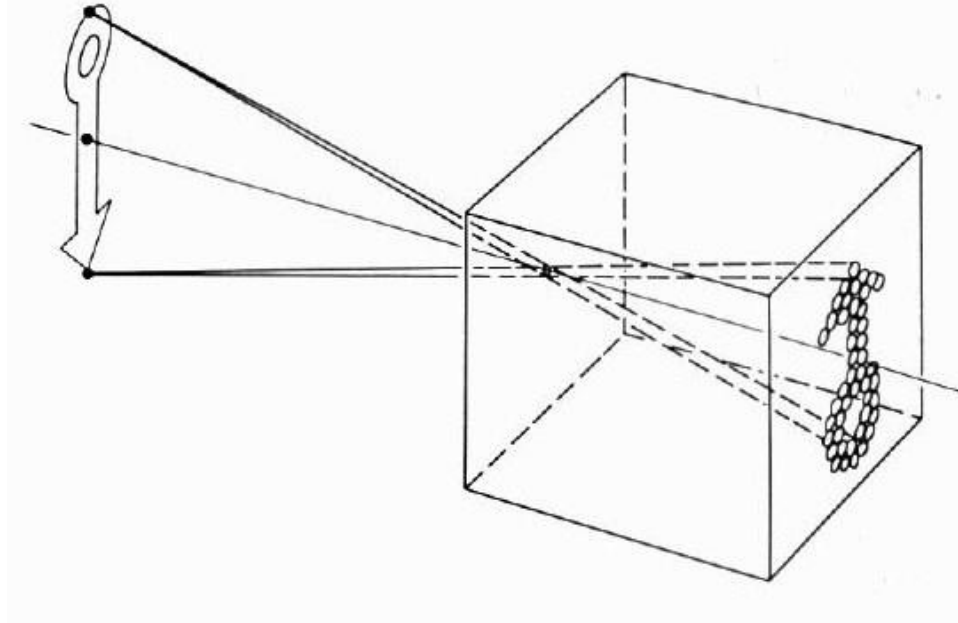
Pinhole camera



Add a barrier to block off most of the rays

- This reduces blurring
- The opening known as the **aperture**
- How does this transform the image?

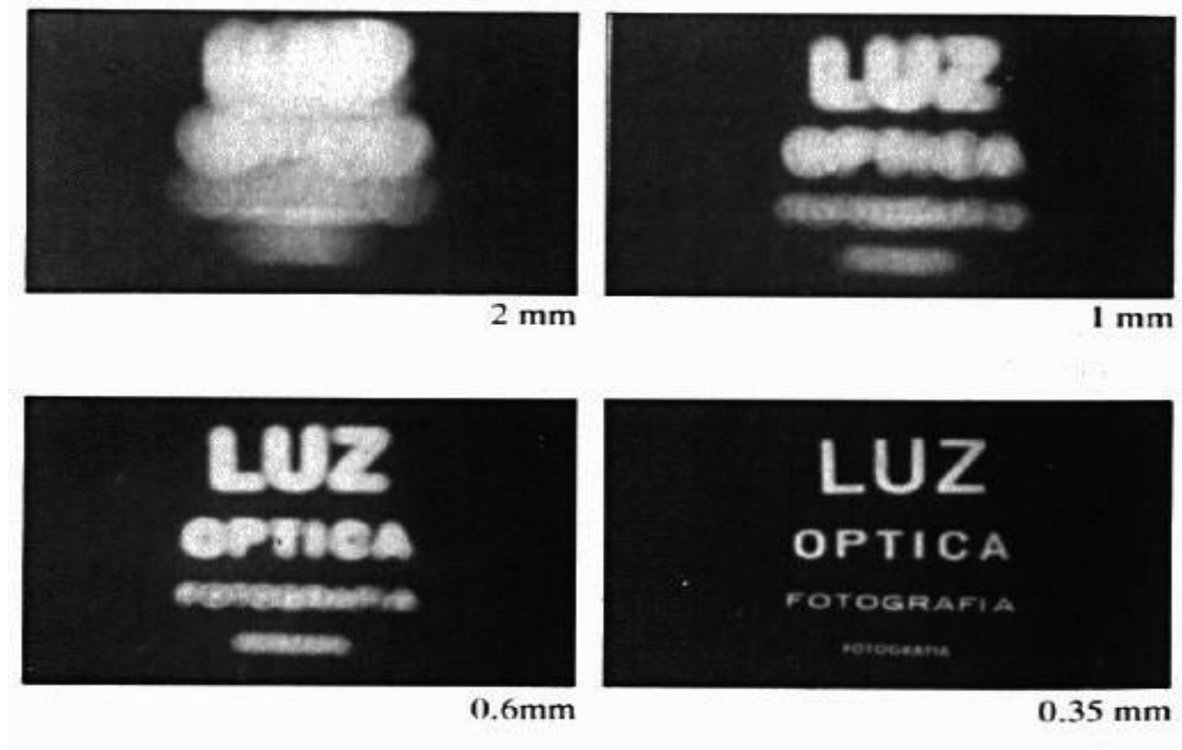
Camera Obscura



The first camera

- Known to Aristotle
- How does the aperture size affect the image?

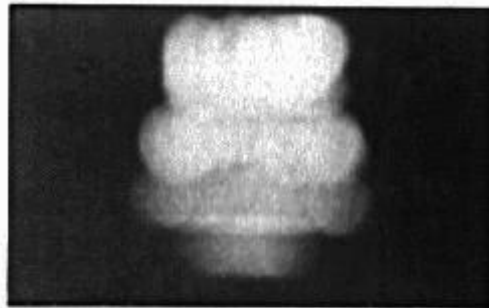
Shrinking the aperture



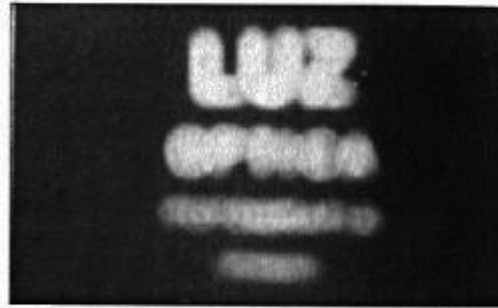
Why not make the aperture as small as possible?

- Less light gets through
- *Diffraction* effects...

Shrinking the aperture



2 mm



1 mm



0.6mm



0.35 mm

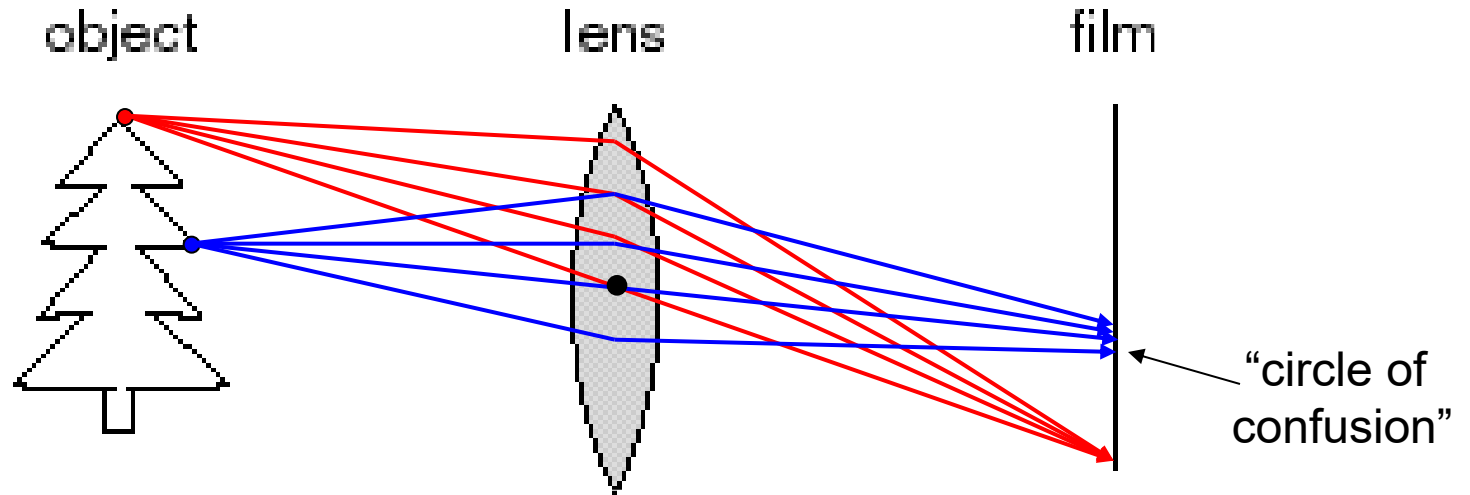


0.15 mm



0.07 mm

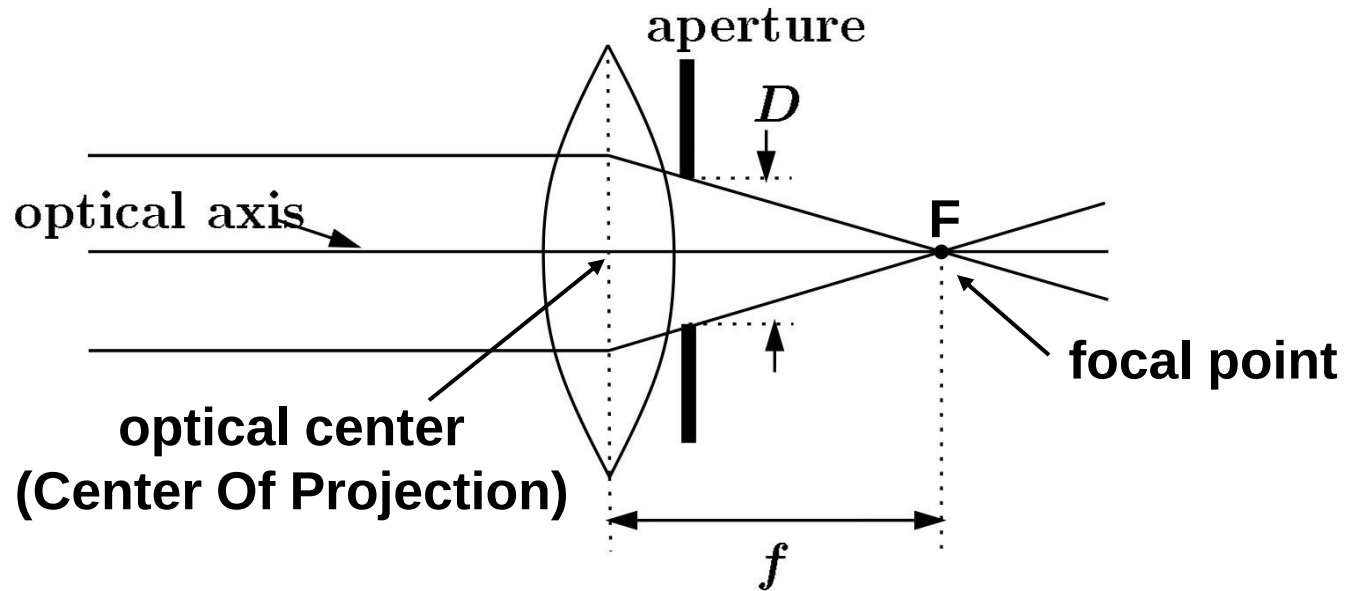
Adding a lens



A lens focuses light onto the film

- There is a specific distance at which objects are “in focus”
 - other points project to a “circle of confusion” in the image
- Changing the shape of the lens changes this distance

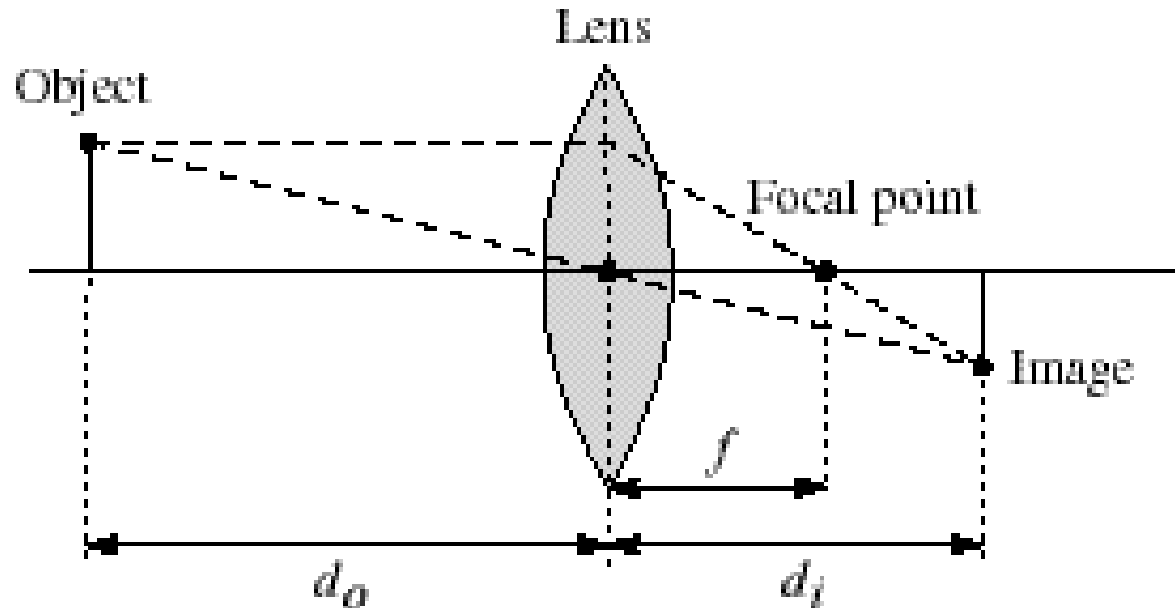
Lenses



A lens focuses parallel rays onto a single focal point

- focal point at a distance f beyond the plane of the lens
 - f is a function of the shape and index of refraction of the lens
- Aperture of diameter D restricts the range of rays
 - aperture may be on either side of the lens
- Lenses are typically spherical (easier to produce)

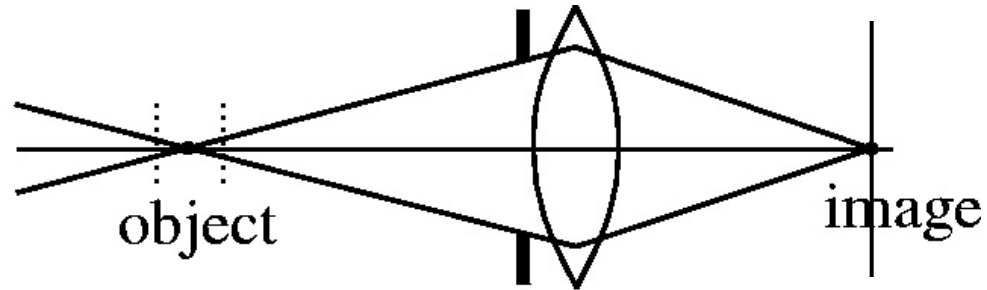
Thin lenses



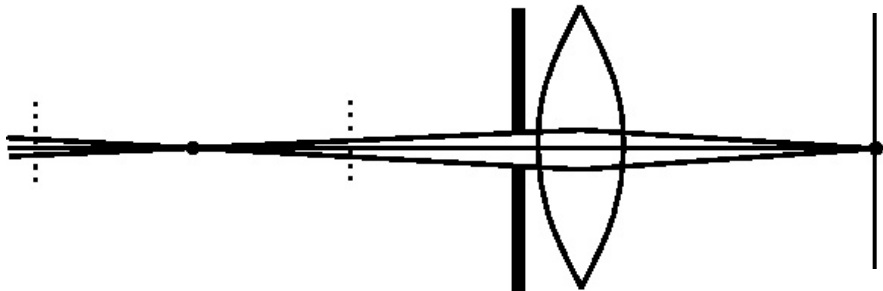
Thin lens equation:
$$\frac{1}{d_o} + \frac{1}{d_i} = \frac{1}{f}$$

- Any object point satisfying this equation is in focus
- What is the shape of the focus region?
- How can we change the focus region?
- Thin lens applet: http://www.phy.ntnu.edu.tw/java/Lens/lens_e.html (by Fu-Kwun Hwang)

Depth of field



$f / 5.6$

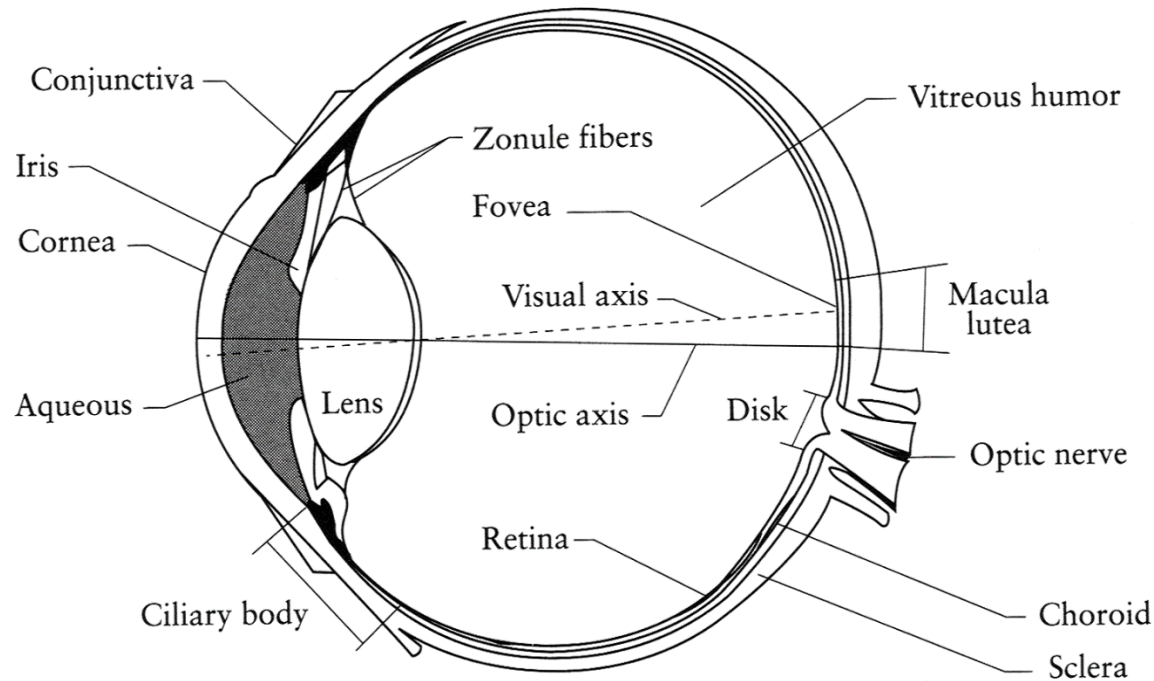


$f / 32$

Changing the aperture size affects depth of field

- A smaller aperture increases the range in which the object is approximately in focus

The eye



The human eye is a camera

- **Iris** - colored annulus with radial muscles
- **Pupil** - the hole (aperture) whose size is controlled by the iris
- What's the "film"?
 - photoreceptor cells (rods and cones) in the **retina**

Digital camera



A digital camera replaces film with a sensor array

- Each cell in the array is a **Charge Coupled Device**
 - light-sensitive diode that converts photons to electrons
 - other variants exist: CMOS is becoming more popular
 - CMOS: power-efficient, cheaper; CCD: higher image quality

Issues with digital cameras

Noise

- big difference between consumer vs. SLR-style cameras
- low light is where you most notice [noise](#)

Compression

- creates [artifacts](#) except in uncompressed formats (tiff, raw)

Color

- [color fringing](#) artifacts from [Bayer patterns](#)

Blooming

- charge [overflowing](#) into neighboring pixels

In-camera processing

- oversharpening can produce [halos](#)

Interlaced vs. progressive scan video

- [even/odd rows from different exposures](#)

Are more megapixels better?

- requires higher quality lens
- noise issues

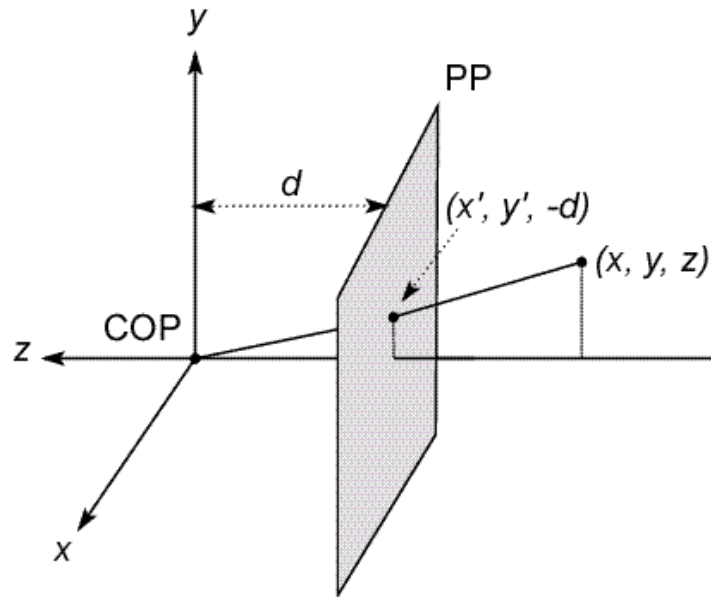
Stabilization

- compensate for camera shake (mechanical vs. electronic)

More info online, e.g.,

- <http://electronics.howstuffworks.com/digital-camera.htm>
- <http://www.dpreview.com/>

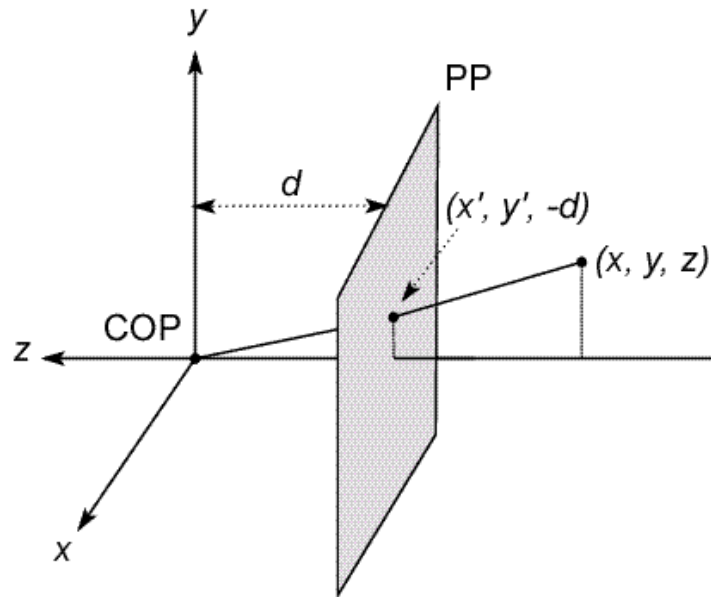
Modeling projection



The coordinate system

- We will use the pin-hole model as an approximation
- Put the optical center (**C**enter **O**f **P**rojection) at the origin
- Put the image plane (**P**rojection **P**lane) *in front* of the CO
- The camera looks down the *negative* z axis
 - we need this if we want right-handed-coordinates

Modeling projection



Projection equations

- Compute intersection with PP of ray from (x,y,z) to COP
- Derived using similar triangles

$$(x, y, z) \rightarrow \left(-d\frac{x}{z}, -d\frac{y}{z}, -d\right)$$

- We get the projection by throwing out the last coordinate:

$$(x, y, z) \rightarrow \left(-d\frac{x}{z}, -d\frac{y}{z}\right)$$

Homogeneous coordinates

Is this a linear transformation?

- no—division by z is nonlinear

Trick: add one more coordinate:

$$(x, y) \Rightarrow \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

homogeneous image
coordinates

$$(x, y, z) \Rightarrow \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

homogeneous scene
coordinates

Converting *from* homogeneous coordinates

$$\begin{bmatrix} x \\ y \\ w \end{bmatrix} \Rightarrow (x/w, y/w)$$

$$\begin{bmatrix} x \\ y \\ z \\ w \end{bmatrix} \Rightarrow (x/w, y/w, z/w)$$

Perspective Projection

Projection is a matrix multiply using homogeneous coordinates:

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1/d & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ -z/d \end{bmatrix} \Rightarrow \left(-d\frac{x}{z}, -d\frac{y}{z} \right)$$

divide by third coordinate

This is known as **perspective projection**

- The matrix is the **projection matrix**

Perspective Projection

How does scaling the projection matrix change the transformation?

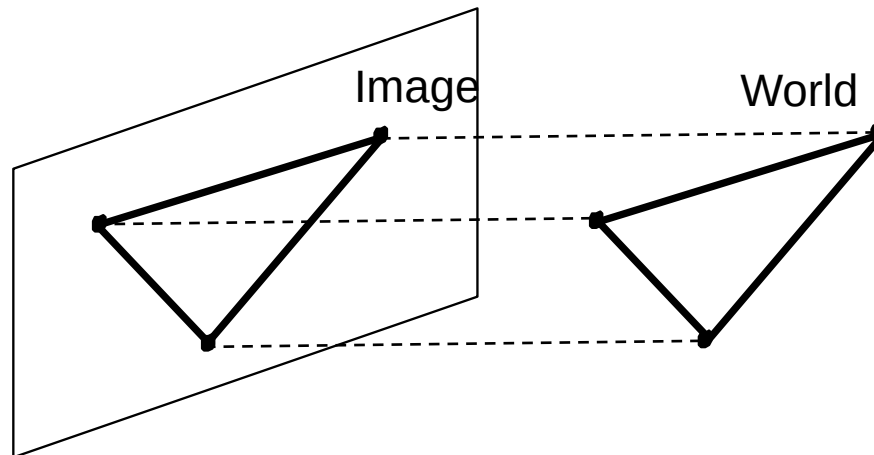
$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & -1/d & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ -z/d \end{bmatrix} \Rightarrow \left(-d\frac{x}{z}, -d\frac{y}{z}\right)$$

$$\begin{bmatrix} -d & 0 & 0 & 0 \\ 0 & -d & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} -dx \\ -dy \\ z \end{bmatrix} \Rightarrow \left(-d\frac{x}{z}, -d\frac{y}{z}\right)$$

Orthographic projection

Special case of perspective projection

- Distance from the COP to the PP is infinite



- Good approximation for telephoto optics
- Also called “parallel projection”: $(x, y, z) \rightarrow (x, y)$
- What’s the projection matrix?

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} \Rightarrow (x, y)$$

Variants of orthographic projection

Scaled orthographic

- Also called “weak perspective”

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1/d \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix} = \begin{bmatrix} x \\ y \\ 1/d \end{bmatrix} \Rightarrow (dx, dy)$$

Affine projection

- Also called “paraperspective”

$$\begin{bmatrix} a & b & c & d \\ e & f & g & h \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ z \\ 1 \end{bmatrix}$$

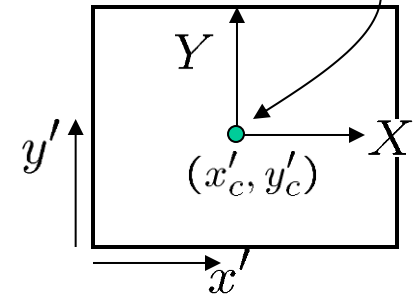
Camera parameters

A camera is described by several parameters

- Translation **T** of the optical center from the origin of world coords
- Rotation **R** of the image plane
- focal length **f**, principle point (x'_c, y'_c) , pixel size (s_x, s_y)
- blue parameters are called “**extrinsics**,” red are “**intrinsics**”

Projection equation

$$\mathbf{X} = \begin{bmatrix} sx \\ sy \\ s \end{bmatrix} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{\Pi} \mathbf{X}$$



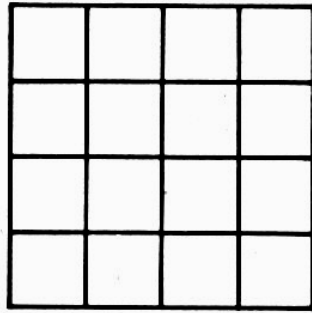
- The projection matrix models the cumulative effect of all parameters
- Useful to decompose into a series of operations

$$\mathbf{\Pi} = \underbrace{\begin{bmatrix} -fs_x & 0 & x'_c \\ 0 & -fs_y & y'_c \\ 0 & 0 & 1 \end{bmatrix}}_{\text{intrinsics}} \underbrace{\begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix}}_{\text{projection}} \underbrace{\begin{bmatrix} \mathbf{R}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}}_{\text{rotation}} \underbrace{\begin{bmatrix} \mathbf{I}_{3 \times 3} & \mathbf{T}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}}_{\text{translation}}$$

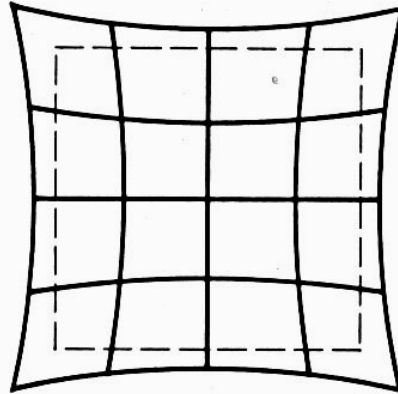
identity matrix

- The definitions of these parameters are **not** completely standardized
 - especially intrinsics—varies from one book to another

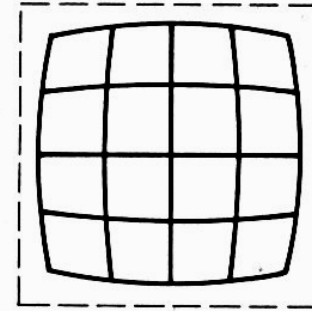
Distortion



No distortion



Pin cushion



Barrel

Radial distortion of the image

- Caused by imperfect lenses
- Deviations are most noticeable for rays that pass through the edge of the lens

Correcting radial distortion



from [Helmut Dersch](#)

Modeling distortion

Project $(\hat{x}, \hat{y}, \hat{z})$
to “normalized”
image coordinates

$$x'_n = \hat{x} / \hat{z}$$
$$y'_n = \hat{y} / \hat{z}$$

Apply radial distortion

$$r^2 = x_n'^2 + y_n'^2$$
$$x'_d = x'_n(1 + \kappa_1 r^2 + \kappa_2 r^4)$$
$$y'_d = y'_n(1 + \kappa_1 r^2 + \kappa_2 r^4)$$

Apply focal length
translate image center

$$x' = f x'_d + x_c$$
$$y' = f y'_d + y_c$$

To model lens distortion

- Use above projection operation instead of standard projection matrix multiplication

360 degree field of view...



Basic approach

- Take a photo of a parabolic mirror with an orthographic lens (Nayar)
- Or buy one a lens from a variety of omnicam manufacturers...
 - See <http://www.cis.upenn.edu/~kostas/omni.html>

Projective Geometry

Adapted from original slides in Computer Vision by Szeliski

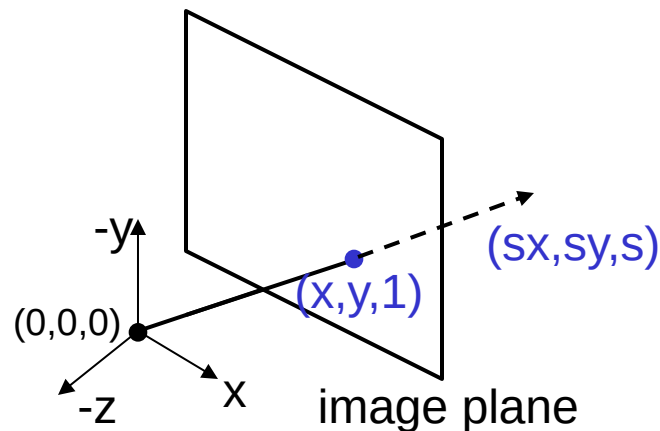
The projective plane

Why do we need homogeneous coordinates?

- represent points at infinity, homographies, perspective projection, multi-view relationships

What is the geometric intuition?

- a point in the image is a *ray* in projective space



- Each *point* (x,y) on the plane is represented by a *ray* (sx,sy,s)
 - all points on the ray are equivalent: $(x, y, 1) \cong (sx, sy, s)$

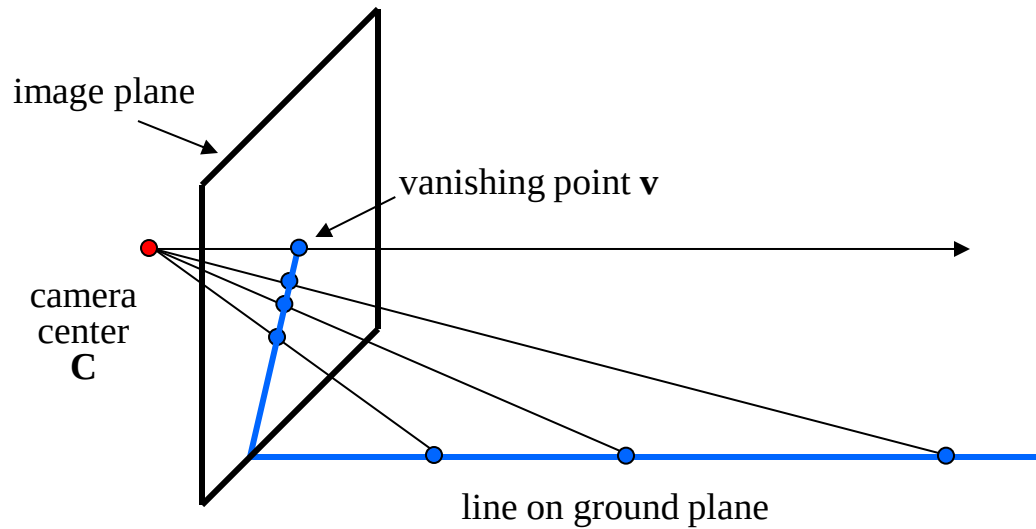
3D to 2D: “perspective” projection

Matrix Projection:
$$\mathbf{p} = \begin{bmatrix} wx \\ wy \\ w \end{bmatrix} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{P}\mathbf{P}$$

What is *not* preserved under perspective projection?

What IS preserved?

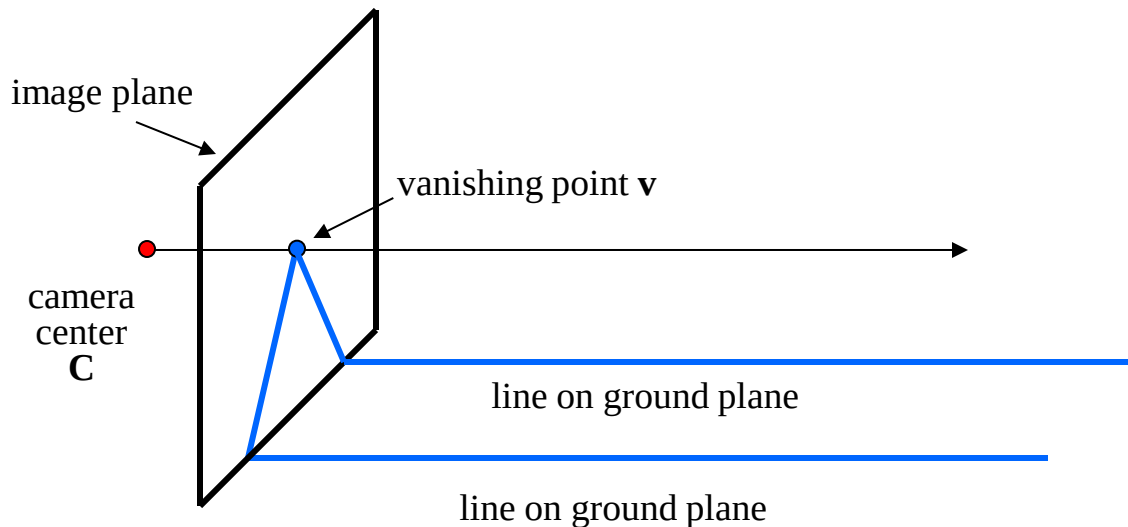
Vanishing points



Vanishing point

- projection of a point at infinity

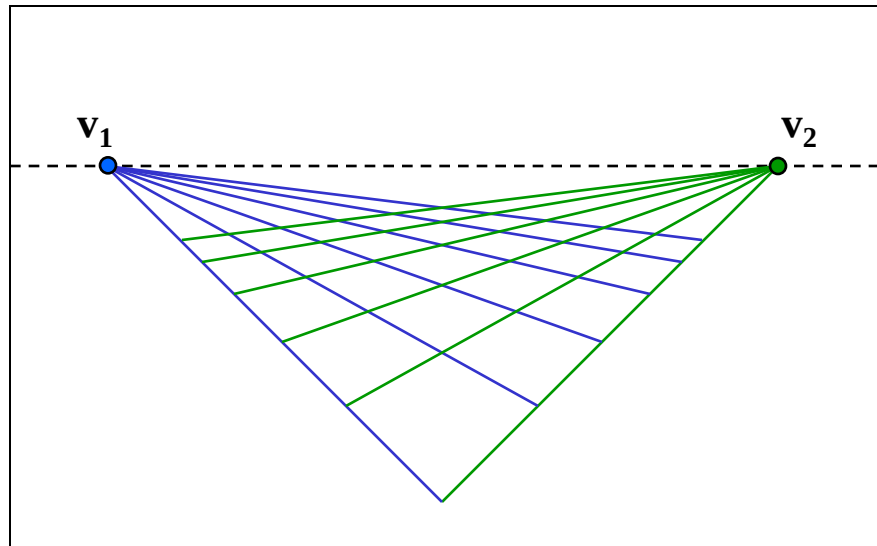
Vanishing points



Properties

- Any two parallel lines have the same vanishing point v
- The ray from C through v is parallel to the lines
- An image may have more than one vanishing point
 - in fact every pixel is a potential vanishing point

Vanishing lines



Multiple Vanishing Points

- Any set of parallel lines on the plane define a vanishing point
- The union of all of vanishing points from lines on the same plane is the *vanishing line*
 - For the ground plane, this is called the *horizon*

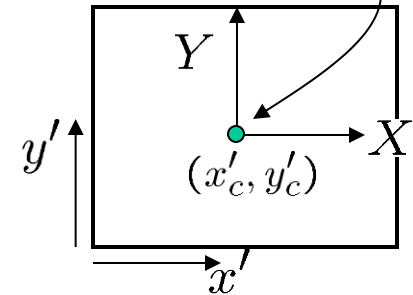
Camera parameters

A camera is described by several parameters

- Translation **T** of the optical center from the origin of world coords
- Rotation **R** of the image plane
- focal length **f**, principle point (x'_c, y'_c) , pixel size (s_x, s_y)
- blue parameters are called “extrinsics,” red are “intrinsics”

Projection equation

$$\mathbf{X} = \begin{bmatrix} sx \\ sy \\ s \end{bmatrix} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{\Pi} \mathbf{X}$$



- The projection matrix models the cumulative effect of all parameters
- Useful to decompose into a series of operations

$$\mathbf{\Pi} = \begin{bmatrix} -fs_x & 0 & x'_c \\ 0 & -fs_y & y'_c \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} \mathbf{R}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{I}_{3 \times 3} & \mathbf{T}_{3 \times 1} \\ \mathbf{0}_{1 \times 3} & 1 \end{bmatrix}$$

intrinsics projection rotation translation

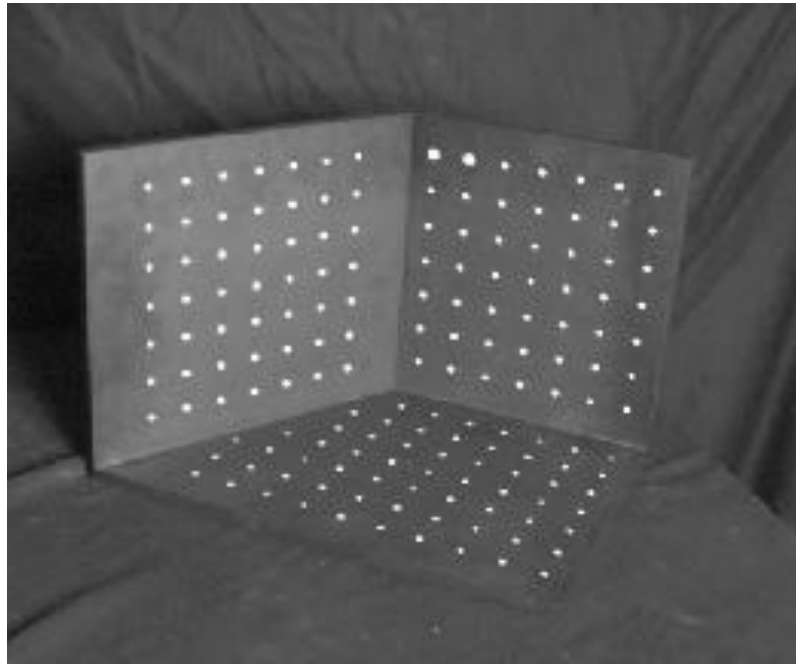
identity matrix

- The definitions of these parameters are **not** completely standardized
 - especially intrinsics—varies from one book to another

Estimating the projection matrix

Place a known object in the scene

- identify correspondence between image and scene
- compute mapping from scene to image



$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} \approx \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

Camera calibration

Goal: estimate the camera parameters

- Version 1: solve for projection matrix

$$\mathbf{X} = \begin{bmatrix} wx \\ wy \\ w \end{bmatrix} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{P}\mathbf{X}$$

- Version 2: solve for camera parameters separately
 - intrinsics (focal length, principle point, pixel size)
 - extrinsics (rotation angles, translation)
 - radial distortion

Direct linear calibration

$$\begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix} \cong \begin{bmatrix} m_{00} & m_{01} & m_{02} & m_{03} \\ m_{10} & m_{11} & m_{12} & m_{13} \\ m_{20} & m_{21} & m_{22} & m_{23} \end{bmatrix} \begin{bmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{bmatrix}$$

$$u_i = \frac{m_{00}X_i + m_{01}Y_i + m_{02}Z_i + m_{03}}{m_{20}X_i + m_{21}Y_i + m_{22}Z_i + m_{23}}$$

$$v_i = \frac{m_{10}X_i + m_{11}Y_i + m_{12}Z_i + m_{13}}{m_{20}X_i + m_{21}Y_i + m_{22}Z_i + m_{23}}$$

$$u_i(m_{20}X_i + m_{21}Y_i + m_{22}Z_i + m_{23}) = m_{00}X_i + m_{01}Y_i + m_{02}Z_i + m_{03}$$

$$v_i(m_{20}X_i + m_{21}Y_i + m_{22}Z_i + m_{23}) = m_{10}X_i + m_{11}Y_i + m_{12}Z_i + m_{13}$$

$$\begin{bmatrix} X_i & Y_i & Z_i & 1 & 0 & 0 & 0 & 0 & -u_iX_i & -u_iY_i & -u_iZ_i & -u_i \\ 0 & 0 & 0 & 0 & X_i & Y_i & Z_i & 1 & -v_iX_i & -v_iY_i & -v_iZ_i & -v_i \end{bmatrix} \begin{bmatrix} m_{00} \\ m_{01} \\ m_{02} \\ m_{03} \\ m_{10} \\ m_{11} \\ m_{12} \\ m_{13} \\ m_{20} \\ m_{21} \\ m_{22} \\ m_{23} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Direct linear calibration

$$\begin{bmatrix}
 X_1 & Y_1 & Z_1 & 1 & 0 & 0 & 0 & 0 & -u_1 X_1 & -u_1 Y_1 & -u_1 Z_1 & -u_1 \\
 0 & 0 & 0 & 0 & X_1 & Y_1 & Z_1 & 1 & -v_1 X_1 & -v_1 Y_1 & -v_1 Z_1 & -v_1 \\
 & & & & & & & \vdots & & & & \\
 X_n & Y_n & Z_n & 1 & 0 & 0 & 0 & 0 & -u_n X_n & -u_n Y_n & -u_n Z_n & -u_n \\
 0 & 0 & 0 & 0 & X_n & Y_n & Z_n & 1 & -v_n X_n & -v_n Y_n & -v_n Z_n & -v_n
 \end{bmatrix}
 \begin{bmatrix}
 m_{00} \\
 m_{01} \\
 m_{02} \\
 m_{03} \\
 m_{10} \\
 m_{11} \\
 m_{12} \\
 m_{13} \\
 m_{20} \\
 m_{21} \\
 m_{22}
 \end{bmatrix}
 =
 \begin{bmatrix}
 0 \\
 0 \\
 \vdots \\
 0 \\
 0
 \end{bmatrix}$$

Can solve for m_{ij} by linear least squares
 - look at solution for homographies.

Solving Homogeneous Linear Systems

The Core Concept: Constrained Optimization

If we want to minimize Ax but avoid the useless "trivial solution" ($x = 0$), we must constrain the size of x . The standard approach is to require x to be a unit vector, meaning its length is 1:

$$\|x\|^2 = x^T x = 1$$

To make Ax as close to the zero vector as possible, we minimize its squared Euclidean norm, $\|Ax\|^2$. Using linear algebra properties, we can rewrite this norm:

$$\|Ax\|^2 = (Ax)^T (Ax) = x^T A^T Ax$$

So, our goal is to find the vector x that minimizes $x^T A^T Ax$ subject to the constraint $x^T x = 1$.

Step 1: Compute the Matrix $A^T A$

Multiply the transpose of your coefficient matrix A by A itself.

- If A is an $m \times n$ matrix, $A^T A$ will always be a **square ($n \times n$), symmetric, and positive semi-definite** matrix.
- These properties are crucial because they guarantee that all eigenvalues of $A^T A$ are real numbers and are greater than or equal to zero.

Solving Homogeneous Linear Systems

Step 2: Set up the Eigenvalue Problem

To solve our constrained optimization problem, we use the method of **Lagrange Multipliers**.

We define the Lagrangian function $\mathcal{L}(x, \lambda)$ with a multiplier λ :

$$\mathcal{L}(x, \lambda) = x^T A^T A x - \lambda(x^T x - 1)$$

To find the minimum, we take the derivative with respect to x and set it to zero:

$$\frac{\partial \mathcal{L}}{\partial x} = 2A^T A x - 2\lambda x = 0$$

Dividing by 2 gives us the classic **eigenvalue equation**:

$$A^T A x = \lambda x$$

This proves that any valid solution x *must* be an eigenvector of $A^T A$.

Step 3: Identify the Minimum Eigenvalue

Now, let's substitute this back into our original minimization objective ($x^T A^T A x$):

$$x^T (A^T A x) = x^T (\lambda x) = \lambda(x^T x)$$

Since we constrained x to be a unit vector ($x^T x = 1$), the expression simplifies beautifully:

$$x^T A^T A x = \lambda$$

This means the value of our objective function is exactly equal to the eigenvalue λ . Therefore, to minimize the objective function, **we must choose the smallest possible eigenvalue, $\lambda_{\min}$** .

Solving Homogeneous Linear Systems

Step 4: Find the Corresponding Eigenvector

1. Find the smallest eigenvalue $\lambda_{\min}$ by solving the characteristic equation $\det(A^T A - \lambda I) = 0$ (or using numerical methods like the Power Method or SVD for large matrices).
2. Find the null space of $(A^T A - \lambda_{\min} I)$ to get the corresponding eigenvector x .
3. Normalize x so that $\|x\| = 1$. This vector x is your optimal solution.

Connection to Singular Value Decomposition (SVD)

In practice, computing $A^T A$ directly can introduce numerical instability.

The SVD of A is defined as:

$$A = U\Sigma V^T$$

Where:

- U contains the eigenvectors of AA^T .
- V contains the eigenvectors of $A^T A$.
- Σ contains the singular values (σ), which are the square roots of the eigenvalues of $A^T A$ ($\lambda = \sigma^2$).

Because the columns of V are sorted in order of descending singular values, **the very last column of V** corresponds to the smallest singular value (and thus the lowest eigenvalue of $A^T A$). This makes the last column of V the exact non-trivial solution to $Ax = 0$.

Direct linear calibration

Advantage:

- Very simple to formulate and solve

Disadvantages:

- Doesn't tell you the camera parameters
- Doesn't model radial distortion
- Hard to impose constraints (e.g., known focal length)
- Doesn't minimize the right error function

For these reasons, *nonlinear methods* are preferred

- Define error function E between projected 3D points and image positions
 - E is nonlinear function of intrinsics, extrinsics, radial distortion
- Minimize E using nonlinear optimization techniques
 - e.g., variants of Newton's method (e.g., Levenberg Marquart)

Camera calibration

Goal: estimate the camera parameters

- Version 1: solve for projection matrix

$$\mathbf{X} = \begin{bmatrix} wx \\ wy \\ w \end{bmatrix} = \begin{bmatrix} * & * & * & * \\ * & * & * & * \\ * & * & * & * \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \\ 1 \end{bmatrix} = \mathbf{P}\mathbf{X}$$

- Version 2: solve for camera parameters separately
 - intrinsics (focal length, principle point, pixel size)
 - extrinsics (rotation angles, translation)
 - radial distortion

(Slides 15-18: Self-study)

Finding the scale factors...

Let's assume that the camera is reasonable

- Square pixels
- Image plane parallel to sensor plane
- Principal point in the center of the image

$$\mathbf{\Pi} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1/f \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \end{bmatrix} \begin{bmatrix} r_{11} & r_{12} & r_{13} & 0 \\ r_{21} & r_{22} & r_{23} & 0 \\ r_{31} & r_{32} & r_{33} & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & t_1 \\ 0 & 1 & 0 & t_2 \\ 0 & 0 & 1 & t_3 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$= \begin{bmatrix} r_{11} & r_{12} & r_{13} & t'_1 \\ r_{21} & r_{22} & r_{23} & t'_2 \\ r_{31}/f & r_{32}/f & r_{33}/f & t'_3/f \end{bmatrix} \quad \begin{bmatrix} t'_1 \\ t'_2 \\ t'_3 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix} \begin{bmatrix} t_1 \\ t_2 \\ t_3 \end{bmatrix}$$

$$= \begin{bmatrix} a \mathbf{v}_X & b \mathbf{v}_Y & c \mathbf{v}_Z & d \mathbf{o} \end{bmatrix}$$

Solving for f

$$\begin{bmatrix} v_{x1} & v_{y1} & v_{z1} & o_1 \\ v_{x2} & v_{y2} & v_{z2} & o_2 \\ v_{x3} & v_{y3} & v_{z3} & o_3 \end{bmatrix} = \begin{bmatrix} r_{11}/a & r_{12}/b & r_{13}/c & t'_1/d \\ r_{21}/a & r_{22}/b & r_{23}/c & t'_2/d \\ r_{31}/af & r_{32}/bf & r_{33}/cf & t'_3/df \end{bmatrix}$$

$$\begin{bmatrix} v_X & v_Y & v_Z & o \end{bmatrix}$$

$$\begin{bmatrix} v_{x1} & v_{y1} & v_{z1} & o_1 \\ v_{x2} & v_{y2} & v_{z2} & o_2 \\ fv_{x3} & fv_{y3} & fv_{z3} & fo_3 \end{bmatrix} = \begin{bmatrix} r_{11}/a & r_{12}/b & r_{13}/c & t'_1/d \\ r_{21}/a & r_{22}/b & r_{23}/c & t'_2/d \\ r_{31}/a & r_{32}/b & r_{33}/c & t'_3/d \end{bmatrix}$$

Orthogonal
vectors

Orthogonal
vectors

$$v_{x1}v_{y1} + v_{x2}v_{y2} + f^2v_{x3}v_{y3} = 0 \quad \longrightarrow \quad f = \sqrt{\frac{v_{x1}v_{y1} + v_{x2}v_{y2}}{-v_{x3}v_{y3}}}$$

Solving for a, b, and c

$$\begin{bmatrix} v_{x1}/f & v_{y1}/f & v_{z1}/f & o_1/f \\ v_{x2}/f & v_{y2}/f & v_{z2}/f & o_2/f \\ v_{x3} & v_{y3} & v_{z3} & o_3 \end{bmatrix} = \begin{bmatrix} r_{11}/a & r_{12}/b & r_{13}/c & t'_1/d \\ r_{21}/a & r_{22}/b & r_{23}/c & t'_2/d \\ r_{31}/a & r_{32}/b & r_{33}/c & t'_3/d \end{bmatrix}$$

Norm
= 1/a

Norm
= 1/a

Solve for a, b, c

- Divide the first two rows by f, now that it is known
- Now just find the norms of the first three columns
- Once we know a, b, and c, that also determines R

How about d?

- Need a reference point in the scene

Solving for d

$$\begin{bmatrix} wu \\ wv \\ w \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} & t'_1/d \\ r_{21} & r_{22} & r_{23} & t'_2/d \\ r_{31} & r_{32} & r_{33} & t'_3/d \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ H \\ 1 \end{bmatrix}$$

Suppose we have one reference height H

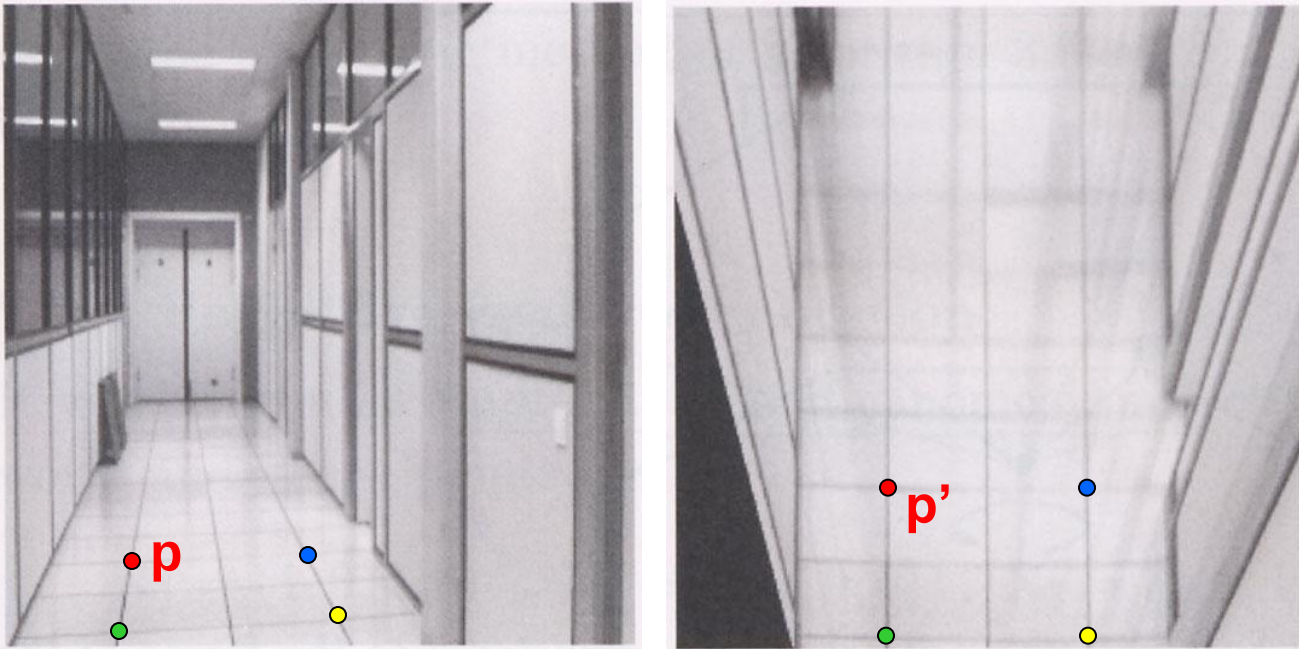
- E.g., we know that (0, 0, H) gets mapped to (u, v)

$$u = \frac{r_{13}H + t'_1/d}{r_{33}H + t'_3/d} \qquad d = \frac{t'_1 - ut'_3}{ur_{33}H - r_{13}H}$$

Finally, we can solve for t

$$\begin{bmatrix} t_1 \\ t_2 \\ t_3 \end{bmatrix} = \begin{bmatrix} r_{11} & r_{12} & r_{13} \\ r_{21} & r_{22} & r_{23} \\ r_{31} & r_{32} & r_{33} \end{bmatrix}^T \begin{bmatrix} t'_1 \\ t'_2 \\ t'_3 \end{bmatrix}$$

Image rectification



To unwarp (rectify) an image

- solve for homography $\mathbf{H}$ given $\mathbf{p}$ and $\mathbf{p}'$
- solve equations of the form: $w\mathbf{p}' = \mathbf{H}\mathbf{p}$
 - linear in unknowns: w and coefficients of $\mathbf{H}$
 - $\mathbf{H}$ is defined up to an arbitrary scale factor
 - how many points are necessary to solve for $\mathbf{H}$?

Solving for homographies

$$\begin{bmatrix} x'_i \\ y'_i \\ 1 \end{bmatrix} \cong \begin{bmatrix} h_{00} & h_{01} & h_{02} \\ h_{10} & h_{11} & h_{12} \\ h_{20} & h_{21} & h_{22} \end{bmatrix} \begin{bmatrix} x_i \\ y_i \\ 1 \end{bmatrix}$$

$$x'_i = \frac{h_{00}x_i + h_{01}y_i + h_{02}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

$$y'_i = \frac{h_{10}x_i + h_{11}y_i + h_{12}}{h_{20}x_i + h_{21}y_i + h_{22}}$$

$$x'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{00}x_i + h_{01}y_i + h_{02}$$

$$y'_i(h_{20}x_i + h_{21}y_i + h_{22}) = h_{10}x_i + h_{11}y_i + h_{12}$$

$$\begin{bmatrix} x_i & y_i & 1 & 0 & 0 & 0 & -x'_i x_i & -x'_i y_i & -x'_i \\ 0 & 0 & 0 & x_i & y_i & 1 & -y'_i x_i & -y'_i y_i & -y'_i \end{bmatrix} \begin{bmatrix} h_{00} \\ h_{01} \\ h_{02} \\ h_{10} \\ h_{11} \\ h_{12} \\ h_{20} \\ h_{21} \\ h_{22} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Solving for homographies

$$\begin{array}{c}
 \begin{bmatrix}
 x_1 & y_1 & 1 & 0 & 0 & 0 & -x'_1 x_1 & -x'_1 y_1 & -x'_1 \\
 0 & 0 & 0 & x_1 & y_1 & 1 & -y'_1 x_1 & -y'_1 y_1 & -y'_1 \\
 & & & & & \vdots & & & \\
 x_n & y_n & 1 & 0 & 0 & 0 & -x'_n x_n & -x'_n y_n & -x'_n \\
 0 & 0 & 0 & x_n & y_n & 1 & -y'_n x_n & -y'_n y_n & -y'_n
 \end{bmatrix}
 \begin{bmatrix}
 h_{00} \\
 h_{01} \\
 h_{02} \\
 h_{10} \\
 h_{11} \\
 h_{12} \\
 h_{20} \\
 h_{21} \\
 h_{22}
 \end{bmatrix}
 =
 \begin{bmatrix}
 0 \\
 0 \\
 \vdots \\
 0 \\
 0
 \end{bmatrix}
 \\
 \mathbf{A} \qquad \qquad \mathbf{h} \qquad \qquad \mathbf{0} \\
 2n \times 9 \qquad \qquad 9 \qquad \qquad 2n
 \end{array}$$

Defines a least squares problem: minimize $\|\mathbf{A}\mathbf{h} - \mathbf{0}\|^2$

- Since $\mathbf{h}$ is only defined up to scale, solve for unit vector $\hat{\mathbf{h}}$
- Solution: $\hat{\mathbf{h}}$ = eigenvector of $\mathbf{A}^\top \mathbf{A}$ with smallest eigenvalue
- Works with 4 or more points

Derivation of the Fundamental Structure from Motion Equations

EE 243

1 Introduction and Notation Reference

In computer vision, the fundamental equations of motion relate the 3D velocity of a point in space to its corresponding 2D velocity (optical flow) on the image plane under a perspective projection model.

1.1 Notation Dictionary

Symbol	Definition
$\mathbf{P} = [X, Y, Z]^T$	3D coordinates of a point in the camera-centered coordinate frame
f	Focal length of the camera system
$\mathbf{x} = [x, y]^T$	2D coordinates of the projected point on the image plane
u, v	Components of optical flow (velocity on the image plane, equivalent to $\dot{x}, \dot{y}$)
$\mathbf{T} = [T_X, T_Y, T_Z]^T$	Linear translational velocity vector of the camera
$\boldsymbol{\omega} = [\omega_X, \omega_Y, \omega_Z]^T$	Angular rotational velocity vector of the camera

Table 1: Variables used in the Structure from Motion formulation.

2 Step 1: 3D Rigid Body Kinematics

Let a 3D point in the scene relative to the camera coordinate frame be denoted by $\mathbf{P}$. If the camera moves through space with a translational velocity $\mathbf{T}$ and a rotational velocity $\boldsymbol{\omega}$, the relative velocity $\dot{\mathbf{P}}$ of the point with respect to the camera observer is given by the kinematic rigid body equation:

$$\dot{\mathbf{P}} = -\mathbf{T} - \boldsymbol{\omega} \times \mathbf{P} \quad (1)$$

Expanding the cross product vector $\boldsymbol{\omega} \times \mathbf{P}$:

$$\begin{bmatrix} \dot{X} \\ \dot{Y} \\ \dot{Z} \end{bmatrix} = - \begin{bmatrix} T_X \\ T_Y \\ T_Z \end{bmatrix} - \begin{bmatrix} \omega_Y Z - \omega_Z Y \\ \omega_Z X - \omega_X Z \\ \omega_X Y - \omega_Y X \end{bmatrix} \quad (2)$$

Breaking this down into individual scalar equations for each 3D coordinate axis yield:

$$\dot{X} = -T_X - \omega_Y Z + \omega_Z Y \quad (3)$$

$$\dot{Y} = -T_Y - \omega_Z X + \omega_X Z \quad (4)$$

$$\dot{Z} = -T_Z - \omega_X Y + \omega_Y X \quad (5)$$

3 Step 2: Pinhole Perspective Projection Model

Under an ideal pinhole camera approximation, a 3D scene point projects onto the 2D image plane at location (x, y) according to the following perspective scaling relations:

$$x = f \frac{X}{Z}, \quad y = f \frac{Y}{Z} \quad (6)$$

4 Step 3: Temporal Differentiation (Image Coordinate Velocity)

To compute the optical flow coordinates $u = \dot{x}$ and $v = \dot{y}$, we must differentiate the perspective equations with respect to time using the calculus quotient rule.

4.1 Derivation for Horizontal Optical Flow (u)

Differentiating $x = f \frac{X}{Z}$ yields:

$$u = \dot{x} = \frac{d}{dt} \left(f \frac{X}{Z} \right) = f \frac{\dot{X}Z - X\dot{Z}}{Z^2} = \frac{f\dot{X}}{Z} - \frac{fX\dot{Z}}{Z^2} \quad (7)$$

We factor out the spatial component by utilizing the identity $X = \frac{xZ}{f}$:

$$u = \frac{f\dot{X} - x\dot{Z}}{Z} \quad (8)$$

Now, substituting the 3D velocity component equations (Equations 3 and 5) for $\dot{X}$ and $\dot{Z}$:

$$u = \frac{f(-T_X - \omega_Y Z + \omega_Z Y) - x(-T_Z - \omega_X Y + \omega_Y X)}{Z} \quad (9)$$

Distributing terms across the common denominator Z :

$$u = \frac{-fT_X + xT_Z}{Z} - f\omega_Y + f\omega_Z \left(\frac{Y}{Z} \right) + x\omega_X \left(\frac{Y}{Z} \right) - x\omega_Y \left(\frac{X}{Z} \right) \quad (10)$$

By reapplying the basic projection definitions ($\frac{Y}{Z} = \frac{y}{f}$ and $\frac{X}{Z} = \frac{x}{f}$), we can cleanly eliminate the unobservable 3D coordinates X and Y :

$$u = \frac{T_Z x - T_X f}{Z} + x\omega_X \left(\frac{y}{f} \right) - f\omega_Y - x\omega_Y \left(\frac{x}{f} \right) + f\omega_Z \left(\frac{y}{f} \right) \quad (11)$$

Factoring out the angular velocity components ω_X , ω_Y , and ω_Z reveals the final structured layout for the horizontal image plane tracking:

$$u = \frac{T_Z x - T_X f}{Z} + \omega_X \left(\frac{xy}{f} \right) - \omega_Y \left(f + \frac{x^2}{f} \right) + \omega_Z y \quad (12)$$

4.2 Derivation for Vertical Optical Flow (v)

Similarly, differentiating $y = f \frac{Y}{Z}$ with respect to time yields:

$$v = \dot{y} = \frac{d}{dt} \left(f \frac{Y}{Z} \right) = f \frac{\dot{Y}Z - Y\dot{Z}}{Z^2} = \frac{f\dot{Y}}{Z} - \frac{fY\dot{Z}}{Z^2} \quad (13)$$

Substituting the identity $Y = \frac{yZ}{f}$:

$$v = \frac{f\dot{Y} - y\dot{Z}}{Z} \quad (14)$$

Now, substituting the 3D velocity component equations (Equations 4 and 5) for $\dot{Y}$ and $\dot{Z}$:

$$v = \frac{f(-T_Y - \omega_Z X + \omega_X Z) - y(-T_Z - \omega_X Y + \omega_Y X)}{Z} \quad (15)$$

Distributing across the common denominator Z :

$$v = \frac{-fT_Y + yT_Z}{Z} - f\omega_Z \left(\frac{X}{Z} \right) + f\omega_X + y\omega_X \left(\frac{Y}{Z} \right) - y\omega_Y \left(\frac{X}{Z} \right) \quad (16)$$

Once more, transforming our 3D position fields back to known camera coordinates via ($\frac{Y}{Z} = \frac{y}{f}$ and $\frac{X}{Z} = \frac{x}{f}$):

$$v = \frac{T_Z y - T_Y f}{Z} + f\omega_X + y\omega_X \left(\frac{y}{f} \right) - y\omega_Y \left(\frac{x}{f} \right) - f\omega_Z \left(\frac{x}{f} \right) \quad (17)$$

Grouping the components systematically by their respective rotational terms:

$$v = \frac{T_Z y - T_Y f}{Z} + \omega_X \left(f + \frac{y^2}{f} \right) - \omega_Y \left(\frac{xy}{f} \right) - \omega_Z x \quad (18)$$

Note on Sign Conventions: Depending on the native handedness convention adopted for the frame rotation orientation (clockwise vs. counter-clockwise configurations around the focal primary optical axis), the system parameterizes the final rotational twist scalar directly as $+\omega_Z x$.

5 Final Structure from Motion Summary Equations

Gathering our final results, we establish the standard matching framework for visual odometry and optical scene flow fields:

$$u = \frac{T_Z x - T_X f}{Z} + \omega_X \left(\frac{xy}{f} \right) - \omega_Y \left(f + \frac{x^2}{f} \right) + \omega_Z y \quad (19)$$

$$v = \frac{T_Z y - T_Y f}{Z} + \omega_X \left(f + \frac{y^2}{f} \right) - \omega_Y \left(\frac{xy}{f} \right) + \omega_Z x \quad (20)$$

Structure from Motion

Structure from motion

Given many images, how can we

- a) figure out where they were all taken from?
- b) build a 3D model of the scene?



This is the **structure from motion (SfM)** problem

Structure from motion

- **Input: images with 2D points in correspondence**
 $p_{i,j} = (u_{i,j}, v_{i,j})$
- **Output**
 - **structure:** 3D location x_i for each point p_i
 - **motion:** camera parameters R_j, t_j (and possibly intrinsic parameters too)
- **Objective function:** minimize *reprojection error*

Basic SfM Math Covered in Class

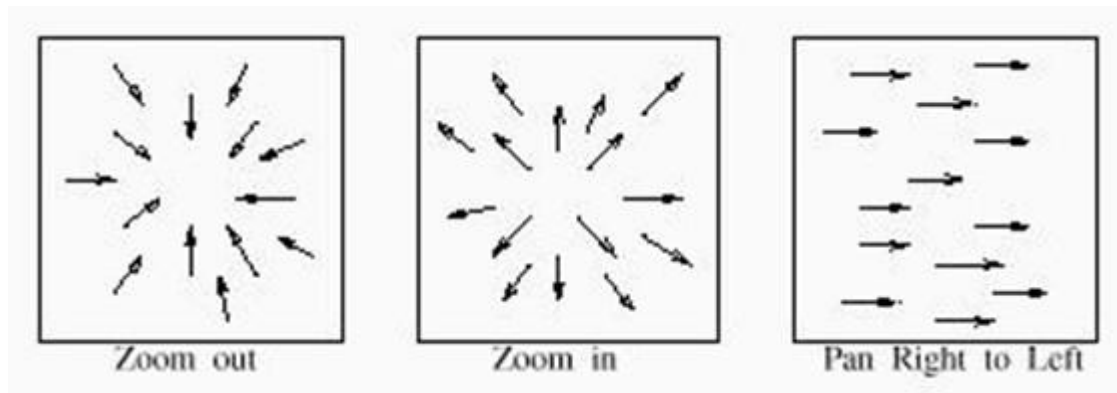
SfM Equations

$$u = \frac{T_Z x - T_X f}{Z} + \omega_X \left(\frac{xy}{f} \right) - \omega_Y \left(f + \frac{x^2}{f} \right) + \omega_Z y$$

$$v = \frac{T_Z y - T_Y f}{Z} + \omega_X \left(f + \frac{y^2}{f} \right) - \omega_Y \left(\frac{xy}{f} \right) + \omega_Z x$$

Notation defined in class

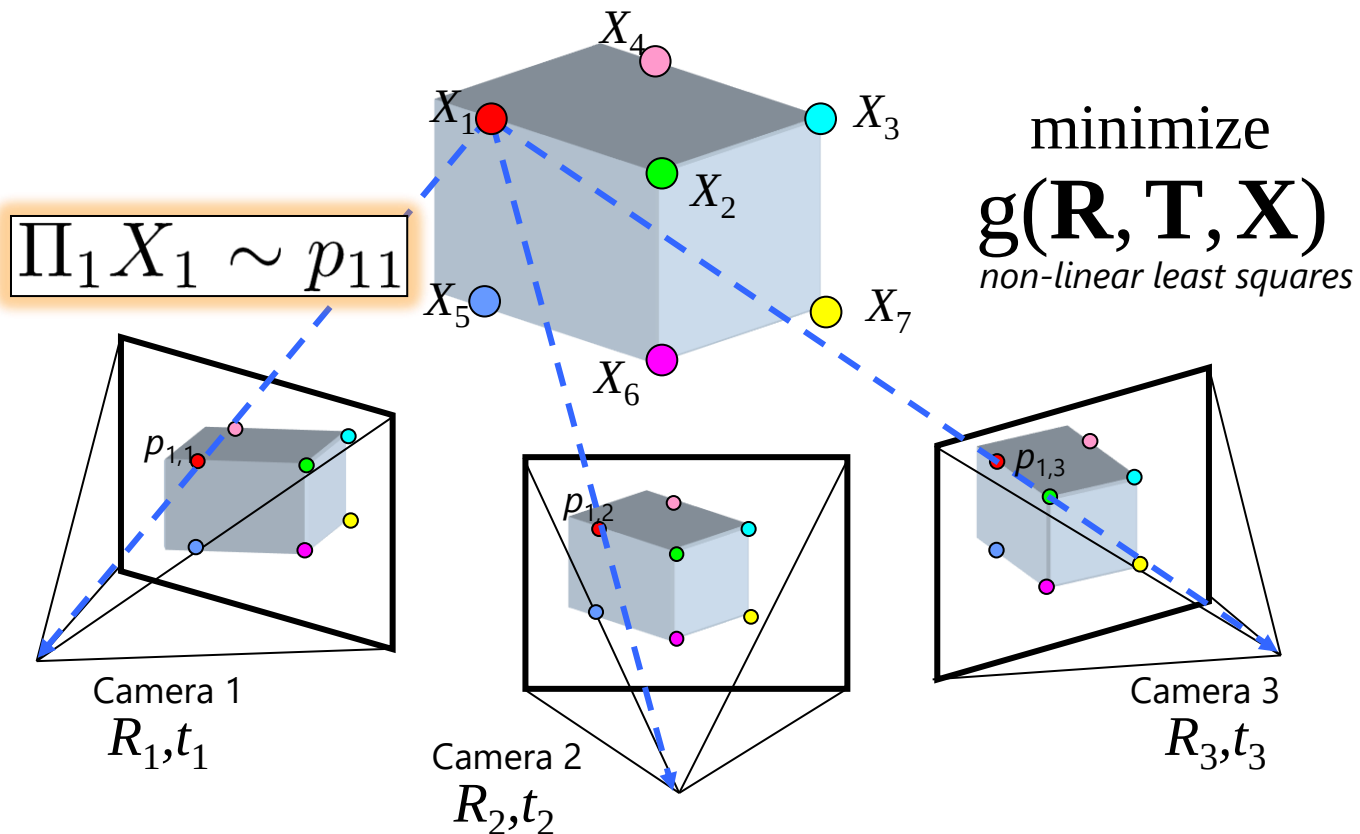
Focus of Expansion/Contraction



FOE/FOC

$T_z = 0$, FOE at infinity

Reprojection Error



Bundle Adjustment

- Minimize sum of squared reprojection errors:

$$g(\mathbf{X}, \mathbf{R}, \mathbf{T}) = \sum_{i=1}^m \sum_{j=1}^n \underbrace{w_{ij}}_{\substack{\text{indicator variable:} \\ \text{is point } i \text{ visible in image } j?}} \cdot \left\| \underbrace{\mathbf{P}(\mathbf{x}_i, \mathbf{R}_j, \mathbf{t}_j)}_{\substack{\text{predicted} \\ \text{image location}}} - \underbrace{\begin{bmatrix} u_{i,j} \\ v_{i,j} \end{bmatrix}}_{\substack{\text{observed} \\ \text{image location}}} \right\|^2$$

- Minimizing this function is called *bundle adjustment*
 - Optimized using non-linear least squares, e.g. Levenberg-Marquardt algorithm

Structure from motion: limitations

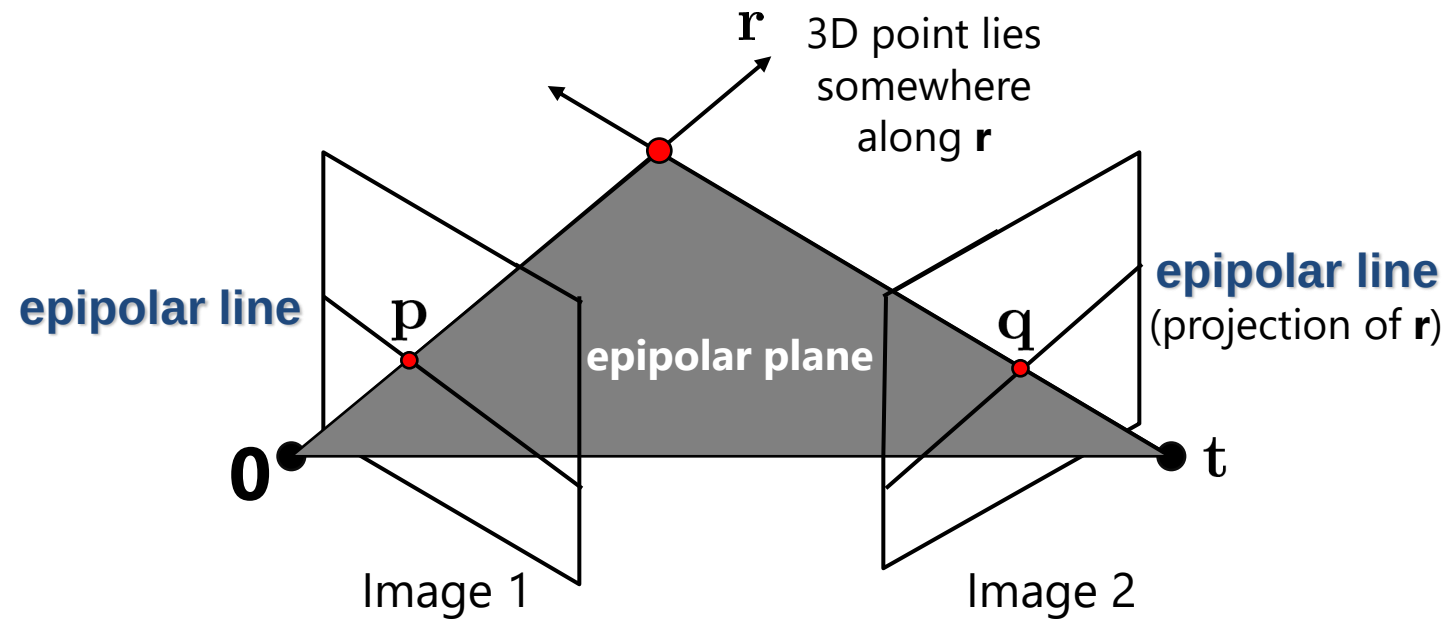
- If nothing is known (only images are available), there is an arbitrary projective ambiguity.
- Even with fully calibrated camera, we cannot recover the absolute scale.
- Need good feature tracker

Two View (Stereo) Reconstruction

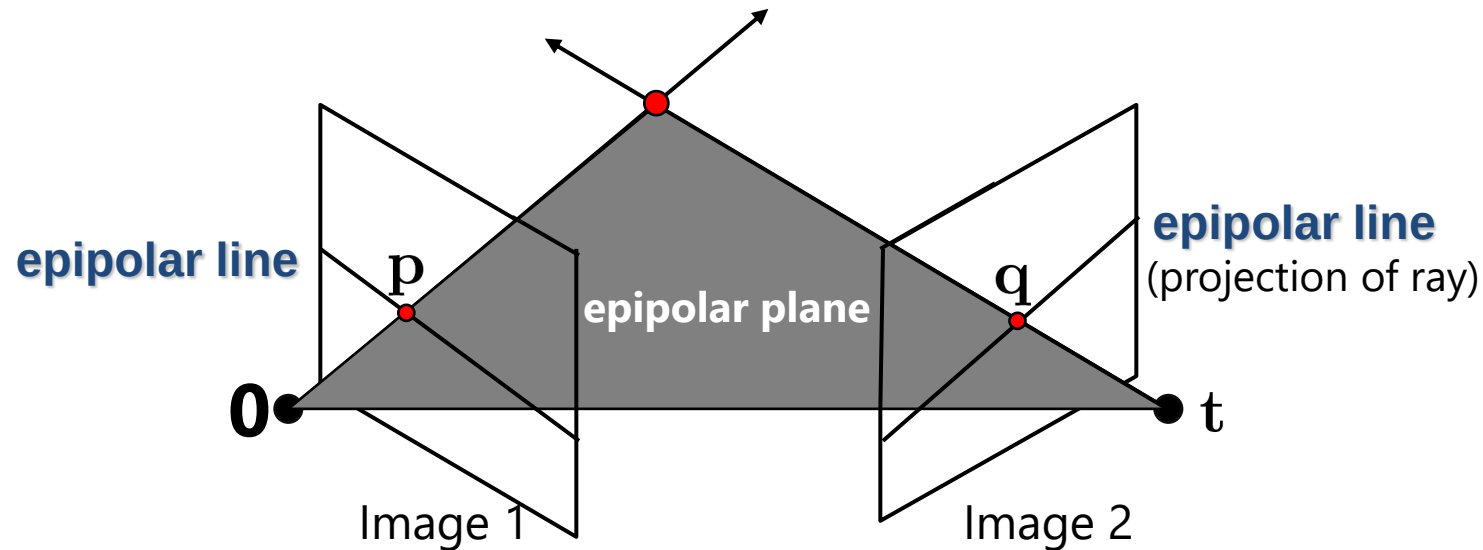


Adapted from Szeliski's book

Stereo: epipolar geometry

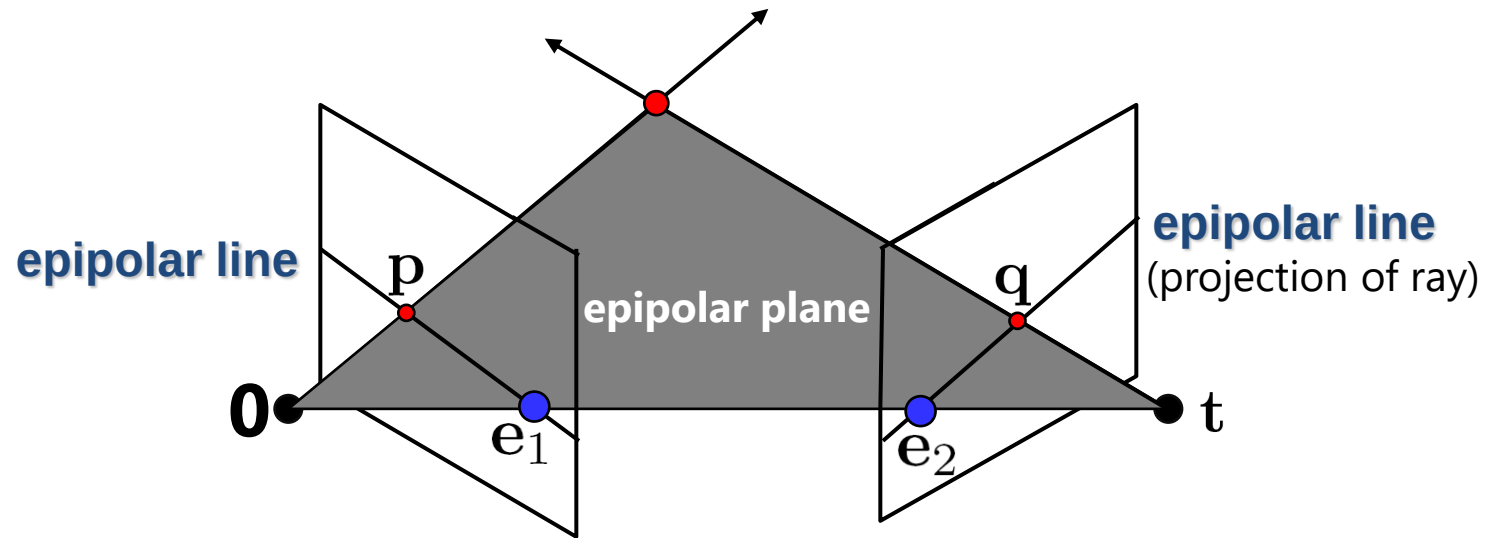


Fundamental Matrix



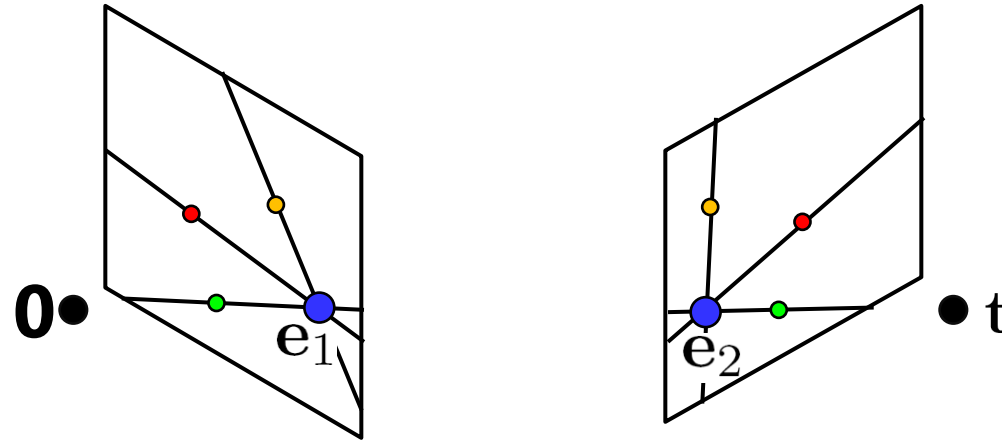
- This *epipolar geometry* of two views is described by a very special 3x3 matrix $\mathbf{F}$, called the *fundamental matrix*
- $\mathbf{F}$ maps (homogeneous) *points* in image 1 to *lines* in image 2!
- The epipolar line (in image 2) of point $\mathbf{p}$ is: $\mathbf{Fp}$
- *Epipolar constraint* on corresponding points: $\mathbf{q}^T \mathbf{Fp} = 0$

Fundamental Matrix



- Two Special points: $\mathbf{e}_1$ and $\mathbf{e}_2$ (the *epipoles*): projection of one camera into the other

Fundamental Matrix



- Two Special points: $\mathbf{e}_1$ and $\mathbf{e}_2$ (the *epipoles*): projection of one camera into the other
- All of the epipolar lines in an image pass through the epipole
- Epipoles may or may not be inside the image

Fundamental Matrix

- $\mathbf{F}\mathbf{p}$ is the epipolar line associated with $\mathbf{p}$
- $\mathbf{F}^T\mathbf{q}$ is the epipolar line associated with $\mathbf{q}$
- $\mathbf{F}\mathbf{e}_1 = \mathbf{0}$ and $\mathbf{F}^T\mathbf{e}_2 = \mathbf{0}$
- $\mathbf{F}$ is rank 2
- How many parameters does $\mathbf{F}$ have?

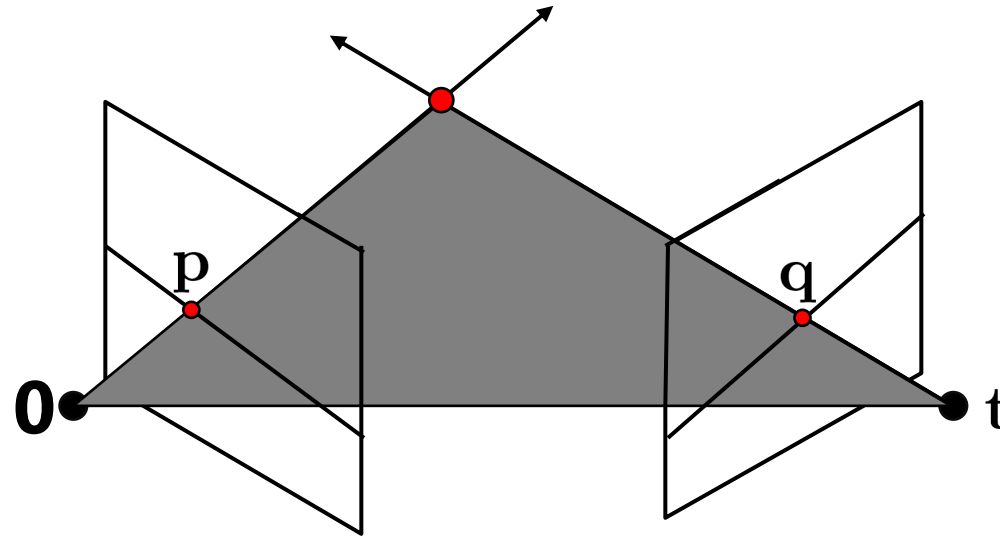
Example – Epipolar lines



Demo

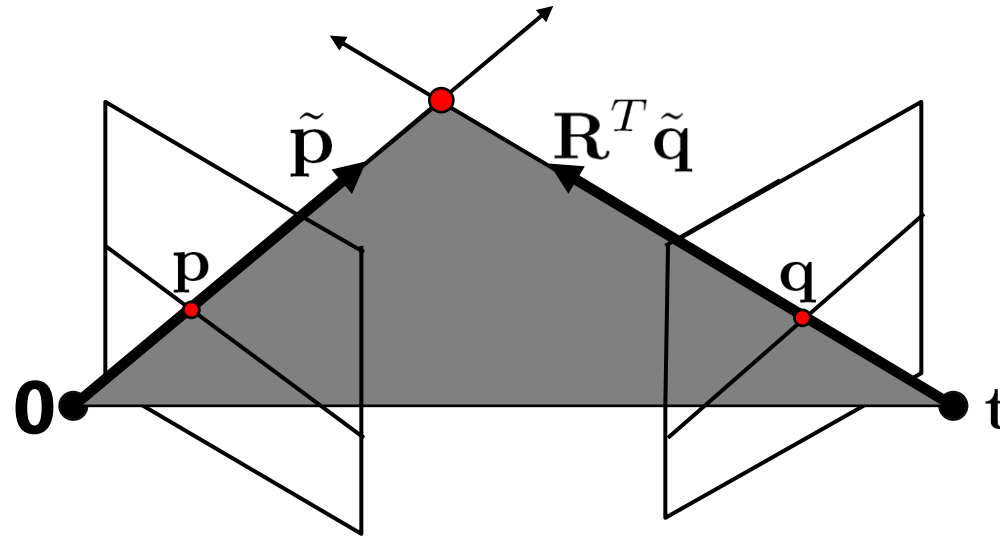
[https://www.cs.cornell.edu/courses/cs5670/2022sp/demos/
FundamentalMatrix/?demo=demo1](https://www.cs.cornell.edu/courses/cs5670/2022sp/demos/FundamentalMatrix/?demo=demo1)

Fundamental Matrix - Derivation



- Why does **F** exist?
- Let's derive it...

Fundamental Matrix - Derivation



$\mathbf{K}_1$: intrinsics of camera 1

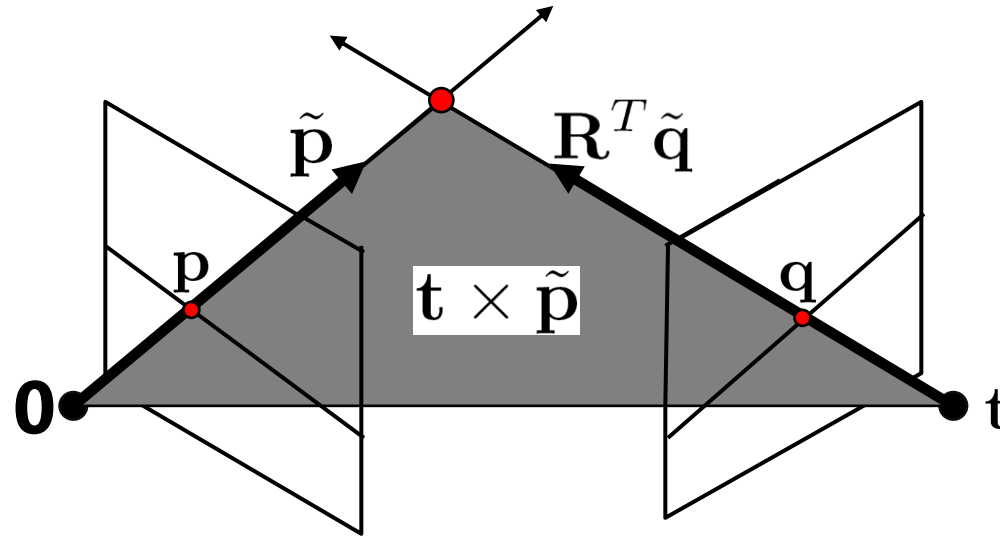
$\mathbf{K}_2$: intrinsics of camera 2

$\mathbf{R}$: rotation of image 2 w.r.t. camera 1

$\tilde{p} = \mathbf{K}_1^{-1} p$: ray through p in camera 1's (and world) coordinate system

$\tilde{q} = \mathbf{K}_2^{-1} q$: ray through q in camera 2's coordinate system

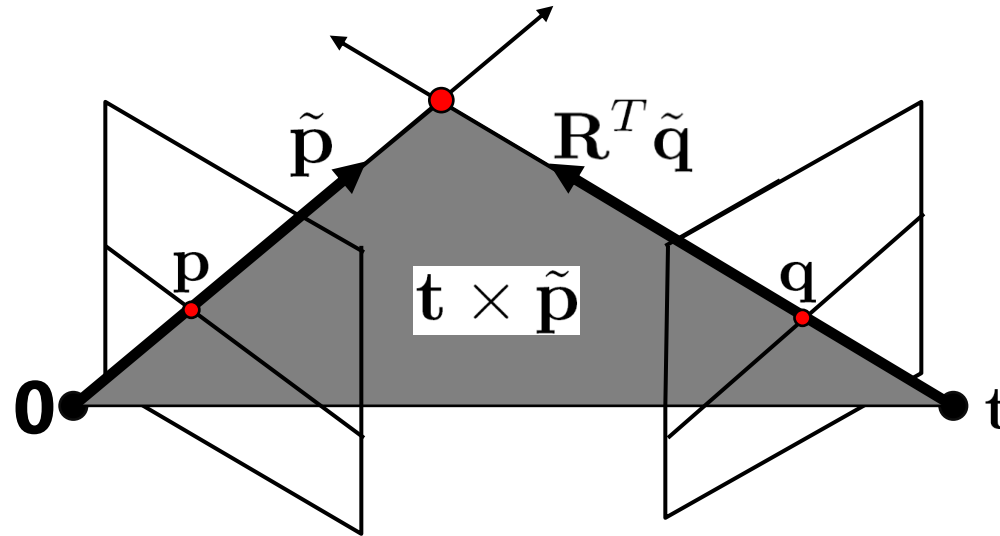
Fundamental Matrix - Derivation



- $\tilde{\mathbf{p}}, \mathbf{R}^T \tilde{\mathbf{q}}$, and $\mathbf{t}$ are coplanar
- epipolar plane can be represented as with its normal $\mathbf{t} \times \tilde{\mathbf{p}}$

$$(\mathbf{R}^T \tilde{\mathbf{q}})^T (\mathbf{t} \times \tilde{\mathbf{p}}) = 0$$

Fundamental Matrix - Derivation



$$(\mathbf{R}^T \tilde{\mathbf{q}})^T (\mathbf{t} \times \tilde{\mathbf{p}}) = 0$$



$$\tilde{\mathbf{q}}^T \mathbf{R} (\mathbf{t} \times \tilde{\mathbf{p}}) = 0$$

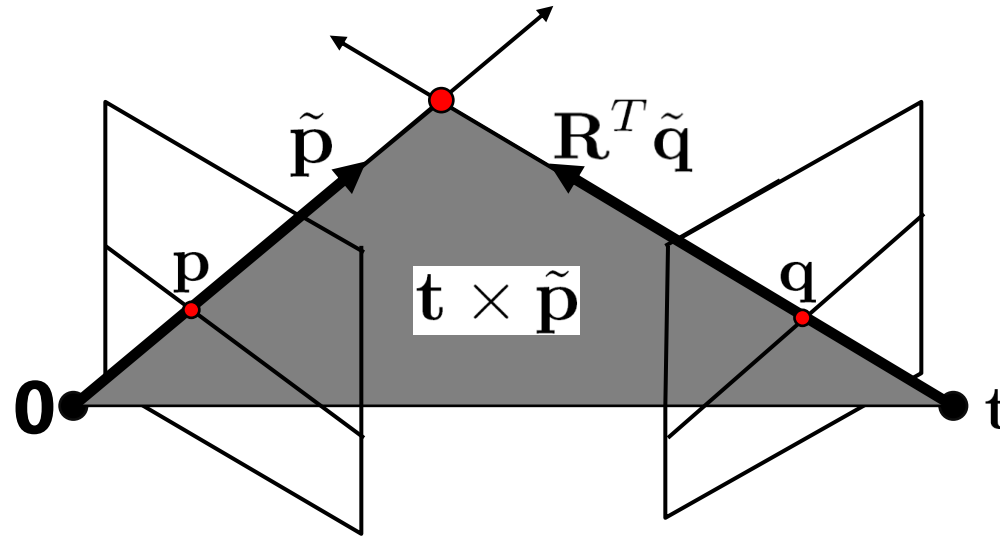
Cross-product as linear operator

Useful fact: Cross product with a vector $\mathbf{t}$ can be represented as multiplication with a (*skew-symmetric*) 3x3 matrix

$$[\mathbf{t}]_{\times} = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix}$$

$$\mathbf{t} \times \tilde{\mathbf{p}} = [\mathbf{t}]_{\times} \tilde{\mathbf{p}}$$

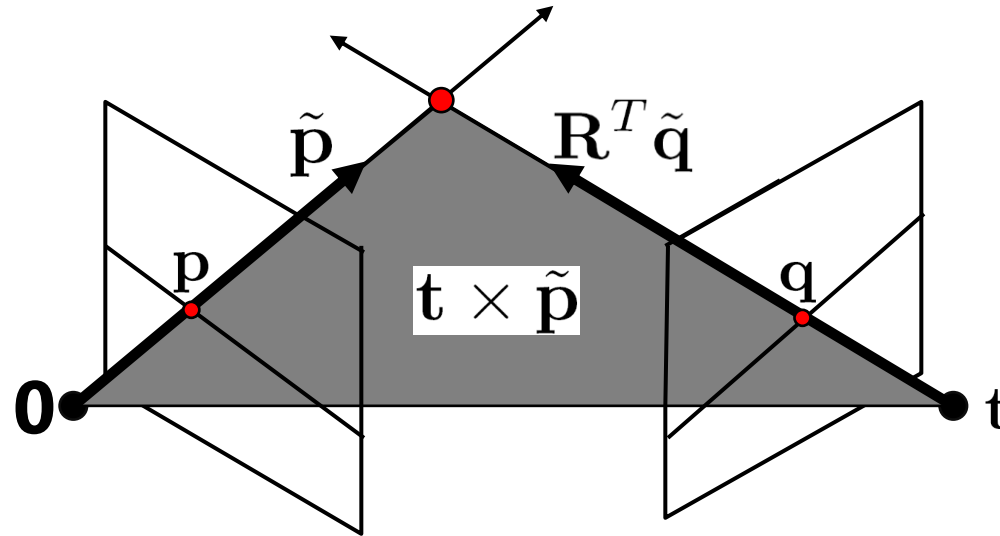
Fundamental Matrix - Derivation



- One more substitution:
 - Cross product with $\mathbf{t}$ (on left) can be represented as a 3x3 matrix

$$[\mathbf{t}]_{\times} = \begin{bmatrix} 0 & -t_z & t_y \\ t_z & 0 & -t_x \\ -t_y & t_x & 0 \end{bmatrix} \quad \mathbf{t} \times \tilde{\mathbf{p}} = [\mathbf{t}]_{\times} \tilde{\mathbf{p}}$$

Fundamental Matrix – Calibrated Cameras

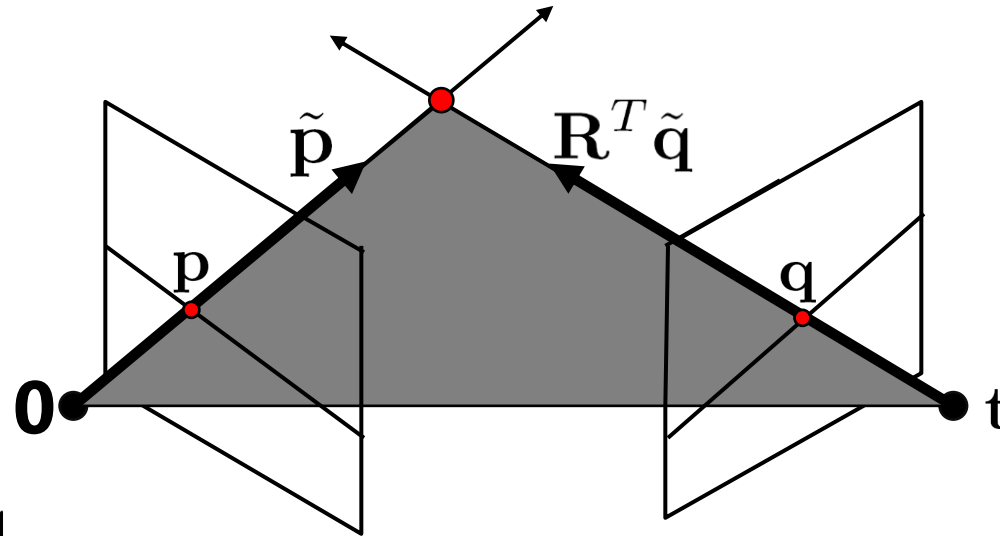


$$\tilde{\mathbf{q}}^T \mathbf{R}(\mathbf{t} \times \tilde{\mathbf{p}}) = 0$$



$$\tilde{\mathbf{q}}^T \mathbf{R} [\mathbf{t}]_{\times} \tilde{\mathbf{p}} = 0$$

Fundamental Matrix – Calibrated Cameras



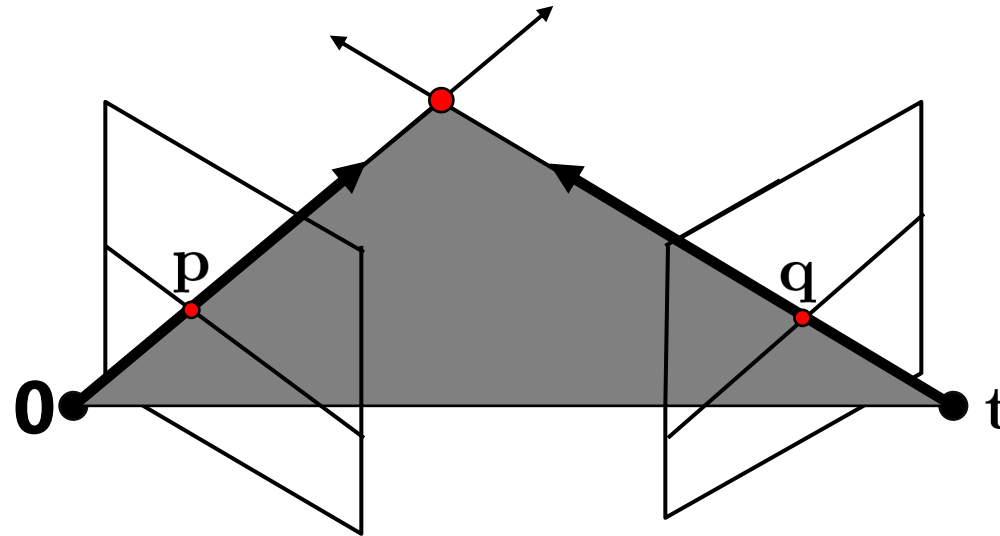
$\tilde{\mathbf{p}} = \mathbf{K}_1^{-1} \mathbf{p}$: ray through $\mathbf{p}$ in camera 1's (and world) coordinate system

$\tilde{\mathbf{q}} = \mathbf{K}_2^{-1} \mathbf{q}$: ray through $\mathbf{q}$ in camera 2's coordinate system

$$\underbrace{\tilde{\mathbf{q}}^T \mathbf{R} [\mathbf{t}]_{\times}}_{\mathbf{E}} \tilde{\mathbf{p}} = 0 \quad \tilde{\mathbf{q}}^T \mathbf{E} \tilde{\mathbf{p}} = 0$$

$\mathbf{E} \nwarrow$ the Essential matrix

Fundamental Matrix – Uncalibrated Cameras



$\mathbf{K}_1$: intrinsics of camera 1

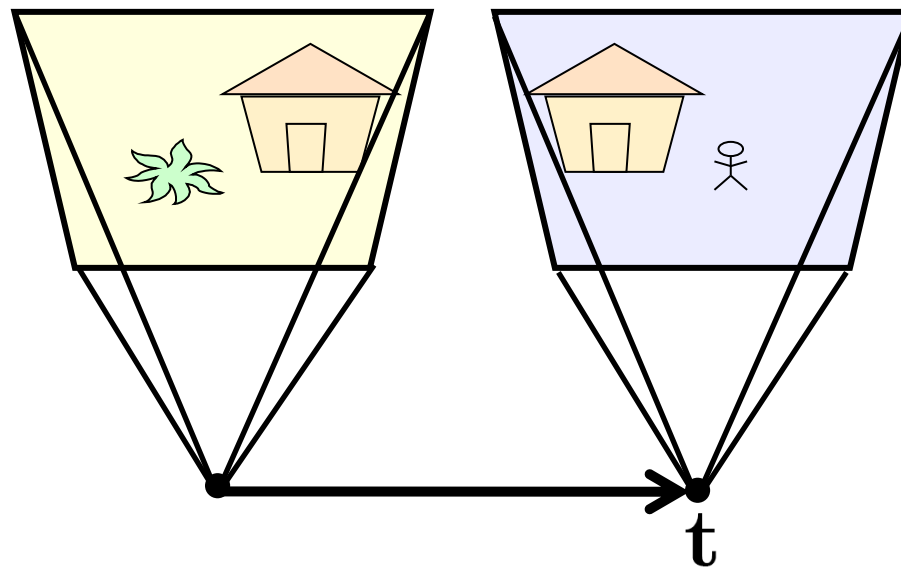
$\mathbf{K}_2$: intrinsics of camera 2

$\mathbf{R}$: rotation of image 2 w.r.t. camera 1

$$\mathbf{q}^T \underbrace{\mathbf{K}_2^{-T} \mathbf{R} [\mathbf{t}]_{\times} \mathbf{K}_1^{-1}}_{\mathbf{F}} \mathbf{p} = 0$$

$\mathbf{F}$ ← the Fundamental matrix

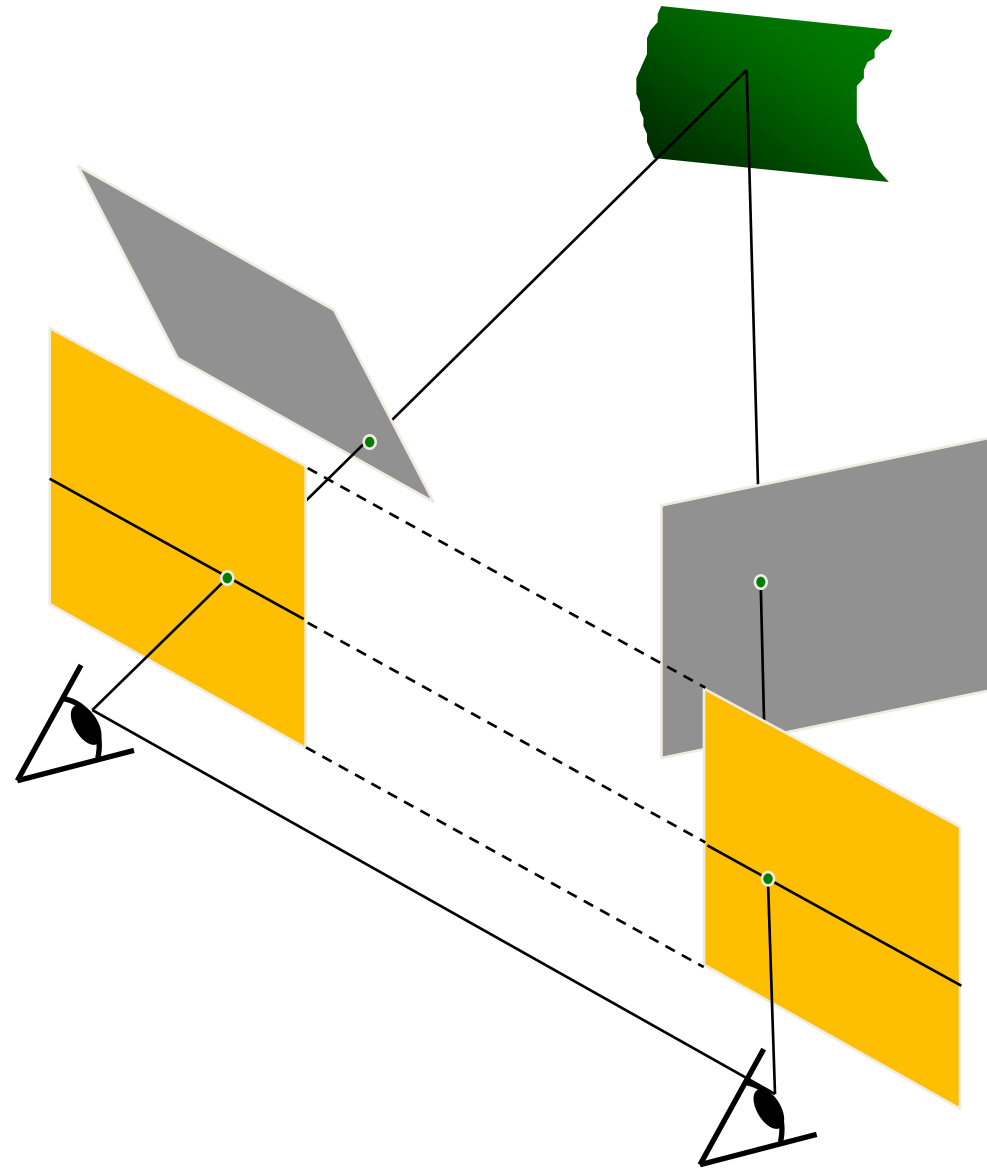
Rectified case



$$\mathbf{R} = \mathbf{I}_{3 \times 3}$$
$$\mathbf{t} = \begin{bmatrix} 1 & 0 & 0 \end{bmatrix}^T$$
$$\mathbf{E} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & -1 \\ 0 & 1 & 0 \end{bmatrix}$$

Rectification

- reproject image planes onto a common plane
 - plane parallel to the line between optical centers
- pixel motion is horizontal after this transformation
- two homographies, one for each input image reprojection





Original stereo pair



After rectification

Estimating $\mathbf{F}$



- If we don't know $\mathbf{K}_1$, $\mathbf{K}_2$, $\mathbf{R}$, or $\mathbf{t}$, can we estimate $\mathbf{F}$ for two images?
- Yes, given enough correspondences

Estimating $\mathbf{F}$

- The fundamental matrix $\mathbf{F}$ is defined by

$$\mathbf{x}'^T \mathbf{F} \mathbf{x} = 0$$

for any pair of matches $\mathbf{x}$ and $\mathbf{x}'$ in two images.

- Let $\mathbf{x} = (u, v, 1)^T$ and $\mathbf{x}' = (u', v', 1)^T$, $\mathbf{F} = \begin{bmatrix} f_{11} & f_{12} & f_{13} \\ f_{21} & f_{22} & f_{23} \\ f_{31} & f_{32} & f_{33} \end{bmatrix}$
each match gives a linear equation

$$uu'f_{11} + vu'f_{12} + u'f_{13} + uv'f_{21} + vv'f_{22} + v'f_{23} + uf_{31} + vf_{32} + f_{33} = 0$$

Estimating F – 8 point algorithm

$$\begin{bmatrix} u_1 u_1' & v_1 u_1' & u_1' & u_1 v_1' & v_1 v_1' & v_1' & u_1 & v_1 & 1 \\ u_2 u_2' & v_2 u_2' & u_2' & u_2 v_2' & v_2 v_2' & v_2' & u_2 & v_2 & 1 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\ u_n u_n' & v_n u_n' & u_n' & u_n v_n' & v_n v_n' & v_n' & u_n & v_n & 1 \end{bmatrix} \begin{bmatrix} f_{11} \\ f_{12} \\ f_{13} \\ f_{21} \\ f_{22} \\ f_{23} \\ f_{31} \\ f_{32} \\ f_{33} \end{bmatrix} = 0$$

- Like with homographies, instead of solving $\mathbf{A}\mathbf{f} = 0$, we seek unit length $\mathbf{f}$ to minimize $\|\mathbf{A}\mathbf{f}\|$: least eigenvector of $\mathbf{A}^T \mathbf{A}$.

Estimating $\mathbf{F}$ – 8 point algorithm

- $\mathbf{F}$ should have rank 2
- To enforce that $\mathbf{F}$ is of rank 2, $\mathbf{F}$ is replaced by $\mathbf{F}'$ that minimizes $\|\mathbf{F} - \mathbf{F}'\|$ subject to the rank constraint.
- This is achieved by SVD. Let $\mathbf{F} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, where

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}, \quad \text{let } \mathbf{\Sigma}' = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

then $\mathbf{F}' = \mathbf{U}\mathbf{\Sigma}'\mathbf{V}^T$ is the solution (closest rank-2 matrix to $\mathbf{F}$)

Estimating F – 8 point algorithm

```
% Build the constraint matrix
A = [x2(1,:)'.*x1(1,:) '  x2(1,:)'.*x1(2,:) '  x2(1,:) '  ...
      x2(2,:)'.*x1(1,:) '  x2(2,:)'.*x1(2,:) '  x2(2,:) '  ...
      x1(1,:) '           x1(2,:) '           ones(npts,1)  ];

[U,D,V] = svd(A);

% Extract fundamental matrix from the column of V
% corresponding to the smallest singular value.
F = reshape(V(:,9),3,3)';

% Enforce rank2 constraint
[U,D,V] = svd(F);
F = U*diag([D(1,1) D(2,2) 0])*V';
```

Estimating F – 8 point algorithm

- Pros: linear, easy to implement and fast
- Cons: susceptible to noise

Practical Considerations in Estimating Fundamental Matrix

Problem with 8-point algorithm

$$\begin{bmatrix}
 u_1 u_1' & v_1 u_1' & u_1' & u_1 v_1' & v_1 v_1' & v_1' & u_1 & v_1 & 1 \\
 u_2 u_2' & v_2 u_2' & u_2' & u_2 v_2' & v_2 v_2' & v_2' & u_2 & v_2 & 1 \\
 \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \vdots \\
 u_n u_n' & v_n u_n' & u_n' & u_n v_n' & v_n v_n' & v_n' & u_n & v_n & 1
 \end{bmatrix}
 \begin{bmatrix}
 f_{11} \\
 f_{12} \\
 f_{13} \\
 f_{21} \\
 f_{22} \\
 f_{23} \\
 f_{31} \\
 f_{32} \\
 f_{33}
 \end{bmatrix} = 0$$

~ 10000 ~ 10000 ~ 100 ~ 10000 ~ 10000 ~ 100 ~ 100 ~ 100 1

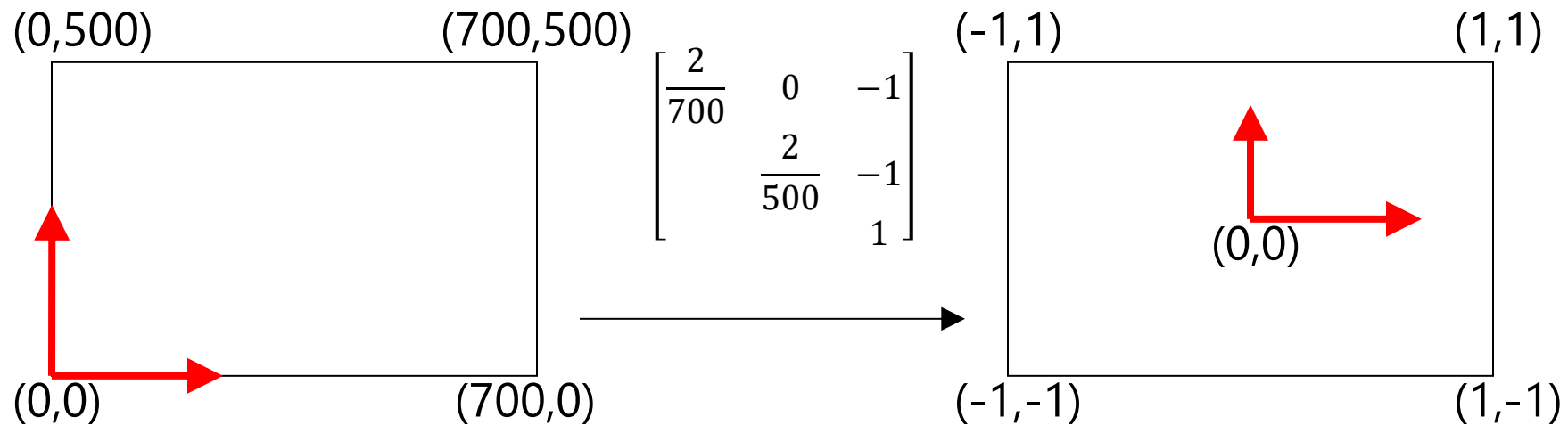


Orders of magnitude difference
 between column of data matrix
 → least-squares yields poor results

Normalized 8-point algorithm

normalized least squares yields good results

Transform image to $\sim [-1,1] \times [-1,1]$



Normalized 8-point algorithm

- Transform input by $\hat{\mathbf{x}}_i = \mathbf{T}\mathbf{x}_i$ $\hat{\mathbf{x}}'_i = \mathbf{T}\mathbf{x}'_i$
- Call 8-point on $\hat{\mathbf{x}}_i, \hat{\mathbf{x}}'_i$ to obtain $\hat{\mathbf{F}}$
- $\mathbf{F} = \mathbf{T}'^T \hat{\mathbf{F}} \mathbf{T}$

$$\mathbf{x}'^T \mathbf{F} \mathbf{x} = 0$$
$$\boxed{\hat{\mathbf{x}}'^T \mathbf{T}'^{-T}} \boxed{\mathbf{F} \mathbf{T}^{-1} \hat{\mathbf{x}}} = 0$$

$\hat{\mathbf{F}}$

Normalized 8-point algorithm

```
[x1, T1] = normalise2dpts(x1);  
[x2, T2] = normalise2dpts(x2);
```

```
A = [x2(1,:)'.*x1(1,:) '   x2(1,:)'.*x1(2,:) '   x2(1,:) '   ...  
      x2(2,:)'.*x1(1,:) '   x2(2,:)'.*x1(2,:) '   x2(2,:) '   ...  
      x1(1,:) '              x1(2,:) '              ones(npts,1)  ];
```

```
[U,D,V] = svd(A);
```

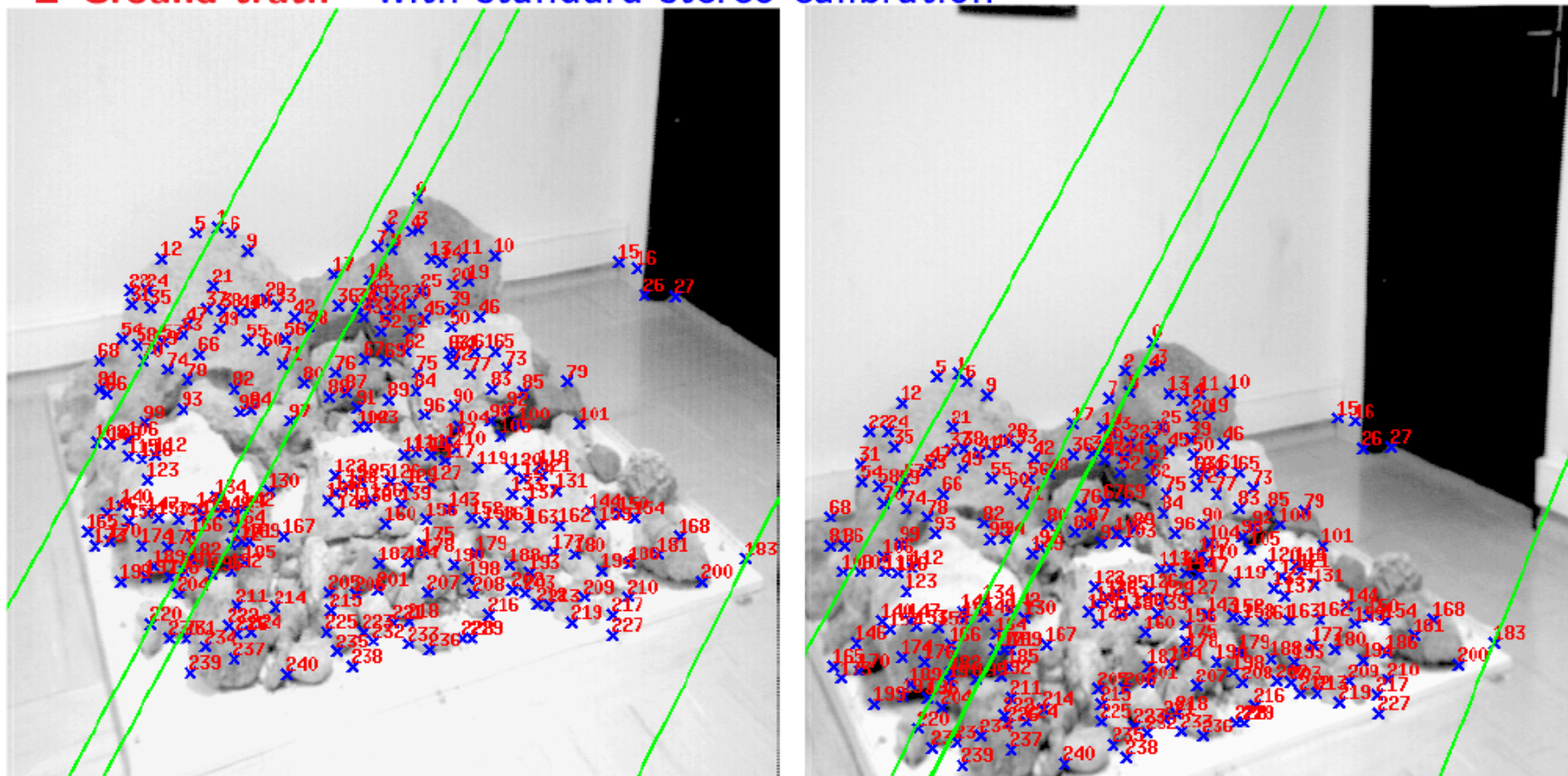
```
F = reshape(V(:,9),3,3)';
```

```
[U,D,V] = svd(F);  
F = U*diag([D(1,1) D(2,2) 0])*V';
```

```
% Denormalise  
F = T2'*F*T1;
```

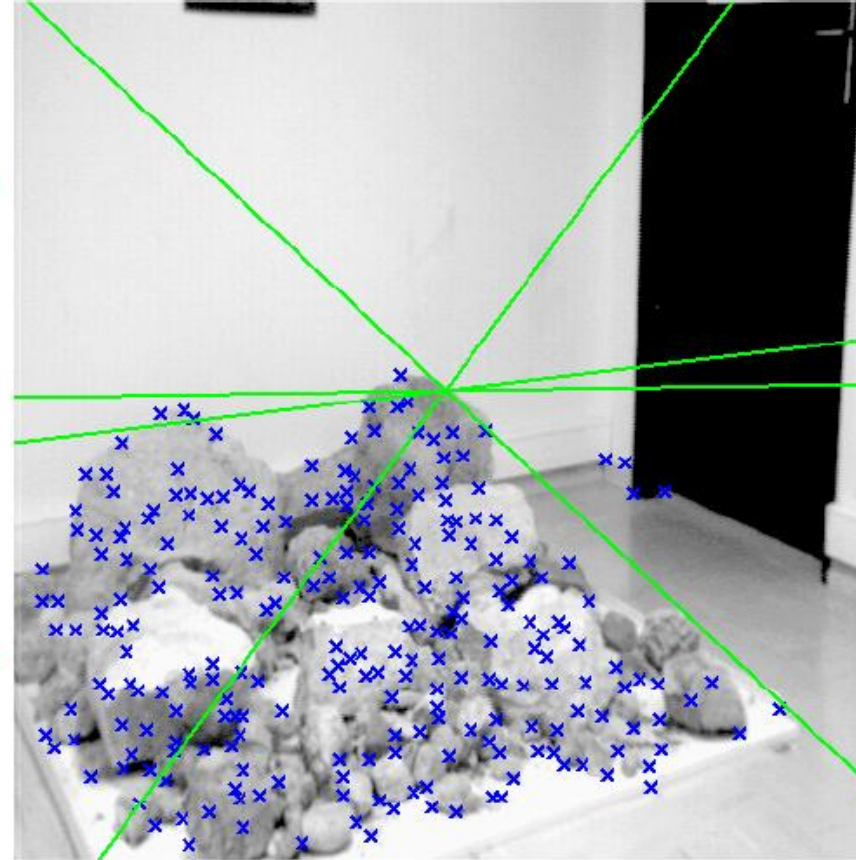
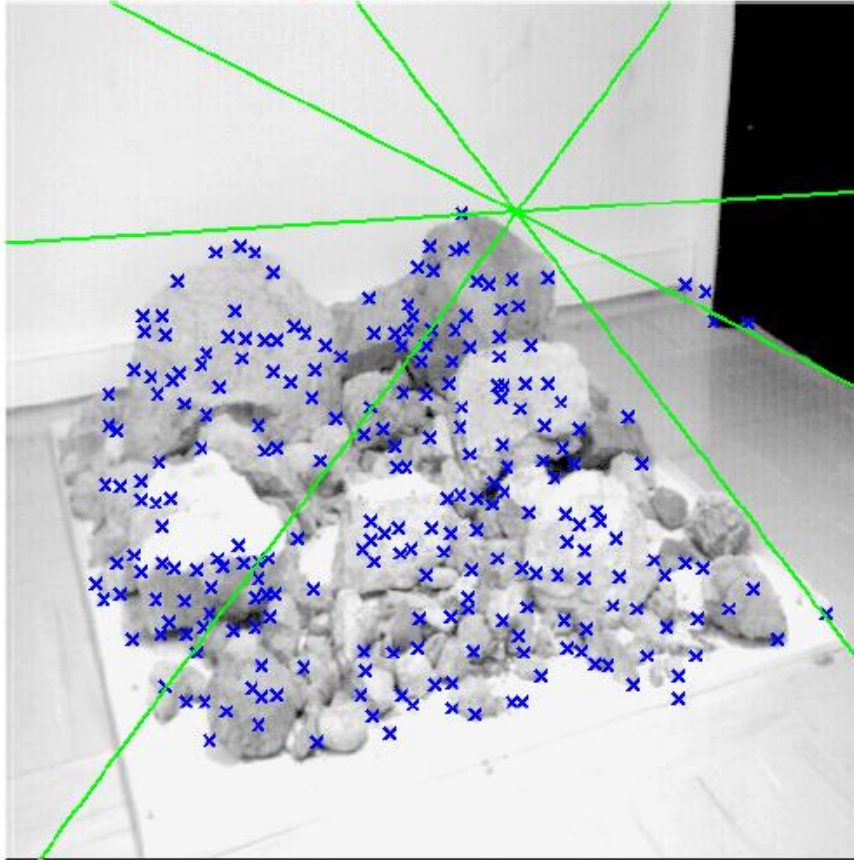
Results (ground truth)

■ Ground truth with standard stereo calibration



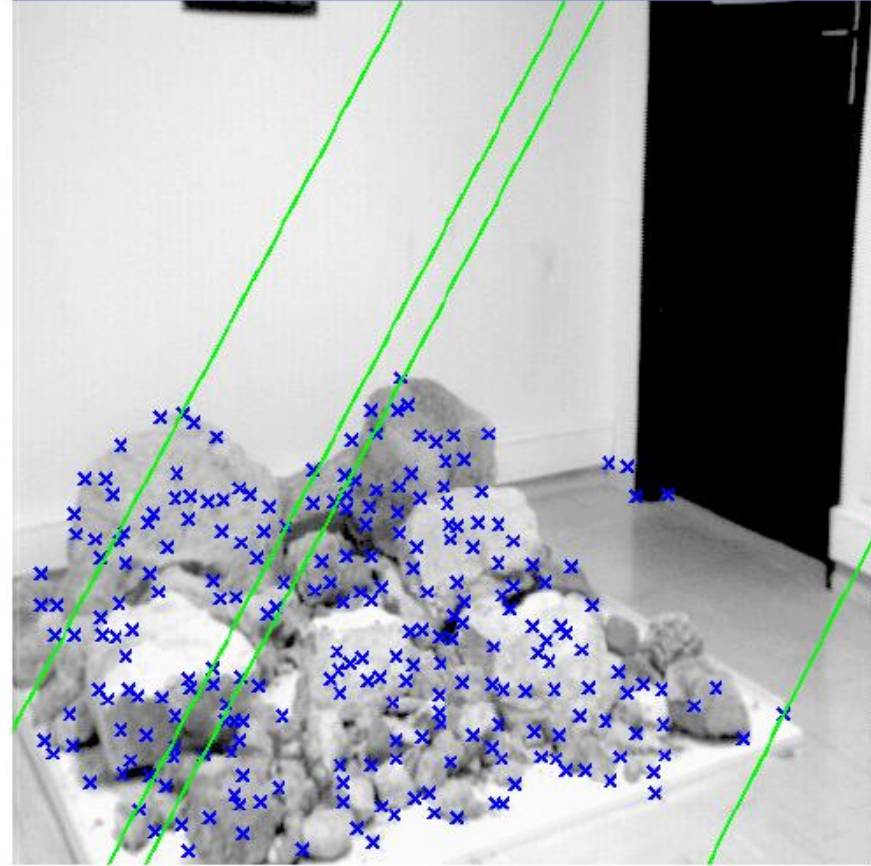
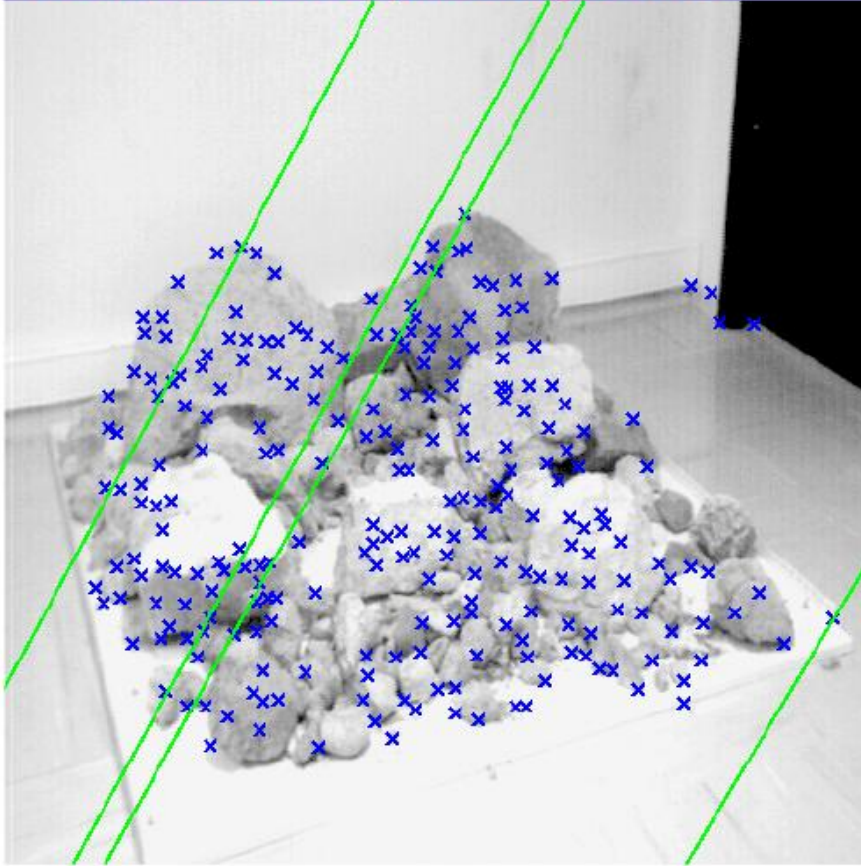
Results (ground truth)

■ 8-point algorithm



Results (normalized 8-point algorithm)

■ Normalized 8-point algorithm



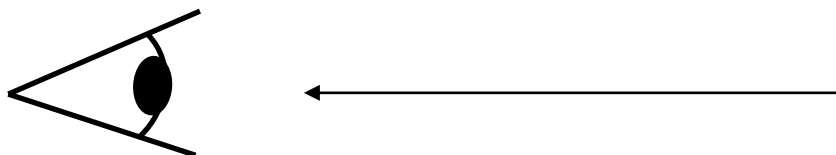
What about more than two views?

- The geometry of three views is described by a $3 \times 3 \times 3$ tensor called the *trifocal tensor*
- The geometry of four views is described by a $3 \times 3 \times 3 \times 3$ tensor called the *quadrifocal tensor*
- After this it starts to get complicated...

What is light?

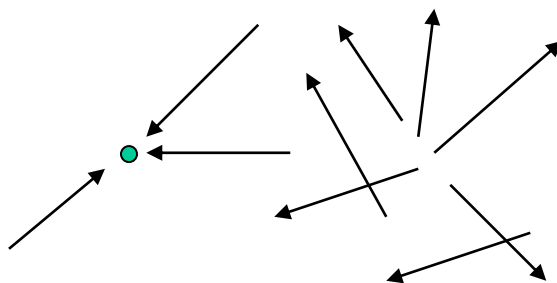
Electromagnetic radiation (EMR) moving along rays in space

- $R(\lambda)$ is EMR, measured in units of power (watts)
 - λ is wavelength



Light field

- We can describe all of the light in the scene by specifying the radiation (or “**radiance**” along all light rays) arriving at every point in space and from every direction



$$R(X, Y, Z, \theta, \phi, \lambda, t)$$

The light field

$$R(X, Y, Z, \theta, \phi, \lambda, t)$$

- Known as the **plenoptic function**
- If you know R , you can predict how the scene would appear from any viewpoint.

Additional Reading

The plenoptic function

- **Original reference:** E. Adelson and J. Bergen, "[The Plenoptic Function and the Elements of Early Vision](#)," in M. Landy and J. A. Movshon, (eds) Computational Models of Visual Processing, MIT Press 1991.
- L. McMillan and G. Bishop, "[Plenoptic Modeling: An Image-Based Rendering System](#)", Proc. SIGGRAPH, 1995, pp. 39-46.

The light field

- M. Levoy and P. Hanrahan, "[Light Field Rendering](#)", Proc SIGGRAPH 96, pp. 31-42.
- S. J. Gortler, R. Grzeszczuk, R. Szeliski, and M. F. Cohen, "[The lumigraph](#)," in Proc. SIGGRAPH, 1996, pp. 43-54.

The interaction of light and matter

What happens when a light ray hits a point on an object?

- Some of the light gets absorbed
 - converted to other forms of energy (e.g., heat)
- Some gets transmitted through the object
 - possibly bent, through “refraction”
- Some gets reflected
 - as we saw before, it could be reflected in multiple directions at once

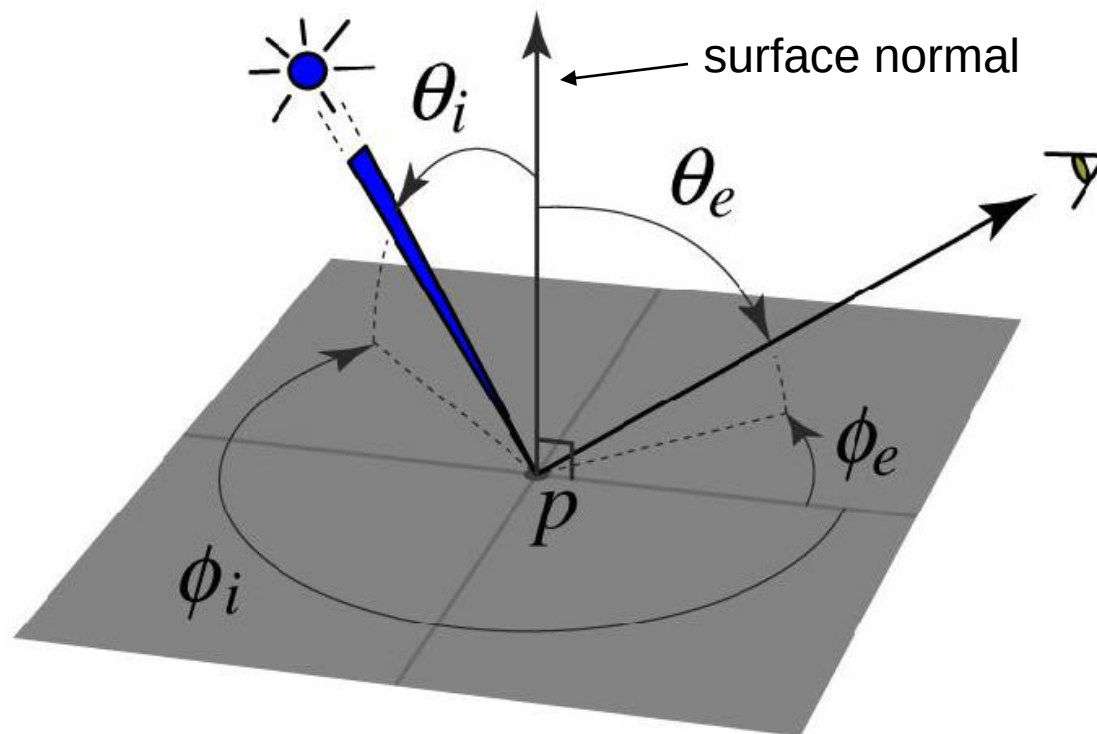
Let's consider the case of reflection in detail

- In the most general case, a single incoming ray could be reflected in all directions. How can we describe the amount of light reflected in each direction?

The BRDF

The Bidirectional Reflection Distribution Function

- Given an incoming ray (θ_i, ϕ_i) and outgoing ray (θ_e, ϕ_e)
what proportion of the incoming light is reflected along outgoing ray?



Answer given by the BRDF: $\rho(\theta_i, \phi_i, \theta_e, \phi_e)$

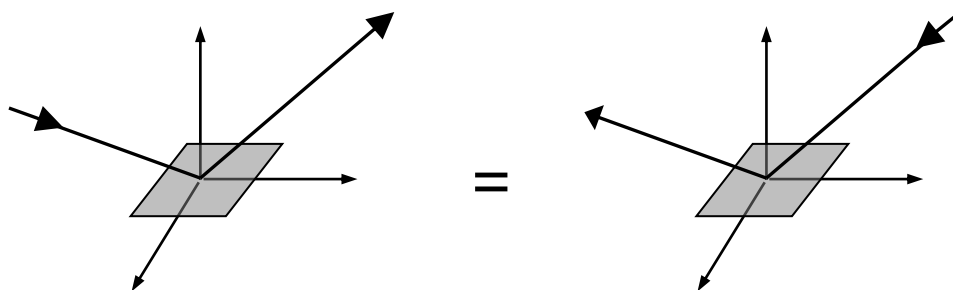
Constraints on the BRDF

Energy conservation

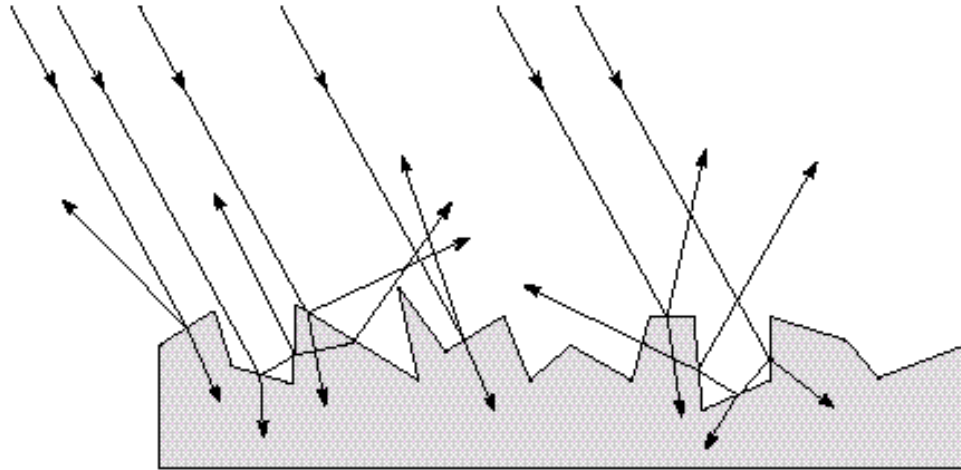
- Quantity of outgoing light $\leq$ quantity of incident light
 - integral of BRDF ≤ 1

Helmholtz reciprocity

- reversing the path of light produces the same reflectance



Diffuse reflection



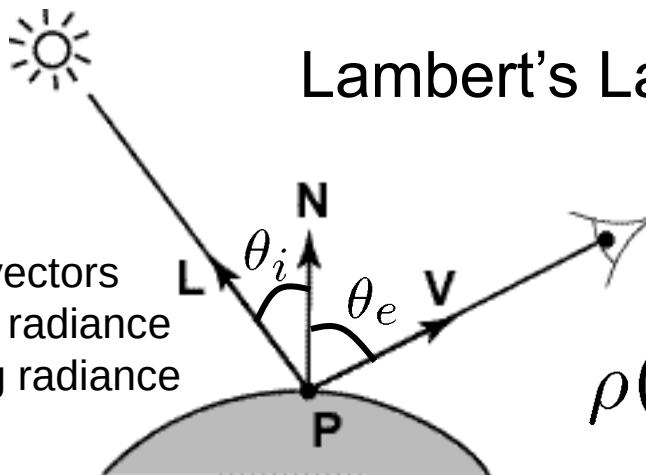
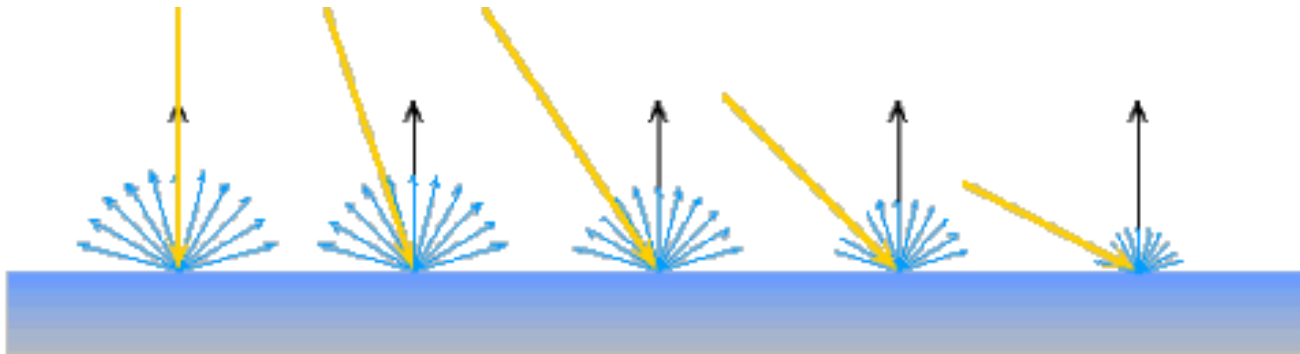
Diffuse reflection

- Dull, matte surfaces like chalk or latex paint
- Microfacets scatter incoming light randomly
- Effect is that light is reflected equally in all directions

Diffuse reflection

Diffuse reflection governed by **Lambert's law**

- Viewed brightness does not depend on viewing direction
- Brightness *does* depend on direction of illumination
- This is the model most often used in computer vision



$\mathbf{L}$, $\mathbf{N}$, $\mathbf{V}$ unit vectors
 I_e = outgoing radiance
 I_i = incoming radiance

Lambert's Law: $I_e = k_d \mathbf{N} \cdot \mathbf{L} I_i$

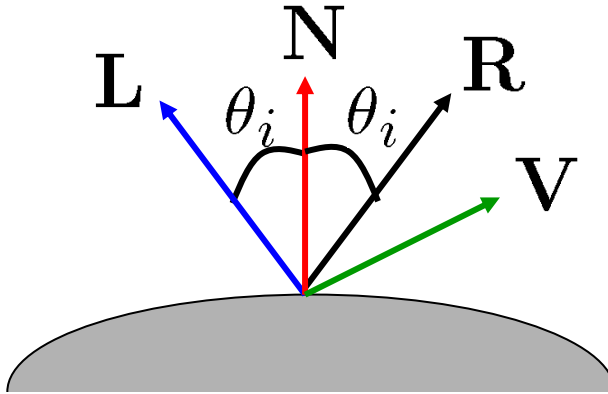
k_d is called **albedo**

BRDF for **Lambertian surface**

$$\rho(\theta_i, \phi_i, \theta_e, \phi_e) = k_d \cos \theta_i$$

Specular reflection

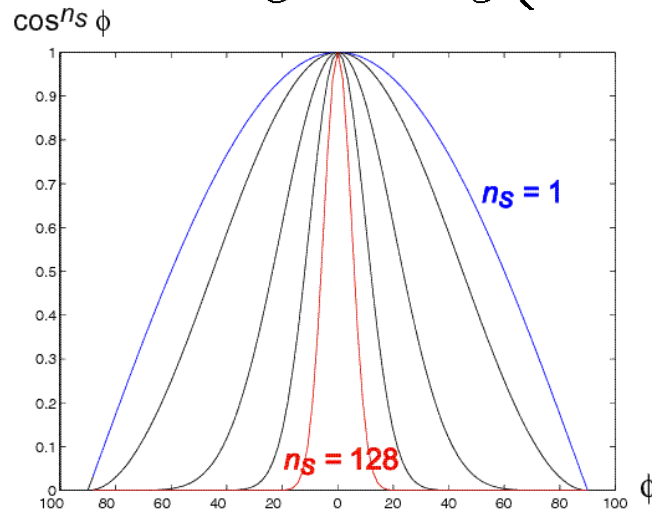
For a perfect mirror, light is reflected about **N**



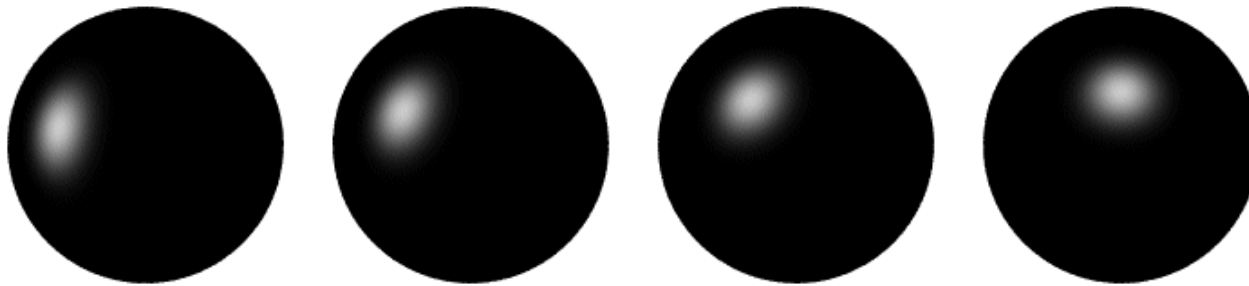
$$I_e = \begin{cases} I_i & \text{if } \mathbf{V} = \mathbf{R} \\ 0 & \text{otherwise} \end{cases}$$

Near-perfect mirrors have a **highlight** around **R**

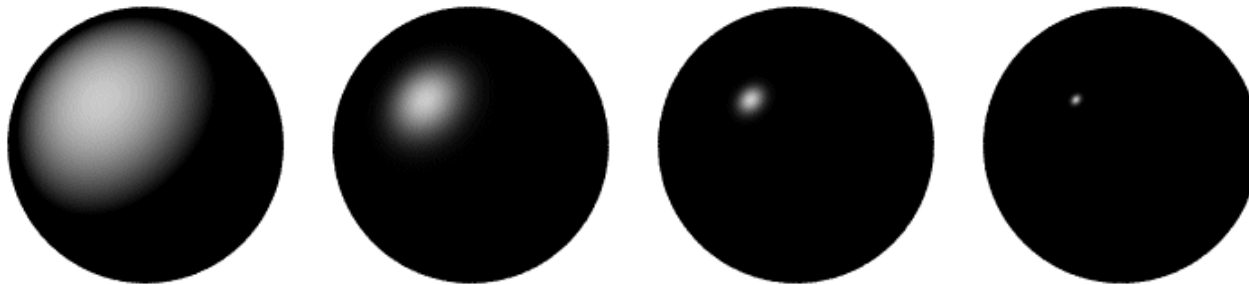
- common model: $I_e = k_s (\mathbf{V} \cdot \mathbf{R})^{n_s} I_i$



Specular reflection



Moving the light source



Changing n_s

Phong illumination model

Phong approximation of surface reflectance

- Assume reflectance is modeled by three components
 - Diffuse term
 - Specular term
 - Ambient term (to compensate for inter-reflected light)

$$I_e = k_a I_a + I_i \left[k_d (\mathbf{N} \cdot \mathbf{L})_+ + k_s (\mathbf{V} \cdot \mathbf{R})_+^{n_s} \right]$$

$\mathbf{L}$, $\mathbf{N}$, $\mathbf{V}$ unit vectors

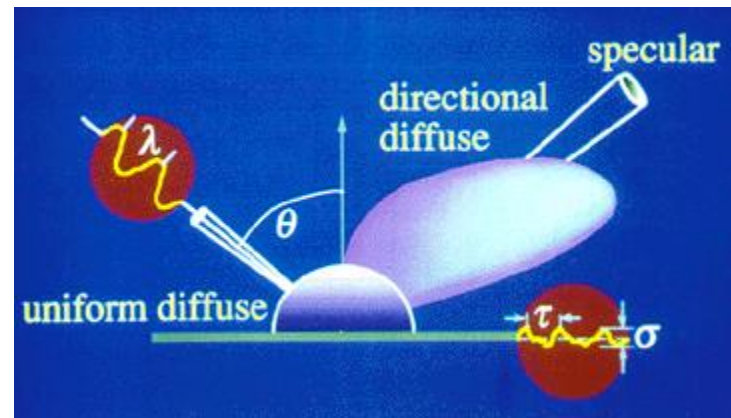
I_e = outgoing radiance

I_i = incoming radiance

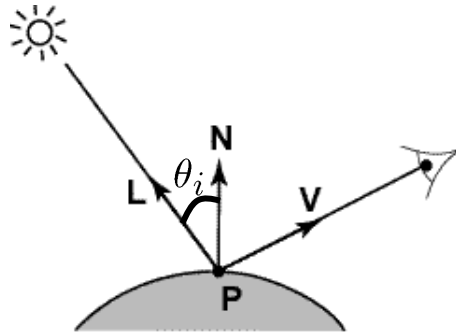
I_a = ambient light

k_a = ambient light reflectance factor

$(x)_+ = \max(x, 0)$



Shape from shading



Suppose $k_d = 1$

$$\begin{aligned} I &= k_d \mathbf{N} \cdot \mathbf{L} \\ &= \mathbf{N} \cdot \mathbf{L} \\ &= \cos \theta_i \end{aligned}$$

You can directly measure angle between normal and light source

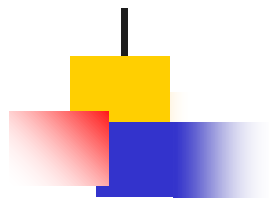
- Not quite enough information to compute surface shape
- But can be if you add some additional info, for example
 - assume a few of the normals are known (e.g., along silhouette)
 - constraints on neighboring normals—“integrability”
 - smoothness
- Hard to get it to work well in practice
 - plus, how many real objects have constant albedo?

Diffusion Models in CV

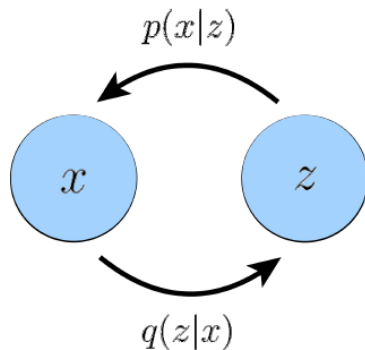


Amit K Roy-Chowdhury

Acknowledgments: <https://calvinyluo.com/2022/08/26/diffusion-tutorial.html#evidence-lower-bound>
<https://arxiv.org/abs/2403.18103>

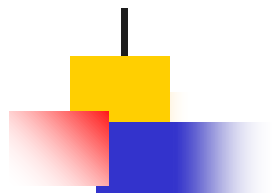


Standard VAE



$$\ln p(\mathbf{x}) \geq \underbrace{\mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\ln p(\mathbf{x}|\mathbf{z})]}_{\text{Reconstruction Error}} - \underbrace{\mathbb{E}_{\mathbf{z} \sim q_{\phi}(\mathbf{z}|\mathbf{x})} [\ln q_{\phi}(\mathbf{z}|\mathbf{x}) - \ln p(\mathbf{z})]}_{\text{KL Divergence}}$$

Objective:
$$\arg \max_{\phi, \theta} \mathbb{E}_{q_{\phi}(\mathbf{z}|\mathbf{x})} [\log p_{\theta}(\mathbf{x} | \mathbf{z})] - \mathcal{D}_{\text{KL}}(q_{\phi}(\mathbf{z} | \mathbf{x}) || p(\mathbf{z}))$$

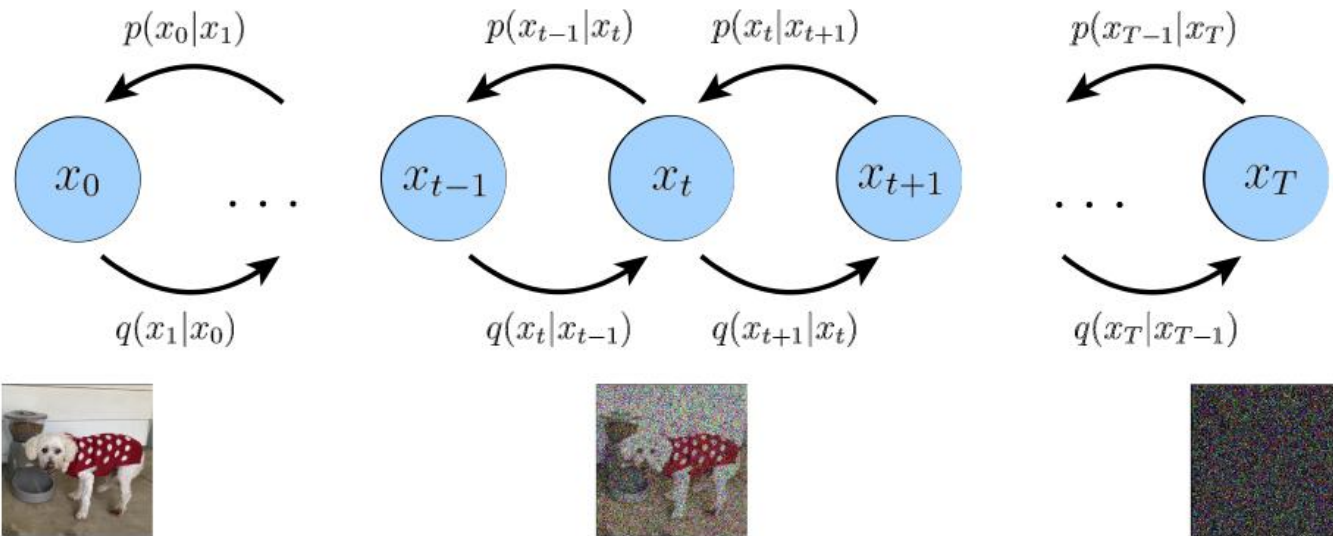


Markov Chain

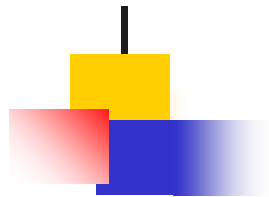
A discrete-time Markov chain $\{X_n | n = 0, 1, \dots\}$ is a discrete-time, discrete-value random sequence such that given $X_0, \dots, X_n$, the next random variable X_{n+1} depends only on X_n through the transition probability

$$P[X_{n+1} = j | X_n = i, X_{n-1} = i_{n-1}, \dots, X_0 = i_0] = P[X_{n+1} = j | X_n = i] = P_{ij}.$$

Variational Diffusion Models



- Learn $p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$
- Sample Gaussian noise from $p(\mathbf{x}_T)$ and denoise based on $p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$ for T steps



Variational Diffusion Models (VDMs)

Latent dimension is the same as data dimension: both data and latent variables as $\mathbf{x}_t$

$$q(\mathbf{x}_{1:T} \mid \mathbf{x}_0) = \prod_{t=1}^T q(\mathbf{x}_t \mid \mathbf{x}_{t-1})$$

Structure of the latent encoder at each step is a linear Gaussian model

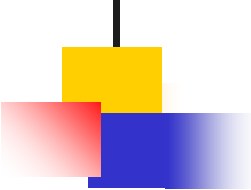
$$q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I})$$

α_t evolves such that the latent at the final timestep is a standard Gaussian.

$$p(\mathbf{x}_{0:T}) = p(\mathbf{x}_T) \prod_{t=1}^T p_{\theta}(\mathbf{x}_{t-1} \mid \mathbf{x}_t)$$

where,

$$p(\mathbf{x}_T) = \mathcal{N}(\mathbf{x}_T; \mathbf{0}, \mathbf{I})$$



Variational Diffusion Models (VDMs)

Denoising Diffusion Probabilistic Models (DDPM)

Because calculating the true exact posterior $q(x_{t-1}|x_t)$ is mathematically intractable (it requires knowing the entire data distribution), we use a neural network to parameterize and approximate a Gaussian distribution.

The formal mathematical expression for the reverse denoising process is:

$$p_{\theta}(x_{t-1}|x_t) = \mathcal{N}(x_{t-1}; \mu_{\theta}(x_t, t), \Sigma_{\theta}(x_t, t))$$

During implementation, instead of predicting μ_{θ} entirely from scratch, the neural network is trained to predict the noise $\epsilon_{\theta}(x_t, t)$ that was injected at that step. The mean is then reconstructed using the network's noise prediction via this formula:

$$\mu_{\theta}(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{1 - \alpha_t}{\sqrt{1 - \bar{\alpha}_t}} \epsilon_{\theta}(x_t, t) \right)$$

Where $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$ represents the cumulative noise schedule up to step t .

- $p_{\theta}(x_{t-1}|x_t)$: The learned probability distribution of the cleaner image x_{t-1} given the noisier image x_t .
- $\mu_{\theta}(x_t, t)$: The **mean** of the distribution, predicted by your neural network (typically a U-Net or a Diffusion Transformer). It represents the model's best guess of what the slightly denoised image features look like at timestep t .
- $\Sigma_{\theta}(x_t, t)$: The **variance** of the distribution. In the foundational DDPM paper, this is often treated as a fixed, non-learned time-dependent constant ($\sigma_t^2 \mathbf{I}$) proportional to the noise schedule, though modern variants sometimes predict it to improve sample speed and quality.

Denoising Diffusion Restoration Models (DDRM)

The core mathematical breakthrough of DDRM is how it handles the corruption. If your degraded image y is modeled as:

$$y = Hx + z$$

(Where H is the degradation operator like blurring or masking, x is the clean image, and z is noise).

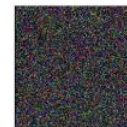
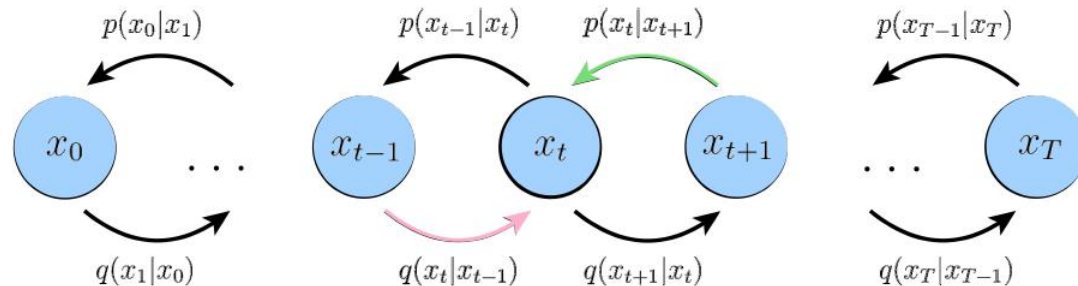
- **For uncorrupted/accessible frequencies** ($\sigma_i > 0$): It takes the known data from y and denoises it slightly, regularizing it with the model's prediction ϵ_{θ} .
- **For completely lost frequencies** ($\sigma_i = 0$, like missing pixels in a mask): It relies 100% on the pre-trained diffusion model's prediction to "hallucinate" and generate the missing details from scratch.

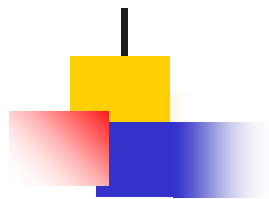


Additional Reading

ELBO for VDMs

$$\begin{aligned}
 \log p(\mathbf{x}) &\geq \mathbb{E}_{q(\mathbf{x}_{1:T}|\mathbf{x}_0)} \left[\log \frac{p(\mathbf{x}_{0:T})}{q(\mathbf{x}_{1:T} | \mathbf{x}_0)} \right] \\
 &= \underbrace{\mathbb{E}_{q(\mathbf{x}_1|\mathbf{x}_0)} [\log p_\theta(\mathbf{x}_0 | \mathbf{x}_1)]}_{\text{reconstruction term}} - \underbrace{\mathbb{E}_{q(\mathbf{x}_{T-1}|\mathbf{x}_0)} [\mathcal{D}_{\text{KL}}(q(\mathbf{x}_T | \mathbf{x}_{T-1}) || p(\mathbf{x}_T))]}_{\text{prior matching term}} \\
 &\quad - \underbrace{\sum_{t=1}^{T-1} \mathbb{E}_{q(\mathbf{x}_{t-1}, \mathbf{x}_{t+1}|\mathbf{x}_0)} [\mathcal{D}_{\text{KL}}(q(\mathbf{x}_t | \mathbf{x}_{t-1}) || p_\theta(\mathbf{x}_t | \mathbf{x}_{t+1}))]}_{\text{consistency term}}
 \end{aligned}$$





VDM Distribution

$$\mathbf{x}_t \sim q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) \quad q(\mathbf{x}_t \mid \mathbf{x}_{t-1}) = \mathcal{N}(\mathbf{x}_t; \sqrt{\alpha_t} \mathbf{x}_{t-1}, (1 - \alpha_t) \mathbf{I})$$

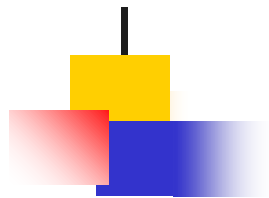
$$\mathbf{x}_t = \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon} \quad \text{with } \boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{\epsilon}; \mathbf{0}, \mathbf{I})$$

$$\mathbf{x}_{t-1} = \sqrt{\alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}} \boldsymbol{\epsilon} \quad \text{with } \boldsymbol{\epsilon} \sim \mathcal{N}(\boldsymbol{\epsilon}; \mathbf{0}, \mathbf{I})$$

Conditional Distribution:

$$q_{\phi}(\mathbf{x}_t \mid \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t \mid \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I}),$$

$$\text{where } \bar{\alpha}_t = \prod_{i=1}^t \alpha_i.$$



VDM Distribution - Proof

$$\begin{aligned}
 \mathbf{x}_t &= \sqrt{\alpha_t} \mathbf{x}_{t-1} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1} \\
 &= \sqrt{\alpha_t} (\sqrt{\alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_{t-1}} \boldsymbol{\epsilon}_{t-2}) + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1} \\
 &= \underbrace{\sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{\alpha_t} \sqrt{1 - \alpha_{t-1}} \boldsymbol{\epsilon}_{t-2}}_{\mathbf{w}_1} + \sqrt{1 - \alpha_t} \boldsymbol{\epsilon}_{t-1}.
 \end{aligned}$$

Sum of two Gaussians is a Gaussian

Mean is zero

Covariance: $\mathbb{E}[\mathbf{w}_1 \mathbf{w}_1^T] = [(\sqrt{\alpha_t} \sqrt{1 - \alpha_{t-1}})^2 + (\sqrt{1 - \alpha_t})^2] \mathbf{I}$
 $= [\alpha_t(1 - \alpha_{t-1}) + 1 - \alpha_t] \mathbf{I} = [1 - \alpha_t \alpha_{t-1}] \mathbf{I}$

Thus:

$$\begin{aligned}
 \mathbf{x}_t &= \sqrt{\alpha_t \alpha_{t-1}} \mathbf{x}_{t-2} + \sqrt{1 - \alpha_t \alpha_{t-1}} \boldsymbol{\epsilon}_{t-2} \\
 &= \sqrt{\alpha_t \alpha_{t-1} \alpha_{t-2}} \mathbf{x}_{t-3} + \sqrt{1 - \alpha_t \alpha_{t-1} \alpha_{t-2}} \boldsymbol{\epsilon}_{t-3} \\
 &= \vdots \\
 &= \left(\sqrt{\prod_{i=1}^t \alpha_i} \right) \mathbf{x}_0 + \left(\sqrt{1 - \prod_{i=1}^t \alpha_i} \right) \boldsymbol{\epsilon}_0.
 \end{aligned}$$

Define $\bar{\alpha}_t = \prod_{i=1}^t \alpha_i$

Then: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \boldsymbol{\epsilon}_0.$



$\mathbf{x}_t \sim q_{\phi}(\mathbf{x}_t | \mathbf{x}_0) = \mathcal{N}(\mathbf{x}_t | \sqrt{\bar{\alpha}_t} \mathbf{x}_0, (1 - \bar{\alpha}_t) \mathbf{I})$